VIBE CODING 教學案例 04

海上許願神社
從一個 HTML 檔
到上線的 3D 網站

水面、船、日夜、繪馬、登入、資料庫，一層一層疊上去。
每疊一層，先證明下面那層沒有壞。

成品　海中央一座 low-poly 的神社：點水面起漣漪、三艘船跟著浪起伏、白天黃昏夜晚會轉換、可以讀繪馬與祈願、附上照片，有背景音樂，點信裡的連結登入，還有一支只有帶著登入權杖才拿得到資料的 API。

使用技術　Three.js・Astro・React・Web Audio・Supabase（Auth、RLS、Storage、Edge Functions）。介面用 React，3D 場景不用 React Three Fiber

時間　閱讀約 5.2 小時，自己動手約 17.8 小時（估計值）

對象　沒有程式背景，不需要看懂 shader 或 JavaScript

完成等級　階段 18 = Learning Prototype / 階段 20 = 部署上線（目前做到這裡）/ 跑完兩個帳號互測 = Functional Prototype（尚未完成）

核心　相接的零件從同一組數字算出來；「需要登入」不等於權限控管

案例四：海上許願神社

一個很重的 3D 畫面，加上會員系統，每一步怎麼確認做對了

> 這份手冊是寫給沒有程式背景的人的。
>
> 你不需要看懂 shader，也不需要會寫 JavaScript。手冊裡會出現一些程式碼和數字。它們是用來確認結果的，不用背。

這個案例要回答的問題：

> 畫面越做越精緻，改的地方越來越多。要怎麼確定這一次的修改沒有把別的東西弄壞？

每一次修改都要問這三個問題：

> 這個數字從哪裡來？跟它接在一起的東西（例如柱子頂端和屋頂底面）是不是用同一個常數算的？改完之後，拿什麼證明其他東西沒有變？

系統架構

下面這張架構圖分成三塊。左邊是網站：使用者打開 wishingshrine.com 看到的東西。3D 畫面和按鈕都在使用者自己的電腦上跑。中間是 Supabase：繪馬、祈願、照片、會員資料都存在這裡。這個網站沒有自己的伺服器。右邊是外面的程式與服務：替網站寄登入信的 Resend，以及別人寫的程式。

別人的程式可以呼叫對外 API，也可以拿網頁裡的公開金鑰，直接讀寫資料表。真正擋住他們的，是資料庫裡的 RLS 規則：沒登入只能讀繪馬牆，自己的紀錄只有自己讀得到。

黑色實線不用登入。朱紅虛線要先登入。藍色實線是服務和服務之間的往來。圖上的專有名詞後面都接了一句白話，寫它在這個網站裡做什麼。

海上許願神社　系統架構

黑色實線：不用登入也能做。
朱紅虛線：要先登入。網頁每次向 Supabase 拿資料，都會附上登入權杖。
藍色實線：服務和服務之間的往來，表示誰把東西交給誰。

網站　使用者在瀏覽器裡看到的部分

繪馬、祈願、照片都不存在這裡，也不存在 Vercel，全部存在中間的 Supabase。

Astro　src/pages/　src/layouts/

Astro：把程式整理成瀏覽器能直接打開的網頁檔案。
首頁：先放載入畫面，再放 3D 畫面和 React 介面。
/api：另一個網頁，寫給其他程式的開發者看的 API 說明。
頁面上可以直接按「試呼叫」。
網頁標題、分享到 LINE 時的預覽圖，也在這裡設定。
做好的檔案放在 Vercel。別人打開網址，就從 Vercel 下載這些檔案。
它在你改完程式、上線之前就先做好，不在使用者的電腦上執行。

React 互動應用　src/app/

React：別人寫好的介面程式。資料一改，畫面上用到它的地方自動更新。
繪馬卡片（讀、按住祈願、寫、附一張照片）、御神籤
控制列（時段、天氣、視角、建築）、登入與帳號、聲音面板
照片上傳前先縮小，並刪掉照片裡記錄的拍攝地點（GPS）
state.js：網頁開著時暫時記住的狀態，例如現在開著哪張卡片。
關掉網頁就沒了。繪馬不存在這裡，存在 Supabase。
runtime.js：把按鈕的動作轉給 3D 引擎和 Supabase
services/：向 Supabase 讀寫資料的程式
願望一律當純文字顯示。有人在願望裡寫程式碼，也不會被執行。
一直在動的東西（海浪、卡片晃動）由 Three.js 自己更新，不經過 React。

↓ 介面叫引擎做事（例：按「夜晚」，海變暗）
↑ 引擎通知介面（例：有人拉了鈴，播鈴聲）

3D 引擎　src/engine/

Three.js：別人寫好的 3D 繪圖程式。海、船、天氣、鏡頭都靠它畫。
小船與船夫、小魚、海豚、海鷗、星空、拉鈴
海上教堂（網址加 ?theme=church），以及走進教堂後的室內
`sim/` 物理　`scene/` 場景　`camera/` 鏡頭　`render/` 畫面　`interior/` 室內
它不碰按鈕和文字。要和介面互通，只走上面這兩條線。

靜態檔　public/

直接下載、不經過程式處理的檔案：音樂、環境音、櫓聲、鈴聲、
網站圖示、分享到 LINE 時的預覽圖。

.env　設定檔

放連到 Supabase 要用的網址和公開金鑰。
每個人接自己的 Supabase 專案、填自己的 .env，所以它不上傳到 GitHub。
`PUBLIC_SUPABASE_URL`　`PUBLIC_SUPABASE_ANON_KEY`

公開金鑰（anon key）
只告訴 Supabase「這是海上許願神社的網頁」，不代表任何一個人。
它本來就會跟著網頁送到每個人的電腦，所以不是祕密。
是誰在讀寫，看登入權杖。能讀寫什麼，看資料庫的規則（RLS）。

登入：寄信、拿權杖
讀繪馬牆
掛繪馬、祈願、抽籤
上傳照片
看自己的紀錄

Supabase　替網站存資料、檢查權限的服務

這一邊叫「後端」：不在使用者的電腦上。這個網站沒有自己的伺服器，後端全部用 Supabase。

Auth　會員登入

使用者輸入信箱，點收到的信裡的連結，就登入了。信由 Resend 寄出。

登入權杖（JWT）
登入後，Supabase 給網頁的一串字，用來證明「我是誰」。
網頁之後每次拿資料都附上它。
1 小時過期，網頁會自動換一張新的。

資料庫（Postgres）　存繪馬、祈願、籤的表格

`wishes`	繪馬牆：所有人讀得到，登入才能新增
`wish_owners`	誰掛了哪一塊：只有本人讀得到
`prayers`	誰祈願過哪一塊：每人每塊一次
`omikuji_draws`	誰在哪天抽到哪支籤：每人每天一支
`omikuji_fortunes`	籤文：60 支籤的內容，可以換新版

觸發器（trigger）：資料寫進去時，資料庫自動執行的程式。
這裡用來記下是誰掛的、祈願時票數加一、同一個人 10 分鐘最多掛 5 塊。

RLS（每一筆資料的讀寫規則）
規則存在資料庫裡，不在網頁裡。拿到網頁和公開金鑰的人，也改不到。
每一次讀寫，資料庫先看登入權杖，認出是誰（程式裡寫成 auth.uid()）。
再照規則決定這一筆能不能讀、能不能改。
例：沒登入只能讀繪馬牆；「誰掛了哪一塊」只有本人讀得到。

Storage　存照片的地方

繪馬的照片都放在叫 ema-images 的資料夾（Supabase 叫它 bucket）。
知道網址就看得到，但沒有人能列出全部的照片。
只收 WebP、JPEG 兩種照片格式，2 MB 以內，而且只能放進自己的資料夾。
檔名是「帳號編號／隨機字串.webp」。資料庫的 wishes 只記這段路徑。

Edge Functions　對外 API

放在 Supabase 上執行的小程式。別的網站、App 會呼叫它。
呼叫：程式向一個網址送出要求，拿回資料。
`GET /v1/wishes`　`POST /v1/wishes`　`POST /v1/wishes/<id>/prayers`
`GET /v1/me`
GET 是讀，POST 是新增。<id> 是哪一塊繪馬；/v1/me 是自己的紀錄。
有附權杖，就用那個人的身分查；沒附，就當成沒登入的訪客。
兩種都一樣要過 RLS 的規則。

寄登入信
用到使者的分身
讀繪馬牆
附上權杖

外面的程式與服務

不在這個專案裡，但會和它往來

別的程式碰得到、碰不到什麼

底下這三樣都在 Supabase 裡。

碰得到：資料表的自動 API
Supabase 替每張表自動做的讀寫網址，
這個網站自己也在用。它沒寫進 API 文件，
但沒有關掉，拿著公開金鑰就能直接用。
擋住別人亂讀亂改的，是 RLS 的規則。

碰不到：service_role 金鑰
能無視所有 RLS 規則的金鑰。
只留在 Supabase 後台，網頁和 .env 都沒有。

碰不到：誰掛了哪一塊（wish_owners）
繪馬牆上看不到是哪個帳號掛的。
這張表只有本人讀得到，API 也不回傳。

Resend　寄信服務

替 Supabase 寄登入信。
Supabase 內建的寄信每小時只能寄 2 封左右，
而且只寄給專案成員（能進 Supabase 後台
的人，不是網站的會員）。

SMTP
電子郵件傳送的標準做法。
Supabase 照這個做法把信交給 Resend。
寄件人：`no-reply@mail.wishingshrine.com`

網域與 DNS
網域是網站的名字：wishingshrine.com。
DNS 記錄網域的設定，這個網站的在 Vercel。

DKIM、SPF、DMARC
加在 DNS 裡的三筆資料。收信的一方用它們
確認信真的是這個網域寄的，
登入信才不會被丟進垃圾信匣。

其他網站、App、curl

curl：在終端機（打指令的視窗）裡，
直接向網址要資料的指令。
讀繪馬牆：不用登入。
掛繪馬、祈願、看自己的紀錄：
要附上登入權杖。
一律走 HTTPS（加密的連線）。

開發與上線

你和 Claude Code 一起改程式。Claude Code 照 CLAUDE.md、docs/ 和開發日誌做事。改完先在 localhost:4324 看過，這是只有你自己的電腦看得到的預覽網址。
git 替程式存版本；GitHub 是放程式碼的網站。推到 GitHub 之後，Vercel 會自動抓新版本，更新 wishingshrine.com。網域也在 Vercel 買，DNS 也在 Vercel 設定。
資料表與規則：supabase/migrations/ 裡的 .sql 檔，貼進 Supabase 後台（管理這個專案的網頁）的 SQL Editor 執行。
對外 API：supabase/functions/ 裡的程式，貼進 Supabase 後台的 Edge Functions。登入信：把 Resend 的帳號資料填進 Supabase 的 SMTP 設定。

目錄

開始之前

- 學員路線圖
- 這個案例做到最後，完成的是什麼？
- 全課地圖
- 開始之前
  - 你需要準備的東西
  - 這個案例需不需要安裝東西
  - 聲音素材
  - 環境檢查（五分鐘）
  - 認識 Claude Code Desktop 的畫面
  - 接上 GitHub（階段 20 才會用到）
- 這份手冊怎麼用
  - 照著做的時候
  - 用 ChatGPT 寫規格，再交給 Claude

隨時可以翻回來查的

- 四張操作卡
  - 操作卡 1 和 AI 一起做事的七個步驟
  - 操作卡 2 哪些事 AI 可以自己做，哪些要先問你
  - 操作卡 3 一個能用的指令，有六個構件
  - 操作卡 4 卡住的時候怎麼辦

階段

1. 先看成品，再做第一版海上神社
2. 盤點、寫規格、拆檔
   - Checkpoint 1（做完階段 2）
3. 做水面與光線
4. 做漣漪與船的浮力
5. 做白天、黃昏、夜晚與燈籠

6. 把柱子、木樁、繪馬架接上
7. 做繪馬卡片、字型與介面
8. 搬進 Astro，切開引擎與介面，介面改用 React
    - Checkpoint 2（做完階段 8）
9. 做風、花瓣、樹與燈火
10. 做時段轉場與鏡頭
11. 比較介面方案，修掉縮小時的空白

- Checkpoint 3（做完階段 11）

12. 做三種船、雷雨與環境音
13. 接上背景音樂
14. 做登入介面，後端先用假的
15. 接上 Supabase 的信箱連結登入
16. 只有登入才拿得到的資料
17. 做對外 API，繪馬牆改讀資料庫
18. 繪馬附一張照片

- 一輪修正：拖拉、帳號名稱、小雨、游標、漁夫

19. 資安檢查
20. 部署到 Vercel
    - 20-8 掛自己的網域（選做）
    - 20-9 接上 Resend 寄登入信（選做）

- 上線之後：換曲子、旗子的字、海浪聲

21. 延伸：御神籤

做完之後

- 完成清單
- 離開這個範例，我會不會自己做？
- 常見卡關
- 名詞白話對照表
- 各階段預估時間

🗺️ 學員路線圖

起點	一個空資料夾，以及你在階段 1 看過的成品
產出	一個 3D 神社網站：可以旋轉、縮放、點水面起漣漪、讀繪馬與祈願。有背景音樂與信箱登入。一支要帶登入權杖才拿得到資料的 API，一支公開的對外 API `v1`。繪馬可以附一張照片。一個部署在 Vercel 的正式網站。一份 `docs/DEVLOG.md` 開發日誌
核心判斷	改完之後拿什麼證明「其他東西沒變」：逐像素比對、Node 測試、接合處的特寫截圖。階段 2 建立，之後每個階段各用一次

這個案例要你自己判斷的，主要是最後那一列。程式碼看不懂不影響。

> 關於「階段」：階段是這份手冊的章節單位。這個案例有 20 個，加上延伸的階段 21，以及兩章沒有編號的〈一輪修正〉〈上線之後〉。照順序做。

這個案例做到最後，完成的是什麼？

```
階段 18 結束          ▓▓▓
```

全課地圖

階段	產出	重點
1	跑起來的成品＋第一版海上神社	給參考連結時，讓 AI 評估要不要照做
2	`CLAUDE.md`、`docs/` 五份規格、拆成約 20 個檔案的 `src/`	先盤點再動手，拆檔前後逐像素比對
3	四道波的水面、岸邊泡沫、色調映射	波浪公式只有一份，CPU 和 shader 共用
4	點水面的漣漪、會跟著浪起伏的三艘船	船讀的是同一片水面，數學用 Node 測
5	白天、黃昏、夜晚，入夜才亮的燈籠	時段是一個 0 到 1 的數字，天氣是乘數
6	柱子、橋墩、繪馬架、船影都接上	柱子高度用屋頂的曲線算，木樁位置用碼頭板的寬度算
7	繪馬卡片、字型、縮放、介面規範 `UI_GUIDE.md`	字疊在 3D 上的對比用腳本量
8	搬進 Astro 的專案、`createShrine()` 介面、React 介面	先切開引擎與介面，再換 React；3D 不重寫
9	共用的風、會落水的花瓣、會擺動的樹	測試場地要和真實場景一樣
10	臨界阻尼的鏡頭、五個鏡位、時段轉場	SwiftShader 量不出來的效能，回報成量不出來
11	可以即時切換的介面方案 A 與 C	對比對著實際畫面量；顏色過渡用 `@property`
12	三種船、雷雨與閃電、環境音	同一種「沒接上」第四次出現，寫進 `CLAUDE.md`
13	五首背景音樂（三個時段各一首、兩首手動選）、等功率交叉淡化、聲音面板	不裝 Howler，音樂用串流；先查授權
14	登入視窗與假後端	介面只綁六個函式，後端之後再換
15	接上 Supabase 的信箱連結登入、開發用的測試帳號	公開金鑰放前端，`service_role` 不放；依賴後台的設計先打開後台確認；登入連結改用 implicit
16	三張資料表、RLS、對外 API `me`	權限寫在資料庫；API 用呼叫者的權杖
17	對外 API `v1`（五個端點）、使用量限制、讀資料庫的繪馬牆	使用量限制放在資料庫；Verify JWT 要關；分頁用時間＋id
18	繪馬附一張照片、`ema-images` 儲存桶	類型、大小、資料夾在伺服器擋；照片去 EXIF；公開儲存桶不建 select policy
一輪修正	拖拉不選字、左側拖得動、小雨、游標、漁夫	回報寫「在哪裡」；看不到的外框用 `elementFromPoint` 找

階段	產出	重點
19	互測程式 `rls-check.mjs`、清測試資料的 SQL	用另一個帳號做不該做的事，被拒絕才算通過（互測這一版沒跑）
20	GitHub 私人專案、Vercel 上的正式網站	推之前看檔案清單；部署成功後看實際的請求；`PUBLIC_` 變數補了要重新部署
上線之後	新的曲子、「潮音寄願」的旗子、交叉使用的海浪聲	「隱藏」要移出 `public/`；先量再決定哪段循環
21（延伸）	籤筒、和紙、60 張籤文、`omikuji_draws`、`v1` 的御神籤端點	籤號由資料庫決定；一天一支靠唯一約束；測試不碰正式資料

每一個動作背後的判斷，寫在各階段的〈📖 為什麼這樣做〉。

開始之前

你需要準備的東西

- 一台電腦（Mac 或 Windows 都可以），裝好 Claude Code Desktop：到 claude.com/download 下載安裝，用你的 Claude 帳號登入
- 一個付費的 Claude 方案。**免費方案打不開 Code 分頁**，需要 Pro、Max、Team 或 Enterprise 其中一種
- Windows 使用者：第一次打開 Code 分頁之前要先裝 Git for Windows，裝完把應用程式關掉重開
- Node.js（階段 1 跑成品時就要用，成品是 Astro 專案；你自己的專案從階段 8 開始用。下面〈環境檢查〉會確認，沒裝的話照那裡的方法裝）
- Chrome 或 Edge（要用開發者工具：F12 打開，會用到 Console、Network）
- 一個 Supabase 帳號（階段 15 之後才要。免費方案就夠。會用到後台左側選單的 SQL Editor、Authentication、Edge Functions）
- 一個收得到信的信箱（階段 15、20 收登入信）
- 終端機：Mac 的「終端機」，Windows 的 PowerShell（階段 16 跑 curl、階段 19 跑互測）
- 你自己的環境音與背景音樂（階段 12、13 才要，見下方〈聲音素材〉）
- 一個 GitHub 帳號與一個 Vercel 帳號（階段 20 才要。免費方案就夠。GitHub 先照下面〈接上 GitHub〉登入一次）

兩個會反覆出現的觀念

這裡各用一句話先講完，用到時手冊會再解釋一次：

- **主迴圈**：瀏覽器每畫一張新畫面就呼叫你一次的機制，一般螢幕一秒 60 次。水面、船、鏡頭這些會動的東西都在這裡更新。
- **公開金鑰與 RLS**：打包進網頁的那把金鑰本來就公開，任何人按 F12 都看得到。擋住別人的是資料庫的規則（RLS）。階段 15、16 會實際設定。

這個案例需不需要安裝東西

你自己的專案在階段 1 到 7 不用裝任何東西。專案是一個 HTML 檔加上 `src/` 裡的模組，用 importmap 從網路載入 three.js。從階段 8 開始，套件改用 npm 安裝，理由寫在階段 8。

階段 1 跑成品時例外：成品已經是 Astro 專案，Claude 會先幫它裝好套件。

> **名詞**：Astro 是把你寫的程式整理成網頁檔案的工具。在這個案例裡，它產生首頁和 `/api` 兩個網頁，兩頁的標題、圖示、分享到 LINE 的預覽圖寫在同一份版型裡。
> 你需要知道到什麼程度：知道它需要先安裝套件，啟動之後會有一個本機網址。其他由 Claude 處理。

聲音素材

這個案例要五個環境音、五首背景音樂。課程附件不提供這些檔案，你要自己準備。

成品資料夾裡也沒有它們。打開成品時，聲音面板顯示載入失敗是正常的，3D 場景照常運作。

你可以用 Suno 或其他 AI 音樂生成工具，把下面的指令貼進去生成。用之前先看那個工具的使用條款，確認生成的檔案可以用在公開的網站上。免費方案常常只能個人使用，這一點要先確認。

生成之後，檔名照下表改好，放進專案裡對應的資料夾：

用途	檔名	放在	哪一步用到
海浪，安靜的底層	`sea-1.mp3`	`public/audio/`	階段 12
海浪，有一道小浪的底層	`sea-3.mp3`	`public/audio/`	〈上線之後〉（階段 12 先用這一個代替參考成品當時的 sea-2）
偶爾出現的大浪	`wave-crash.mp3`	`public/audio/`	〈上線之後〉
雨	`rain.mp3`	`public/audio/`	階段 12
雷雨	`rain-thunder.mp3`	`public/audio/`	階段 12
白天的曲子	`floating-shrine-day.mp3`	`public/audio/music/`	階段 13
黃昏的曲子	`glass-wind-chimes.mp3`	`public/audio/music/`	階段 13
夜晚的曲子	`floating-shrine-night.mp3`	`public/audio/music/`	階段 13
手動選的曲子 1	`frost-on-the-windowpane.mp3`	`public/audio/music/`	階段 13

用途	檔名	放在	哪一步用到
手動選的曲子 2	`after-the-rain.mp3`	`public/audio/music/`	階段 13

檔名是參考成品用的。你可以換成自己的名字，跟 Claude 說清楚哪個檔案是做什麼用的就好。

兩段海浪的底層要能循環，開頭和結尾的音量要接得起來。生成兩次，分別存成 `sea-1.mp3`、`sea-3.mp3`：

貼給 AI

```
Calm ocean waves gently lapping on a small rocky island, no music, no melody, no voices, no birds.
Steady and quiet, suitable as a seamless loop. Length about 8 to 10 seconds.
The start and the end must have the same loudness so it loops without a jump.
Export as mp3.
```

偶爾出現的大浪不用循環。一道浪從小漲到大、打上岸就結束：

貼給 AI

```
A single larger ocean wave rolling in and crashing on rocks, no music, no melody, no voices.
It starts quiet, builds up, and ends with the crash. Length about 10 seconds.
This clip will be played occasionally, not looped. Export as mp3.
```

雨聲不能有雷：

貼給 AI

```
Steady light-to-medium rain falling on water and wooden roofs, no thunder, no music, no melody, no voices.
Even intensity from start to end, suitable as a seamless loop. Length about 10 seconds. Export as mp3.
```

雷雨要讓雷在前幾秒出現。網站會在雷響的那一刻打閃電，雷的位置固定，比較好對準：

貼給 AI

```
Heavy rain with one or two rolls of thunder, no music, no melody, no voices.
The first thunder should start within the first 3 seconds. Length about 8 to
10 seconds.
This clip will be played occasionally, not looped. Export as mp3.
```

背景音樂五首。白天、黃昏、夜晚各一首，另外兩首給使用者手動選。把 `〈…〉` 換成那一首的情境，例如 `daytime`、`warm sunset`、`night`、`frosty winter morning`、`quiet after the rain`：

貼給 AI

```
Quiet Japanese ambient music for a 〈daytime / warm sunset / night / frosty winter
morning / quiet after the rain〉 scene at a small shrine on the sea.
Soft instruments such as koto, shakuhachi, piano, wind chimes or pads. Slow tempo,
no drums, no vocals.
It is background music for a website, so it must not draw attention to itself.
Length 2 to 4 minutes. The ending should flow back into the beginning so it can
loop. Export as mp3.
```

參考成品一開始用過一首商業錄音，後來換成自己生成的曲子。這是授權的反例，見階段 13 的〈⚖️ 動手前先想〉與〈上線之後〉。

環境檢查（五分鐘）

先做完這一步再開始。 環境沒弄好就往下走，後面每一步都會出錯，而且錯誤訊息看起來跟環境無關。

動手

打開 Claude Code Desktop，按左上角的 **Code** 分頁，開一個新對話。**Project folder** 隨便選一個空資料夾就好。它在輸入框上方那一排，位置見下一節的定位圖。把這段貼進去：

貼給 AI

你只需要修改：沒有。整段照貼，四個問題一個都不要刪。

```
幫我檢查這台電腦能不能做網站開發。請逐項回報，先不要幫我安裝任何東西：

1. Node.js 有沒有裝？版本是多少？（我需要 22.12 以上）
2. npm 能不能正常執行？
3. 我對目前這個資料夾有沒有寫入權限？
4. 能不能連到 npm 的伺服器？（隨便查一個套件的版本就好）

每一項都用「可以」或「不行」開頭。
不行的話告訴我：問題是什麼、對一個沒有程式背景的人來說最簡單的解法是什麼。
```

四項都「可以」就準備好了。

常見狀況

訊息	意思	怎麼辦
`command not found: npm`	這台電腦沒有裝 Node.js	到 nodejs.org 下載 **LTS** 版本（長期支援版，比較穩），一路按下一步
版本是 18、20 這種數字	太舊了	同上，重裝一次 LTS 版本
Windows 上 Code 分頁打不開	少了 Git for Windows	到 git-scm.com 安裝，裝完把應用程式重開
`EACCES` 或 `permission denied`	那個資料夾不讓你寫東西	換一個資料夾，例如在桌面上新建一個
安裝一直卡住不動	網路擋住了 npm	換一個網路（手機熱點通常可以）。公司電腦的話問一下 IT
Views 選單裡沒有 Browser 面板	應用程式版本太舊	選單列的 **Claude → Check for Updates**（Windows 是 **Help → Check for Updates**）
硬碟空間不足	一個專案大約要 200–300MB	清一些空間再來

認識 Claude Code Desktop 的畫面

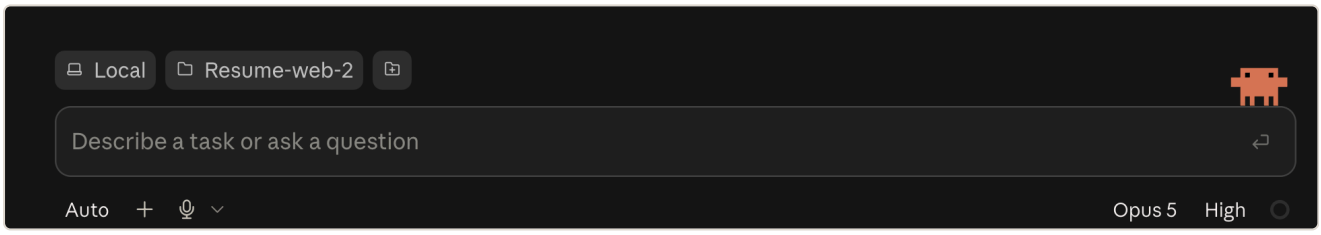
打開應用程式之後，**視窗左上角有 Home 和 Code 兩個分頁。這個案例只會用到 Code 分頁。**

（`Chat` 和 `Cowork` 不是分頁。它們是 Home 畫面輸入框右下角的兩個切換鈕，這個案例不會用到。）

🖥️ 介面定位圖：主要會碰到六個地方

主要會用到下面六個位置，找不到某一項時回來看這張表。Views 選單裡還有一個 Terminal，只有手冊寫「你自己打」的指令才會用到。

> **你要看的是什麼**：① 在**視窗最左上角**，和 Home 並排。打開應用程式時會停在 **Home**，**每次都要先按一下 Code**，這份手冊講的所有東西才會出現在你眼前。



> **你要看的是什麼**：② ③ ④ 三個設定全部集中在**視窗最下方這一條**，不在任何選單裡。左上角的 `Local` 是 Environment。旁邊的資料夾名稱（圖上是 `Resume-web-2`）是 Project folder，再旁邊那個小按鈕可以換資料夾。輸入框**下面**那一排的最左邊（圖上是 `Auto`）就是權限模式選單。大部分時間要把它切成 **Plan** 或 **Manual**。右下角的 `Opus 5` 是模型，用預設的就好。

#	名稱	在哪裡找	用途
①	**Code**（分頁）	視窗**左上角**，和 Home 並排（見上面第一張圖）	整個案例只待在這個分頁。開啟時停在 Home，要先按 Code
②	**Environment**	輸入框**上方**那一排的第一個（上圖是 `Local`）	選 **Local**，檔案才會建在你自己的電腦上
③	**Project folder**	緊接在 Environment 右邊（上圖是 `Resume-web-2`）	指定 Claude 只能在哪個資料夾裡工作
④	**模式選單**（Plan / Manual …）	輸入框**下方**那一排的最左邊（上圖是 `Auto`），隨時可切	手冊寫「切到 Plan」的地方先用 **Plan** 看計畫，確認後切 **Manual**。其餘直接用 **Manual**

#	名稱	在哪裡找	用途
⑤	**Views → Browser**	選單列的 **Views**（上圖沒拍到）	直接預覽你做的網站，可以拖到對話框旁邊並排
⑥	**Views → Diff**	同一個 Views 選單（上圖沒拍到）	看 Claude 這次改了哪幾行。**這是你最主要的驗收工具**

開始一個對話之前，先設定三件事

位置都見上面的定位圖，這裡只講要選什麼：

1. **Environment** 選 `Local`
2. **Project folder** 選那一階段手冊指定的資料夾
3. **模式選單** 見下一段

模型用預設的就好。

權限模式：決定 AI 什麼時候要先問你

模式選單在輸入框**下方**那一排的最左邊（見上面的介面定位圖），隨時可以切換。

模式	白話	用在哪
Plan	只規劃，不動手	手冊寫「切到 Plan」的地方用它，先看 Claude 打算怎麼做
Manual	每一個改動都先問你	確認計畫之後執行用這個。其餘時間都用它
Accept edits	自動接受改檔，但執行指令還是會問	熟了之後可以換成這個，比較快
Auto	自動執行，背後有安全檢查	這個案例用不到
Bypass permissions	什麼都不問	不要用

> **Plan 模式只套用在目前這個對話**，開新對話要重選。其他模式會記住你在這個資料夾的選擇。

> 舊版本的介面把這幾個模式叫做 Ask permissions、Auto accept edits、Plan mode。名字不同，意思一樣。

三個好用的面板

從 **Views** 選單打開，可以拖到旁邊並排著看：

- **Diff**：Claude 這次改了哪幾行，一行一行對照。**這是你最主要的驗收工具**
- **Browser**：直接預覽你做的網站，不用另外開瀏覽器
- **Terminal**：自己打指令的地方。手冊寫「你自己打」的指令才會用到

Claude 動手之前會問你

在 Manual 模式下，Claude 每次要改檔案或執行指令都會停下來。它會跳出一張卡片等你決定。

看不懂那張卡片在說什麼的時候，不要按允許。 直接在對話框問它：「你剛剛那個動作是要做什麼？為什麼需要？」

接上 GitHub（階段 20 才會用到）

階段 20 要讓 Claude 建一個 GitHub 專案，再把程式碼推上去。它用的是 GitHub 官方的指令列工具 `gh`。這一節把它裝好、登入一次。之後不用再做。

> **名詞**：`gh` 是 GitHub 官方的指令列工具。Claude 用它建專案、推程式碼，不用開網頁。
> 你需要知道到什麼程度：知道它要先登入一次。登入在瀏覽器裡完成，密碼不會經過 Claude。

📋 貼給 AI（裝 gh 並登入）

貼給 AI

你只需要修改：沒有。整段照貼。**第 1 點「我同意之後再裝」不要刪。**

```
幫我安裝 GitHub 的指令列工具 gh，然後登入我的 GitHub 帳號。

1. 先檢查 gh 是不是已經裝了。沒有的話，告訴我你要用什麼方式安裝、
   要執行什麼指令，我同意之後再裝
2. 登入用瀏覽器的方式：gh auth login --web --git-protocol https --hostname github.com
   它會印出一組一次性代碼和一個網址，請把這兩個原樣給我看
3. 我在瀏覽器完成之後，用 gh auth status 確認登入成功
```

你自己做（在瀏覽器完成登入）

1. 打開 Claude 給你的網址（`https://github.com/login/device`）
2. 用你的 GitHub 帳號登入，貼上那組代碼，照畫面指示授權
3. 回到 Claude Code，跟它說「好了」

> **它要你輸入密碼或 token 怎麼辦**：不要在對話框裡輸入。回它「請改用瀏覽器登入的方式」。

✅ 怎麼知道做對了

- [ ] `gh auth status` 顯示已經登入 github.com，後面是你的帳號名稱

🆘 如果沒有成功

你可能會看到	通常是因為	怎麼辦
Mac 上 Claude 說沒有 Homebrew	這台電腦沒有 Homebrew 這個安裝工具	到 cli.github.com 下載安裝檔自己裝，裝完回來從第 2 點繼續
瀏覽器說代碼無效	代碼過期了，或用了上一次的代碼	「代碼過期了，請重新執行一次登入」
Windows 或公司電腦裝不了 `gh`	安裝可能需要 IT 或系統管理員權限	「請說明用 winget 安裝 gh 的方式；如果裝不了，教我改用 GitHub 網站手動建立專案」

這一步失敗可以跳過嗎：可以，只影響階段 20。

四張操作卡

這四張卡不是這個案例的內容，是和 AI 一起做事的通用方法。 四個案例裡完全一樣，你之後做自己的專案也用得上。建議印出來，或者截圖存在手機裡。

不用現在背下來。先掃過一遍知道有這四張，遇到下面這些情況再翻回來：

情況	操作卡
要開始一件新的工作，不知道從哪一步下手	**操作卡 1**：七個步驟的順序
Claude 說它要做某件事，你不確定該不該讓它做	**操作卡 2**：哪些可以放手、哪些要你先點頭
想不出指令怎麼寫，或者寫了但它答非所問	**操作卡 3**：一個能用的指令有哪六塊
卡住超過 20 分鐘	**操作卡 4**：卡住的三步處理法
做到後面，Claude 開始違反前面講好的規則	**操作卡 4**：最後一節〈對話太長的時候怎麼辦〉

手冊各階段的 🆘 只處理那一步特有的問題。**手冊沒寫到的情況，回來看這四張。**

操作卡 1｜和 AI 一起做事的七個步驟

```
Step 1  先把需求講清楚
        要做什麼 / 不做什麼 / 有什麼限制 / 怎麼算完成

Step 2  新的、大的改動，先讓 AI 規劃，不要直接改
        什麼時候切到 Plan：
          要做新功能、要大範圍修改、規格還沒談定
        切到 Plan 模式，並明講：
        「先不要修改任何檔案。請告訴我你準備怎麼做、
          會改哪些檔案、每一步的目的，以及可能的風險。」
        手冊寫「這一步用 Manual」的階段，
        是照已經談定的規格或計畫做，直接跳到 Step 4

Step 3  看過計畫，再切回 Manual 讓它執行
        看三件事：有沒有誤解需求 / 有沒有多做不需要的事 /
        有沒有要碰不該碰的檔案或外部資源
        確認完一定要切回 Manual，Plan 模式下它不會動任何檔案
```

```
Step 4  一次只做一小段
        執行 → 看結果 → 驗收 → 確認 → 再往下一步
        不要一次叫 AI 重做整個專案

Step 5  照手冊的驗收條件檢查
        不要用「好像可以」「看起來差不多」
        要用看得到的結果、量得到的數字、跑得出來的測試
        不知道該看哪裡、該怎麼量，就把這一句貼過去：
        「請列出這次改動裡，我在畫面上看不出來、
          但會出錯的地方，以及我要怎麼試。」

Step 6  失敗時不要猜
        把錯誤訊息整段貼給 Claude，再加一句：
        「請先不要修改任何東西。先告訴我最可能的三個原因，
          以及你準備先檢查哪一個。」

Step 7  修完之後重新驗收一次
        「沒有錯誤訊息」不等於「做對了」
```

操作卡 2｜哪些事 AI 可以自己做，哪些要先問你

層級	內容
AI 可以自己做	讀取專案檔案 / 修改本機檔案 / 執行 build（打包） / 執行測試 / 執行檢查 / 分析錯誤 / 建立本機的存檔紀錄（commit）
執行前要你確認	安裝新的工具 / 改動電腦的全域設定 / 大量刪除檔案 / 存取專案資料夾以外的地方
一定要你點頭，它才能做	建立 GitHub 專案 / 第一次把程式碼推上去 / 建立雲端資源 / 部署到公開網址 / 修改正式資料 / 付費 / 發送訊息 / 刪除遠端資源

第三層的原則是：**對外的、要花錢的、不可逆的，AI 不能自己決定。**

重點不是「一定要你親手按」，而是**它動手之前你必須先看到它要做什麼**。這幾件事做下去就收不回來，你要有機會在執行前喊停。正確的說法是：

> 請你先準備好需要的內容與指令，把即將進行的對外操作說明給我聽，我確認之後你再執行。

操作卡 3｜一個能用的指令，有六個構件

手冊上的每一個 Prompt 都是照這六個構件組出來的。**要寫自己的指令時，照同一個順序填。**

#	構件	寫什麼
1	要做什麼	一句話講完
2	不要做什麼	負面清單。這一項最常被漏掉，也最有效
3	限制是什麼	技術、數值、要遵守哪份文件
4	先規劃還是先動手	「先不要改任何檔案，先告訴我你打算怎麼做」
5	怎麼算完成	看得到、量得到、跑得出來的條件
6	哪裡要停下來問我	對外的、要花錢的、要裝東西的

手冊上的 Prompt 怎麼改成你自己的

- `〈…〉` 裡的內容**一定要換掉**
- 其餘的可以照你的專案改寫，但**六個構件盡量都留著**
- **不確定哪裡能動**：手冊上每一段可以貼的 Prompt，上面都有一行「**你只需要修改**：…」。照那一行做就好，沒被點到的部分整段照貼，不用看懂

三個馬上可以用的句型

貼給 AI

先不要寫程式碼。用沒有程式背景的人聽得懂的方式，
告訴我你打算怎麼做、分成哪幾步、每一步在解決什麼問題。
我確認之後你再寫。

貼給 AI

只做 **〈這一段〉**，其他的等一下再說。

貼給 AI

有錯先告訴我是什麼，不要直接改。

操作卡 4｜卡住的時候怎麼辦

依序做這三步

第 1 步　照那個階段的「🆘 如果沒有成功」做一次

第 2 步　把下面的 Recovery Prompt 貼給 Claude，一次只處理一個原因

第 3 步　還是不行 → 先跳過

每個階段的 🆘 都會註明「這一步失敗可以跳過嗎」

卡超過 20 分鐘還沒解決，繼續卡下去沒有意義

Recovery Prompt

貼給 AI

我照著教材做到這一步，但沒有得到預期的結果。

我沒有程式背景。

教材說我應該看到：

〈貼上手冊裡那一階段的驗收條件〉

我實際看到的是：

〈描述畫面，或整段貼上錯誤訊息〉

請先不要修改任何東西。先告訴我：

1. 最可能是哪三個原因
2. 你準備先檢查哪一個

3. 為什麼是這一個
4. 你需要我提供什麼資訊

一次只處理一個原因。

「整段貼上錯誤訊息」不要偷懶。 真正有用的線索，常常不在最紅、最顯眼的那一行。它在上面幾行不起眼的路徑或版本號裡。

「先不要修改任何東西」也是刻意的。 讓 AI 自己想辦法繞過去，它可能會裝一堆你不知道的東西，或是改掉某個設定。問題會藏得更深。先知道原因，再讓它改。

對話太長的時候怎麼辦

症狀：做到後面，Claude 開始違反前面講好的規則，或是重做已經做過的事。對話越長，前面講過的內容在它眼裡佔的份量越低。

情況	做法
同一件工作還沒做完，對話已經很長	在對話框輸入 `/compact`。它會把前面的對話壓縮成摘要，接著做
這一件做完了，要開始下一件	先確認談好的規則都寫進 `CLAUDE.md`，再輸入 `/clear` 或開一個新對話

`/clear` 之後，Claude 只看得到 `CLAUDE.md` 和專案裡的檔案。談好但沒寫進檔案的決定，換了對話就不見了。接著上一段的 Prompt，例如「照你剛剛列的做」，要和前一段在同一個對話裡做。中間不要 `/clear`。換對話之前先問一句：

貼給 AI

我要開一個新對話繼續做。

這個對話裡談定、但還沒寫進 CLAUDE.md 或任何檔案的決定，請列出來。

我確認之後，請把需要保留的寫進 CLAUDE.md。

這份手冊怎麼用

每個階段最多分成三塊：

```
┌ ⚖️ 動手前先想（有些階段才有）──────
│ 這一步要做的判斷，以及判斷的依據
└──────────────────────────────────

┌ 🛠️ 動手做 ──────────────────────
│ 🧰 這一步的產出
│ 📋 貼給 AI
│ ✅ 怎麼知道做對了
│ 🆘 如果沒有成功
└──────────────────────────────────

┌ 📖 為什麼這樣做 ────────────────
│ 🎯 這一步的重點
│ 🧭 這一步在做什麼
│ 🧩 它在建構流程的位置
│ 💡 那段指令為什麼那樣寫
└──────────────────────────────────
```

先想判斷，再動手，原理在後。讀完階段開頭那幾行提示和〈⚖️ 動手前先想〉就可以動手，做完再回頭讀為什麼。

〈🛠️ 動手做〉照時間順序排。每一節是一次來回：

1. 貼什麼，或看到畫面之後回哪一句
2. Claude 找到的原因（症狀 → 原因）
3. 改了什麼，結果是什麼

📋 區塊裡的指令是做參考成品時實際用的指令，錯字也照原文。標了「(整理版指令)」的，是照 Claude 當時的做法事後整理出來的，不是當時傳的原文。沒有實際跑過的項目（例如階段 19 的兩個帳號互測），「你應該看到」都標了「預期」。照著跑時，拿你自己的結果對照。手冊另外整理的指令，會寫成「你也可以加一句」。

長的規格指令（Phase 1 到 4、介面、聲音）照原文放，有英文的放英文。前面用一兩句中文說明它在要求什麼。 `〈附上截圖〉` 代表那一則要附截圖。你貼的時候換成你自己的截圖。 Prompt 裡其他的 `〈…〉` 是要換

成你自己內容的地方。

每個階段做完，請 Claude 在 `docs/DEVLOG.md` 追加一筆。怎麼寫見階段 2。

照著做的時候

📋 有兩種，照做的方法不一樣：

- **規格類**：一整段需求或規格（例如 1–3、2–2、3–1、13–2）。照「你只需要修改」那一行改完就貼
- **回饋類**：看到畫面之後回的一兩句，通常附截圖（例如 1–4、6–2、12–3）。先看你自己的畫面：有一樣的問題，附上你自己的截圖再貼；沒有就跳過那一節。你看到別的問題，照同一個寫法（在哪裡、看起來怎樣）寫給 Claude

標著「你自己做」的小節，是你要親手在瀏覽器或後台操作，不用貼給 Claude。每個階段做完，照 ✅ 一項一項檢查。沒過的話，先找 🆘 裡對應的那一列。

用 ChatGPT 寫規格，再交給 Claude

這個案例的幾份長規格（2–1、2–3、3–1、4–2、9–2、10–1、13–2）是先把畫面截圖給 ChatGPT 看，討論要改什麼，請它寫成規格，再貼給 Claude。

先想一想：ChatGPT 只看過截圖，沒看過程式。它寫的規格裡，哪些東西最可能和你的專案對不上？

什麼時候找第二個 AI

- 要寫一份很長的規格，自己不知道從哪裡寫起
- 想聽第二個意見，例如畫面哪裡不自然、還缺什麼
- 手上有一堆截圖，要整理成一條一條的問題

怎麼請它看

附上目前的截圖，說清楚現在的問題，以及哪些東西不能動。下面這段可以照抄給 ChatGPT：

貼給 AI

你只需要修改：「不能動的東西」換成你自己的。

> 附圖是我正在做的網站目前的畫面。我會把你寫的規格貼給另一個能改程式的 AI（Claude）。
>
> 請先列出你從截圖看到的問題，每一項寫「在畫面哪裡、看起來怎樣、應該要怎樣」。
>
> 不能動的東西：**（例如：等角視角、整體配色、已經做好的繪馬卡片）**。
>
> 規格請分成幾節，每一節寫驗收條件。驗收要寫看得到或量得到的東西，不要寫「感覺自然」。
>
> 你看不到程式，所以不要指定套件、檔名或數字。需要數字的地方寫「請依現況決定，並說明理由」。

交給 Claude 之前

ChatGPT 看不到程式，規格裡的套件、檔名、數字、做法都可能和專案不一樣。所以先請 Claude 盤點，指出衝突，再動手：

貼給 AI

> 下面這份規格是另一個 AI 看截圖寫的，它看不到程式。
>
> 動手之前，請先對照 CLAUDE.md、docs/ 和現在的程式，列出規格和現況不一樣的地方：專案沒有的套件、對不上的檔名或數字、已經做過的項目、和既有規則衝突的要求。
>
> 每一項寫你建議怎麼處理，等我確認再動手。

ChatGPT 寫的規格，和專案對不上的例子

這個案例會遇到的一個例子：階段 4、13 的規格要求用 React、TypeScript、Zustand、Howler.js。階段 4 時，專案是原生網頁加 three.js，這些一個都沒有裝。階段 13 時，介面已經在階段 8 換成 React，其他還是沒有裝。沒有先盤點就照做，Claude 會先裝一堆用不到的套件，再把已經能跑的程式改寫一遍。

名詞：React 是一套寫網頁介面的工具。你寫「卡片現在是什麼狀態」，它負責把畫面改成對應的樣子。React Three Fiber（R3F）是把 three.js 包成 React 寫法的套件。
你需要知道到什麼程度：知道 React 管按鈕、卡片、表單這類介面。R3F 底層一樣是 three.js，換過去畫面不會多任何東西。

其他幾份規格也有類似的出入，例如浪的公式正負號寫反、要做的項目前面已經做過。做到那幾個階段時，手冊會在那裡提醒你。

不先盤點的代價，階段 2–1 會實際看到。

階段 1　先看成品，再做第一版海上神社

📖 讀完約 10 分鐘　🛠️ 動手約 40 分鐘

- **這一步一定要懂**：這個網站要用本機網址打開，不能用檔案的方式打開。給 AI 參考連結時，要讓它先評估適不適合，不是照參考硬做。
- **這一步可以先不懂**：WebGL、WebGPU、shader 各自怎麼運作。
- **做完後你要能用自己的話回答**：指令裡附了一個 WebGPU 流體模擬的參考，Claude 為什麼沒有照著做？講出至少兩個理由。

⚖️ 動手前先想

水面要用 WebGPU 流體模擬，還是 WebGL 的風格化水面？

下面 1–3 附的參考是 Codrops 的 WebGPU 流體模擬。Claude 評估後會建議不用：

理由	具體是什麼
瀏覽器支援不齊	不支援 WebGPU 的瀏覽器會看不到水面
成本高	流體模擬每一幀都要解方程式
風格不合	寫實的流體和 low-poly 的神社放在一起不協調

> **名詞**：**WebGL** 是瀏覽器裡直接操作顯示卡的介面，Three.js 是它的包裝。**WebGPU** 是比較新的同類介面。
> 你需要知道到什麼程度：知道 WebGPU 比較新、能做的事比較多，但不是每個瀏覽器都支援。這個案例全部用 WebGL。

> **名詞**：**shader**（著色器）是一小段跑在顯示卡上、決定每個像素顏色的程式。
> 你需要知道到什麼程度：知道水面的樣子是它畫的，而且它和 JavaScript 是兩份程式。兩份各寫一次公式，就會有一天對不起來。

> **名詞**：**等角視角**是一種沒有透視的鏡頭：遠的東西不會變小，平行線永遠平行。
> 你需要知道到什麼程度：知道這個場景用的是這種鏡頭（`OrthographicCamera`）。所以畫面看起來像一個微縮模型。階段 11 會遇到它的一個副作用。

> 改用的做法是幾道正弦波疊在一起的風格化水面。水面、船、小島共用同一組波浪公式，船和島才會跟著同一片浪起伏。

🛠️ 動手做

🧰 這一步的產出

- 在瀏覽器裡跑起來的成品
- 一個你自己的資料夾，裡面有第一版海上神社（一個 HTML 檔）

狀態	參考畫面
這一步做完：海上的神社	 海鋪滿畫面，鳥居柱腳有白浪。三艘船長得一樣，樹只有一棵松，介面是白色的紙卡。
中途的版本：場景接近成品，介面還是舊的	 和上一張比，水面多了浪的層次，船、樹、介面都換過。這張的介面是階段 11 之前的版本。

📋 貼給 AI（1–1：把成品跑起來）

1. 到課程頁面下載 `case-04-wishes.zip`，解壓縮到桌面
2. 開一個新對話，Project folder 選那個資料夾
3. 貼上下面這句

> 貼給 AI
>
> **你只需要修改**：沒有。整句照貼。
>
> 幫我把這個網站跑起來，給我本機網址。

4. 把 Claude 給你的網址（`http://localhost:4324/`）貼到 Chrome 打開

這個資料夾是看目標用的。從 1–3 開始，你會在一個全新的空資料夾裡從頭做一次。

你自己做（1–2：在成品上試五件事）

每一件都對應後面的一個階段。

#	動作	會看到	對應階段
1	點船旁邊的水面	一圈漣漪擴散出去，船被推起、傾斜	4
2	右上角切到「夜晚」	天色 4 到 5 秒慢慢變暗，燈籠一盞接一盞亮	5、10
3	點左邊的繪馬牆	卡片沿著吊繩飛出來，可以按住「一起祈願」	7
4	網址後面加 `?wx=雷雨` 再打開	下雨，等幾秒會打雷、閃電	12
5	按「一起祈願」	跳出登入視窗。登入碼填 `123456`	14

第 5 件用的是假的登入，因為成品的 `.env` 是空的。階段 15 才會接上真的 Supabase。

成品比手冊多了四樣東西：參拜小船與船夫、晴夜海上的星光、燈火閃爍、碼頭接到石階。手冊沒有這些步驟，你的版本沒有它們是正常的。

📋 貼給 AI（1-3：做一座海上的神社）

在桌面建一個空資料夾，例如 `wishing-shrine`，開一個新對話，Project folder 選它。

貼給 AI

你只需要修改：參考連結可以換成你找到的。「如果你覺得太複雜，或風格不符合，可以調整」這一句一定要留，理由在 💡 。

```
請幫我做一個 3D 的海上許願神社網站：

- 神社蓋在海中央的小島上，前方有一座很大的鳥居，旁邊有幾艘小船漂浮
- 小島跟著海浪微微浮沉
- 材質與光影要精緻、吸睛
- 水波的效果可以參考：https://tympanus.net/codrops/2025/02/26/webgpu-fluid-simulations-high-performance-real-time-rendering/
  如果你覺得太複雜，或風格不符合，可以調整
- 神社裡的繪馬牆可以點開，看到裡面的許願內容

先做成一個能用瀏覽器打開的 HTML 檔，給我本機網址。
```

你應該看到：Claude 先評估參考連結，說明不照 WebGPU 流體模擬做的理由，類似〈⚖️ 動手前先想〉那張表。然後做出一個單一的 HTML 檔。水面改用 WebGL 的風格化做法：幾道正弦波疊加、岸邊與柱腳的白浪、淺水偏青綠、陽光反光。

📋 貼給 AI（1-4：第一版做出來後，檢查這幾件事）

用 Claude 給的本機網址打開第一版，一項一項看。有一樣的問題，附上你自己的截圖，貼那一項的指令；沒有就跳過。

1. 浪太亮、想要一點霧

看海面的高光和反光是不是太搶眼。指令後半是繪馬卡片的要求，留到階段 7 處理，可以先刪掉。

貼給 AI

```
海浪的光影可以稍微弱一點點，然後可以打開霧氣的感覺。

另外許願的部分我預期跟課程就無關，他可以是個獨立的許願卡片，然後可以做出一些很棒的互動，
```

你應該看到：高光、陽光陰影、天空反射都調弱，海上多了幾團飄動的霧。

2. 海只佔中間一塊、霧在海面上留下色帶

先想一想：海是一個方塊，四周露出背景，看起來像桌上的模型還是一片海？霧如果在海面上留下一條條硬邊的水平色帶，要修霧還是先拿掉？

貼給 AI

另外要不要乾脆把海洋擴散到整個畫面

貼給 AI

如果覺得霧氣影響太嚴重可以先移除霧氣

你應該看到：飄動的霧拿掉，只留往地平線變淡的距離霧。海面變成 150 × 150，在霧裡接到地平線。原本的方塊剖面版還在，網址加 `?diorama` 可以切回去。第二句「如果覺得霧氣影響太嚴重可以先移除」先給了退路。Claude 不用硬修一個效果不好的東西。

3. 船身看得到藍色

放大看船：從上面看，船裡是不是有一片藍色的水？

貼給 AI

〈附上截圖〉

船身上不應該看得到藍色，很像沈船了

原因是船吃水太深，浪頭漫過甲板。Claude 當時的修法是把船抬高。這個修法到階段 6 會被推翻：抬高只是把問題藏起來，後來船影就對不上了。

4. 畫面最底下有一段白色

先分清楚：那段白色是網頁的一部分，還是你在看的那個視窗的一部分？

貼給 AI

〈附上截圖〉

為什麼底下會有一段白色的呀

你應該看到：Claude 先量畫布大小，實測是 898 × 647，和視窗一樣大。它確認頁面背景不是純白，回答那段白色是桌面應用程式瀏覽器面板下方的工具列。不改任何程式。這一句問的是「為什麼」，不是「把它修掉」。問原因，Claude 才會先查，不會為了一個不存在的問題改程式。

5. 船還是透進水裡，順便加微風與材質

貼給 AI

你只需要修改：換成你的截圖和你要的效果。

〈附上截圖〉

* 如附圖，船還是會透到水裡面，請協助我優化調整。
* 另外我想要實作微風吹拂的效果
* 協助我優化材質的感覺，包含石頭紋路、木頭紋路等等

你應該看到：

問題	原因	修法
船還是透進水裡	船身是一個開口的碗，看得到裡面	加甲板、船舷與兩片座板
船邊白浪像水灌進船裡	白浪圈沒有跟著船身方向轉	白浪改用船自己的座標，跟著船轉
風吹出的水面亮點像下雪	亮點太密	門檻提高到 0.88
程式紋理一載入就報錯 `Cannot access 'rand' before initialization`	紋理在亂數函式宣告之前就用了它	把亂數函式移到最前面

微風吹動樹、紙垂、布旗、繪馬、花瓣與水面。石紋、木紋、屋瓦、砂利紋理都由程式產生，沒有用圖片檔。

Claude 自己截圖時，另外找到兩個問題。遠處海面有一圈一圈的雲狀白斑，原因是浪頭泡沫的門檻太低，改成雜訊條紋、門檻提高。海面邊緣有波紋凸起，原因是邊緣衰減用錯了座標軸。你的版本如果也有，貼一句「遠處海面有一圈一圈的白斑，請找原因，先不要改」。

✅ 怎麼知道做對了

- [ ] 成品在 Chrome 裡跑起來，五件事都試過
- [ ] 你的資料夾裡有一個海上神社的 HTML 檔，用本機網址打開，水面在動
- [ ] Claude 說明了水面用什麼技術做，以及為什麼沒有照 WebGPU 的參考做
- [ ] 點繪馬牆會打開一個顯示願望的面板
- [ ] 放大看船，看不到船裡的水

🆘 如果沒有成功

你可能會看到	通常是因為	怎麼辦
畫面一片空白，只有背景色	用 `file://` 的方式打開了，瀏覽器擋掉了模組之間的引用	跟 Claude 說「請給我本機網址，不要用檔案的方式打開」
程式改了，畫面卻沒變	Claude 桌面應用程式的瀏覽器面板顯示的是檔案的舊快照	一律在 Chrome 打開 `http://localhost:…` 的網址
Claude 直接照著 WebGPU 的參考寫流體模擬	指令裡少了「可以調整」那一句	「請先評估 WebGPU 流體模擬在支援度、成本、風格上適不適合這個場景，評估完再決定做法」

這一步失敗可以跳過嗎：1–1 可以，但盡量想辦法看到成品。1–3 不行，後面每一階段都改這個檔案。

📖 為什麼這樣做

🎯 這一步的重點

給 AI 參考資料時，也給它不照做的空間，並要求它說出理由。

🧭 這一步在做什麼

先看成品，再做一個能動的第一版。

🧩 它在建構流程的位置

流程開始之前。這一步的產出是一個 HTML 檔。階段 2 把它拆成 `src/` 裡約 20 個檔案，之後的階段改的是那些檔案。

💡 那段指令為什麼那樣寫

1–3 的「如果你覺得太複雜，或風格不符合，可以調整」，讓 Claude 有理由不照參考連結做。

少了這一句，AI 通常會照你給的參考硬做。做出來的東西你看不出哪裡不對，等到別人的瀏覽器打不開才知道。

1–4 的每一項都很短，大多附了截圖。寫的是看到的現象（「很像沈船了」「底下會有一段白色」），沒有指定修法。修法由 Claude 找原因之後決定，所以第 4 項才會查出那段白色根本不是網頁的問題。

階段 2　盤點、寫規格、拆檔

📖 讀完約 15 分鐘　🛠️ 動手約 45 分鐘

- **這一步一定要懂**：文件要對應實際的程式，不能對應還沒跑過的草稿。「畫面完全沒變」要用逐像素比對證明，而且要先量出雜訊底線。
- **這一步可以先不懂**：`.claude/rules/` 的規則範圍怎麼設定、拆檔時每個模組怎麼命名。
- **做完後你要能用自己的話回答**：拆檔前後比對，第一次有 0.75% 的像素不同。Claude 是怎麼找到原因的？

⚖️ 動手前先想

文件要對應實際程式，還是對應還沒跑過的草稿？

對應實際程式。草稿沒跑過，裡面的東西可能根本不能用。文件照草稿寫，下一次修改的人（包括 Claude）會照著錯的文件做。

「畫面完全沒變」要怎麼證明？

> **名詞**：**逐像素比對**是把兩張截圖的每一個像素拿來比，算出有多少比例不一樣。
> 你需要知道到什麼程度：知道 0% 代表兩張完全相同。比對的腳本由 Claude 寫，你只看那個百分比。

> **名詞**：**draw call** 是 CPU 叫顯示卡畫一次東西的次數。
> 你需要知道到什麼程度：知道這個數字變了，代表畫面上多畫或少畫了東西。截圖看起來一樣時也是這樣。

下面 2–4 的四個步驟就是方法。做的時候你會看到 0.75% 的差異。0.75% 不是 0%，但也不能直接放過。先證明同一個版本跑兩次是 0%，0.75% 才確定是某個東西在變，接著才去找是哪個。

🛠️ 動手做

🧰 這一步的產出

```
CLAUDE.md                      ← 專案定位、技術架構、工程原則、工作流程
.claude/rules/                 ← rendering / animation / performance / interaction
四份細部規則
docs/ART_DIRECTION.md          ← 美術方向
docs/SIMULATION_SPEC.md        ← 波浪、漣漪、浮力、花瓣、風、相機的數學與驗收標準
docs/IMPLEMENTATION_PLAN.md    ← 功能盤點：已實作 / 部分 / 未實作
docs/DEVLOG.md                 ← 開發日誌，之後每一階段追加一筆
src/                           ← 從單一 HTML 拆出來的約 20 個檔案
```

畫面上不會有任何變化。這一步要做的，是證明它沒有變化。

📋 貼給 AI（2-1：四階段的英文大規格）

這是階段 1 做完之後貼的第一份大規格。它分成 Phase 1 到 4：視覺、環境、互動、品質。

貼給 AI

你只需要修改：這一段不建議照貼。它是 2-2 的反例，放在這裡讓你看到發生了什麼。

```
Act as a senior Creative Developer, Three.js engineer, and 3D
Technical Artist.

Upgrade my existing Japanese shrine isometric low-poly 3D scene into a
premium interactive 3D diorama. Preserve the current shrine, torii, cherry
blossom tree, boats, stone lanterns, rocky island, and overall stylized
visual language. Inspect the actual project structure, rendering engine,
available geometry and materials before modifying anything. Do not rewrite or
replace working components unnecessarily.

Design goals: cinematic but minimal, cohesive art direction, soft atmospheric
lighting, subtle and believable environmental motion, polished interaction,
stable frame rate. Avoid photorealistic ocean shaders, exaggerated bloom,
visual clutter, unrealistic waves, excessive particles and excessive
camera movement.
```

```
PHASE 1 — VISUAL: Improve composition, camera target, lighting,
physically coherent stylized materials, AO/contact shadow, color
management, roof details, shore transition and selective glow. Keep the
orthographic/isometric composition.

PHASE 2 — ENVIRONMENT: Implement stylized GPU water with layered low-
amplitude waves, normal perturbation and shoreline foam mask. Add cherry
blossom instanced particles with variable spawn phase and wind trajectories.
Add distinct phase-based boat floating animation, lantern emissive glow and
carefully controlled bloom.

PHASE 3 — INTERACTION: Add pointer/touch water ripple using raycasting and a
lightweight heightfield or analytic ripple shader. Add damped pointer-
parallax camera movement, click-to-focus points of interest, and a
day/sunset/night transition which interpolates lighting, environment, water
palette, window emission and lantern intensity. Preserve camera limits and
allow reset.

PHASE 4 — QUALITY: Use pixel ratio caps, batched/instanced particles,
compressed textures where appropriate, quality presets, visibility-aware
rendering and mobile-friendly fallbacks. Respect prefers-reduced-motion.
Avoid expensive reflections and shadows on low-end devices. Provide
before/after comparison and basic performance profiling.

Suggested stack only if compatible with existing architecture: Three.js,
React Three Fiber, Drei, @react-three/postprocessing, GSAP, Zustand,
custom GLSL.

Deliver incrementally. For each phase identify files changed, implementation,
test procedure and measured performance where available. Prioritize visual
cohesion and motion subtlety over effect quantity.
```

Claude 讀完就開始照四個 Phase 改程式，一大段直接寫進了場景檔，還沒跑過。這是反例：大規格一貼進去，AI 沒盤點就開始改。看到 Claude 開始大改程式，就照 2-2 先叫它停下來。

📋 貼給 AI（2–2：先盤點，再建規範）

先不要做效果，先盤點現況、建立協作規範。把模式切到 Plan（只規劃、不動手），貼上這段：

貼給 AI

你只需要修改：最後一句「我預期的文件結構可能如附圖」。有自己的圖就附上，沒有就刪掉。其餘照貼。「重要要求」那幾點與「第一階段只處理架構盤點與文件建立」是這一步的範圍。

（附上截圖）

請先完整檢視目前的 3D Shrine Diorama 專案架構、套件、Shader、動畫邏輯、場景物件與檔案組織。

接著依照現有專案，建立一套適合長期 AI Coding 協作的專案規範。

請完成以下事項：

1. 在專案根目錄建立或更新 `CLAUDE.md`，記錄專案定位、實際技術架構、工程原則、視覺方向與開發工作流程。
2. 建立 `.claude/rules/`，將 Rendering、Animation、Performance 等細部規則拆開管理，並依適用檔案設定適當的規則範圍。
3. 建立 `docs/ART_DIRECTION.md`，定義目前神社場景的 Low-poly / Isometric 美術方向、構圖、材質、光影、色彩與動態設計。
4. 建立 `docs/SIMULATION_SPEC.md`，完整記錄 Gerstner Waves、Ripple、Boat Buoyancy、Cherry Blossom Particle、Camera Damping 等演算法、數學模型、座標系統、參數與驗收標準。
5. 建立 `docs/IMPLEMENTATION_PLAN.md`，依照現有程式完成度，整理視覺優化、環境動畫、互動效果、日夜模式與效能優化的實作階段。

重要要求：

* 先盤點現況，明確區分已實作、部分實作和尚未實作的功能。
* 不要假設圖片中的技術架構已全部安裝。
* 保留現有可正常運作的功能。
* 不要直接開始大規模重構。
* 不要將整份演算法文件複製進 CLAUDE.md。
* 文件內容必須對應實際程式碼，而非單純的理想架構。
* 如果發現技術選型有更合適的方案，先提出原因與取捨。
* 完成後提供文件樹、架構現況、技術缺口與優先開發建議。

第一階段只處理架構盤點與文件建立，確認規格後再進入效果實作。

我預期的文件結構可能如附圖

Claude 收到之後做了兩件事：

1. 把場景檔還原到最後一個驗證過的版本。2-1 那段沒跑過的程式另存成 `drafts/ocean-shrine-upgrade-UNTESTED.html`，只當參考
2. 逐行對照程式，寫出 `CLAUDE.md`、`.claude/rules/` 四份、`docs/` 三份

寫文件時，Claude 在程式裡找到三個畫面上看不太出來的錯。看答案之前，先想三個問題：

1. 船被浪從船頭頂起時，船頭應該往上還是往下？你在畫面上看到的是哪一個？
2. 同一段鏡頭推近，在每秒 30 張畫面的筆電和每秒 60 張的桌機上，會花一樣久嗎？
3. 三道浪疊在一起，整片海多久會完全重複一次？

問題	後果	怎麼察覺	可以照抄的請求或規格
船的俯仰正負號反了。three.js 繞 X 軸轉正角時，船頭那一端往下	水在船頭比較高時，船頭反而往下壓	在船頭正前方點一下起漣漪，看波峰到船頭時船頭往上還是往下	「請列出船的俯仰、橫搖的正負號定義，並用一道只從船頭來的浪驗證船頭會抬起」
好幾個平滑過渡寫成「每幀乘固定比例」，例如相機推近每幀 `* 0.06`	60 fps 和 30 fps 的速度差一倍：60 fps 約 0.27 秒，30 fps 約 0.54 秒	同一段過渡在 30 fps 和 60 fps 各錄一次，比較花了幾秒	「請列出所有每幀乘固定比例的過渡，改成和幀率無關的寫法」
三道波的角速度 0.9、1.1、1.6 都是 0.1 的倍數	整片海每 62.8 秒（2π ÷ 0.1）完全重複一次	盯著同一片浪數秒數；或請 Claude 算出三道浪週期的最小公倍數	規格寫「幾道浪的角速度不要有簡單的比例，例如不要都是 0.1 的倍數」

最後它會列出幾個要你決定的問題（實際做的時候是五個，見 2-3）。

📋 貼給 AI（2–3：回答問題，並評估另一份規格）

在同一則訊息裡回答 Claude 問的問題，接著貼一份英文規格（Advanced Rendering），請 Claude 評估。

貼給 AI

你只需要修改：前五行換成你對 Claude 那五個問題的回答。規格換成你手上的那一份。「另外幫我評估這段實作的指令」照貼。

```
1. 可動
2. 你覺得有需要嗎？
3. 好像可以先移除
4. 可以沒問題
5. 我會在建立一個


另外幫我評估這段實作的指令

Advanced Rendering, Animation & Interaction Specification
You are not only implementing visual animations. You must implement a
coherent, physically inspired, stylized 3D environment where the water,
boats, particles, lighting, and camera respond consistently to shared
simulation states.
Preserve the existing scene's low-poly / isometric art direction. The
objective is premium stylized visual quality, not photorealism.
1. Shared Simulation Architecture
Create a centralized simulation system containing:

* Global elapsed time and bounded frame delta.
* Shared water surface function.
* Global wind field.
* Interaction events.
* Day / sunset / night transition state.
* Quality configuration.
* Pause, visibility and reduced-motion states.

Avoid implementing each effect with unrelated timing logic.
Use deterministic seeds when randomness is required.
```

Maintain a consistent coordinate system and document the transformation between world coordinates, local water coordinates, and simulation UV coordinates.

2. Water Surface Shader

Implement a custom stylized water shader using 3–5 layered traveling waves.

Height function:

$h(x,z,t) = \Sigma A_i \sin(k_i \cdot (x,z) - \omega_i t + \phi_i)$

Requirements:

* Low wave amplitude.
* Multiple directional wave components.
* Smooth surface displacement.
* Analytical or finite-difference normals.
* Subtle Fresnel reflection.
* Color gradient based on depth, surface direction and lighting.
* Small-scale normal perturbation.
* No exaggerated photorealistic ocean waves.

Expose a corresponding CPU-side `getWaterHeight(x,z,time)` function for buoyancy calculations.

The water shader must support adding the interaction ripple field to the base wave height.

3. Interactive Ripple System

Start with an analytical ripple model:

$r(d,t) = A \exp(-\lambda t) \exp(-\mu(d-ct)^2) \cos(k(d-ct))$

Support:

* Mouse and touch raycasting.
* World-to-water coordinate conversion.
* Multiple concurrent ripple sources.
* Time-based amplitude attenuation.
* Smooth propagation and disappearance.
* Small ripples generated by falling petals.
* Large but visually controlled ripples generated by user interaction.

Use a bounded ripple source pool to prevent unbounded growth.

If analytical ripples do not satisfy the required interference behavior or

interaction density, introduce a GPU heightfield simulation using the damped wave equation:

`∂²h/∂t² + γ∂h/∂t = c²∇²h`

Use ping-pong GPU render targets, fixed simulation time steps and appropriate stability checks.

Do not introduce GPU simulation unless its visual benefit justifies the performance and implementation complexity.

4. Boat Buoyancy and Orientation

Boats must respond to the water simulation.

For each boat:

* Sample the shared water height at its center.
* Sample multiple positions along the longitudinal and lateral axes.
* Calculate target vertical position.
* Derive pitch and roll from the sampled height differences.
* Smooth motion using critically damped interpolation.
* Apply unique resting offsets and different initial phases through their world positions.
* Keep rotation angles small and believable.

For GPU-only ripple fields, use a low-cost approximation or controlled asynchronous data transfer. Do not perform synchronous per-frame GPU readbacks.

A nearby user-generated wave should visibly influence the closest boat.

5. Cherry Blossom Particle Simulation

Use GPU-friendly instancing or particle buffers.

Motion model:

`v_next = v + (gravity + wind(position,time) - drag × v) × dt`

`p_next = p + v_next × dt`

Requirements:

* Spatially correlated wind field.
* Low-frequency noise.
* Variable rotation speed.
* Randomized but deterministic seeds.
* Per-particle lifetime.
* Frustum-aware or bounded particle management.

* Water collision detection.
* Optional small ripple generation on collision.
* Smooth opacity fade.

Avoid excessive particle density.

6. Lighting and Day / Night States

Implement three visual presets:

* Day: warm natural sunlight, turquoise water, minimal lantern intensity.
* Sunset: warm directional light, gentle long shadows, amber highlights.
* Night: deep blue atmospheric lighting, warm lantern emission, restrained bloom.

Use a centralized transition state `dayCycleProgress`.

Interpolate:

* Directional light intensity, direction and color.
* Ambient / environment lighting contribution.
* Water palette.
* Background color and atmospheric treatment.
* Lantern emissive intensity.
* Shrine window emission.
* Bloom contribution.

Do not abruptly replace materials or backgrounds during transitions.

Avoid unnecessarily rebuilding environment maps every frame.

7. Camera Motion System

Use a damped spring model:

`x'' + 2ζωx' + ω²(x-target) = 0`

Use approximately critical damping as the initial tuning target.

Requirements:

* Preserve orthographic composition.
* Smooth pointer parallax.
* Bounded camera orbit.
* Named cinematic camera presets.
* Click-to-focus interactions.

* Return-to-home animation.
* Pause idle motion during direct manipulation.
* Respect reduced-motion preferences.

8. Render Quality

Implement:

* Contact shadows and restrained ambient occlusion.
* Selective bloom where available, without overexposing large surfaces.
* Color management and consistent tone mapping.
* Controlled exposure.
* Lightweight fog or atmospheric gradients only where visually appropriate.
* Consistent visual treatment across water, shrine, boats and particles.

Do not add effects merely because the rendering engine supports them.

9. Performance Engineering

Define quality presets:

High:

* Full water shader.
* More detailed ripple field.
* Higher particle density.
* Enhanced postprocessing.
* Higher rendering resolution.

Medium:

* Reduced particle density.
* Lower simulation resolution.
* Reduced reflection quality.
* Limited postprocessing.

Low:

* Simplified water.
* Analytical ripples.
* Minimal particles.

```
* Reduced shadows.
* No expensive reflection passes.

Cap device pixel ratio appropriately.
Avoid unnecessary React re-renders inside the animation loop.
Profile GPU cost, CPU frame time, draw calls and memory usage.
10. Acceptance Criteria
The implementation is complete only when:

1. Water exhibits subtle non-repeating motion.
2. Water lighting and coloration remain consistent with the overall stylized
art direction.
3. User interaction generates visible, smooth and dissipating ripples.
4. Boats move consistently with sampled water displacement.
5. Cherry blossom particles exhibit believable wind-driven motion.
6. Day/night transitions update the entire environment coherently.
7. Camera movement feels smooth, intentional and restrained.
8. The scene remains visually balanced and recognizable.
9. The experience is usable on desktop and mobile.
10. Performance is measured and reported, not merely claimed.

Implement changes incrementally.
For each feature:

* Identify impacted files and scene objects.
* Explain the chosen algorithm.
* Implement the effect.
* Validate the visible result.
* Test performance.
* Adjust parameters to match the art direction.
* Report limitations and tradeoffs.

Prioritize visual cohesion, interaction responsiveness and stable performance
over unnecessary technical complexity.
```

前五行對應的問題：

#	Claude 問的	實際的回答	結果
1	要不要把單一 HTML 拆成 `src/` 底下的模組	可動	拆
2	要不要加 `package.json`，讓截圖與量測腳本進專案	你覺得有需要嗎？	Claude 建議不加。量測腳本放暫存目錄，給人看的量測是 `?stats` 與開發者工具
3	要不要換成 Gerstner 波	好像可以先移除	不做。目前的浪高下看不出差別，船、花瓣、白浪卻都要跟著改
4	預設時段用黃昏	可以沒問題	黃昏
5	後端誰做	我會在建立一個	自己另外建立

Claude 如果用英文回，再傳一句：

貼給 AI

請你都用中文給我討論！

評估的結論：這份規格八成和既有方向一致，九處要改。最重要的幾處：

- 規格的 `−ωt` 會讓浪反向，保留 `+ωt`
- 「水面不重複」那條驗收現在就不過（62.8 秒週期）
- 規格沒考慮天氣，時段和天氣會互相蓋掉，所以天氣改成乘數（階段 5）
- 花瓣漣漪會把點擊漣漪擠掉，所以分兩個池（階段 4）
- 「感覺平順」不能驗收，換成數字

📋 貼給 AI（2–4：開工拆檔）

確認之後，只要回兩個字：

貼給 AI

開工

他能只打兩個字，是因為拆檔的做法 Claude 已經寫進 `docs/IMPLEMENTATION_PLAN.md`。如果你的計畫裡沒有寫比對的方法，你也可以加這一段：

貼給 AI

```
照 IMPLEMENTATION_PLAN 的提案，把單一 HTML 拆成 src/ 底下的模組。
這一步畫面不能有任何改變。

1. 拆檔之前，先在「減少動態效果」模式下截一張全景（時間停在 0，畫面靜止）
2. 拆完之後，同樣條件再截一張，逐像素比對
3. 比對之前，先把拆檔前的版本跑兩次互相比對，量出雜訊底線
4. 同時記錄每一幀的 draw call 次數，拆檔前後要一樣

差異不是 0% 的話，先找出原因再改，不要調門檻讓它通過。
```

結果：920 行的單一 HTML 拆成約 20 個模組（`src/sim`、`src/scene`、`src/ui` 等）。比對的過程：

1. 第一次比對，0.75% 的像素不同
2. 先把拆檔前的版本跑兩次互比，差異 0%，代表不是隨機雜訊
3. 找到會自己變動的東西：「模擬祈願」。它模擬其他人在祈願，時機每次都不同，票數和光脈衝跟著不同
4. 關掉模擬祈願再比：拆檔前後 0%，draw call 都是 360

📋 貼給 AI（2–5：開始寫開發日誌）

貼給 AI

```
另外提醒，因為最終這個會要整理成教學文件喔～～所以中間的步驟都要有記憶起來
```

Claude 建立 `docs/DEVLOG.md`，從對話紀錄回推，補記之前每一步。格式和 `CLAUDE.md` 的改法，就是下面這段寫的。

你也可以加這一段，直接指定格式：

貼給 AI

建立 docs/DEVLOG.md，把到目前為止的每一步補記上去。每一筆記五件事：

需求、判斷、做法、踩到的問題、驗證。

「踩到的問題」一律寫成「症狀 → 原因 → 修法」。

被推翻的判斷也要記，不要只記成功的結果。

然後在 CLAUDE.md 的工作流程加上一條：每一步完成時，在 DEVLOG.md 追加一筆。

✅ 怎麼知道做對了

- [ ] `docs/IMPLEMENTATION_PLAN.md` 有一張現況表，每一項標著已實作、部分或未實作
- [ ] Claude 在盤點時至少找到一個程式裡的問題（實際做的時候找到三個）
- [ ] 你的五個決定寫進了 `docs/IMPLEMENTATION_PLAN.md`，不是只留在對話裡
- [ ] 拆檔前後逐像素比對 0%，draw call 兩邊一樣（實測當時是 360）
- [ ] `docs/DEVLOG.md` 存在，`CLAUDE.md` 的工作流程寫著要追加日誌

🆘 如果沒有成功

你可能會看到	通常是因為	怎麼辦
Claude 的文件寫了很多「將會」「建議」，但程式裡沒有	它照著外部規格寫，不是照程式寫	「文件要對應實際程式。請逐項標明這一項在哪個檔案的哪一段，找不到的標成未實作」
拆檔前後有零點幾 % 的像素不同	畫面上有會自己變動的東西	「請先把拆檔前的版本跑兩次互比。如果兩次也不同，找出是哪個東西每次都在變，把它關掉再比」
Claude 在盤點時已經改了一大段程式	沒有切到 Plan 模式，或像 2-1 一樣先貼了要動手的規格	「請先停下來。剛才改的程式跑過了嗎？沒跑過的話，還原到最後一個驗證過的版本，把改的部分另存成草稿」
Claude 用英文回覆	你貼的規格是英文	「請你都用中文給我討論！」

這一步失敗可以跳過嗎：不行。後面每一個階段都在比對「改之前」和「改之後」。

📖 為什麼這樣做

🎯 這一步的重點

建立一套證明「沒壞」的方法：先量雜訊底線，再排除會自己變動的東西，最後才比對。

🧭 這一步在做什麼

盤點、寫規範、拆檔。三件事都不改畫面。

2–2 把順序改成「先盤點、再寫文件、確認規格後再實作」。後面所有階段都照這個順序。

盤點找到的三個錯都不會出現錯誤訊息。只有逐行對照程式寫文件時才看得到。

🧩 它在建構流程的位置

```
第一版原型 ──▶ 【盤點、規範、拆檔】 ──▶ 水面（階段 3）
```

💡 那段指令為什麼那樣寫

2–2 的「重要要求」每一條都在擋一種實際會發生的狀況：

要求	擋的是什麼
不要假設圖片中的技術架構已全部安裝	參考圖列了 React Three Fiber、GSAP、Zustand，專案裡一個都沒有
文件內容必須對應實際程式碼	文件寫成理想架構，和程式對不起來
先提出原因與取捨	AI 自己決定換框架
第一階段只處理架構盤點與文件建立	一邊寫文件一邊改程式，兩件事的錯混在一起。2–1 就是這樣

2–4 的「不要調門檻讓它通過」：比對不過時，最省事的做法是把門檻從 0% 放寬到 1%。那會把真的差異一起放過去。

🧭 Checkpoint 1（做完階段 2）

口頭回答就好，三題都講得出來就往下走。

1. 為什麼不能照著還沒跑過的草稿寫文件？
2. 拆檔前後比對有 0.75% 不同。第一步要先做什麼？
3. 「每幀乘固定比例」在 30 fps 和 60 fps 下會有什麼差別？

> 第 2 題答不出來，回階段 2 的〈⚖️ 動手前先想〉再看一次。

階段 3　做水面與光線

📖 讀完約 12 分鐘　🛠️ 動手約 45 分鐘

- **這一步一定要懂**：波浪公式只有一份，畫水面的 shader 和算船高度的程式都從它來。新加的東西不能用共用的亂數 `rand()`。
- **這一步可以先不懂**：Fresnel、法線、泛光的數學。照 3–1 貼就好。
- **做完後你要能用自己的話回答**：加了一張屋頂貼圖之後，岩石和船為什麼全部換了位置？

⚖️ 動手前先想

新加的東西能不能用共用的亂數 `rand()`？

場景裡的紋理、岩石、布旗、船、雨，依序從同一條 `rand()` 拿亂數。同一個種子，每次拿到的序列都一樣，岩石和船的位置才會固定。

先想一想：中間多一個東西也來拿一次亂數，後面的岩石和船會怎樣？下面 3–1 會加一張屋頂貼圖，做到那裡就會看到答案，寫在 3–1 那張表的第二列。

色調映射用挑的還是用比的？

> **名詞**：**色調映射**是把 3D 計算出來的亮度，壓進螢幕能顯示的範圍。
> 你需要知道到什麼程度：知道換一種色調映射，整個畫面的顏色都會變。所以要並排比過再選。

左邊 ACES 把鳥居的朱漆推成偏橘，石台壓成死白。右邊 AgX 讓整個黃昏發灰。中間 Neutral 的朱漆與石台最接近原本的顏色。

🛠️ 動手做

🧰 這一步的產出

```
src/sim/water.js      ← 波浪參數與 getWaterHeight()，水面 shader 的公式由這裡產生
src/scene/sea.js      ← 水面材質：岸邊泡沫、柱腳白浪
src/render/post.js    ← 選擇性泛光與色調映射
```

畫面上應該看到：水面有四道不同方向的浪，岩石邊有一圈貼著岩石形狀的泡沫。燈籠會發光，但不會整片過曝。

📋 貼給 AI（3-1：Phase 01 視覺與水面基礎）

這是實際用的 Phase 01 指令原文。

貼給 AI

你只需要修改：沒有。整段照貼。第一節「先檢查現有程式」與最後兩句是這一步的範圍，不要刪。第一節第 1 點提到 React Three Fiber，你的專案沒有的話，Claude 會回報沒有，不影響。

接下來，請依照專案既有的 `CLAUDE.md`、`.claude/rules/` 與 `docs/` 技術規格，正式開始第一階段的視覺與水面系統優化。

本次任務的目標，是將目前的 Low-poly Isometric 神社，從基本可運作的 3D 模型，提升成具有高品質光影、細緻材質與自然水面動態的互動場景。

一、先檢查現有程式

請先：

1. 確認目前 Three.js / React Three Fiber 版本與渲染模式。
2. 盤點攝影機、場景光源、水面 Mesh、Shader、材質及後製效果。
3. 確認既有動畫是否有統一時間來源。
4. 檢查是否已存在可重用的水面高度函式。
5. 找出目前影響畫面品質的主要問題。

請不要直接推翻既有架構或重新建模。

二、建立統一的 Water Simulation

實作低振幅、多方向的 Gerstner Wave 水面。

使用：

`h(x,z,t) = Σ Aᵢ sin(kᵢ · (x,z) − ωᵢt + ϕᵢ)`

要求：

* 使用 3–5 組波浪，參數可調整。
* 保持平靜、柔和的水面風格。
* 使用正確的世界座標與時間來源。
* 將水面運動與視覺法線擾動分開。
* 使用解析偏導數或有限差分計算法線。
* 提供 CPU 可重用的 `getWaterHeight(x,z,time)`。
* 保留後續加入互動漣漪的擴充介面。
* 不要在這個階段實作複雜的 GPU Heightfield。

如現有水面 Shader 已符合需求，請優先擴充而非重寫。

三、改善水面視覺

加入：

* 低對比度的流動水紋。
* 柔和的 Fresnel 效果。
* 適度的水面反光。
* 水深或岸線相關的顏色變化。
* 岩石與鳥居柱體周圍的柔和泡沫。
* 帶有緩慢變化的泡沫 Noise。

泡沫應優先使用岸線遮罩、距離場或適合既有幾何結構的方案。

請避免使用大量隨機白色斑塊模擬泡沫。

四、提升環境光影品質

改善：

* Directional Light
* Ambient / Environment Light
* Contact Shadows
* Ambient Occlusion
* Tone Mapping
* Color Management
* Selective Bloom

請特別檢查：

1. 黑色屋頂是否能保有材質細節。

2. 鳥居紅色是否過度飽和。

3. 神社木質材料是否有合理的受光層次。

4. 岩石是否有足夠的體積感。

5. 水面與岸邊的光影是否一致。

6. 燈籠發光是否過曝。

不要為了追求電影感而過度增加 Bloom 或景深。

五、優化視覺構圖

維持 Orthographic / Isometric 風格。

檢查：

* 神社與鳥居之間的視覺平衡。
* 水面留白。
* 船隻的相對位置。
* 屋頂與鳥居的視覺重量。
* 場景在桌面與手機上的完整度。

除非確實有必要，否則不要大幅修改既有場景物件位置。

六、實作效能與品質控制

* 避免在每幀重新建立 Geometry 或 Material。
* 避免在 `useFrame` 裡持續觸發 React State 更新。
* 使用合理的 Delta Time。
* 控制 Shadow Map 與 Render Target 解析度。
* 支援桌面與手機的不同渲染品質。
* 保留既有正常的互動功能。
* 不要引入沒有必要的新套件。

七、視覺驗收

請提供 Before / After 對照。

至少檢查：

* 原始預設視角。
* 神社屋頂與建築細節。
* 鳥居與水面交界。
* 岩石周圍的泡沫。
* 三艘船所在水域。

```
* 手機版場景構圖。

若有 Browser 或 Screenshot 能力，請實際渲染並檢查，而不是只確認 Build 成功。
八、完成條件
本階段完成時應達到：

1. 水面呈現自然、低振幅的持續運動。
2. 整體光影明顯比原本更精緻。
3. 水面與場景視覺風格一致。
4. 泡沫位置合理，不會隨機散布。
5. 可以共用水面高度函式。
6. 沒有明顯 Shader 錯誤。
7. 原本的場景與互動不受破壞。
8. 已記錄效能測試結果及尚未完成的部分。

完成後請更新 `docs/IMPLEMENTATION_PLAN.md`，標記實際完成項目，並列出 Phase 02 所需的擴充介面。
請不要在本階段提前實作櫻花粒子、船隻物理、日夜切換或 Cinematic Camera。
本階段以「視覺品質與水面基礎」為唯一核心目標。
```

Claude 做完附了 7 組改動前後的截圖。過程中它自己找到四個問題。

要提早發現，可以請 Claude「每次改動前後各截一張全景，對照岩石和船的位置有沒有變」。

症狀	原因	修法
屋頂瓦紋完全看不到	屋頂曲面的貼圖座標用的是場景單位，瓦片方向和密度都錯	屋頂專用貼圖，換算重複次數
加了新貼圖之後，岩石和船全部換位置	新貼圖多呼叫了共用的 `rand()`，後面每個物件拿到的亂數都往後移一格	新貼圖用自己的亂數。調換建構順序也會讓序列錯開，所以 `CLAUDE.md` 寫明了建構順序不能隨便動
岸邊泡沫是一個規則的橢圓，和岩石之間有空隙	用橢圓近似小島的形狀	啟動時從正上方拍一次小島剪影，算出到岸線的距離，存成貼圖
泛光加上去，高畫質的 draw call 從 362 跳到 565	泛光要多畫一次場景，連海也畫了	海面擋不到任何發光物，泛光那一趟不畫海。三角形數回到 285k，比修改前少

色調映射是 Claude 自己並排比過三種才選的：Neutral、曝光 0.85。比較圖見〈⚖️ 動手前先想〉。如果你想確定 AI 會這樣做，你也可以在 3–1 後面加一句：

貼給 AI

色調映射請不要直接選一個。用同一個畫面、同一個曝光，
`ACES`、`Neutral`、`AgX` 三種各截一張並排給我看，
並說明每一種讓朱漆、石台、黃昏天色各變成什麼樣。

✅ 怎麼知道做對了

- [ ] 水面有好幾道不同方向的浪，看一分鐘找不到明顯重複的節奏
- [ ] 岸邊泡沫貼著每顆岩石的形狀，不是一個規則的橢圓
- [ ] Claude 給了改動前後的對照截圖，不是只回報 build 成功
- [ ] 岩石和船的位置跟這一步之前一樣（截圖對照）
- [ ] 網址加 `?stats`，記下 draw call 與三角形數，之後每一步都拿來比

🆘 如果沒有成功

你可能會看到	通常是因為	怎麼辦
加了新貼圖之後，岩石和船全部換位置	新貼圖呼叫了共用的 `rand()`	「新加的東西請用它自己的亂數，不要呼叫共用的 rand()」
屋頂的瓦紋完全看不到	屋頂曲面的貼圖座標方向和重複次數錯了	「屋頂的瓦紋看不到，請檢查屋頂的貼圖座標方向與重複次數」
岸邊泡沫是一個橢圓，和岩石之間有空隙	用橢圓近似小島的形狀	「泡沫要貼著每顆岩石。請改用從正上方拍一次小島剪影、算出到岸線距離的做法」
draw call 突然變多很多	泛光要多畫一次場景	「泛光那一趟請只畫會發光的東西，海面不用畫」
Claude 順手做了櫻花或船的物理	指令最後一句被刪了	「請先只做 Phase 01。櫻花粒子、船隻物理、日夜切換、運鏡都留到之後」

這一步失敗可以跳過嗎：不行。階段 4 的船和漣漪都讀這片水面。

📖 為什麼這樣做

🎯 這一步的重點

一個數值只有一個來源。波浪參數只寫在 `src/sim/water.js`，shader 的公式由同一份陣列產生。

🧭 這一步在做什麼

把水面從「看起來在動」改成「有一個公式、別的程式可以讀」。

水面是 shader 畫的，船的高度是 JavaScript 算的。兩邊如果各寫一次波浪公式，改了一邊忘了改另一邊，船就會浮在空中或沉進水裡。所以 shader 的那段公式由 `water.js` 的陣列產生，CPU 那邊用同一個陣列算 `getWaterHeight()`。

這一步還改了兩件事：

- 頂點只吃波浪高度，細部雜訊只進法線。細小的水紋只改變光怎麼反射，不改變水面高度，船就不會被細紋晃得發抖
- 四道波的角速度不成比例。階段 2 盤點時發現原本的三道波每 62.8 秒重複一次，這裡一起改掉

🧩 它在建構流程的位置

```
【水面公式 water.js】 ──┬──▶ shader 畫水面
                        └──▶ getWaterHeight() ──▶ 船、漣漪、花瓣（階段 4、9）
```

💡 那段指令為什麼那樣寫

3-1 的最後兩句限制了範圍：這個 Phase 只做水面與光影，後面的功能先不做。大規格的每一個 Phase 都寫得很完整，AI 讀到後面的功能，很容易順手一起做。一次做太多，出問題時你分不出是哪一個改動造成的。階段 2-1 那份英文規格沒有這兩句，Claude 就四個 Phase 一起寫了。

第七節要求「若有 Browser 或 Screenshot 能力，請實際渲染並檢查，而不是只確認 Build 成功」。build 成功只代表程式沒有語法錯誤，不代表畫面是對的。

這一步一次就做完，因為規格在階段 2 已經寫好，Phase 01 的指令又列了要檢查的截圖。Claude 照清單自己截圖，四個問題都是它在截圖裡找到的。

階段 4　做漣漪與船的浮力

📖 讀完約 15 分鐘　🛠️ 動手約 50 分鐘

> - **這一步一定要懂**：船的反應來自它讀到的水面高度，不是點擊時播一段搖晃動畫。浮力的數學放在不碰 three.js 的檔案裡，才能用 Node 測試。
> - **這一步可以先不懂**：漣漪公式、阻尼彈簧的推導。
> - **做完後你要能用自己的話回答**：漣漪經過時，船高一開始只變 0.009。原因是什麼？

⚖️ 動手前先想

船的浮力能不能寫成 3D 程式以外的純數學？

可以。浮力只需要「某個位置、某個時間的水面高度」，不需要 three.js。寫在 3D 程式裡的話，只能開瀏覽器用眼睛看。階段 2 盤點時發現的俯仰正負號反了，就是這樣一直沒被看出來。

放在 `src/sim/buoyancy.js`、不 import three，就能用 Node 餵固定的時間間隔測試：

測試	驗什麼
正負號	水在船頭比較高時，船頭往上抬
30 / 60 / 120 fps	同樣的 5 秒，三種幀率結果一致
船舷餘裕	最壞的浪加上最大傾角，浪不會漫過甲板
梯度一致性	CPU 算的水面斜率和 shader 一致

> **名詞**：**臨界阻尼彈簧**是一種追目標的方式：起步慢、中途加速、到了不會衝過頭來回晃。
> 你需要知道到什麼程度：知道它比「每幀往目標靠近一點」穩，而且和幀率無關。船、階段 10 的鏡頭、時段轉場都用它。

下面 4–3 會要船「自己搖幾下」，做法是把阻尼比調到 1 以下，讓它衝過頭。4–3 也會加兩個測試。一個和 1000 步的數值積分比對，一個驗最壞傾角加上衝過頭時的船舷餘裕。

🛠️ 動手做

🧰 這一步的產出

```
src/sim/ripples.js       ← 漣漪：兩個池（點擊、花瓣），CPU 與 GPU 同一組參數
src/sim/buoyancy.js      ← 船的浮力：5 點取樣、阻尼彈簧。不 import three
tests/sim.test.mjs       ← 用 Node 跑的數學測試
docs/PHASE_02_PROMPT.md ← 對齊實際架構後的 Phase 02 指令
```

畫面上應該看到：點空曠的水面起一圈漣漪。點在船旁邊，船被推起、傾斜，過一下恢復。

📋 貼給 AI（4–1：計畫裡預留的接口要做什麼）

Phase 01 做完，`docs/IMPLEMENTATION_PLAN.md` 第 8.5 節列了一張「預留的接口」。看不懂那一節，就直接問：

貼給 AI

你只需要修改：8.5 換成你的計畫裡看不懂的那一節。

```
8.5 具體要做的工作有什麼？
```

Claude 的回答：8.5 本身不是工作，是 Phase 01 在程式裡留好的空接口，目前都回傳 0。要把它們接上的工作有五項：點水面漣漪、船的浮力、櫻花瓣、燈籠、畫質參數。它建議順序是先做船的浮力，再做漣漪，最後花瓣。也指出一個計畫上的矛盾：漣漪原本排在 Phase 3，但花瓣落水要用漣漪池，所以漣漪要提前。

📋 貼給 AI（4-2：Phase 02 互動漣漪與船的浮力）

這份是事先寫好的 Phase 02 指令，約 7,000 字。它寫的是 React Three Fiber 的架構，但專案裡沒有裝。他沒有自己改，而是在原文前面加了一句，請 Claude 把 4-1 的建議一起併進去：

貼給 AI

你只需要修改：第一句照貼。底下的規格換成你的 Phase 02。檔案路徑（`docs/ART_DIRECTION.md` 等）要和你階段 2 建的一樣。

```
這是原本 Phase 2 的 Prompt，請結合你剛剛的建議一起新增調整

Phase 02 — Interactive Water Ripples & Boat Physics
Role & Objective
請作為 Senior Creative Developer、Three.js / React Three Fiber Engineer、GPU Shader Engineer 與 Technical Artist，接續目前神社場景的開發。
本階段目標是讓現有的水面系統，從單純的視覺動畫，升級成能夠回應使用者與場景物件的互動系統。
核心體驗：

1. 使用者點擊或觸碰水面，可以產生自然向外擴散的漣漪。
2. 不同位置、不同強度的漣漪可以同時存在並互相疊加。
3. 三艘木船會根據所在位置的實際水面高度與法線產生浮動、Pitch 與 Roll。
4. 使用者在船隻附近製造水波時，船隻會出現自然的物理反應。
5. 整體效果保持 Low-poly / Isometric 的美術方向，不應變成寫實的物理模擬器。

本階段優先確保數學模型一致、互動自然、動畫順暢與視覺品質，不追求不必要的物理真實度。
01. Project Inspection
開始實作前，請先閱讀：

* `CLAUDE.md`
* `.claude/rules/`
* `docs/ART_DIRECTION.md`
* `docs/SIMULATION_SPEC.md`
* `docs/IMPLEMENTATION_PLAN.md`

並檢查現有專案：

* Three.js / R3F 版本。
* 水面 Mesh 與 Shader。
```

* Phase 01 已完成的水面高度函式。
* 水面材質、Normal 與 Uniform。
* Water Plane 使用的座標系。
* 三艘船的模型結構、Pivot 與 Transform。
* Canvas 的 Pointer Events。
* Camera Controls。
* 現有 Animation Loop。
* Performance / Quality Presets。

如果 Phase 01 的實作與文件不同，請以實際程式為準，先說明差異。
禁止為了完成新效果而重新建立一套獨立的水面系統。
應優先擴充既有 Water Simulation。
02. Shared Water Simulation
建立或完善統一的水面採樣介面。
建議 API：

```
interface WaterSample {
  height: number;
  normal: THREE.Vector3;
  velocity?: THREE.Vector3;
}

interface WaterSimulation {
  sample(
    worldX: number,
    worldZ: number,
    time: number
  ): WaterSample;
}
```

水面位移應包含：
[
H(x,z,t)=H_{base}(x,z,t)+H_{ripple}(x,z,t)
]
其中：

* `H_base`：Phase 01 的基礎波浪。
* `H_ripple`：本階段新增的互動漣漪。

要求：

* 共用同一個世界座標系。
* Shader 與 CPU Sample 必須使用一致的公式與參數。
* 所有互動事件使用相同的模擬時鐘。
* 不要以 React State 每幀同步大量水面數據。
* 如果使用 GPU 模擬，必須明確設計 CPU 與 GPU 的資料一致性策略。

不要假設可以免費取得 GPU Render Target 中的每個水面高度值。

03. Interactive Ripple Shader

3.1 Analytical Ripple Model

第一版優先使用解析式 Ripple Model，而不是直接導入 GPU Heightfield。

建議公式：

[
r(d,\tau)=A e^{-\lambda\tau}
e^{-\mu(d-c\tau)^2}
\cos(k(d-c\tau))
]

其中：

* (d)：水面取樣位置與漣漪中心的距離。
* (\tau)：漣漪產生後經過的時間。
* (A)：初始振幅。
* (\lambda)：時間衰減。
* (\mu)：波前寬度控制。
* (c)：傳播速度。
* (k)：空間頻率。

僅在 `τ >= 0` 時套用該漣漪。

實作時使用與場景尺度一致的參數，並避免因高頻波紋造成畫面鋸齒與閃爍。

3.2 Ripple Events

建立漣漪事件結構：

```
interface RippleEvent {
  id: number;
  position: {
    x: number;
    z: number;
  };
  startTime: number;
  amplitude: number;
  speed: number;
  decay: number;
  width: number;
  frequency: number;
  duration: number;
}
```

支援：

* Mouse Click
* Touch Tap
* Multiple Simultaneous Ripples
* Automatic Ripple Expiration
* Different Ripple Intensities

所有漣漪必須有生命週期。

請設定 Ripple Pool 最大容量，避免使用者連續點擊造成事件無限累積。

3.3 Raycasting

使用 Three.js Raycaster 或既有 R3F Pointer Events 將使用者輸入映射到水面。

要求：

1. 點擊水面才觸發漣漪。
2. 點擊神社、鳥居或其他非水面物件時，不應穿透觸發水波。
3. 點擊 UI 時不應觸發 Canvas 互動。
4. 支援滑鼠、觸控與必要的 Pointer Capture。
5. 能正確處理 Camera Orbit、Drag 與 Click 的差異。
6. 拖曳攝影機不能意外觸發大量漣漪。

請建立 Drag Threshold。

例如：

* Pointer Down 記錄座標。
* Pointer Up 時比對位移。
* 在合理位移與時間範圍內判斷為 Click / Tap。
* 若超過門檻，視為 Camera Drag。

具體門檻應考量 CSS Pixels 與不同裝置的輸入差異。

3.4 Ripple Visual Design

漣漪必須：

* 平滑向外擴散。
* 逐漸降低振幅。
* 產生柔和的法線與明暗變化。
* 能與原本水面波形疊加。
* 避免形成白色實心圓盤。
* 不應像 UI 圓形動畫貼在水面上。

若水面位移幅度很小，可以透過 Normal Perturbation 增強可見度。

但不能為了提高辨識度而破壞原本的水面材質風格。

04. Boat Buoyancy Simulation

目前場景中有三艘木船。

請辨識每艘船的模型結構，建立可共用的 `useBoatBuoyancy()` 或相應的 Simulation Controller。

4.1 Water Sampling

每艘船至少取樣：

* 船隻中心
* 船頭
* 船尾
* 左側
* 右側

所有取樣點必須根據船隻目前的世界座標、方向與尺寸計算。

不可假設船隻長軸永遠與世界 X 軸平行。

4.2 Vertical Displacement

船隻的目標高度：

[

y_{target}=H(x,z,t)+y_{offset}

]

其中 `y_offset` 為模型與水面的合理吃水偏移。

使用 Damped Interpolation，而不是直接把船隻 Y 座標指定為水面高度。

4.3 Pitch & Roll

依照船頭與船尾的水面高度差，推算 Pitch。

依照左右兩側的水面高度差，推算 Roll。

建議使用：

[

\theta_{pitch}\approx \arctan

\left(\frac{h_{bow}-h_{stern}}{L}\right)

]

[

\theta_{roll}\approx \arctan

\left(\frac{h_{right}-h_{left}}{W}\right)

]

其中：

* `L`：船頭至船尾的取樣距離。
* `W`：左右側的取樣距離。

旋轉軸與正負號必須依據實際船隻局部座標系決定。

建議將船隻目標姿態轉換成 Quaternion，再透過平滑插值更新。

4.4 Damped Motion

船隻不可直接追蹤水面採樣值，否則可能產生不自然的高頻抖動。

使用阻尼彈簧：

[

\ddot{x}+2\zeta\omega\dot{x}+\omega^2(x-x_{target})=0

]

建議：

* 初始阻尼比接近 Critical Damping。
* 垂直位移與旋轉可以使用不同的回應速度。
* 限制最大 Pitch / Roll。

* 避免超出合理水位。
* 使用 Frame-rate Independent 的更新方式。

請根據實際模型尺寸調整振幅與角度上限。

4.5 Three Boats, Different Behaviors

三艘船不可使用完全相同的浮動節奏。

應透過：

* 實際水面位置
* 船隻朝向
* 船隻尺寸
* 不同的阻尼參數
* 初始狀態

自然產生不同的反應。

不要單純對每艘船加入不同的 Random Sine Wave，取代水面採樣。

05. Boat and Ripple Interaction

這是本階段最重要的互動驗收。

當使用者在船隻附近點擊水面時：

1. 產生漣漪。
2. 漣漪向外傳播。
3. 波前抵達船隻附近。
4. 船隻的水面採樣結果發生變化。
5. 船隻產生自然的上下位移與傾斜。
6. 波紋衰減後，船隻逐漸恢復到基礎波浪的浮動狀態。

禁止使用點擊事件直接觸發船隻預製搖晃動畫，假裝船隻受到水波影響。

船隻應透過共用水面函式間接感受到漣漪。

為了維持美術品質，可對船隻反應做適度振幅限制與濾波。

06. Water Contact Visuals

請優化船隻與水面的接觸感。

建議加入：

* 船體附近極輕微的水面擾動。
* 船隻與水面之間合理的接觸陰影。
* 船隻移動時細緻的水波反應。

* 船隻周圍柔和且低對比的泡沫或亮度變化。

避免：

* 每艘船下方都有固定白色光圈。
* 船隻周圍出現過大的水花。
* 船隻像漂浮在水面上方。
* 船隻移動時產生不合理的水面拖影。

如果目前水面材質不支援完整動態接觸效果，請採用效能合理且視覺一致的近似方案。

07. Performance & Architecture

本階段應避免新增不必要的第三方物理引擎。

優先使用：

* Three.js Math Utilities
* React Three Fiber
* Existing Custom GLSL
* Shared Simulation Clock
* Lightweight CPU Sampling
* Bounded Ripple Pool

不需要為三艘船導入完整剛體物理系統。

請控制：

* Ripple Uniform 數量。
* Shader Loop 上限。
* 每幀 CPU Sampling 次數。
* Pointer Events 的處理成本。
* Object Allocation。
* React Re-render。

請對 Low / Medium / High 品質設定不同的 Ripple 容量或視覺細節。

不要讓低階裝置因多次點擊而出現明顯卡頓。

08. Debug & Tuning Controls

請建立開發環境專用的調整面板，使用現有工具；如需新增 Leva 或其他套件，先評估必要性。

至少能調整：

* Base Wave Amplitude
* Ripple Amplitude
* Ripple Speed
* Ripple Frequency
* Ripple Decay
* Ripple Width
* Maximum Active Ripples
* Boat Buoyancy Offset
* Boat Vertical Damping
* Boat Rotation Damping
* Maximum Pitch / Roll

另外提供 Debug Mode：

* 顯示 Ripple Origin。
* 顯示 Ripple Influence Radius。
* 顯示 Boat Sampling Points。
* 顯示 Boat Target Height。
* 顯示水面法線或船隻目標姿態。

Debug Visuals 不得出現在正式使用者介面。

09. Test Scenarios

請實際測試以下情境。

Scenario A — Single Ripple

點擊空曠水面。

預期：

* 漣漪向外平滑擴散。
* 不出現明顯鋸齒。
* 漣漪自然衰減。
* 沒有異常閃爍。

Scenario B — Multiple Ripples

連續點擊不同位置。

預期：

* 漣漪正確疊加。

* 不互相覆蓋或突然消失。
* Ripple Pool 正常清理過期事件。
* 沒有明顯效能下降。

Scenario C — Ripple Near Boat

點擊木船旁邊的水面。

預期：

* 水波向船隻傳播。
* 船隻的浮動、Pitch、Roll 受到水波影響。
* 反應與波紋抵達時間一致。
* 反應平滑、合理。

Scenario D — Camera Drag

拖曳旋轉場景。

預期：

* Camera Controls 正常。
* 不會意外連續產生漣漪。
* UI 互動不會觸發水波。

Scenario E — Different Devices

測試桌面與手機。

預期：

* Pointer 與 Touch 正常。
* 操作延遲不明顯。
* 視覺品質一致。
* 不會因裝置差異導致座標錯誤。

Scenario F — Long Running Simulation

讓場景持續運作至少五分鐘，並進行多次水面點擊。

預期：

* 不發生記憶體持續異常增長。
* 船隻姿態不會累積偏移。
* Ripple Events 正常清理。

* FPS 不出現持續性的明顯下降。

10. Visual Acceptance Criteria

請確保：

1. 水面保持原有柔和、乾淨的藝術風格。
2. 漣漪是水面材質的一部分，而非獨立的 UI 效果。
3. 船隻真正受到水面高度變化影響。
4. 三艘船的運動自然且不完全同步。
5. 互動效果細緻，不過度戲劇化。
6. 鳥居、神社、石燈籠與水面之間沒有新的穿模或渲染問題。
7. 相機控制與 UI 操作正常。
8. 原本 Phase 01 的光影、材質、泡沫與效能沒有明顯退步。

請注意：數學模型正確不代表視覺效果合格。

需要實際觀察動畫、調整參數，確保符合場景的美術方向。

11. Deliverables

請完成：

1. Interactive Ripple System。
2. Shared Water Sampling API。
3. Boat Buoyancy Controller。
4. Boat–Water Interaction。
5. Ripple and Buoyancy Debug Tools。
6. Performance Optimization。
7. Desktop / Mobile Interaction Verification。
8. Documentation Update。

完成後：

* 更新 `docs/SIMULATION_SPEC.md`，記錄實際採用的算法、參數與限制。
* 更新 `docs/IMPLEMENTATION_PLAN.md`，標註 Phase 02 完成狀態。
* 說明新增或修改的檔案。
* 提供效能測試結果。
* 列出尚未解決的問題。

如具備 Browser Automation 或 Screenshot 工具，請實際觀察渲染結果。

除了 Before / After 截圖，也請盡可能提供固定鏡頭的動態測試結果，以確認波紋傳播與船隻反應的時間關係。

本階段禁止提前開發日夜切換、櫻花粒子、整體 UI 重設計或複雜 Cinematic Camera。

請優先完成 Phase 02，確保它是一套穩定、可擴充、視覺品質一致的水面互動系統。

Claude 沒有馬上動手。兩分鐘後它回了一份對齊實際架構的版本，存成 `docs/PHASE_02_PROMPT.md`，逐項差異寫在文件最後：

- 拿掉 React Three Fiber、`useBoatBuoyancy()`、React state。專案是原生 three.js，照原文寫會讓人以為要引入 React
- 水面採樣改成回傳純數字。`src/sim/` 不 import three，才能在 Node 裡測；三艘船每幀採樣 15 次，用 `Vector3` 就是每幀 15 個新物件
- 漣漪池分成點擊與花瓣兩組，花瓣那組這次只留接口。漣漪參數改成常數，每個事件只記位置、時間、強度
- 船的取樣點、旋轉順序、正負號直接寫死，因為現況的錯就是正負號反了
- 不加 Leva，網址加 `?debug` 才建一個原生的小面板
- 補兩個可以量的情境：G 檢查正負號，H 檢查 30、60、120 fps 結果一致

最後一句是「確認這份沒問題的話，說一聲我就照它開工」。看過沒問題，貼：

貼給 AI

開工

看第一列之前先想一想：漣漪的波長和船身差不多長時，船頭在波峰，船尾會在哪裡？船的平均高度會變多少？要量的話，請 Claude「在 Node 裡讓一道漣漪經過船，印出船高與傾斜的最大變化」。規格可以寫「漣漪的波長要明顯比船長 2.2 長」。

症狀	原因	修法
漣漪經過，船高只變 0.009	漣漪波長 2.09 和船長 2.2 差不多。船頭在波峰時船尾在波谷，平均高度互相抵消	用 Node 掃參數：波長拉長到 3.7、衰減變慢、旋轉彈簧變快。船高變化 0.049、傾斜 3°
漣漪像一個灰白色的甜甜圈	長波長只裝得下一圈，波峰又加亮	物理那層不變，外觀另外加三倍頻率的細紋，只進法線、不改高度

症狀	原因	修法
船邊的白浪一直都在	場景用真實時間，彈簧用夾住的時間，低於 20 fps 時船永遠落後水面	場景、漣漪、彈簧統一用同一個模擬時間 `simTime`
時間統一之後，白浪還是偶爾冒出來	用「船速減水面速度」判斷，基礎波浪加速時彈簧追得慢就會觸發	改成只看漣漪本身的升降速度
點介面面板，後面的水面起漣漪	面板設成讓點擊穿透	面板改成擋住點擊
6 個漣漪同時存在時掉 31% fps	每個像素都把 14 個名額算完	CPU 先只留還活著的漣漪，shader 離波前太遠就跳過。降到掉 20%

驗證：Node 測試（正負號、三種幀率結果一致、船舷餘裕、斜率一致）。瀏覽器逐幀記錄船的狀態，從點擊到波前抵達船，實測 1.1 秒，理論 1.14 秒。5 分鐘長跑，記憶體與物件數沒有增加。

測試本身也錯過兩次。手機觸控「沒反應」，其實是點到了除錯面板。「拖曳不產生漣漪」的檢查一開始無效：漣漪池是滿的，新漣漪只是取代舊的，數總數看不出有沒有新增。

📋 貼給 AI（4–3：船要有自己的角度）

先看畫面：三艘船是不是大部分時間幾乎是平的？是的話，貼：

貼給 AI

> 我覺得船應該要也有自己的漂浮角度，不是都完全不變喔～

症狀 → 原因：船的姿態全部來自水面，基礎波浪又長又緩。船向也完全固定，看起來像釘在水上。Claude 加了三種船自己的動態，疊加在水面採樣之上，不取代它：

動態	數字
固有傾角（像載重不同）	最多約 3°
繞錨點隨風擺盪	風大時最多約 17°，一個來回約 100 秒
會自己搖幾下的旋轉彈簧	阻尼比 0.45，每艘有自己的週期

瀏覽器記錄到同一時刻三艘的擺盪角約 −15°、+12°、+9°。Node 測試從 7 項加到 8 項。`?debug` 面板多一個阻尼比的滑桿，可以自己調手感。

✅ 怎麼知道做對了

- [ ] 點空曠水面，一圈漣漪擴散、慢慢消失
- [ ] 點在船旁邊，波紋到了船才動，不是一點下去就動
- [ ] 拖曳旋轉場景，放開之後水面沒有漣漪
- [ ] 點右上角的介面，後面的水面沒有漣漪
- [ ] 請 Claude 跑 `node --test tests/sim.test.mjs`，全部通過
- [ ] 三艘船各自有不同的傾角，會自己輕輕搖幾下

兩圈漣漪是淡淡的細紋，不是白色的實心圓環。這是「甜甜圈」那一列修完之後的樣子。

🆘 如果沒有成功

你可能會看到	通常是因為	怎麼辦
漣漪經過，船幾乎沒動	漣漪波長和船長差不多，前後抵消	「請用 Node 掃漣漪的波長、衰減與彈簧速度，找出船反應最明顯的組合」
漣漪像白色的圓環	長波長只裝得下一圈，波峰又加亮	「物理那層不要改。外觀另外加一層更細的紋路，只影響光的反射，不改高度」
船邊的白浪一直都在	時間來源不只一個	「場景、漣漪、彈簧請統一用同一個模擬時間（simTime）」
點介面面板，後面的水面起漣漪	面板讓點擊穿透	「點介面時不要產生漣漪，面板要擋住點擊」

你可能會看到	通常是因為	怎麼辦
Claude 做了一段「點擊時船搖一下」的動畫	指令的第 05 節被刪了	「船要透過共用的水面函式間接感受到漣漪，不要用點擊事件直接觸發船的搖晃」

這一步失敗可以跳過嗎：4–3 可以。4–2 不行，階段 9 的花瓣落水會用到漣漪池。

📖 為什麼這樣做

🎯 這一步的重點

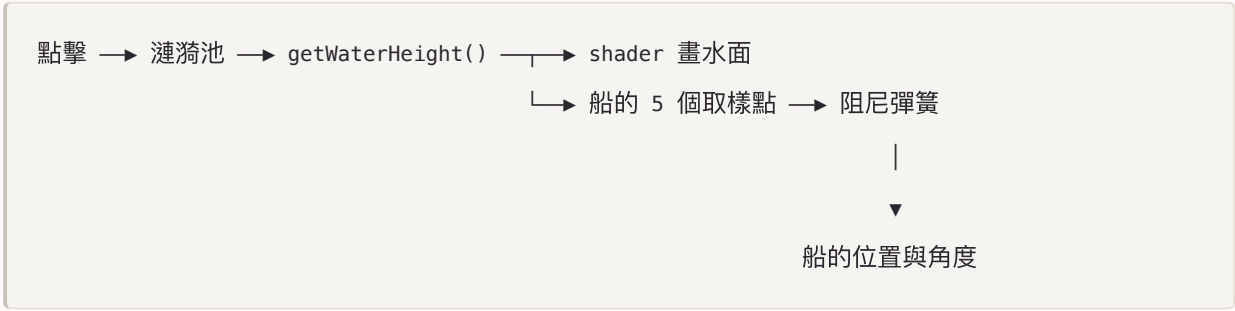
船不知道有人點了水面。它只知道自己腳下的水面高度變了。

🧭 這一步在做什麼

讓水面能被點，讓船讀得到這個變化。

漣漪加進 `getWaterHeight()` 之後，所有讀水面高度的東西都會跟著受影響。船、之後的花瓣、柱腳的白浪都是。

🧩 它在建構流程的位置



💡 那段指令為什麼那樣寫

這段指令最重要的一句在 4–2 原文第 05 節：「禁止使用點擊事件直接觸發船隻預製搖晃動畫，假裝船隻受到水波影響」。那是最快做出「看起來對」的方法。效果在你點的那一下完全正確，但它和水面沒有關係。點在船的另一邊，船一樣會搖。之後加花瓣落水、加雨，船都不會有反應。

第 09 節的六個測試情境，讓「做完了」有一個你能檢查的定義。情境 F 的五分鐘長跑，是為了抓「記憶體慢慢變大」這種短時間看不出來的問題。

原文第 01 節寫了「如果 Phase 01 的實作與文件不同，請以實際程式為準，先說明差異」。這一句讓 Claude 有理由在動手前先改寫這份指令。

計畫文件看不懂的地方，直接問「具體要做的工作有什麼」，Claude 會把它翻成檔案與步驟。

階段 5　做白天、黃昏、夜晚與燈籠

📖 讀完約 8 分鐘　🛠️ 動手約 30 分鐘

- **這一步一定要懂**：時段是一個 0 到 1 的數字，按鈕只改它的目標值。天氣是乘在時段上的係數，兩者不會互相蓋掉。
- **這一步可以先不懂**：每一盞燈的亮度曲線怎麼調。
- **做完後你要能用自己的話回答**：黃昏時海上出現一個奇怪的方塊。那是什麼？怎麼修的？

🛠️ 動手做

🧰 這一步的產出

```
src/sim/daycycle.js          ← 時段數值 0（白天）到 1（夜晚）
src/scene/timeofday.js       ← 光源、水色、天色跟著時段數值變
src/scene/nightLanterns.js ← 兩盞提燈、五盞漂浮燈籠、兩盞點光源
```

畫面上應該看到：多一組白天 / 黃昏 / 夜晚按鈕。切到夜晚，燈籠一盞接一盞亮，海上浮出幾盞漂浮燈籠。

📋 貼給 AI（5-1：確認有沒有夜晚模式）

不確定有沒有夜晚模式，就直接問：

貼給 AI

> 確認有夜晚模式了嗎？

Claude 的回答是還沒有。目前固定是黃昏，只有晴、薄霧、小雨三種天氣。三組色盤寫在 `docs/ART_DIRECTION.md`，和天氣疊加的規則寫在 `SIMULATION_SPEC.md`。但兩者都還沒接進程式。它列出要做的四件事：

- 一個 0 到 1 的時段數值
- 天氣改成乘在時段之上
- 一排切換按鈕

- 夜晚加兩盞點光源

最後問要不要接著做。

📋 貼給 AI（5–2：入夜才亮的燈籠）

確認還沒有夜晚模式之後，貼：

> 貼給 AI
>
> **你只需要修改**：沒有。整句照貼。
>
> 接著做！另外我想要製作幾個燈籠，晚上會打開

結果：

- 時段切換約 2 秒，從白天到夜晚會經過黃昏的顏色。天氣乘在時段之上，夜晚也會下雨、起霧
- 紅色提燈兩盞，掛在本殿屋簷外緣
- 漂浮燈籠五盞（灯籠流し），在鳥居內外的水面上跟著浪起伏。點水面的漣漪也會推動它們。成品後來減成三盞，拿掉的兩盞擋在參拜小船的航道上
- 點光源兩盞，夜晚照亮參道與碼頭
- 天色暗下來時，燈一盞接一盞亮

Claude 截圖時自己找到兩個問題：

症狀	原因	修法
屋簷下的提燈看不到	掛得比屋簷下緣還高，從斜上方看被屋頂整個擋住	移到屋簷外緣、往下掛
黃昏時海上有一個奇怪的方塊	漂浮燈籠白天沒點亮，看起來像箱子	白天不出現，入夜才浮現

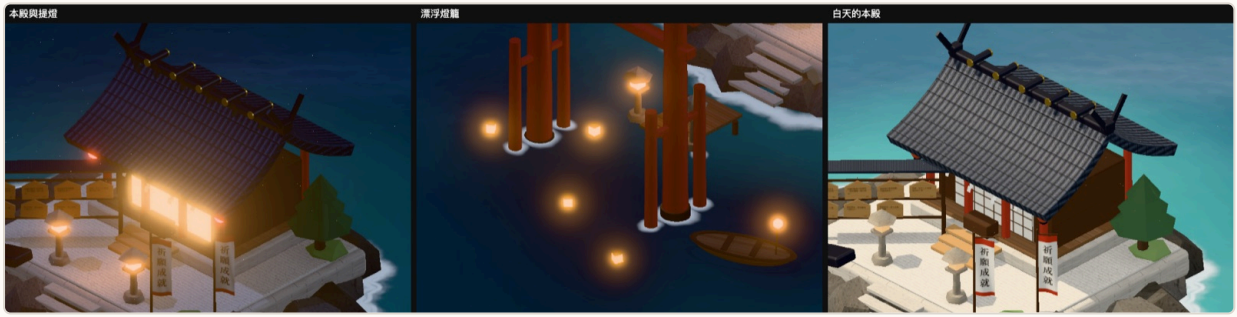
如果你沒有問 5–1，Claude 做出來的天氣可能會把時段蓋掉。例如夜晚一下雨，就變回白天的亮度。你也可以加這一段。這是 Claude 在階段 2 評估規格時就決定的做法：

貼給 AI

時段請用一個 0（白天）到 1（夜晚）的數字表示，按鈕只改目標值，畫面慢慢追過去。
天氣不要直接設定光源的亮度，改成乘在時段結果上的係數，
這樣夜晚下雨是「夜晚的亮度再暗一點」，不會跳回白天。

✅ 怎麼知道做對了

- [ ] 三個按鈕都能切換，畫面是慢慢變的，不是一下跳過去
- [ ] 夜晚時，本殿的提燈、碼頭的石燈籠、海上的漂浮燈籠都亮了
- [ ] 白天和黃昏時，海上看不到漂浮燈籠
- [ ] 夜晚等到下雨，畫面還是夜晚，只是更暗

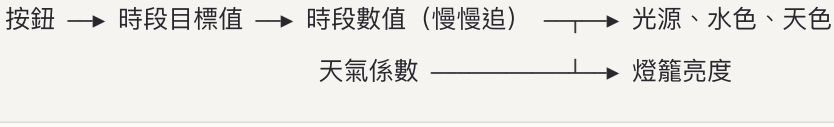


左邊夜晚的本殿，紙門透出暖光，屋簷外緣掛著提燈。中間是夜晚才浮出來的漂浮燈籠。右邊是同一個角度的白天，提燈沒亮。

🆘 如果沒有成功

你可能會看到	通常是因為	怎麼辦
屋簷下的提燈看不到	提燈被屋頂擋住	「提燈從斜上方看不到，請移到屋簷外緣、往下掛」
黃昏時海上有一個奇怪的方塊	漂浮燈籠白天沒點亮，看起來像箱子	「漂浮燈籠白天和黃昏不要出現，入夜才浮上來」
夜晚一下雨，畫面突然變亮	天氣直接設定了光源亮度，蓋掉時段	貼 5–2 最後那一段

> 這一步失敗可以跳過嗎：可以先跳過燈籠。時段本身不行，階段 10 的轉場與階段 13 的選曲都跟著它。

> 📖 **為什麼這樣做**
>
> ### 🎯 這一步的重點
>
> 光、水、天、燈都從同一個時段數字算出來。
>
> ### 🧭 這一步在做什麼
>
> 把「現在是幾點」變成一個 0 到 1 的數字。
>
> 所有會跟著時段變的東西，都在主迴圈裡讀這個數字。按鈕不直接改光源，只改「目標」。這樣中途再按一次別的時段，畫面會從現在的位置轉過去，不會先跳回起點。
>
> ### 🧩 它在建構流程的位置
>
> 
>
>
> ### 💡 那段指令為什麼那樣寫
>
> 這一步的指令只有一句。它能用，是因為兩件事已經先做了。階段 2 把時段與天氣的規則寫進 `docs/`，5–1 又讓 Claude 把要做的事列出來。
>
> 如果你跳過了階段 2，這一句話會讓 Claude 自己決定時段怎麼表示。那個決定會影響階段 10 和 13。

階段 6　把柱子、木樁、繪馬架接上

📖 讀完約 10 分鐘　🛠️ 動手約 35 分鐘

- **這一步一定要懂**：柱子、木樁這類要接在別的東西上的，位置直接用對方的常數算。例如後排柱子的高度用屋頂的曲線算，不要另外寫一個固定高度。驗收要看接合處的特寫，全景看不出 0.2 單位的縫。
- **這一步可以先不懂**：屋頂的曲線怎麼算、陰影貼圖怎麼接進自己寫的 shader。
- **做完後你要能用自己的話回答**：船的影子為什麼「又」不見了？為什麼說之前的影子是假的？

⚖️ 動手前先想

柱子、木樁的位置，要不要用屋頂、碼頭板的常數算？

要。這個案例裡同一種「沒接上」出現了四處：

哪裡	兩邊各寫的數字
本殿後排柱	柱子一樣高，屋頂底面是曲面，後排空出 0.25
碼頭木樁	木樁在 ±1.5，碼頭板只到 ±1.3
繪馬架	轉 45° 之後前柱在 z 5.55，砂利邊緣只到 ±5.4
屋形船的篷（階段 12）	柱子在 z ±0.5，篷的半徑是 0.66，柱頂懸在篷裡面

改成從對方的常數推出來。之後改屋頂的曲線、碼頭板的寬度、篷的半徑，柱子和木樁會跟著動。

「看起來有影子」是不是真的有影子？

不是。做到 6–4 時影子會不見，原因是水面從來沒有接收陰影。之前那圈暗色橢圓讓它看起來有，6–3 縮小橢圓之後，缺的東西才看得出來。

🛠️ 動手做

🧰 這一步的產出

不會多出新檔案。本殿、碼頭、繪馬牆、船的程式各改一段。

畫面上應該看到：

- 柱子頂到屋頂
- 碼頭木樁接到板子
- 石燈籠站在碼頭上
- 船在水上有影子

這些問題實際上是在階段 5 到階段 9 之間陸續發現的。手冊把它們集中在這裡，順序照實際時間。

你自己做（6–1：找出沒接上的地方）

把畫面放大到最大，轉一圈。找這四種地方：

看哪裡	沒接上的樣子
本殿的柱子頂端	柱子和屋頂之間有空隙，後排比前排明顯
碼頭的木樁	木樁站在板子外側，頂端沒碰到板子
繪馬牆的柱腳	柱子懸在石台邊緣外
船和水面	船看起來浮在水面上方，水上沒有影子

找到的每一個都截一張局部放大的圖。

📋 貼給 AI（6–2：柱子、橋墩、繪馬架沒接上）

6–1 找到哪一個，才貼對應的那一段，附上你自己的截圖。

柱子沒接上屋頂

貼給 AI

你只需要修改：換成你自己找到的問題。一定要附截圖。

（附上截圖）

有一些小細節例如柱子沒有連接到天路版這種

「天路版」是錯字，意思是天花板，也就是屋頂底面。Claude 看截圖就知道指的是本殿的柱子。

症狀 → 原因：屋頂底面是凹曲面，柱子卻全部一樣高，後排空出 0.25。修法：用屋頂的 Bézier 曲線算出每一排柱子的高度。再加一根連接柱頂的朱紅橫樑（頭貫）。

橋墩沒接上

貼給 AI

〈附上截圖〉

另外還有一個是橋墩沒有接上，中間有間隙

症狀 → 原因：木樁放在 ±1.5，碼頭板只到 ±1.3。木樁站在板子外側，頂端也沒碰到板子。修法：木樁移到板子下方，每邊三根，加兩條縱樑把木樁和木板接起來。水面的柱腳浪花改成從同一份 `DOCK_POSTS` 產生，木樁移了，浪花跟著移。

繪馬架沒貼好

貼給 AI

〈附上截圖〉

一個很小的是這個架子沒有完整的貼好，整個繪馬架可以稍微縮小一點

症狀 → 原因：繪馬牆轉 45° 之後，前柱落在 z 5.55，超出砂利邊緣（±5.4）。修法：整個繪馬架縮成 0.8 倍、中心往內移，四周都留 0.5 以上。

📋 貼給 AI（6-3：船影的位置、燈在海上、燈太大）

先看船旁邊的影子和碼頭上的石燈籠。有一樣的問題才貼。

船影位置、燈在海上

兩個問題附兩張圖，用「附圖一」「附圖二」分開講：

貼給 AI

〈附上截圖〉

〈附上截圖〉

附圖一這種船影的位置好像不太正確

附圖二的燈在海上不應該

問題	原因	修法
船影的位置不對	船浮太高，船底只沾到水；影子卻用整個船身的大小畫，從斜上方看就錯開了	掃描不同吃水的最壞餘裕，吃水從 0.38 改成 0.2；影子改用水面實際切過船身的大小
石燈籠站在海上	碼頭寬 ±1.3，燈籠放在 ±1.7，底座懸在水面上	移到碼頭板上

第一列推翻了階段 1–4 的修法。當時船身看得到藍色，修法是把船抬高。那是用抬高掩蓋問題。船要浮多高，你可以先問自己：最大的浪加上漣漪，水面最高會到哪裡？船舷要比它高多少才看不到船裡的水？這一次改成船身封頂，再算清楚最壞情況下的餘裕。三艘船跑 60 秒、每 4 秒在船旁打一個漣漪，水面仍離船舷 0.22。要自己量的話，可以請 Claude「讓三艘船跑 60 秒，每 4 秒在船旁打一個漣漪，回報水面離船舷最近是多少」。規格可以寫「水面到船舷至少留 0.2」。

燈太大

貼給 AI

〈附上截圖〉

這兩個燈感覺體積要小一點，不然目前好像佔了橋上很大的體積

燈籠移上碼頭之後，佔掉了走道。Claude 把它縮成島上燈籠的 0.6 倍，靠在碼頭兩側邊緣，中間留出走道。

📋 貼給 AI（6–4：船又沒有影子）

貼給 AI

〈附上截圖〉

目前看起來船又沒有陰影了..

症狀 → 原因：水面是自己寫的 shader，從來沒有接收太陽的陰影。之前看起來有影子，是船邊一圈假的暗色橢圓。6–3 把它縮成實際水線大小之後，幾乎看不到了。

修法：水面接上 three.js 內建的陰影，影子濃淡跟著太陽強度（薄霧、雨、夜晚變淡）。船、鳥居、碼頭、小島的影子都落在水上了。低畫質沒有陰影貼圖，不受影響。

📋 貼給 AI（6–5：把規則寫進 CLAUDE.md）

同一種「沒接上」出現第四次（階段 12 的船篷）時，用的是這一句：

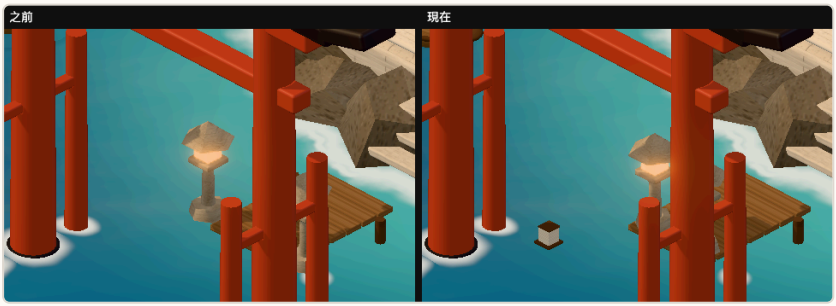
> 貼給 AI
>
> 這種對上的應該也要更新回 `CLAUDE.md`

你不用等到第四次。6–2 修完，就可以貼上面那一句。如果你想把規則寫得更明確，可以加這一段。這是照 `CLAUDE.md` 最後寫進去的內容整理的：

> 貼給 AI
>
> 請在 `CLAUDE.md` 的工程原則加一條：
>
> 要接在別的東西上的物件（柱子、木樁、架子），位置用對方的常數算出來，例如柱子高度用屋頂的曲線算，不要另外寫一個固定的數字。
>
> 驗證時一定要附接合處的特寫截圖。
>
> 並把這次修過的地方（哪兩樣東西、原本差多少）列在那一條底下當例子。

✅ 怎麼知道做對了

- [ ] 放到最大轉一圈，6–1 表上四個地方都接上了
- [ ] 每一個修過的地方，Claude 都給了一張接合處的特寫截圖
- [ ] 船在水上有影子，而且切到薄霧、雨、夜晚時影子變淡
- [ ] `CLAUDE.md` 有那條工程原則

狀態	參考畫面
石燈籠	 局部放大。左邊「之前」，石燈籠在碼頭板子外面，底下是海。右邊「現在」，燈籠縮小後移回碼頭上。

狀態	參考畫面
碼頭 木樁	 局部放大。木樁在碼頭板子正下方，頂端接著板子。水上看得到碼頭和燈籠的影子。

🆘 如果沒有成功

你可能會看到	通常是因為	怎麼辦
修完前排柱子，後排還是空 0.25	屋頂底面是凹曲面，柱子卻一樣高	「柱子高度請從屋頂的曲線算出每一排的高度，不要每排一樣高」
木樁還是沒接上	木樁的位置和碼頭板的寬度各寫一個數字	「木樁請放在板子正下方，位置從碼頭板的寬度算出來」
繪馬牆轉 45° 之後柱子懸空	轉完之後前柱超出石台邊緣	「轉完之後請檢查四個柱腳都在石台內，四周留 0.5 以上」
船的影子只是一圈暗色橢圓	水面沒有接收太陽的陰影	「水面請接上 three.js 內建的陰影，影子濃淡跟著太陽強度變」
Claude 把船抬高來解決「浪漫進船裡」	用抬高掩蓋問題	「不要抬高船。請把船身封頂，算出最壞情況下的船舷餘裕」

這一步失敗可以跳過嗎：可以先跳過。但每一個沒接上的地方，之後換鏡頭特寫時都會被看到。

📖 為什麼這樣做

🎯 這一步的重點

柱子的高度用屋頂的曲線算，木樁的位置用碼頭板的寬度算。屋頂或碼頭板改了，柱子和木樁跟著動，不會再空出縫。

🧭 這一步在做什麼

這一步的幾個問題，原因一樣：兩樣要接在一起的東西，各自寫了自己的位置。

🧩 它在建構流程的位置

> 水面、船、時段 ──▶ 【接合處的修正】 ──▶ 介面（階段 7）

💡 那段指令為什麼那樣寫

這一步的 6 則回饋都只有一兩句，每一則都附了截圖。Claude 看得到截圖裡哪裡沒接上，比你用文字描述位置準確。

6–5 把規則寫進 `CLAUDE.md`，因為對話會被清掉，`CLAUDE.md` 不會。實際做的時候，前三處都只修了那一處：修好柱子之後，碼頭木樁、繪馬架還是各寫各的數字。寫進 `CLAUDE.md` 之後，每一次對話開始時都會讀到這一條。

回報時用字要準：「又」告訴 Claude 這是之前有過、後來不見的東西。兩個問題附兩張圖，就寫「附圖一」「附圖二」分開講。

階段 7　做繪馬卡片、字型與介面

📖 讀完約 12 分鐘　🛠️ 動手約 45 分鐘

> - **這一步一定要懂**：字疊在 3D 場景上，讀不讀得清楚要對著實際畫面量。使用者輸入的文字只能用 `textContent` 或 canvas 放上畫面。
> - **這一步可以先不懂**：Design Token 怎麼命名、Web Animations API 的語法。
> - **做完後你要能用自己的話回答**：螢幕報讀器把祈願數「4」讀成「34」，原因是什麼？

⚖️ 動手前先想

使用者寫的文字，用什麼方式放上畫面？

只經過 `textContent` 和 canvas 的 `fillText`，不放進 `innerHTML`。

> **名詞**：`innerHTML` 是把一段文字當成網頁程式碼放進畫面的寫法。`textContent` 是只當成純文字放進去。
> 你需要知道到什麼程度：知道使用者寫的繪馬只能用後者。用前者的話，有人在繪馬裡寫一段程式，看到那塊繪馬的人都會執行它。

這一條在階段 15 會更重要。登入狀態存在瀏覽器裡，頁面上只要有一個地方能執行別人的程式，登入狀態就拿得走。

🛠️ 動手做

🧰 這一步的產出

```
docs/UI_GUIDE.md          ← 介面規範：Design Token、字級、動態、四個區域、無障礙
tests/ui-contrast.mjs     ← 用腳本量文字對比
```

畫面上應該看到：四個角落各一塊介面。左上標題、右上祈願數與時段、左下操作說明、右下「寫一張繪馬」。點繪馬牆會打開一張繪馬卡片，可以翻面、傾斜、按住祈願。

📋 貼給 AI（7–1：繪馬做成獨立的卡片）

這一句是階段 1–4 那則訊息的後半：

貼給 AI

另外許願的部分我預期跟課程就無關，他可以是個獨立的許願卡片，然後可以做出一些很棒的互動，

結果：願望改成生活願望。點繪馬，卡片翻面飛入，跟著滑鼠傾斜反光。按住 1.1 秒祈願：圓環填滿、火花、光點飛進本殿。也可以寫一張新的繪馬掛上牆。

📋 貼給 AI（7–2：字體、厚度、看不見與模糊的字）

打開繪馬卡片，看字體、卡片厚度，以及疊在場景上的字讀不讀得清楚。有一樣的問題才貼。

字體、厚度、天氣

貼給 AI

你只需要修改：字體換成你要的風格。

（附上截圖）

字體上我希望用例如小冢明體這類型的，不要用標楷體。

同時整體可以有厚度的立體感覺。

另外當然天氣偶爾有一些變化

「小冢明體」是小塚明朝。小塚明朝沒有網頁字型版本，Claude 改用 Noto Serif TC（思源宋體），字集涵蓋繁體中文。

厚度：卡片做 11 層木色側邊，傾斜時看得到板厚；按鈕加實色的底邊。天氣：晴、薄霧、晴、小雨循環，雨有雨絲與水面雨點。

底下的字看不見

貼給 AI

〈附上截圖〉

底下這裡的文字其實也看不見

左下角的說明文字直接疊在海上，看不清楚。Claude 先調了文字的底色與雨的濃度。

字看起來模糊

貼給 AI

你只需要修改：換成你的截圖。最後一句是順便問的，可以刪。

〈附上截圖〉

目前文字呈現上會有些模糊，在設計上想問有沒有更適合的模式。

另外想知道你都是使用 `WebGL?`

症狀 → 原因：文字後面加了一層模糊的光暈，想讓字從海面上浮出來。結果在忙碌的背景上糊成一片。修法：每一塊文字放在幾乎不透明的紙面板上，文字不再依賴背後是什麼。 Claude 也提了另外兩種做法讓你選。一種是只在畫面上下緣淡出的漸層條。另一種是減少文字，說明顯示一次就淡出。階段 11 的 A 方案用的就是漸層的方向。

📋 貼給 AI（7–3：縮放）

貼給 AI

幫我開放可以放大縮小的功能

Claude 做了滾輪、雙指、`+` `-` `0` 鍵的縮放。打開來試，如果放大不了，貼：

貼給 AI

```
我看沒有辦法 zoom in
```

原因有三層，三種症狀和修法列在本階段的〈🆘 如果沒有成功〉。

📋 貼給 AI（7-4：繪馬上的印）

附三張圖：你的繪馬，以及兩張真的御朱印當參考。

貼給 AI

〈附上截圖〉

〈附上截圖〉

〈附上截圖〉

這種應該要是類似附圖二、附圖三這種的御珠印

「御珠印」是御朱印。繪馬上原本只有一個細框方塊寫著「願」。改成朱紅雙框方印「願池神社」加菱形「奉納」副印，用 SVG 濾鏡做墨色不均。

📋 貼給 AI（7-5：介面精修）

這是實際用的 UI / Typography 指令原文。內容是設計方向、Design Token、四個區域、動態、響應式、無障礙。還有和 3D 場景的整合、驗收與工作流程。

貼給 AI

你只需要修改：五個色碼可以換成你的。「先提出一個新的版面配置與設計方向，再分階段實作」與第 8 節的十點驗收照貼。

```
接下來處理這個！


UI / Typography & Interaction Polish
請依照既有的 `CLAUDE.md`、`.claude/rules/`、`docs/ART_DIRECTION.md` 與
`docs/IMPLEMENTATION_PLAN.md`，規劃並執行 Shrine Diorama 的介面視覺優化。
本階段聚焦在 3D 場景周邊的 HTML / React UI、Typography、資訊層級、排版、動態設計與使用
者操作體驗。
```

核心目標是讓整體介面和 3D Diorama 呈現一致的高質感設計，而不是讓 UI 看起來像一個覆蓋在 3D 畫面上的傳統網站。

1. Design Direction

採用 Japanese Editorial Minimalism 的設計方向。

視覺關鍵字：

* Japanese minimalism
* Editorial typography
* Premium digital craftsmanship
* Warm paper texture
* Soft atmospheric surfaces
* Refined motion
* Calm, intentional interactions

避免：

* 過度強烈的 Glassmorphism
* 過大的卡片與陰影
* 大量裝飾線條
* 過度使用漸層
* 科技感霓虹效果
* 一般 SaaS Dashboard 風格
* 大量 Floating Cards

UI 應該融入神社的空間體驗。

2. Design Tokens

請建立統一的色彩與字體變數。

初始參考色彩：

* Paper: #F7F3EB
* Sand: #D7C9AF
* Ink: #26231E
* Vermilion: #C9412E
* Water: #7D9DA1

這些是建議起點，請根據實際場景的 3D 渲染結果進行微調。

字體建議：

* Display / Heading: Noto Serif TC
* Body / UI: Noto Sans TC
* Numeric display: Serif font with tabular numerals where appropriate

請建立：

* Typography scale
* Font weight scale
* Line height scale
* Spacing system
* Border radius
* Border colors
* Shadow tokens
* Motion duration tokens
* Responsive breakpoints

確保繁體中文字型載入與字重正確。

3. UI Hierarchy & Layout

重新規劃畫面四個主要區域。

Top Left — Shrine Identity

保留：

* 海上許願神社
* 簡短場景說明

建議：

* 標題保持主視覺識別。
* 精簡文案，避免大面積文字卡。
* 使用較小的 Editorial Layout。
* 可以增加小型的日文或英文輔助標記，但不得影響中文主標題。
* 說明內容可以漸進顯示。

Top Right — Wish Counter

目前主要內容：

* 已送達的祈願總數
* 願望狀態或場景狀態

請重新設計數字層級：

* 數字具有精緻字體比例。
* 說明文字低對比。
* 減少卡片尺寸。
* 更新時提供克制的數字動畫。

不得捏造數據；使用既有資料來源。

Bottom Left — Interaction Guide

將目前固定顯示的長段操作提示，改成精簡、可展開的操作指南。

建議：

* 初次進入場景顯示簡短提示。
* 使用者首次完成操作後收起。
* 保留 Help 按鈕重新展開。
* Desktop 與 Mobile 顯示不同操作說明。

Bottom Right — Primary Action

重新設計「寫一張繪馬」CTA。

要求：

* 延續朱紅色識別。
* 符合場景的儀式感。
* 明確的 Hover / Press / Focus 狀態。
* 加入適當的 Icon 與轉場。
* 保持良好的文字對比。
* 不要加入誇張的閃光或粒子效果。

4. Motion Design System

建立一致的 UI Motion Tokens。

建議初始值：

* Micro Interaction: 150–220ms
* Card Transition: 250–400ms

* UI Entrance: 450–600ms
* Modal / Ema Transition: 500–800ms

使用適當的 Ease Out 或 Spring Motion。
避免所有元件同時出現或同時移動。
優先使用 CSS Transform 與 Opacity。
對需要跨元件協調的轉場，可以使用既有 GSAP。
尊重 `prefers-reduced-motion`。
5. Responsive Design
Desktop：

* 保留 3D 主場景的完整視覺空間。
* UI 盡可能靠近畫面邊緣。
* 避免遮擋神社與鳥居。
* 資訊密度維持適中。

Tablet：

* 適當縮減 UI 尺寸。
* 保留主要操作入口。

Mobile：

* 標題資訊更簡潔。
* 祈願數字可以合併到標題區。
* CTA 固定在可安全操作的位置。
* 使用 safe-area inset。
* 避免大量浮動卡片。
* 操作說明優先採用可展開模式。

6. Accessibility
請確保：

* 文字對比度符合 WCAG AA 的適用標準。
* 鍵盤可以操作所有主要按鈕。
* Focus State 明確。
* 觸控目標具足夠尺寸。

* 正確使用語意化 HTML。
* 動畫支援 Reduced Motion。
* 不使用顏色作為唯一狀態提示。

7. Integration With 3D Scene

所有 UI 必須與現有 R3F / Three.js Canvas 正確整合。

特別注意：

* Canvas 與 UI 的 Pointer Event 分配。
* 互動區域的 z-index。
* 3D Drag 與 UI Click 不得衝突。
* 開啟繪馬書寫介面時，需要暫停或限制場景操作。
* 關閉介面時應恢復原本的 Camera Control 狀態。
* 避免 UI 動畫造成 Canvas 不必要的重新渲染。

8. Acceptance Criteria

本階段完成時：

1. UI 和 3D 場景具有一致的視覺設計。
2. 3D 神社仍然是整體畫面的第一視覺焦點。
3. 四個資訊區域具有明確層級。
4. 文字閱讀性良好。
5. CTA 具有精緻且直覺的互動回饋。
6. Desktop / Tablet / Mobile 排版正常。
7. UI 不會擋住場景的重要物件。
8. 操作說明簡潔、容易重新開啟。
9. Keyboard / Touch / Reduced Motion 正常。
10. 原有 3D 場景、動畫與功能沒有回歸問題。

9. Implementation Workflow

請先盤點既有的 UI Components、CSS 與 Design Tokens。

優先沿用現有架構，不要為了單一頁面的視覺重整導入不必要的大型 UI Framework。

先提出一個新的版面配置與設計方向，再分階段實作。

請提供：

* UI 結構修改摘要
* Typography 與 Design Token 設定

```
* Desktop / Mobile 截圖
* Before / After 比較
* 互動與無障礙檢查結果
* 尚未完成的部分

完成後更新 `docs/IMPLEMENTATION_PLAN.md`，並補充必要的 UI 規範文件。
```

Claude 先寫 `docs/UI_GUIDE.md`，再改介面。沒有引入任何 UI 框架。

- 左上：大卡片縮成小紙籤，眉標、標題、一句說明依序淡入。手機只留標題
- 右上：祈願數一行，朱紅等寬數字。數字更新時舊的往上滑出、新的從下滑入
- 時段切換從上方正中央移到右上。正中央在鏡頭推近時會壓到本殿屋頂
- 對比、Tab 順序、觸控目標大小、卡片開著時背後的按鈕不能被 Tab 選到，都寫成腳本檢查

> **名詞**：**對比**是文字顏色和背景顏色的亮度差，用一個比值表示。4.5 是一般文字要達到的最低標準。
> 你需要知道到什麼程度：知道 4.5 以下的字，有些人會讀不清楚。數字由腳本算，你只看有沒有過 4.5。

做的過程踩到兩個問題。第一個是桌機上滑鼠版和觸控版的說明同時出現。原因是 CSS 選擇器的優先權輸給另一條規則，修法是提高選擇器優先權。第二個是報讀器把祈願數讀成「34」。跳動動畫中，舊數字「3」和新數字「4」同時存在。修法是舊數字對報讀器隱藏。

做到一半，可以這樣回報：

貼給 AI

```
《附上截圖》
我看畫面現在是空的椰
```

程式沒壞。瀏覽器面板開的是 `file://` 分頁。用 `file://` 打開的頁面不能載入程式的其他檔案，所以 3D 場景沒跑起來，只剩介面。改回本機網址就正常了。

📋 貼給 AI（7-6：卡片的吊繩斷掉）

貼給 AI

《附上截圖》

我想說點擊的時候，應該要從上面掛下來的...但現在看起來中間斷掉的感覺

症狀 → 原因：吊繩只有 46px，懸在半空，和卡片的穿繩孔之間還有空隙。修法：吊繩從畫面上方一路垂到孔、打一個結。打開與換張時，卡片繞繩子頂端輕輕擺盪（±3°、週期 1.6 秒）。

✅ 怎麼知道做對了

- [ ] 點繪馬牆，卡片翻面飛入，滑鼠移動時卡片會傾斜
- [ ] 按住「一起祈願」約 1 秒，光點飛進本殿，數字加 1
- [ ] 觸控板捏合、滾輪、`+` `-` `0` 鍵都能縮放
- [ ] 請 Claude 跑 `tests/ui-contrast.mjs`，每一個文字的對比都 ≥ 4.5
- [ ] 只用 Tab 鍵能走完所有按鈕；卡片開著時，背後的按鈕 Tab 不到

卡片上的字是不透明的紙面，不依賴背後的海。左下角是御朱印「願池神社」。下方那一列是祈願按鈕、這張繪馬的祈願數、關閉。紅色吊繩一路垂到卡片的穿繩孔，是 7-6 修完的樣子。這張是階段 11 之前的介面。

🆘 如果沒有成功

你可能會看到	通常是因為	怎麼辦
字看起來糊	文字外面加了發光暈	「文字不要依賴背景。請改成不透明的紙面板」
縮放程式寫了，畫面卻放大不了	看的是桌面應用程式瀏覽器面板裡的舊快照	在 Chrome 打開本機網址 `http://localhost:5201/…`
觸控板捏合幾乎沒反應	Mac 觸控板的捏合送的是滾輪事件加 Ctrl 鍵，數值很小	「觸控板捏合幾乎沒反應，請放大捏合時的倍率」。參考成品放大 8 倍
放大之後被拉回原本的大小	每幀的相機更新蓋掉了使用者的縮放	「使用者的縮放請存成獨立的數值，不要被每幀的更新蓋掉」。參考成品存成 `userZoom`
報讀器把「4」讀成「34」	數字跳動時，舊數字和新數字同時存在	「跳動動畫中的舊數字請對報讀器隱藏」
卡片打開時像是中間斷掉	吊繩太短	「吊繩請從畫面上方一路垂到穿繩孔」

這一步失敗可以跳過嗎：介面精修可以先跳過。繪馬卡片不行，階段 14 的登入是從卡片觸發的。

📖 為什麼這樣做

🎯 這一步的重點

對比、Tab 順序、觸控目標大小，都用腳本量，不靠眼睛。

🧭 這一步在做什麼

讓 3D 場景以外的部分也能讀、能用。

短回饋處理的是看得到的單一問題（字糊、縮放沒反應、繩子斷掉）。長規格處理的是整套規則（Design Token、四個區域、無障礙）。規則寫成 `UI_GUIDE.md` 之後，每一次改介面都有東西可以對照。

🧩 它在建構流程的位置

```
3D 場景 ──▶ 【四個角落的介面 + 繪馬卡片】 ──▶ 搬進 Astro、介面改用 React（階段 8）
```

💡 那段指令為什麼那樣寫

7–5 第 6 節列了「文字對比度符合 WCAG AA 的適用標準」。Claude 的做法是寫成腳本量，不是用眼睛看。用 Design Token 算出來的對比，是字的顏色對面板的顏色。但這個網站有一部分字直接

> 疊在 3D 場景上，背後是會動的海和天空。只有對著實際渲染出來的畫面量，數字才對得上使用者看到的東西。階段 11 會看到差多少。
>
> 7–5 第 9 節寫了「先提出一個新的版面配置與設計方向，再分階段實作」。理由和階段 2 一樣：規範先寫進檔案，實作才有東西可以對照。
>
> 想要的效果，給一個例子和一個排除，例如「小塚明體，不要標楷體」。不確定怎麼改時，問「有沒有更適合的做法」，不要直接指定參數。

階段 8　搬進 Astro，切開引擎與介面，介面改用 React

📖 讀完約 20 分鐘　🛠️ 動手約 90 分鐘

> - **這一步一定要懂**：3D 場景不重寫，從外面掛進去。引擎與介面只透過一組函式和事件溝通。每搬完一步都要比對畫面，而且基準要選對。
> - **這一步可以先不懂**：Astro 的設定檔、React 元件的寫法、`dispose()` 要清掉哪些顯示卡的資源。
> - **做完後你要能用自己的話回答**：Astro、React、Three.js、Supabase 各負責什麼？為什麼要先切開引擎與介面，再把介面換成 React？

⚖️ 動手前先想

網站要不要改用 React？什麼時候改？

先數介面有幾種狀態。到階段 7 為止，介面只有繪馬卡片和四個角落的按鈕。用程式一個一個切換按鈕和文字，還管得動。階段 14 之後要做登入、上傳照片、會員紀錄。每一個都有「沒登入、送出中、成功、失敗」幾種狀態。還用一個一個切換的寫法，每多一種狀態，就要記得改到每一個地方。這一步會問兩次。第一次（8-1）介面還簡單，答案是先不改。第二次（8-2）後端要做了，答案是介面改用 React。

3D 場景要不要一起改寫成 React Three Fiber（R3F）？

不要。理由寫在 `docs/MIGRATION_PLAN.md`：

不改成 R3F 的理由	數字或症狀
畫面不會多任何東西	R3F 底層一樣是 Three.js
會打掉已經驗證過的東西	水面、浮力、燈光經過 Node 測試與逐像素比對
每幀的邏輯要搬進 `useFrame`	主迴圈目前有固定的更新順序，搬進去之後順序分散在各元件

換框架之前，要不要先把引擎和介面切開？

要。切開之前，引擎直接把 3D 物件掛在願望資料上（`w.group`、`w.plaque`）。換介面框架時，畫面一變，就分不出是引擎還是介面造成的。所以分三步做，每一步畫面都不能變：先搬進 Astro，再

切開引擎與介面，最後把介面換成 React。遷移計畫和 Claude 的回覆會把這三步叫做 M0、M1、M2。看到這三個代號，指的就是這三步。

🛠️ 動手做

🧰 這一步的產出

```
docs/MIGRATION_PLAN.md    ← 遷移計畫：搬家、切邊界、介面換 React、接上真的資料（計畫裡叫 M0–M3）
package.json              ← 套件版本全部寫死（不加 ^）
src/pages/index.astro     ← 首頁：載入畫面、3D 畫面的位置、React 的入口
src/layouts/              ← 網頁共用的版型：標題、圖示、分享預覽圖
src/engine/engine.js      ← createShrine()：引擎唯一的入口
src/app/                  ← React 介面：繪馬卡片、祈願數、時段與天氣、視角、操作說明
src/app/runtime.js        ← 把介面的動作轉給引擎，把引擎的事件轉給介面
prototypes/b2-vanilla/    ← 搬家前的版本，凍結做對照
```

畫面上不會有任何變化。開的網址從 5201 換成 `http://localhost:4324/`。

Astro 預設的埠號是 4321。搬家時請 Claude 把開發伺服器設成 4324，或記下它實際用的埠號。階段 15 起 Supabase 的 Site URL、Redirect URLs 都要寫這個網址，兩邊不一樣就登入不了。這一步之後，檔案的位置也跟著換：階段 3–7 的 `src/sim/`、`src/scene/` 搬進 `src/engine/sim/`、`src/engine/scene/`，介面搬進 `src/app/`。後面的階段都用新的路徑。

📋 貼給 AI（8-1：第一次問要不要改 React）

這個問題可以早一點問。實際做的時候，第一次是在階段 3 做 Phase 01 的時候：

貼給 AI

> 好奇你覺得有需要擴充到 React 的架構嗎？

Claude 的回答是現在不需要。場景只有一個，狀態十幾個變數，拆成 `src/` 之後邊界已經清楚。改 React Three Fiber 要加建置步驟、重寫整個場景。

> **名詞：建置**（build）是把你寫的程式整理、壓縮成瀏覽器直接能用的檔案，輸出到 `dist/`。你需要知道到什麼程度：知道指令是 `npm run build`，部署出去的是建置的結果。這個案例從階段 8 開始需要它。

📋 貼給 AI（8-2：第二次問，因為要接資料庫）

做完介面精修（階段 7-5），可以再問一次，這次多給一個理由：

> 貼給 AI
>
> **你只需要修改**：沒有。整句照貼。先把模式切到 Plan。
>
> ```
> 剛剛有討論到要不要調整到 React 來實作，畢竟後續要接資料庫、會員登入等等
> ```

Claude 的建議變了：網站外殼（卡片、登入、上傳、會員頁）改 React，3D 場景從外殼掛進去。

📋 貼給 AI（8-3：寫遷移計畫）

> 貼給 AI
>
> ```
> 然後按照你的策略開始準備遷移計畫
> ```

Claude 先盤點引擎直接碰網頁的地方：`main.js` 的畫布插入與字型載入、`state.js` 的 `$`、`pointer.js` 的游標。介面反過來依賴引擎的地方集中在 `card.js` 與 `pointer.js`。這些就是要切開的邊界。計畫寫在 `docs/MIGRATION_PLAN.md`，分 M0 到 M3 四步。

最後 Claude 問了三個決定。

📋 貼給 AI（8-4：回答三個決定，開始搬進 Astro）

> 貼給 AI
>
> **你只需要修改**：三個回答換成你的決定。
>
> ```
> 1. Supabase 即可
> 2. 好啊
> 3. 教學對照可以
> ```

三個問題分別是：後端用什麼、新程式要不要用 TypeScript、舊原型要不要保留做教學對照。結果是：

- 後端用 Supabase。網站用 Astro＋React。Supabase 不限定前端用什麼工具，選 Astro 是因為其他案例也用它
- TypeScript 回答了「好啊」，但參考成品把介面換成 React 時，仍用 JavaScript 寫。3D 引擎維持 JavaScript
- 舊原型凍結，保留做對照

Claude 接著照計畫把專案搬進 Astro，沒有另外的指令。你也可以加這一段：

貼給 AI

```
照 MIGRATION_PLAN 做 M0：把專案搬進 Astro，畫面不能有任何改變。


1. 舊原型改指向一份凍結的引擎快照，之後改引擎不會影響它
2. three 改成 npm 套件，版本寫死，和原本 importmap 的版本一樣，不要升級
3. 開發伺服器用 4324，避開案例三的 4321；關掉 Astro 的開發工具列
4. 減少動態模式、關掉模擬祈願，舊原型和 Astro 版逐像素比對，要 0%
5. draw call 兩邊一樣；繪馬卡片流程、?debug、?stats 正常；astro build 成功
```

搬進 Astro 的結果：舊原型（5201）和 Astro 版（4324）逐像素比對 0%，draw call 都是 582。踩到的問題：檢查 `?debug` 的腳本報錯。腳本裡寫死了搬家前的路徑，程式本身沒問題。測試工具也要跟著搬家。

📋 貼給 AI（8–5：畫面沒更新）

搬進 Astro 之後，如果畫面看起來沒更新，先確認開的是哪個網址。貼：

貼給 AI

```
畫面好像每有更新
```

瀏覽器面板還開在 5201，那是凍結的舊原型，搬進 Astro 之後就不會再變。改開 `http://localhost:4324/` 就看到了。

📋 貼給 AI（8-6：切開引擎與介面）

貼給 AI

好的接下來 M1

要做的事寫在 `MIGRATION_PLAN.md` 裡。你也可以加這一段從計畫整理出來的要求：

貼給 AI

照 MIGRATION_PLAN 做 M1：切開引擎與介面，畫面不能有任何改變。

1. 引擎整段包成 createShrine(container, { wishes, timeOfDay, onEvent })，回傳 API
2. 引擎原本直接改的介面（祈願數、天氣文字、時段按鈕、提示框）改成發事件
3. 介面原本直接碰的引擎內部，改成呼叫 API
4. 願望的文字、票數、「已祈願」屬於介面；引擎只認 id
5. 一頁一個實例：第二次呼叫 createShrine 拿到同一個；dispose() 要清乾淨
6. 和 M1 之前「同條件」的截圖逐像素比對，要 0%

切開引擎與介面時踩到兩個問題：

症狀	原因	修法
第一次比對差 3.1%	基準選了凍結的原型。那一份是加影子與新樹之前的版本，本來就不一樣	改用切開之前最後一張同條件截圖，差異 0%（最大通道差 1，四捨五入）
介面檢查偶爾回報「卡片關閉後還在開啟狀態」	測試腳本固定等 1.5 秒。SwiftShader 負載高時，飛回動畫還沒跑完	測試改成等條件成立，不等固定秒數

📋 貼給 AI（8-7：介面換成 React）

參考成品是在所有功能做完之後，才把介面換成 React。手冊把這一步移到這裡：從階段 9 開始，你都在 React 版上做。下面這段是手冊整理的指令，參考成品當時用的是另一份較長的規格：

貼給 AI

```
照 MIGRATION_PLAN 做 M2：把介面換成 React，3D 引擎不改寫。

1. 首頁只放一個 React 入口；載入畫面和 3D 畫面的位置留在 Astro 頁面裡
2. 繪馬卡片、祈願數、時段與天氣按鈕、視角按鈕、操作說明改成 React 元件
3. 使用者寫的文字一律由 React 顯示，不用 innerHTML
4. 引擎只透過 createShrine() 的函式和事件跟 React 溝通；海浪、鏡頭這類每一幀都在變的數值，不放進 React 的狀態
5. React 把元件重新掛上去時，不能重建 3D 場景；一頁只有一個引擎
6. 和 M2 之前同條件的截圖逐像素比對；讀卡片、按住祈願、寫繪馬、Esc、手機版各走一遍
```

第 5 點的原因：React 在開發時會把元件掛上去兩次，用來抓清理不乾淨的程式。場景沒有防這件事，海面、船和聲音都會變成兩份。

參考成品當時只用瀏覽器實際操作檢查，你照第 6 點比對。當時踩到兩個問題：

症狀	原因	修法
自動測試點了按鈕，沒有反應	3D 畫面一出現就去點。這時場景還在建，按鈕還是停用的	等載入畫面消失，再開始測
測試說用鍵盤打開視窗後，焦點沒有移到按鈕上	React 畫完按鈕的下一幀才移焦點，測試在那之前就檢查了	等焦點移過去再檢查，最多等 3 秒

✅ 怎麼知道做對了

- [ ] `http://localhost:4324/` 打開，和搬家前看起來一樣
- [ ] Claude 回報三步各自的比對結果，每一步都說得出基準是哪一張截圖
- [ ] `package.json` 裡每一個版本號都沒有 `^`
- [ ] 舊原型還能用 5201 打開
- [ ] 請 Claude 列出引擎裡還碰到 `document` 或 `window` 的地方，每一個都說得出理由（參考成品只剩：自己的容器、字型載入、`?debug`）
- [ ] 請 Claude 確認 `src/engine/` 裡沒有任何一個檔案 import React

- [ ] 打開繪馬卡片、按 Esc、切換時段，結果和換 React 之前一樣

🆘 如果沒有成功

你可能會看到	通常是因為	怎麼辦
切開引擎與介面後，比對差幾 %	基準選錯了	「比對的基準要是切開之前、同條件的最後一張截圖，不是凍結的原型」
檢查 `?debug` 的腳本報錯	腳本裡寫死了搬家前的路徑	「測試腳本裡的路徑也要跟著搬家」
測試偶爾失敗、重跑又過	腳本用固定秒數等待	「測試請等條件成立，不要等固定秒數」
`astro build` 失敗	引擎用了 top-level await	「建置目標請設成 esnext」
改了程式，畫面沒變	還開著 5201 的舊原型	改開 `http://localhost:4324/`
開發時海浪聲變兩倍大、船變兩艘	React 把元件掛了兩次，場景也建了兩次	「一頁只能有一個引擎：React 重新掛上元件時，沿用同一個場景」

這一步失敗可以跳過嗎：不行。階段 14、15 的登入要用 Astro 的環境變數（寫在 `.env` 裡的設定，階段 15 會用到），登入視窗也寫成 React 元件。

📖 為什麼這樣做

🎯 這一步的重點

換框架之前，先把兩邊切開。每搬一步，畫面都不能變，比對時只有一個變因。換的是介面，3D 場景不重寫。

🧭 這一步在做什麼

我們用 Astro 組成網站頁面，讓網址、文件與分享資訊有一致的管理方式。React 負責互動介面，依照登入、選取、載入與錯誤等狀態呈現畫面。Three.js 專心處理 3D 場景、相機與場景動畫，Supabase 負責認證、資料與權限。

這樣分工的好處，是新增介面功能時比較容易控制影響範圍，也能分別測試各層。代價是工具與整合規則較多，因此每一層都要有清楚的責任，避免重複管理同一份狀態。

四個工具在這個網站裡實際做的事：

工具	在這個網站裡做什麼	選它的理由	代價
Astro	產生首頁和 `/api` 兩個網頁（`/api` 在階段 16 加上）。網頁標題、圖示、分享到 LINE 的預覽圖寫在同一份版型裡	兩個網頁共用一份版型，標題和預覽圖只寫在一個地方。其他案例也用它	首頁幾乎都是互動內容，只用到 Astro 的一小部分。不用 Astro 也做得出這個網站
React	繪馬卡片、控制列，之後的登入、御神籤	這些畫面都有好幾種狀態：沒登入、送出中、成功、失敗。每種狀態對應一個畫面，不用自己切換按鈕和文字	多一套要懂的東西。首頁要等 JavaScript 下載完，介面才出現。React 本身未壓縮約 210 KB
Three.js	海、船、天氣、鏡頭，每秒更新約 60 次	這部分在換 React 之前就能用，重寫就要重新驗證每個效果。每秒 60 次的更新也不適合經過 React	3D 和介面要自己接線：介面叫引擎做事，引擎通知介面
Supabase	會員登入、存繪馬和照片、對外 API、讀寫規則（階段 15 起）	不用自己架伺服器。誰能讀寫什麼，寫成資料庫裡的規則，網頁改不到	規則要自己寫對。寄信要另外接寄信服務。正式資料要在正式環境測

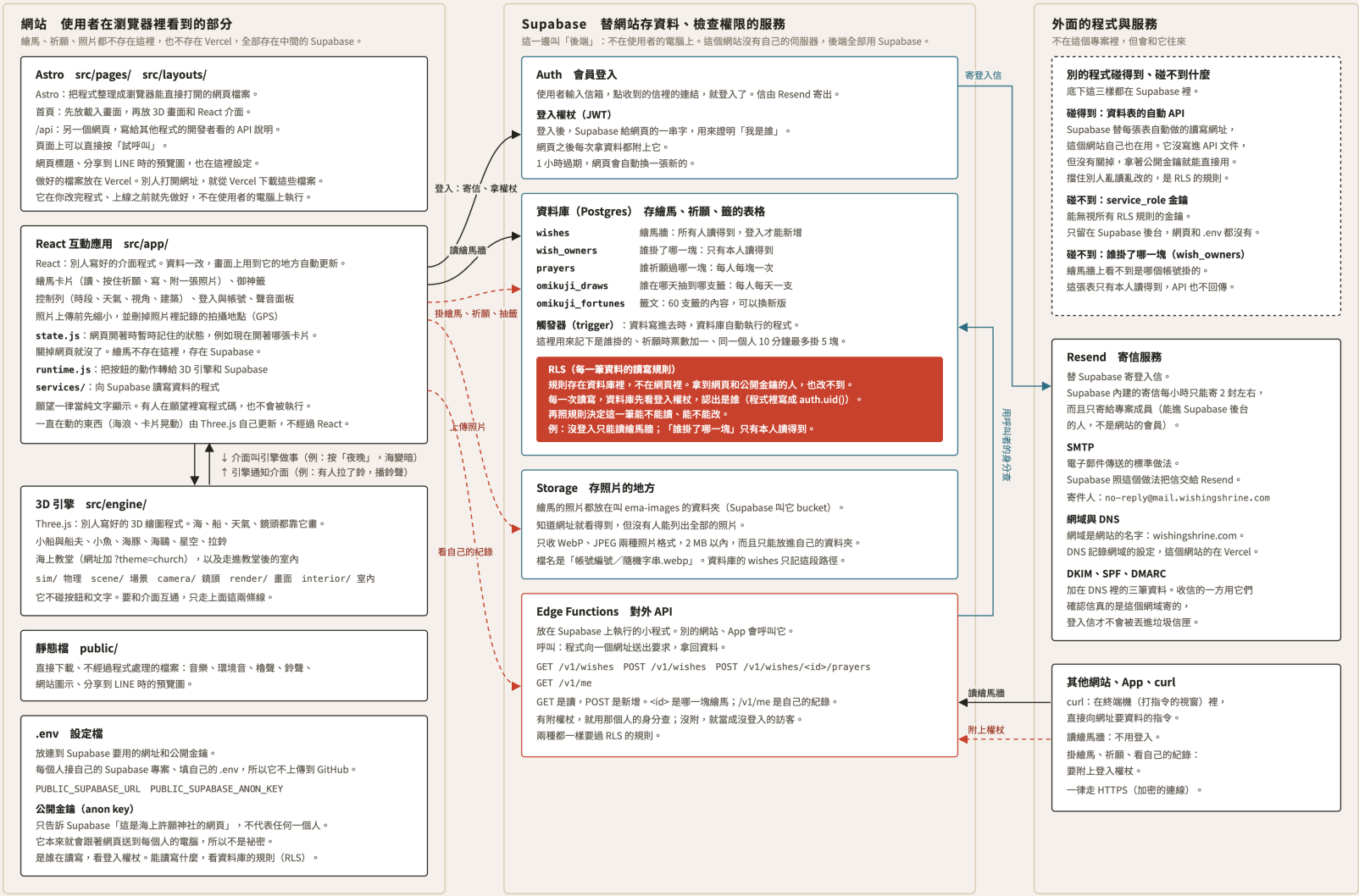
「避免重複管理同一份狀態」在這個網站裡是這個意思：「現在開著哪張卡片」由 React 管，「船現在在哪裡」由 Three.js 管，「誰能讀哪一筆繪馬」由 Supabase 管。兩個地方同時管同一份資料，畫面就會對不上。

「分別測試各層」在這個網站裡是：3D 的數學用 Node 測（階段 4），介面用瀏覽器測，權限用兩個帳號互測（階段 19）。

做到最後，整個網站長這樣。這張就是手冊最前面〈系統架構〉那張圖：

海上許願神社　系統架構

黑色實線：不用登入也能做。
朱紅虛線：要先登入。網頁每次向 Supabase 拿資料，都會附上登入權杖。
藍色實線：服務和服務之間的往來，表示誰把東西交給誰。



開發與上線

你和 Claude Code 一起改程式。Claude Code 照 CLAUDE.md、docs/ 和開發日誌做事。改完先在 localhost:4324 看過，這是只有你自己的電腦看得到的預覽網址。

git 替程式存版本；GitHub 是放程式碼的網站。推到 GitHub 之後，Vercel 會自動抓新版本，更新 wishingshrine.com。網域也在 Vercel 買，DNS 也在 Vercel 設定。

資料表與規則：supabase/migrations/ 裡的 .sql 檔，貼進 Supabase 後台（管理這個專案的網頁）的 SQL Editor 執行。

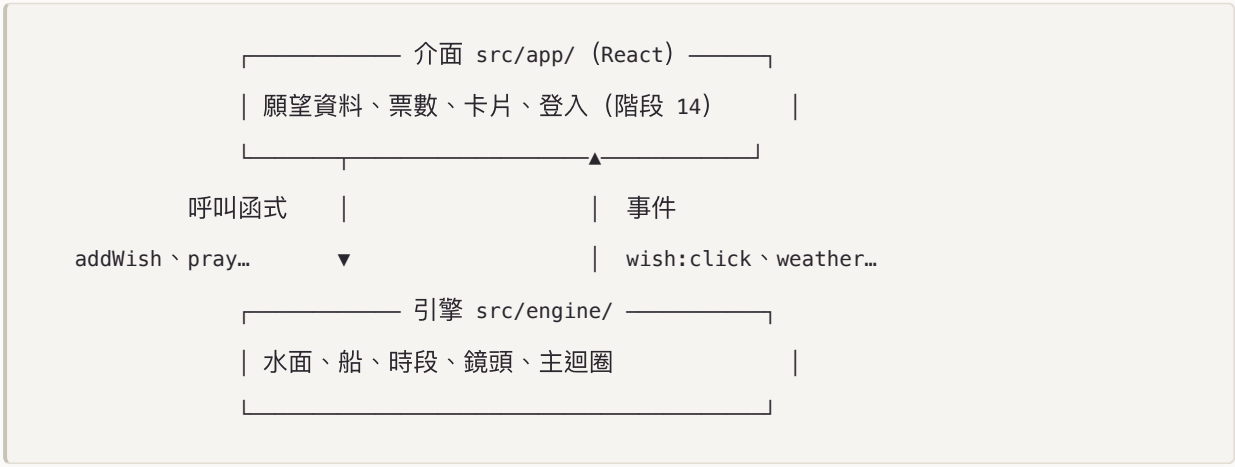
對外 API：supabase/functions/ 裡的程式，貼進 Supabase 後台的 Edge Functions。登入信：把 Resend 的帳號資料填進 Supabase 的 SMTP 設定。

看圖的順序：

- 左邊是網站。最上面是 Astro，下面是 React 介面和 3D 引擎。兩者中間的兩條線，就是 8-6 切出來的邊界：介面叫引擎做事，引擎通知介面。
- 中間是 Supabase，階段 15 起才接上。React 介面透過它登入、讀寫繪馬、上傳照片。
- 右邊是外面的程式與服務：替網站寄登入信的服務，以及別人寫的程式。
- 線的顏色：黑色不用登入，朱紅要先登入，藍色是服務和服務之間的往來。

遷移計畫還有最後一步（M3）：示範願望與模擬祈願換成真的資料。階段 16 把掛繪馬與祈願寫進資料庫，階段 17 讓繪馬牆讀資料庫。

🧩 它在建構流程的位置



切開之後的規則寫進 `CLAUDE.md`。引擎不碰介面的網頁元素。介面不 import 引擎內部的檔案，只用 `createShrine()` 的函式與事件。

搬進 Astro 時另外做的兩件事：

- 版本寫死。three 從 importmap 改成 npm 套件，鎖在 0.185.1。npm 上當時已經有 0.186.1，不升級，避免沒改程式畫面卻變了
- 關掉 Astro 的開發工具列。它會蓋在畫面上，截圖比對時多出差異

💡 那段指令為什麼那樣寫

8-6 的指令很短：「好的接下來 M1」只有六個字。它能用，是因為 8-3 先讓 Claude 寫了 `MIGRATION_PLAN.md`，每一步的內容與驗收都在計畫裡。

「同條件」三個字是被踩過才加進 `CLAUDE.md` 的。

8-7 把要求寫成六點，因為換 React 是三步裡改動最多的。第 4、5 點是這個網站特有的：3D 每秒更新約 60 次，React 會重新掛上元件。這兩件事沒寫，Claude 可能照 React 一般網站的寫法做。

> 同一個問題，條件變了就再問一次，並附上新的理由（例如「後續要接資料庫」）。

🧭 Checkpoint 2（做完階段 8）

口頭回答就好，三題都講得出來就往下走。

1. 船的反應為什麼要從水面高度來，不能用點擊觸發？
2. 本殿柱子、碼頭木樁、繪馬架，三個「沒接上」的共同原因是什麼？
3. 換 React 之前，為什麼要先切開引擎與介面？

> 第 2 題答不出來，回階段 6 的〈🧭 這一步在做什麼〉再看一次那張表。

階段 9　做風、花瓣、樹與燈火

📖 讀完約 12 分鐘　🛠️ 動手約 45 分鐘

- **這一步一定要懂**：全場只有一個風場，每個受風的物件在自己的位置取樣。測試要放在和畫面一樣的位置。
- **這一步可以先不懂**：花瓣的速度積分、樹冠彎曲的 vertex shader。
- **做完後你要能用自己的話回答**：Node 測試都過了。為什麼實際畫面上 30 秒內沒有任何一片花瓣落水？

⚖️ 動手前先想

測試場地能不能和真實場景不一樣？

不能。這一步會遇到兩次測試和畫面對不上。花瓣的 Node 測試用的是一塊空曠的場地，花瓣怎麼飄都會落進海裡。真實場景裡，櫻花樹在小島西緣，風往東吹，樹上掉的花瓣全部落在石台上。測試全過，畫面上 30 秒內沒有一片落水。

陣風會遇到相反的情況：程式沒錯，是測試的判斷方法錯了。測試用「總風力超過門檻」數陣風，風小的時段裡的陣風就會被漏掉。兩件事都要先問同一個問題：測試的條件，和畫面上會發生的事一樣嗎？

🛠️ 動手做

🧰 這一步的產出

```
src/engine/sim/wind.js        ← 風向、慢起伏、陣風、位置差異。全場唯一的風
src/engine/sim/petals.js      ← 花瓣的物件池與運動
src/engine/scene/petals.js    ← 花瓣的畫面（一個 draw call）
src/engine/scene/trees.js     ← 松、櫻花、苔
```

畫面上應該看到：偶爾有幾片櫻花被風吹落。有的落在石台，有的落進海裡起一圈很小的漣漪。樹冠輕輕擺，樹幹不動。

📋 貼給 AI（9-1：樹與櫻花）

附一張樹的截圖：

貼給 AI

〈附上截圖〉

稍後幫我優化樹木和櫻花

Claude 在搬進 Astro 之後做：

- 松：從三個堆疊的圓錐，改成庭園整姿的黑松。彎曲的樹幹、四根往外伸的橫枝、枝端扁平的雲朵枝葉
- 櫻花：從五顆粉紅多面體，改成分枝的樹幹、五簇花團、樹下一片落花
- 每棵樹的零件合併成「枝幹」「葉 / 花」兩個幾何體，零件變多，draw call 反而少（586 → 578）
- 樹用自己的亂數，不動到共用序列，岩石與船的位置不變（階段 3 的規則）

📋 貼給 AI（9-2：Phase 3 環境動態，附草皮截圖）

這是實際用的 Phase 3 指令原文。規格由 ChatGPT 撰寫，做法見〈用 ChatGPT 寫規格，再交給 Claude〉。規格前面的截圖、草皮那一句、「接著處理 Phase 3」那一句，是另外加上去的。

貼給 AI

你只需要修改：截圖與草皮那一句換成你看到的問題，沒有就刪。「有些項目過去已經處理完畢的就不要再重複處理」一定要留，理由在 💡 。

〈附上截圖〉

這塊草皮也需要優化設計

接著處理 Phase 3，有些項目過去已經處理完畢的就不要再重複處理

Living Environment & Ambient Animation

請擔任 Senior Creative Developer、Three.js Technical Artist、Shader Engineer 與 Motion Designer，接續現有的 Shrine Diorama 專案。

本階段目標是讓整個 Low-poly Isometric 神社成為一個有生命力、但安靜而克制的微縮世界。

請先閱讀：

* `CLAUDE.md`
* `.claude/rules/`
* `docs/ART_DIRECTION.md`
* `docs/SIMULATION_SPEC.md`
* `docs/IMPLEMENTATION_PLAN.md`

並檢查 Phase 01、Phase 02 的實際程式與完成狀況。
不要重新建立水面或 Ripple System，所有新的環境動態必須盡可能整合現有系統。
01 — Shared Environmental Simulation
建立可重用的環境模擬狀態。
包含：

* Global Simulation Time
* Wind Direction
* Wind Strength
* Wind Gust
* Wind Noise
* Particle Spawn Rate
* Environmental Motion Intensity

使用統一的時間來源，並支援低動態模式、暫停與品質分級。
風場可使用低頻 Noise：
`wind(position, time) = baseWind + gustStrength × noise(position × spatialScale, time × temporalScale)`
要求：

* 風向具有一致性。
* 風力隨時間緩慢變化。
* 不同位置的風力可以有細微差異。
* 避免完全隨機、快速跳動的風向。
* 提供可由樹葉與粒子共享的 Wind Sampling API。

02 — Cherry Blossom Particle System
替現有櫻花樹加入花瓣飄落動畫。
優先使用：

* THREE.InstancedMesh

* GPU-friendly particle buffers
* Vertex Shader animation
* CPU event pool for limited interactions

粒子運動模型：

`v_next = v + (gravity + wind(position,time) - drag × v) × dt`

`p_next = p + v_next × dt`

花瓣必須具有：

* 不同初始位置
* 不同大小與透明度
* 不同旋轉速度
* 方向一致但略有差異的風場
* 個別生命週期
* 緩慢、自然的下降速度
* 輕微橫向漂移

不要將花瓣做成快速下落的粉紅色粒子雨。

視覺上希望是偶爾有幾片櫻花被風吹落，而不是整棵樹持續大量掉落。

03 — Petal and Water Interaction

讓花瓣落水時能夠與 Phase 02 的 Ripple System 整合。

流程：

1. 花瓣依照風場移動。
2. 偵測花瓣與水面交會。
3. 使用既有 Water Sampling API 取得水面高度。
4. 在落水座標建立低振幅 Ripple Event。
5. 花瓣可短暫漂浮、旋轉或淡出。
6. 超過生命週期後將粒子回收到 Pool。

注意：

* 不需要所有花瓣都生成漣漪。
* 使用事件密度限制。
* 花瓣漣漪強度應顯著小於使用者點擊產生的漣漪。
* 不要讓粒子碰撞引發大量 Shader Uniform 更新。
* 使用合理的碰撞檢查頻率與事件管理方式。

04 — Cherry Blossom Tree Motion

讓櫻花樹冠產生非常細微的風動效果。

請先檢查模型的幾何結構。

優先選擇：

* 局部群組的輕微旋轉
* Vertex Shader Wind Deformation
* 基於頂點高度的 Wind Weight

樹幹應相對穩定，樹冠外側的移動幅度稍大。

如果目前樹冠是大型 Low-poly Mesh，不要為了實作風動而破壞原有幾何造型。

05 — Lantern Flame and Glow

檢查現有石燈籠、船燈與其他發光物件。

建立統一的 Lantern Light Behavior。

要求：

* 微小的亮度起伏
* 低頻 Noise 控制
* 避免完全隨機的閃爍
* 發光材質與實際光源強度協調
* Bloom 必須克制
* 光源之間保留細微節奏差異

可使用：

`intensity(t) = baseIntensity × (1 + variation × smoothNoise(t))`

注意不要對每一個燈籠都加入高成本的即時陰影光源。

06 — Ambient Micro Events

增加少量自然的環境事件，例如：

* 偶爾出現的微風
* 局部櫻花飄動
* 水面反光緩慢變化
* 燈籠亮度細微變化

如果場景原本沒有鳥類、螢火蟲或其他生物，不要自行大量增加新物件。

新增物件必須符合既有 Art Direction。

07 — Animation Timing

將動畫分成三種：

Continuous

持續但非常低調：

* 水面
* 船隻
* 樹冠
* 燈火

Ambient Events

使用有界隨機間隔：

* 櫻花落下
* 微風吹動
* 花瓣落水

Interactive Events

由使用者行為觸發：

* 水面漣漪
* 船隻附近的物理反應

請確保不同動畫具有各自的自然節奏，但共享環境時鐘與相關狀態。

08 — Performance

* 所有花瓣應使用物件池或 GPU-friendly Instancing。
* 不要為每片花瓣建立獨立 React Component。
* 不要在每幀建立大量 Vector3、Quaternion 或 Material。
* 設定 Maximum Active Particles。
* 設定 Maximum Ripple Events。
* 根據畫質分級調整粒子密度。
* 支援 Reduced Motion。
* 頁面不可見時降低或暫停不必要的模擬運算。
* 避免花瓣事件造成大量 CPU-GPU 同步。

09 — Debug Controls

在開發模式提供：

* Wind Strength
* Wind Direction
* Gust Frequency
* Petal Spawn Rate
* Petal Gravity
* Petal Drag
* Petal Rotation Speed
* Tree Sway Intensity
* Lantern Flicker Amount

Debug Controls 不應出現在正式使用者介面。

10 — Acceptance Criteria

本階段完成時：

1. 櫻花能夠隨風自然飄落。
2. 花瓣不會呈現完全同步的運動。
3. 風場具有合理的方向一致性。
4. 花瓣落水可以觸發既有 Ripple System。
5. 樹冠能夠輕微擺動而不破壞模型。
6. 燈籠具有柔和且不突兀的亮度變化。
7. 動畫不會造成過度視覺干擾。
8. 水面與船隻物理仍然正常。
9. Desktop / Mobile 效能穩定。
10. 沒有粒子持續累積、記憶體異常成長或明顯掉幀。

請實際觀察至少 30 秒固定鏡頭的場景動畫，確認環境動態自然且不會過度重複。

完成後：

* 更新 `docs/SIMULATION_SPEC.md`
* 更新 `docs/IMPLEMENTATION_PLAN.md`
* 回報新增與修改的檔案
* 提供測試、效能與視覺驗收結果

本階段不要提前實作日夜模式、Cinematic Camera 或 UI 重設計。

Claude 先盤點：水面、漣漪、花瓣漣漪池、全域風力、新的樹模型都已經有了。這一輪只補缺的部分：

- 風（`sim/wind.js`）：風向、慢起伏、陣風、不同位置差 ±20%。陣風平均 14 秒一陣，間隔只由時間決定。每個受風物件在自己的位置取樣
- 花瓣：物件池、速度積分（重力加風減阻力）、落地停住、落水漂浮。35% 的落水花瓣起很小的漣漪，整片海至少隔 0.6 秒才起一次。一個 draw call，建構時預熱 25 秒
- 樹：從整棵旋轉，改成 vertex shader 依高度彎曲。樹幹底部不動，樹冠擺。不改模型
- 燈火：從兩個正弦相乘（會規律重複），改成低頻的平滑雜訊 `flicker()`，每盞節奏不同
- 分頁看不到時停止主迴圈

草皮：原本是一顆壓扁的綠色多面體。改成苔庭的一塊：6 到 7 團低矮起伏的苔墊、邊緣幾叢細草、一兩顆小石頭。這些合併成一個 draw call。第一版太鮮豔、太集中，看起來像一團生菜。綠色改成偏灰偏黃，苔墊攤平往外鋪。

這一輪踩到的問題：

症狀	原因	修法
30 秒觀察中，從來沒有花瓣落水	櫻花在小島西緣，風往東吹，樹上掉的全部落在石台。Node 測試用的是空曠場地，所以沒抓到	加 30% 從上風處吹來的花瓣，落在小島西側的海上
陣風測試失敗（10 分鐘只數到 19 陣）	測試用「總風力超過門檻」判斷陣風，落在風小時段的陣風被漏掉	直接數陣風的訊號。這次是測試錯，不是程式錯
苔的程式出現警告	二十面體本來就沒有索引，又轉了一次	只在有索引時才轉
高畫質 fps 從約 7.3 掉到 5.4	一開始和長時間測試同時跑，兩個測試搶同一顆 CPU。單獨重量之後還是有落差，原因當時沒確認	階段 10 確認這很可能是測量工具的雜訊

驗證：Node 測試 12 項。固定鏡頭 30 秒，畫面上有 11 到 17 片花瓣、3 圈花瓣漣漪。5 分鐘長跑，記憶體沒有增加。

📋 貼給 AI（9–3：文件盤點）

做到這裡，請 Claude 檢查一次文件：

貼給 AI

你只需要修改：沒有。整句照貼。

```
協助我評估目前更新到現在，是否有需要補充更新到 CLAUDE.md 的規範或者其他文件中的規格
```

Claude 找到的落差：

文件	落差
`SIMULATION_SPEC.md`	還在描述 Phase 01 的方形花瓣和兩個正弦的風
`CLAUDE.md`	該保留的功能清單少了漣漪、船、時段、花瓣。沒寫「新東西用自己的亂數」。驗證規則沒有這幾輪學到的教訓。教訓是：基準要同條件、先量雜訊底線、等條件不等秒數、效能要單獨量。工作流程的編號被之前的腳本改壞
`.claude/rules/animation.md`	還寫著「全場只有一個風力值」
`.claude/rules/performance.md`	畫質表的花瓣欄還是空的
`IMPLEMENTATION_PLAN.md`	回歸檢查只有 8 項，改成 11 項

✅ 怎麼知道做對了

- [ ] 固定鏡頭看 30 秒：看得到花瓣從櫻花樹與上風處飄下來，有幾片落進海裡起小漣漪
- [ ] 樹冠在動，樹幹底部不動
- [ ] 夜晚時，每一盞燈的亮度起伏節奏不同
- [ ] 開「減少動態效果」重新整理：花瓣停在半空與地上，畫面靜止
- [ ] 9–3 之後，Claude 列出文件和程式不一致的地方，並且改好了

🆘 如果沒有成功

你可能會看到	通常是因為	怎麼辦
30 秒內沒有任何一片花瓣落水	樹的位置和風向讓花瓣全部落在陸地上，測試場地和真實場景不一樣	「請在真實場景裡觀察 30 秒，數落水的花瓣。如果是 0，加一些從上風處吹來的花瓣」
陣風的測試失敗	測試用總風力判斷陣風	「請直接數陣風的訊號，不要用總風力判斷」
苔看起來像一團生菜	綠色太鮮豔、太集中	「苔的綠色請偏灰偏黃，往外攤平鋪開」
效能數字比上一步差很多	和其他測試同時跑	「效能請單獨量，不要和其他測試同時跑」

你可能會看到	通常是因為	怎麼辦
Claude 重做了水面或漣漪	「不要再重複處理」那一句被刪了	「過去已經處理完畢的就不要再重複處理。請先盤點哪些已經有了」

這一步失敗可以跳過嗎：可以。花瓣與樹是氛圍，後面的階段不依賴它們。風不行，階段 12 的燈籠受風會用到。

📖 為什麼這樣做

🎯 這一步的重點

測試的樹和風，要放在和畫面上一樣的位置。

🧭 這一步在做什麼

讓場景在你不動的時候也有一點變化。

> **名詞**：InstancedMesh 是讓顯示卡一次畫很多個相同形狀的寫法。
> 你需要知道到什麼程度：知道幾十片花瓣用它只算一次 draw call，一片一片畫就是幾十次。

陣風的間隔只由時間決定，同一個時間點每次都一樣。測試才重現得出來。

🧩 它在建構流程的位置

```
wind.js ──┬──► 樹冠擺動
          ├──► 花瓣 ──► 落水 ──► 花瓣漣漪池 ──► 水面（階段 4）
          ├──► 布旗、紙垂
          └──► 吊燈擺動（階段 12）
```

💡 那段指令為什麼那樣寫

9–2 的「有些項目過去已經處理完畢的就不要再重複處理」是加在規格前面的。

這份 Phase 3 規格是事先寫好的，每一項都假設從零開始。但做到這裡，水面、漣漪、花瓣漣漪池、樹都已經有了。沒有這一句，Claude 可能把已經做過的再做一次，用不一樣的做法蓋掉前面驗證過的版本。

規格最後有一句「請實際觀察至少 30 秒固定鏡頭的場景動畫」。抓到「花瓣不落水」的就是這一條。Node 測試抓不到，30 秒的實際觀察抓到了。

> 做完一段之後問一次「有沒有需要補進文件的」，文件不會自己更新。

階段 10　做時段轉場與鏡頭

📖 讀完約 12 分鐘　🛠️ 動手約 45 分鐘

- **這一步一定要懂**：鏡頭與時段轉場都用臨界阻尼彈簧，中途切換會保留速度。使用者一拖曳，鏡位就取消，閒置運鏡也停下來。
- **這一步可以先不懂**：球座標、最短方向的角度內插。
- **做完後你要能用自己的話回答**：低畫質量兩次，一次 9.4 fps、一次 12.8 fps。這代表什麼？

⚖️ 動手前先想

效能工具量到的差異能不能相信？

這一步量效能時會看到：低畫質一次 9.4 fps、一次 12.8 fps，每幀的中位數都是 83 ms。加不加暈影的對照也分不出來。 SwiftShader 的雜訊大約 ±25%，和要量的差異一樣大。結論是這個工具量不出來，要用真機量。把這種雜訊當成退步，你會去修一個不存在的問題。

真機怎麼量：網址加 `?stats`，畫面上會顯示 fps、draw call、三角形數。這要你自己在自己的電腦和手機上看。

🛠️ 動手做

🧰 這一步的產出

```
src/engine/camera/rig.js      ← 鏡位、彈簧運鏡、閒置運鏡
src/engine/sim/daycycle.js    ← 時段轉場改成臨界阻尼彈簧
```

畫面上應該看到：右上角多一列視角按鈕（全景、本殿、鳥居、水面）。點本殿、鳥居、船，鏡頭會推過去。全景放著 8 秒不動，鏡頭會非常慢地飄。

📋 貼給 AI（10–1：Phase 4 光影與鏡頭）

這是實際用的 Phase 4 指令原文，共 15 節。規格由 ChatGPT 撰寫，做法見〈用 ChatGPT 寫規格，再交給 Claude〉。第一句是另外加在規格前面的。

貼給 AI

你只需要修改：沒有。整段照貼。第一句與最後一句（不要在本階段重做 UI）一定要留。

```
最後一個步驟，同樣應該有不少你再過去的過程已經調整完畢，就不需要再執行囉

Cinematic Experience & Dynamic Lighting
請擔任 Senior Creative Developer、Three.js Rendering Engineer、Technical Artist、Cinematic Motion Designer 與 Interaction Designer。
接續現有 Shrine Diorama 專案，實作具有高品質視覺敘事的 Cinematic Experience。
請先閱讀：

* `CLAUDE.md`
* `.claude/rules/`
* `docs/ART_DIRECTION.md`
* `docs/SIMULATION_SPEC.md`
* `docs/IMPLEMENTATION_PLAN.md`

並檢查 Phase 01–03 的實際成果。
本階段不需要重新設計模型。
目標是透過光影、環境狀態、攝影機、轉場和互動，讓使用者感受到同一座神社在不同時間與觀賞角度下的氛圍。
01 — Cinematic Art Direction
維持：

* Low-poly / Isometric
* Japanese Minimalism
* Soft Atmospheric Lighting
* Premium Miniature Diorama
* Restrained Motion
* Cohesive Color Grading

避免：
```

* 過度寫實的 HDR 效果
* 強烈鏡頭光暈
* 誇張 Bloom
* 大量粒子特效
* 過快的攝影機移動
* 過度戲劇化的日夜轉換

畫面應像一件精緻的數位藝術作品，而不是遊戲引擎技術展示。

02 — Environment State System

建立統一的環境狀態控制系統。

支援：

* Day
* Sunset
* Night

使用單一 `dayCycleProgress` 或等價的時間控制參數。

需要同步控制：

* Directional Light
* Ambient Light
* Environment Lighting
* Background Color
* Water Color
* Water Reflection
* Lantern Intensity
* Shrine Window Emission
* Atmospheric Effects
* Postprocessing Parameters

所有數值須透過一致的時間軸進行平滑轉換。

建議以 GSAP Timeline 或既有動畫系統控制高層次轉場。

不要在每幀透過 React State 更新所有燈光數值。

03 — Day Lighting

白天模式：

* 自然、柔和的方向光

* 青綠色水面
* 適度明亮的天空
* 清晰但不銳利的陰影
* 低強度燈籠發光
* 櫻花保持柔和粉色
* 保留屋頂與鳥居的材質細節

不要讓水面過曝或變成純白。

04 — Sunset Lighting

黃昏模式：

* 暖色方向光
* 橘紅色的邊緣受光
* 水面局部暖色反射
* 柔和且較長的陰影
* 逐漸變深的背景
* 燈籠逐漸亮起
* 神社窗戶開始散發暖光

Sunset 應作為 Day 和 Night 之間的完整視覺狀態，而不只是過渡色。

05 — Night Lighting

夜晚模式：

* 深藍色環境光
* 柔和的水面反射
* 溫暖的石燈籠與船燈
* 神社窗戶透出微弱暖光
* 局部光照強調鳥居與建築輪廓
* 克制的 Bloom
* 保留物件輪廓與空間層次

避免：

* 整個場景變成純黑。
* 大面積沒有細節的藍色陰影。
* 燈籠周圍出現過大的白色光圈。
* 所有物件都套用相同的藍色濾鏡。

夜間效果應透過實際光源、環境光與材質狀態的搭配達成。

06 — Day / Night Transition

建立平滑的模式轉場。

建議轉場時間：

* Day → Sunset：3–5 秒
* Sunset → Night：3–5 秒
* Night → Day：4–6 秒

上述為初始參數，應依實際觀感調整。

要求：

1. 光源色溫逐漸改變。
2. 水面顏色同步轉換。
3. 背景與環境光平滑變化。
4. 燈籠逐步點亮。
5. 神社窗戶發光逐漸增強。
6. Postprocessing 強度依狀態調整。
7. 不允許中途出現畫面閃白或突然變黑。

支援使用者在轉場途中再次切換模式，不得造成 Timeline 衝突或數值跳動。

不要每幀重新生成高成本環境貼圖或陰影資源。

07 — Cinematic Camera System

建立 Camera Presets。

至少包含：

Home View

完整展示神社、鳥居、水域與木船。

Shrine Focus

鏡頭緩慢移向神社建築，突出屋頂、紙門、石階與細節。

Torii Focus

鏡頭移向鳥居，呈現朱紅色結構與水面。

Water View

採用較低的觀賞角度，展示水面漣漪、船隻與周圍光影。

Return Home

平滑回到初始 Isometric 構圖。

不要讓鏡頭穿過建築、鳥居或水面。

08 — Camera Damping

使用具有阻尼的攝影機運動。

可採用：

`x'' + 2ζωx' + ω²(x-target) = 0`

初始使用接近 Critical Damping 的設定。

要求：

* 平滑移動
* 無明顯抖動
* 沒有突兀的旋轉
* Zoom 保持合理
* Preset 切換不造成大幅度畫面跳躍
* 使用 Quaternion 或其他適當方式處理旋轉插值
* Respect Reduced Motion

不要直接線性改變 Euler Angles，造成不自然的旋轉路徑。

09 — Ambient Cinematic Camera

加入極輕微的 Idle Camera Motion。

例如：

* 緩慢的水平視差
* 低振幅的攝影機漂移
* 具有時間間隔的景別微調

但必須遵守：

* 使用者操作時停止 Idle Camera。
* 鏡頭不可持續快速旋轉。
* 不要造成暈眩。
* 不要影響點擊水面。
* 不要影響繪馬書寫流程。
* 支援關閉動態效果。

10 — Scene Interaction

建立場景物件的 Click-to-Focus 機制。

支援：

* Shrine
* Torii
* Boats
* Water

使用 Raycasting 辨識互動物件。
操作原則：

* 水面點擊保留 Phase 02 的 Ripple Interaction。
* 點擊神社或鳥居可切換觀賞鏡位。
* 點擊船隻可以聚焦，但不得破壞浮力動畫。
* 拖曳旋轉應優先於 Click Interaction。
* UI Overlay 不得穿透觸發 3D 點擊。
* 使用者可以隨時返回初始鏡位。

互動回饋應該自然、低調。
11 — Cinematic Postprocessing
視情況加入或調整：

* Selective Bloom
* Ambient Occlusion
* Contact Shadows
* Tone Mapping
* Subtle Vignette
* Depth of Field

使用原則：

* 優先提升空間深度與光影品質。
* 不要過度模糊低多邊形模型。
* 不要使場景有過強的柔焦效果。
* 不要讓畫面色彩失去原本的乾淨感。

若某項後製效果的效能成本高於視覺收益，請提出替代方案。
12 — Motion and Interaction Timing
建立統一的 Motion Tokens。
初始參考：

* Hover Feedback：150–250ms
* Camera Focus：1.2–2.5s
* Camera Return：1.2–2.0s
* Environment Transition：3–6s
* Ambient Camera Motion：低頻、低振幅

所有動畫應具備合理的中斷與重入行為。

使用者直接操作應優先於自動動畫。

13 — Performance

支援：

* High Quality
* Medium Quality
* Low Quality
* Reduced Motion

不同品質可以調整：

* Reflection Complexity
* Shadow Quality
* Bloom Quality
* Atmospheric Effects
* Render Resolution
* Camera Idle Animation
* Particle Density

如果使用者裝置較弱，應優先保留穩定的 Camera Interaction 與 Day/Night Transition，而不是所有視覺後製效果。

14 — Debug Tools

加入開發環境專用控制項：

* Day Cycle Progress
* Sun Direction
* Sun Intensity
* Ambient Intensity
* Lantern Intensity

* Window Emission
* Bloom Strength
* Exposure
* Camera Preset
* Camera Damping
* Idle Motion Strength

允許開發者快速比較三種光照模式。

不要讓 Debug Controls 出現在正式使用者介面。

15 — Acceptance Criteria

完成時必須符合：

1. Day / Sunset / Night 有明確不同的視覺氛圍。
2. 三種光照模式具有一致的美術風格。
3. 日夜轉換平滑，不會突然閃爍。
4. 燈籠與神社窗戶發光合理。
5. 水面會依環境狀態改變視覺呈現。
6. Camera Presets 可以正常切換。
7. 使用者可以隨時中斷自動運鏡。
8. Click-to-Focus 不影響水面互動。
9. 互動與動畫支援桌面及手機。
10. 效能沒有出現明顯退步。

請實際驗收：

* Day Screenshot
* Sunset Screenshot
* Night Screenshot
* Home View
* Shrine Focus
* Torii Focus
* Water View
* Desktop / Mobile

如有瀏覽器自動化能力，請實際檢查渲染畫面及動畫過程。

完成後更新：

```
* `docs/ART_DIRECTION.md`
* `docs/SIMULATION_SPEC.md`
* `docs/IMPLEMENTATION_PLAN.md`

請提供：

* 修改檔案清單
* 功能完成狀態
* Before / After
* 實際效能數據
* 尚未完成或存在風險的部分

請不要在本階段重做整個 UI / Typography 系統，該部分應由獨立的 UI Polish 階段處理。
```

Claude 先盤點。環境狀態系統、三個時段、時段按鈕、夜晚燈籠都已經有了。選擇性泛光、接觸陰影、色調映射也有了。缺的是：

- 轉場太快（約 2 秒）
- 黃昏沒有暖色邊緣光，夜裡鳥居沒有光
- 鏡頭只有一個焦點，而且是「轉頭」；沒有點擊聚焦、沒有閒置運鏡
- 沒有暈影，沒有光照與鏡頭的除錯滑桿

做了什麼：

- 鏡頭的看向點、方位角、仰角、縮放四個量，各自用臨界阻尼彈簧追目標。用的是階段 4 船的浮力那一支。角度在球座標上走，方位角走最短方向，不內插 Euler 角
- 鏡位：全景、本殿、鳥居、水面（低角度）、船。繪馬卡片疊在鏡位上，關掉回到原鏡位
- 拖曳一開始就取消鏡位，任何操作都打斷閒置運鏡。閒置運鏡只在全景、8 秒沒操作時動：±2°、看向點 ±0.4，週期 45 與 60 秒
- 黃昏太陽降低（影子拉長）、背光改暖橘；夜裡鳥居兩柱之間一盞暖燈；暈影用 CSS 疊一層
- 不做景深與螢幕空間的環境光遮蔽。景深會把 low-poly 的硬邊糊掉，等角鏡頭也沒有對焦距離；環境光遮蔽成本高，接觸陰影已經夠

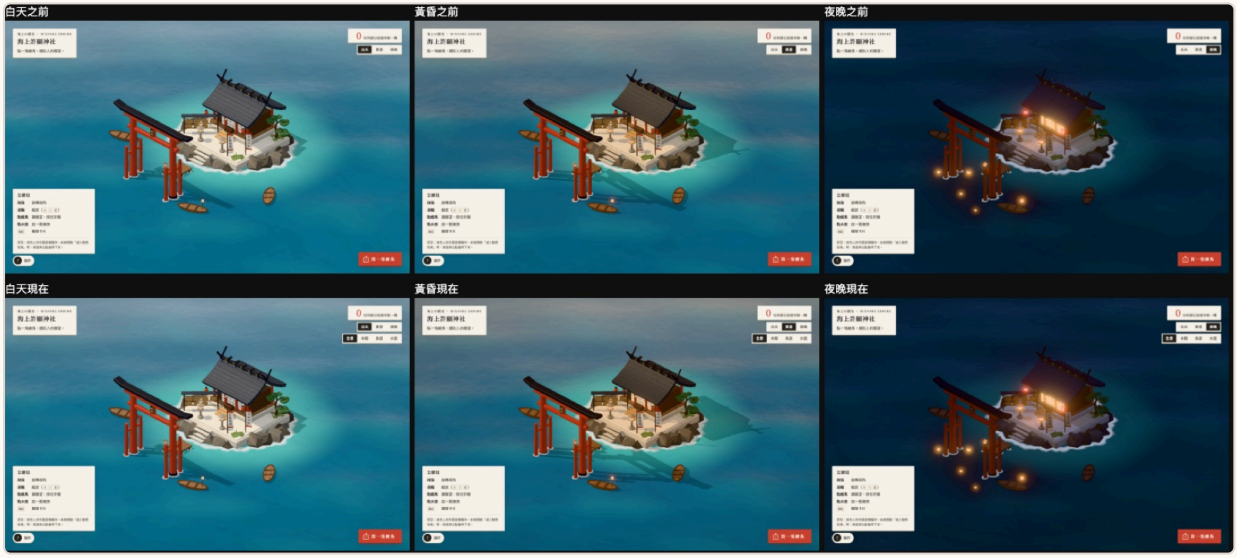
踩到的問題：

症狀	原因	修法
量起來夜晚只比白天暗 20%	取樣框包到了操作指南的白色紙籤	收起指南、只取場景中央：白天 117、夜晚 46
夜晚切白天，第一個取樣點亮度就跳了 20 級	指數逼近在按下的瞬間速度最快	改成臨界阻尼彈簧，起步緩，中途再切換保留速度。測試加上「途中切換每幀變化 < 0.012」

驗證：Node 測試 13 項。四個鏡位在桌機與手機的截圖、時段前後對照、轉場的亮度曲線。鏡頭互動 6 項、減少動態效果。

✅ 怎麼知道做對了

- [ ] 白天切到夜晚，畫面 4 到 5 秒慢慢變暗，起步是緩的，不是一按就跳
- [ ] 轉到一半再按白天，畫面從當下的位置轉回去，不會先跳一下
- [ ] 點本殿、鳥居、船，鏡頭推過去；拖曳一下就取消鏡位；按 `Esc` 或 `0` 回全景
- [ ] 全景放 8 秒不動，鏡頭開始非常慢地飄；動一下滑鼠就停
- [ ] 開「減少動態效果」：沒有閒置運鏡，時段切換直接到位



上排是這一步之前，下排是之後。黃昏的影子變長了，因為太陽降低了。夜晚的鳥居兩柱之間多了一盞暖燈。下排右上角多了一列視角按鈕。

🆘 如果沒有成功

你可能會看到	通常是因為	怎麼辦
夜晚只比白天暗一點點	量亮度的範圍包到了白色的介面	「量亮度時請收起操作指南，只取場景中央」

你可能會看到	通常是因為	怎麼辦
按下時段按鈕的瞬間亮度跳一大格	用了「每幀往目標靠近一個比例」的做法	「時段轉場請改成臨界阻尼彈簧：起步緩，中途切換保留速度」
減少動態的檢查說「畫面還在動」	模擬祈願讓右上角的計數器變了	「請關掉模擬祈願，只比場景中央」
Claude 重做了介面	最後一句被刪了	「不要在本階段重做 UI，那由獨立的介面階段處理」

這一步失敗可以跳過嗎：鏡位可以先跳過。時段轉場不行，階段 11 的介面顏色和階段 13 的選曲都跟著它。

📖 為什麼這樣做

🎯 這一步的重點

量不出來的東西，回報成「量不出來」，不要寫一個看起來像結果的數字。

🧭 這一步在做什麼

讓時段轉場起步緩、可以中途改變主意。讓鏡頭可以推到本殿、鳥居、船，使用者一動就交還控制。

🧩 它在建構流程的位置

```
時段按鈕 ──▶ 時段目標 ──▶ 【臨界阻尼彈簧】 ──▶ 光、水、天、燈、介面顏色（階段 11）
點物件   ──▶ 鏡位目標 ──▶ 【臨界阻尼彈簧】 ──▶ 相機
```

💡 那段指令為什麼那樣寫

第一句「最後一個步驟，同樣應該有不少你再過去的過程已經調整完畢，就不需要再執行囉」。它和階段 9 的那一句作用一樣。這份 15 節的規格，有一半在前面的階段已經做過了。

最後一句是規格本身就有的：「請不要在本階段重做整個 UI / Typography 系統，該部分應由獨立的 UI Polish 階段處理」。它把介面留給階段 11，這一步出問題時只要看光影與鏡頭。

規格要求提供「實際效能數據」。Claude 量了，回報的是「這個工具量不出來」。9.4 和 12.8 寫進文件當成結果，下一個人會以為效能退步了。

階段 11　比較介面方案，修掉縮小時的空白

📖 讀完約 15 分鐘　🛠️ 動手約 50 分鐘

- **這一步一定要懂**：幾個方案共用同一份網頁結構，只差 CSS。對比要對著實際渲染出來的畫面量。會隨時段變的介面顏色，要用 `@property` 註冊才能慢慢過渡。
- **這一步可以先不懂**：`@property` 的語法、怎麼查一個字實際用了哪個字型。
- **做完後你要能用自己的話回答**：畫面縮小時下緣有一塊空白，Claude 第一次加大了海。為什麼沒用？

⚖️ 動手前先想

又是先量還是先修：畫面有空白，是海不夠大，還是相機的參數？

做到 11–2 時，第一個修法（加大海面）會沒用。第二次先量相機，才找到近裁切面。

> **名詞**：**近裁切面**是相機前方一個看不見的平面，比它更靠近相機的東西不畫。
> 你需要知道到什麼程度：知道相機太靠近場景時，前景會被它切掉。畫面上看起來就是一塊空白。

左邊「之前低角度」，畫面下方三分之一是一片米白，海在中間被切斷。中間「現在低角度」，海一路鋪到畫面下緣。右邊是確認全景沒有被影響。

🛠️ 動手做

🧰 這一步的產出

```
src/styles/variants.css    ← 每個方案一段 CSS，用 html[data-ui] 切換
docs/UI_VARIANTS.md        ← 方案的比較、實測對比、建議
```

畫面上應該看到：開發環境的畫面下方有一個切換器。切換方案時，3D 場景、鏡位、時段都不動，只有四個角落的介面換樣子。

📋 貼給 AI（11-1：三個介面方案）

階段 7-5 的介面精修做完，看一下周邊介面：還像一般網站的資訊卡嗎？想換一個方向，貼這份規格。第一句是加在規格前面的。

貼給 AI

你只需要修改：三個方案的方向可以換成你的。「切換 UI Variant 不可重新載入 3D Canvas 或重置場景狀態」與最後一句照貼。

這個調整完畢後協助我優化文字 UI 配置：

UI Art Direction Exploration — Shrine Diorama

請針對目前的海上許願神社重新探索 UI / Typography 的設計方向。

現有 3D 場景已具有明確的 Low-poly Japanese Diorama 美術風格，但周邊 UI 仍然偏向傳統網站資訊卡，缺乏一致的設計語言。

本次不要直接進行大規模重構。

請先設計並實作可比較的三種 UI 視覺方案：

A. Japanese Editorial Minimalism

* 移除大部分背景卡片。
* 以開放式排版、精緻明體、留白與細線建立層次。
* 強調繁體中文的字形品質。
* 3D 場景為第一視覺焦點。

B. Contemporary Japanese Gallery

* 以美術館、展覽作品標籤的視覺語言重新設計。
* 加入小型作品標記、低對比輔助文字與精確網格。

* 減少不必要的裝飾。
* 避免過度模仿傳統日式牌匾。

C. Modern Shrine Ritual

* 將繪馬、奉納、祈願紀錄轉化為 UI 的互動語言。
* 以朱紅、暖白、深墨色為主要色系。
* 保留少量和紙、印章或木牌的視覺元素。
* 避免傳統遊戲介面、過度擬物化與大量裝飾。

三個方案都必須重新規劃：

1. 左上主標題與品牌識別。
2. 右上祈願數字。
3. Day / Sunset / Night 控制。
4. 左下操作說明。
5. 右下「寫一張繪馬」主要入口。

Typography 要求

* 以繁體中文為主要語言。
* 檢查實際使用的中文字型，不可依賴瀏覽器隨機 Fallback。
* 分別建立中文標題、介面文字、英文輔助標籤和數字的排版規則。
* 注意字距、字重、行高和對齊。
* 不要為了製造設計感而大量加入英文或日文。
* 確保所有操作文案清楚、自然且容易理解。

視覺整合要求

* 3D 場景必須是主角。
* 不要用大量不透明卡片遮擋神社或鳥居。
* 避免一般 SaaS Dashboard 的設計語言。
* 使用一致的間距、邊框、陰影與互動狀態。
* 保留良好的可讀性與無障礙對比。
* Desktop 和 Mobile 都需要規劃。
* 避免新增沒有作用的裝飾按鈕。

實作方式

請在開發環境提供 UI Variant Switcher，讓我能在相同的 3D 場景、相同鏡位和相同日夜模式下，快速比較 A、B、C 三種 UI。

切換 UI Variant 不可重新載入 3D Canvas 或重置場景狀態。

每個方案都應該是可實際操作的介面，而不只是靜態圖片。

最後請提供三個方案的 Desktop / Mobile 截圖，說明各自的優缺點，並推薦最適合目前神社美術風格的版本。

本階段不要修改 3D 模型、水面 Shader、船隻物理或 Cinematic Camera。

Claude 的做法：同一份網頁結構，三段 CSS，切換只改 `<html data-ui>` 一個屬性。切換四次之後，3D 畫布還是同一個物件。做排版之前，它先用 Chrome 的開發者協定查每個元素實際繪製的字型。標題、說明、數字、按鈕都由網頁字型繪製，沒有落到系統字型。

踩到的問題：

症狀	原因	修法
A 的字用眼睛看還好，量起來白天只有 1.98	字直接疊在天空上，背後的顏色會變	把文字變透明拍一張，取文字背後最不利的顏色來量；漸層加高加深後最低 5.23
C 的說明按鈕量到 3.49	量測框包到了上緣的朱紅帶，量錯了	量測框內縮 4px，實際是 14.67
手機上 A、B 有按鈕只有 36 到 40px	觸控目標太小	補到 44px

Claude 的建議：以 A 為底，借 C 的繪馬入口與朱印。

📋 貼給 AI（11-2：下方的白色、縮小時的空白）

貼給 AI

你只需要修改：換成你自己看到的問題，附截圖。

（附上截圖）

覺得底下的白色有點太自然，然後上面也可以更簡化背景，另外如附圖放大時會出現沒有渲染的空間

「太自然」的意思是不自然。Claude 拿掉 A 下方整條白色漸層。上方改成只在左上、右上文字後面各一團光暈。

放大縮小時下緣出現沒有渲染的空間，這件事 Claude 第一次修錯了。它以為是海面不夠大，加了一圈外海，沒有用。回頭量相機才找到原因。正交相機離看向點只有 76 單位。低角度加上縮小時，畫面下緣的視線起點落到水面以下，前景被近裁切面切掉。修法是相機拉到 300 單位，霧的距離加上同樣的補償。正交相機的畫面大小和距離無關，所以拉遠之後畫面不會變小。外海那一圈留著，遠景比較完整。

📋 貼給 AI（11–3：印章、只留兩個方案）

看過兩個方案都能操作之後，再決定細節和要留哪幾個。

C 方案的印章

這一句原本是一則訊息的第一點（其他幾點在階段 12）：

> 貼給 AI
>
> **（附上截圖）**
>
> * 第一個這裡的珠印同樣幫我調整

「珠印」是朱印，「同樣」指的是階段 7–4 繪馬上的那種御朱印。C 方案標題旁的印章改成同一種做法。

只留兩個方案

> 貼給 AI
>
> **（附上截圖）**
>
> 保留 A 和 C 即可，其他移除

B 與原本的「現行」方案移除，預設 A。網址加 `?ui=c` 可以切到 C。

📋 貼給 AI（11–4：光暈太重、換時段時先跳色）

切換時段時，盯著畫面上方的光暈看：太重嗎？會不會先跳色再慢慢變？有一樣的問題才貼。

光暈太重

貼給 AI

（附上截圖）

上面這兩塊大霧稍微有點影響太大，可以淡一點點

A 的兩團光暈縮小，濃度 0.92 改成 0.8。改完重量對比，最低是黃昏的 4.63，仍過 4.5。

換時段時光暈先跳色

貼給 AI

（附上截圖）

切換時段，上面這兩塊會突然變色，然後才慢慢淡化變成正常，應該要跟環境一樣慢慢轉變顏色

先想一想：天色 3.5 秒慢慢變，光暈卻一下跳過去。問題是顏色選錯，還是過渡沒有作用？症狀 → 原因：光暈的顏色存成一串 RGB 數字。CSS 不知道那是顏色，沒辦法在兩段文字之間慢慢過渡，只能一下換掉。修法：用 `@property` 把它註冊成顏色，再加 3.5 秒的過渡，和場景的時段轉場對齊。量到切換後 1.2 秒時顏色在中途，5 秒時到位。這一條也寫進了 `CLAUDE.md`（工程原則 8）。你可以這樣請 Claude 查：「請列出所有會隨時段變的 CSS 顏色，切換後 1 秒各截一張，確認每一個都在中途，不是一下跳到終點」。

📋 貼給 AI（11–5：導言改日文、換張動畫）

貼給 AI

幫我優化用詞「點一塊繪馬，讀別人的願望。」或許可以用日文的

然後會馬切換的動畫可以更自然一點，現在有點突兀

- 導言改成「絵馬をめくれば、誰かの願い。」（翻開繪馬，就是某個人的願望）。操作方式在操作說明與滑過的提示裡已經有中文，導言可以只負責氣氛
- 換張動畫：原本舊卡瞬間消失、新卡從旁邊滑進來。改成像掛在繩子上的繪馬被撥了一下。先轉到側面（200 ms），看不到字的那一刻換內容，再從另一側轉回來（560 ms）。連按時不重播轉出

踩到的問題：14 個字裡有 1 個掉到系統字型，因為 Noto Sans TC 沒有這個字。那個字是「絵」，日文的新字體。另外載入 Noto Sans JP，只抓這 14 個字，導言標上 `lang="ja"`。

📋 貼給 AI（11-6：按鈕太簡陋）

這一則指的是 C 方案右下的「寫一張繪馬」：

貼給 AI

（附上截圖）

這個按鈕有點太簡陋

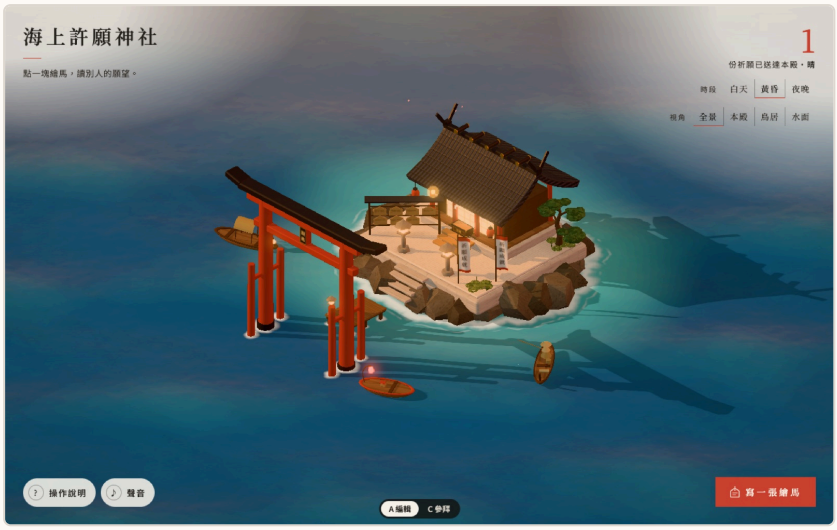
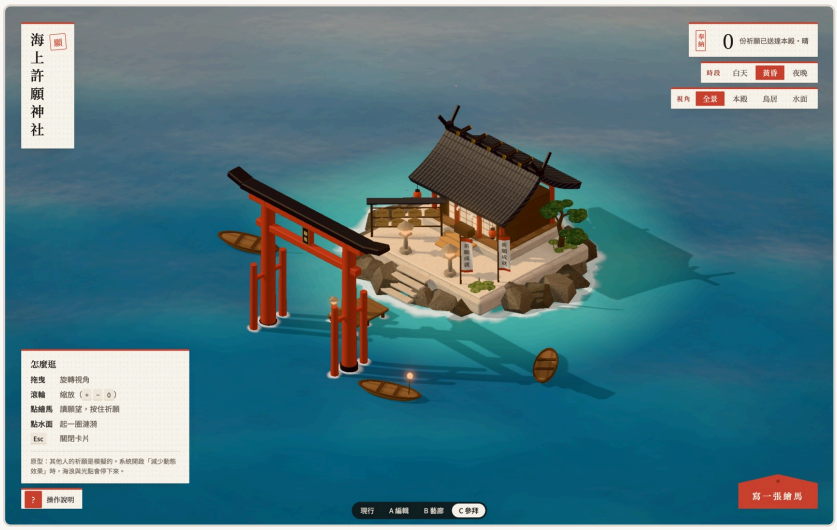
原本只是用 `clip-path` 把朱紅色塊裁成五角形。改成在按鈕裡墊一張 SVG 繪馬：深色屋頂、淡淡的木紋、內框、從上方吊下來的紅白絞繩。不再裁切按鈕，因為 `clip-path` 會連陰影一起裁掉。陰影改用 `drop-shadow`，跟著繪馬的形狀走。滑過時繞著繩頂晃一下（0.9 秒），減少動態時不晃。

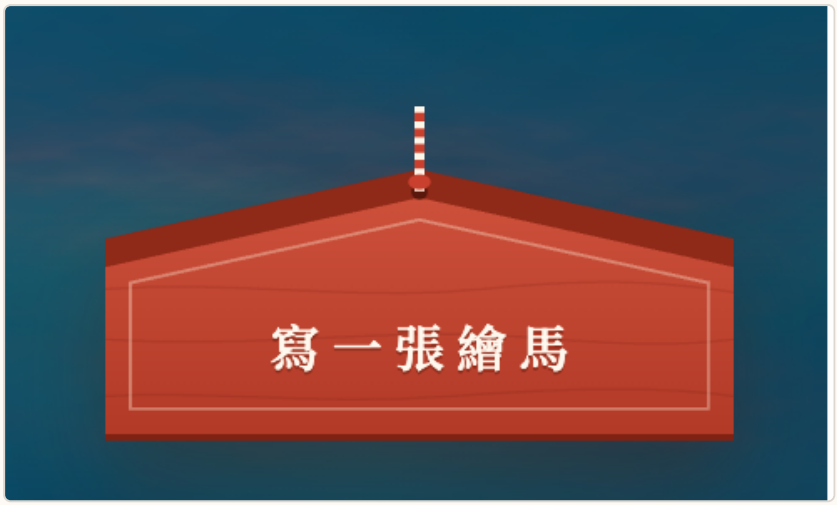
踩到的問題：

症狀	原因	修法
滑鼠移開按鈕時，整顆按鈕重新淡入一次	滑過時只寫了晃動的動畫，原本的進場動畫被換掉，移開時又被當成新的動畫重播	滑過時兩個動畫都列上，進場動畫放第一個
繩子看起來像一隻小眼睛	第一版是兩條往外彎的曲線	改成一條直直往上吊的繩，紅底加白色間隔段

✅ 怎麼知道做對了

- [ ] 切換方案時，場景沒有重新載入（鏡位、時段、漣漪都還在）
- [ ] Claude 給了每個方案、每個時段的實測對比，全部 ≥ 4.5
- [ ] 手機寬度下，所有按鈕都 ≥ 44 × 44
- [ ] 切換時段時，左上與右上的光暈顏色跟著場景慢慢變，不是先跳色
- [ ] 低角度縮到最小，畫面下緣沒有空白

方案	參考畫面
A 編輯	 沒有任何卡片。左上與右上的字後面各有一團很淡的光暈。右上的時段與視角是純文字，選中的底下一道朱紅線。
C 參拜	 左上的社名是直書，旁邊一個朱印「願」。右下的「寫一張繪馬」是五角形。畫面下方的切換器還有 B 和「現行」，11–3 之後移除了。

方案	參考畫面
C 的繪馬按鈕（11–6 之後）	 局部放大。按鈕是一塊吊著的繪馬：深色的屋頂、淡淡的木紋、內框。頂端一條紅白相間的繩子往上吊。改之前就是上一張右下角那顆五角形。

🆘 如果沒有成功

你可能會看到	通常是因為	怎麼辦
頁面整頁 500 錯誤	先改了頁面去引用新的 CSS 檔，檔案後來才建。開發伺服器記住了「找不到檔案」	「請重新啟動開發伺服器」
疊在場景上的字量起來對比不夠	用 Design Token 算，沒對著畫面量	「請把文字變透明拍一張，取文字背後最不利的顏色來量對比」
低角度縮小時，畫面下緣有一塊空白	前景被近裁切面切掉	「畫面下緣的空白，請先檢查相機的近裁切面，不要先加大海面」
切換時段時，光暈先跳色才慢慢變	CSS 不能內插一串 RGB 數字	「請把光暈顏色用 @property 註冊成 color，再加 transition，時間和場景的時段轉場對齊」
日文導言的某一個字字型不一樣	那個字不在中文網頁字型裡	「請只為這幾個字另外載入 Noto Sans JP，導言標上 lang="ja"」

這一步失敗可以跳過嗎：方案比較可以跳過，直接用 A。空白與顏色過渡不行，那是畫面上看得到的錯。

📖 為什麼這樣做

🎯 這一步的重點

同一個症狀可能有兩種原因。第一個修法沒效時，回頭量參數，不要繼續加大修法。

🧭 這一步在做什麼

先做幾個能實際操作的方案，在同一個場景、同一個鏡位下比較，看過再刪。

A 方案沒有卡片，字直接疊在 3D 場景上。背後是會變的天空與海，用 Design Token 算不準。Claude 的量法：把文字變透明拍一張，取文字框背後最不利的 10% 顏色。三個時段、每個方案各量一次。

時段	A 第一次	A 修正後	C
白天	1.98	5.59	6.68
黃昏	未記錄	5.23	6.68
夜晚	未記錄	11.14	6.68

🧩 它在建構流程的位置

```
時段數值（階段 10） ──▶ <html data-tod> ──▶ 介面光暈顏色（@property，3.5 秒）
切換器 ──────────────────────▶ <html data-ui>  ──▶ 介面樣式（A 或 C）
```

💡 那段指令為什麼那樣寫

11–1 的「切換 UI Variant 不可重新載入 3D Canvas 或重置場景狀態」是比較方案的前提。如果每次切換都重新載入，你比的是「載入後的第一秒」。比的不是同一個鏡位、同一個時段下的幾種介面。這也是階段 8 說明的引擎與介面分工：介面換三次，引擎不需要知道排版細節。

11–2 到 11–5 都是一兩句話，大多附截圖。

程度（「淡一點點」）和發生的順序（「突然變色，然後才慢慢淡化」）都寫出來。這樣 Claude 才知道要改顏色還是改過渡。

🧭 Checkpoint 3（做完階段 11）

1. 花瓣的 Node 測試全過，畫面上卻沒有花瓣落水。原因是什麼？
2. SwiftShader 量到 9.4 和 12.8 fps，你會怎麼回報這個結果？
3. A 方案的對比，用 Design Token 算和對著畫面量，為什麼會不一樣？

> 第 2 題答不出來，回階段 10 的〈⚖️ 動手前先想〉再看一次。

階段 12　做三種船、雷雨與環境音

📖 讀完約 12 分鐘　🛠️ 動手約 50 分鐘

- **這一步一定要懂**：船型不能多拿共用的亂數。閃電要對準雷聲檔案裡雷響的時間點，而且要有光敏安全的間隔。
- **這一步可以先不懂**：低頻能量分析怎麼找到雷聲、Web Audio 的節點怎麼接。
- **做完後你要能用自己的話回答**：回報寫「打雷好像沒有閃電」。程式明明有閃電，為什麼看不到？

⚖️ 動手前先想

階段 6 那一題再出現一次：篷的柱子要不要用篷的常數算？

船篷的柱子在 z ±0.5，篷的半徑是 0.66，兩個數字各寫各的。

閃電看不到，是程式沒有，還是看不出來？

做到 12–3 時你會看到：程式有閃電，畫面上卻看不出來。雨天的畫面本來就灰，燈光加亮的幅度會被吃掉。

🛠️ 動手做

🧰 這一步的產出

```
public/audio/                ← 兩段海浪、一段雨、一段雷雨（你自己準備的檔案，見〈聲音素材〉）
src/engine/scene/boats.js    ← 三種船：屋形船、漁舟加蓑衣漁夫、小舟
src/engine/sim/weather.js    ← 新增「雷雨」
public/favicon.svg           ← 網站小圖示
```

畫面上應該看到：三艘船長得不一樣。網址加 `?wx=雷雨`，下大雨，偶爾打雷，打雷時整個畫面亮一下。左下角多一個「聲音」按鈕。

📋 貼給 AI（12–1：船型與環境音）

先照〈聲音素材〉生成四個聲音檔，放進 `public/audio/`。貼這段：

貼給 AI

你只需要修改：四個 `@"…"` 換成你自己的檔案位置（可以在對話框打 `@` 選檔案）。第一點的印章在階段 11–3 處理過，刪掉。「有一個雨聲有雷聲」要照你的檔案改寫，Claude 靠它知道哪一個檔案有雷。

《附上截圖》

`@"/Users/《你的帳號》/Downloads/sea_1.mp3" @"/Users/《你的帳號》/Downloads/sea_2.mp3" @"/Users/《你的帳號》/Downloads/rain_1.mp3" @"/Users/《你的帳號》/Downloads/rain_2.mp3"`

* 第一個這裡的珠印同樣幫我調整
* 另外我希望船可以有不同的樣式！目前三艘都長得完全相同
* 協助我配置我上傳的海浪聲音、下雨的聲音，有一個雨聲有雷聲，所以可以新增一種有打雷的環境

結果：

- 三種船：屋形船、漁舟、小舟。差異全部由船型名稱決定，不多呼叫 `rand()`，後面的雨、花瓣位置不變。浮力的取樣點不變，只把水面的接觸陰影依船的大小調整
- 環境音：兩段海浪循環播放，雨聲跟著天氣的雨量
- 雷：Claude 用低頻能量分析，找到有雷的那個檔案裡雷響在 1.5 到 3 秒。這個檔案不能拿來循環，會每 8 秒打一次雷。改成雷雨期間偶爾播一次，閃電對準檔案裡雷響的起點
- 天氣多一種「雷雨」

📋 貼給 AI（12–2：黃昏、奉納箱、釣竿與漁夫、網站小圖示）

這幾項都是看畫面時想到就提的小修改，一次一項，附截圖。沒有一樣的需求就跳過。

黃昏要更金黃、更紅

貼給 AI

另外黃昏的整體氛圍可以更金黃或者紅色的日落感覺

Claude 把黃昏的光色與天色往金黃、紅色調整。改完重新量了 A、C 兩個方案在黃昏時的文字對比。

奉納箱

貼給 AI

《附上截圖》

另外我想要讓奉納箱更擬真，但我有點擔心空間不夠？

如果擔心空間不夠，直接寫出來。Claude 先量位置：紙門在 z 2.15、前排柱 z 2.6、最上一階 z 3.45，原本的方塊佔 2.63 到 3.38。空間其實夠。外形大小不變，細節往裡長：三角木條、金具、字板。

釣竿與漁夫

貼給 AI

《附上截圖》

這個很棒椰，只是釣竿的魚餌應該要在海裡面，然後可以有一個帶蓑衣的漁夫，可以以形體為主

症狀 → 原因：Claude 原本做的是「櫓」，划船用的長槳。畫面上看起來像釣竿，末端懸在空中。修法：改成真的釣竿，釣線拉到船底以下 0.8，波浪加上橫搖都不會露出線尾。漁夫照指令說的「以形體為主」，只做輪廓。

網站小圖示

貼給 AI

另外幫我設計一個 `Fav Icon` 並且更新上去～

SVG 小圖示：黃昏天色、海平線、朱紅鳥居。16px 時只剩剪影，所以不放字、線條至少 4 單位粗。

📋 貼給 AI（12–3：船篷沒接上、沒有閃電）

貼給 AI

你只需要修改：換成你自己看到的問題，附截圖。

（附上截圖）

小船屋頂和柱沒有連接。另外打雷好像沒有閃電～～

問題	原因	修法
小船屋頂和柱沒有連接	柱子在 z ±0.5、篷的半徑 0.66，柱頂懸在篷裡面	柱子與篷從同一組常數（`RR`、`RY`）算出來，下緣加一根橫木
打雷沒有閃電	第一版只把燈光加亮 2.2 倍，在雨天的灰裡看不出來	燈光加倍，再疊一層冷白的天光（上限 0.45）

閃電另外加了光敏安全：兩次閃電至少隔 1.5 秒，開了「減少動態效果」就不閃。

這是這個案例第四次「沒接上」（前三次在階段 6）。

📋 貼給 AI（12–4：規則寫回 CLAUDE.md）

貼給 AI

這種對上的應該也要更新回 `CLAUDE.md`

「對上」指的是柱子頂到篷的下緣這種接合。Claude 把它寫進 `CLAUDE.md` 的工程原則 7：要接在一起的兩樣東西，位置從同一個常數推出來。原則底下列出四處實例，也規定驗證時要附接合處的特寫截圖。

📋 貼給 AI（12–5：燈籠受風、花瓣落在石頭上）

切到風大的天氣，看吊著的燈籠會不會動；等花瓣落下，看岩石上有沒有花瓣穿過去。

燈籠受風

貼給 AI

燈籠也應該受到風的影響喔～！

吊著的燈改成以吊點為軸：順風偏一點，再加上單擺擺動。擺動的函式 `swing()` 放在 `sim/wind.js`，風還是只有一個來源。船上的燈先抵銷船身的傾斜，繩子永遠朝下。石燈籠不動。

花瓣落在石頭上

貼給 AI

（附上截圖）

櫻花有一些應該是要落到石頭上面

先想一想：花瓣落下時，程式怎麼知道腳下是石台、岩石還是海？症狀 → 原因：地面只有石台那個長方形，外圍岩石上的花瓣穿過岩石掉進海裡。修法：載入時向下打射線，烘焙一張地面高度表（0.25 一格，約 80 ms），岩石頂面也算地面。你可以這樣請 Claude 查：「花瓣判斷『落地』用的是哪一塊範圍？請把它和石台、岩石的範圍畫在同一張俯視圖上給我看」。

📋 貼給 AI（12-6：聲音預設打開）

貼給 AI

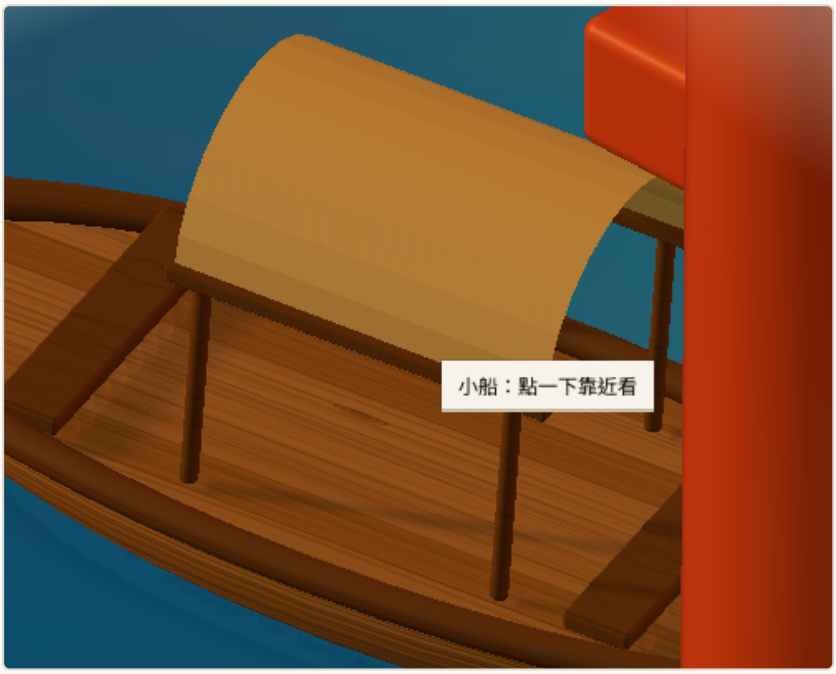
聲音預設打開，可以稍微小聲

預設打開、音量 0.55。瀏覽器要互動過才肯出聲，所以第一次點擊畫面時才真的啟動。如果第一下就是按「聲音」按鈕，代表要靜音。第一版在啟動中把按鈕設成不能按，那一下的點擊被吃掉了，後來改掉。

✅ 怎麼知道做對了

- [ ] 三艘船造型不同，而且岩石、其他物件的位置和這一步之前一樣
- [ ] 船篷的柱子頂到篷的下緣（放大看接合處）
- [ ] `?wx=雷雨` 等幾秒：聽到雷聲的同時，畫面亮一下

- [ ] 開「減少動態效果」：雷雨時不閃
- [ ] 第一次點畫面之後才有聲音；第一下就按「聲音」是靜音
- [ ] 夜晚時，本殿的提燈和船上的燈會順著風擺
- [ ] 瀏覽器分頁上看得到小圖示

狀態	參考畫面
船篷接合處	 局部放大。四根柱子的頂端接在篷的下緣，篷的下緣有一條橫木。修之前柱頂懸在篷裡面。
雷雨與閃電	 右上角的天氣寫著「雷雨」。這一張是閃電亮起的那一刻，整個場景偏亮、偏冷白。

🆘 如果沒有成功

你可能會看到	通常是因為	怎麼辦
換了船型之後，雨和花瓣的位置變了	船型多拿了共用的亂數	「船型的差異請全部由船型名稱決定，不要多呼叫 rand()」
打雷時看不出閃電	燈光加亮的幅度在雨天的灰裡看不出來	「閃電在雨天看不到。請把燈光再加強，再疊一層冷白的天光」
雷聲每 8 秒就響一次	把雷雨的檔案拿來循環播放	「這個檔案不要循環。雷雨期間偶爾播一次，閃電對準檔案裡雷響的時間點」
船篷的柱子懸在篷裡面	柱子與篷各寫一個數字	「柱子與篷請從同一組常數算出來」
第一下按「聲音」沒反應	按鈕在聲音啟動中被設成不能按	「聲音啟動中，按鈕不要設成不能按」
雷聲出現的時間和閃電對不上	你生成的雷不在前幾秒	「請分析 rain-thunder.mp3 裡雷響的時間點，閃電對準它」

這一步失敗可以跳過嗎：可以。後面的階段不依賴船型和雷雨。聲音的開關不行，階段 13 的背景音樂接在同一套聲音系統上。

📖 為什麼這樣做

🎯 這一步的重點

同一種錯誤第四次出現，就把規則寫進 `CLAUDE.md`。

🧭 這一步在做什麼

這一步是一連串的小修改，每一個都來自看畫面時的一句話。

🧩 它在建構流程的位置

天氣（雷雨）⟶ 雨聲音量、偶爾播雷聲 ⟶ 雷響的時間點 ⟶ 閃電

風（階段 9）⟶ 吊燈擺動

💡 那段指令為什麼那樣寫

12-2、12-5 的每一項都只有一兩句。看到一個就傳一個，Claude 正在做別的事也可以照傳。訊息會排隊，Claude 做完手上的事再接著處理。一則只講一件事，處理完可以一則一則對照，不會漏。

> 有擔心就直接寫出來（「有點擔心空間不夠？」），Claude 會先量再回答，不會先動手改。

階段 13　接上背景音樂

📖 讀完約 15 分鐘　🛠️ 動手約 50 分鐘

- **這一步一定要懂**：不裝 Howler，因為它的淡出是線性的。音樂用串流播放，不解碼成整段音訊。放進網站之前先查每一首的授權。
- **這一步可以先不懂**：Web Audio 的節點、壓縮器與限幅器的參數。
- **做完後你要能用自己的話回答**：兩首交叉淡化到正中間時，各自的音量是多少？為什麼不是 0.5？

⚖️ 動手前先想

要不要裝 Howler？音樂要解碼還是串流？

> **名詞**：**等功率交叉淡化**是兩首曲子交替時，讓兩首音量的平方加起來永遠等於 1。
> 你需要知道到什麼程度：知道直接線性淡化，在正中間兩首都是 0.5。聽起來中間會凹下去，約 −3 dB。等功率在正中間是約 0.707。

選擇	決定	理由
Howler.js	不裝	等功率淡化要直接控制音量節點，Howler 的 `fade()` 是線性的
解碼成整段音訊	不用	一首 4 分鐘的立體聲解碼後約 85 MB，五首超過 400 MB
`<audio>` 串流	用	每首一個元素，第一次需要時才建立、之後重用，也只在那時才開始下載（約 5 MB）

參考成品實測的 0.706 與 0.708，就是等功率在正中間的樣子。

這首曲子能不能放進公開的網站與教材？

判斷依據是「它是誰的錄音」。這個案例實際走過一次反例，你不用照做，看它怎麼一步步出問題：

1. 當時用了坂本龍一的 Merry Christmas Mr. Lawrence 當黃昏的曲子。它是有著作權的商業錄音，在自己電腦上試可以，所以只在本機試過。13–4 的第 1 版拿掉了它，13–5 又放回私人連結。這個連結只要分享出去，就是在散布那首錄音

2. 階段 20 推上 GitHub 時選了「全部推上去」，第一次部署就跟著上線了
3. 〈上線之後〉把它移出 `public/`，換成自己生成的曲子
4. 移出之後的提交，Vercel 一開始沒有自動部署（原因見階段 20 的 🆘），正式網站還留著這首。之後的提交部署成功，正式網址上的這首才變成 404

從「只在本機試」到「真的從網站上消失」，中間每一步都要有人確認。這一條寫進了 `CLAUDE.md`，也是課程附件裡沒有這個檔案的原因。你的版本從一開始就用自己有權使用的五首，這四步都不會發生。

🛠️ 動手做

🧰 這一步的產出

```
public/audio/music/          ← 五首背景音樂：白天、黃昏、夜晚各一首，另外兩首手動選（檔名見〈聲音素材〉）
src/shell/audio/config.js ← 曲目、時段對應、轉場秒數、預設音量、響度校正
src/shell/audio/mixer.js   ← 音樂 / 環境 / 音效三條線，交叉淡化，限幅器
src/app/components/Chrome.jsx ← 左下「聲音」面板（React 元件）
tests/audio.test.mjs         ← 選曲、等功率淡化、連切不疊加
docs/AUDIO.md                ← 曲目、響度、狀態、驗收結果
```

畫面上應該看到：點「聲音」打開面板，選曲有「隨時段」加上五首。切時段時，背景音樂慢慢換成那個時段的曲子。

你自己做（13–1：確認五首曲子的授權）

你要準備五首：白天、黃昏、夜晚各一首，另外兩首給使用者手動選。可以照〈聲音素材〉的指令生成。

放進專案之前，先回答每一首：這是誰的錄音？你有沒有權利放進一個公開的網站？

曲子的來源	能不能放進公開網站
你自己錄的	可以
你用 AI 工具生成的	看那個工具的條款。有些免費方案只能個人使用
素材庫明確標示可以商用的	可以，照條款標示出處
你買的 CD、串流平台上的歌	不行。只能在自己電腦上試

參考成品做到這一步時只有三首。白天、夜晚是兩首自己生成的 Floating Shrine。黃昏是坂本龍一那首，屬於表格最後一列，經過見〈⚖️ 動手前先想〉。

最後的成品是這五首：

用途	曲子	`trim`
白天	Floating Shrine · 昼	0.67
黃昏	`glass-wind-chimes.mp3`	0.68
夜晚	Floating Shrine · 夜	0.67
手動選	Frost on the Windowpane	0.62
手動選	雨後の静けさ	0.69

參考成品的五首都是自己生成的。`trim` 是把每一首拉到差不多大聲的倍數，見下面的響度。

📋 貼給 AI（13-2：背景音樂系統）

這是實際用的 Adaptive Audio 指令原文。規格由 ChatGPT 撰寫，做法見〈用 ChatGPT 寫規格，再交給 Claude〉。開頭的三個 `@` 是當時上傳的三首曲子。規格寫的架構是 React、TypeScript、Zustand、Howler.js，專案裡只有 React。Claude 怎麼處理見下面。

貼給 AI

你只需要修改：開頭的三個 `@"…"` 換成你自己的五首曲子。原文寫的是當時的三首，規格裡的「三首」都改成「五首」。在「根據白天、黃昏、夜晚自動選擇對應音樂」後面加一句「另外兩首只在手動選曲時播放」。第一個 `@` 是坂本龍一的商業錄音，不要照著用。規格第 01 節「三首音樂實際所在位置與格式」與第 03 節「請根據實際音檔名稱更新，不要假設檔案已存在」照貼，數字改成五。Claude 會照你的檔名改設定。

```
@"/Users/《你的帳號》/Downloads/merry christmas mr lawrence ryuichi sakamoto.mp3" @"/Users/《你的帳號》/Downloads/Floating Shrine (1).mp3" @"/Users/《你的帳號》/Downloads/Floating Shrine.mp3"
這是三首背景音樂，你可以協助我調整正確的曲名

Adaptive Audio & Immersive Sound Experience
Role & Objective
請作為 Senior Frontend Engineer、Creative Developer、Interaction Designer 與 Sound Experience Designer，為目前的「海上許願神社（Wishing Shrine）」建立完整的背景音樂與環境音效系統。
專案目前使用 React、TypeScript、Three.js / React Three Fiber，並已有白天、黃昏、夜晚
```

三種場景模式。

我已經準備好三首不同的背景音樂。

本階段目標：

1. 整合三首背景音樂。
2. 根據白天、黃昏、夜晚自動選擇對應音樂。
3. 提供自然的 Crossfade 音樂轉場。
4. 允許使用者手動切換音樂。
5. 提供音量、播放、暫停與靜音控制。
6. 為後續的海浪、風鈴、繪馬與祈願音效建立擴充架構。
7. 建立符合整體神社視覺風格的極簡音訊 UI。

核心設計理念：

Sound should enhance the atmosphere, never dominate the experience.

音樂應自然融入神社，而不是讓網站變成傳統音樂播放器。

01 — Inspect Existing Project

請先閱讀：

* `CLAUDE.md`
* `.claude/rules/`
* `docs/ART_DIRECTION.md`
* `docs/IMPLEMENTATION_PLAN.md`

並檢查：

* 目前 React / TypeScript 架構。
* Three.js / R3F Canvas。
* Day / Sunset / Night 狀態管理。
* 是否使用 Zustand。
* 現有 UI Design Tokens。
* 是否已經存在 AudioManager 或音訊相關程式。
* 專案是否已安裝 Howler.js。
* 三首音樂實際所在位置與格式。

優先使用既有架構，不要重新建立重複的場景狀態。

02 — Audio Technology

採用：

* Howler.js
* TypeScript
* React
* Existing Zustand Store（若有）

不需要導入 Tone.js。
利用 Howler.js 管理：

* Audio Loading
* Playback
* Pause / Resume
* Looping
* Volume
* Fade In
* Fade Out
* Crossfade
* Sound Effects
* Audio State

建立統一的 AudioManager。
建議架構：

```
src/
  audio/
    AudioManager.ts
    audioConfig.ts
    audioTypes.ts
    useAudioController.ts

  components/
    audio/
      AudioControls.tsx
      MusicSelector.tsx
```

請依照目前實際專案架構調整，不必強制照搬以上路徑。

03 — Music Configuration

建立三首背景音樂的設定。

例如：

```
type SceneTime = "day" | "sunset" | "night";

interface MusicTrack {
  id: string;
  title: string;
  src: string;
  scene: SceneTime;
  defaultVolume: number;
  loop: boolean;
}
```

初始配置：

```
const tracks: MusicTrack[] = [
  {
    id: "day-ambient",
    title: "Day Ambient",
    src: "/audio/day.mp3",
    scene: "day",
    defaultVolume: 0.4,
    loop: true,
  },
  {
    id: "sunset-piano",
    title: "Sunset Piano",
    src: "/audio/sunset.mp3",
    scene: "sunset",
    defaultVolume: 0.35,
    loop: true,
```

```
  },
  {
    id: "night-ambient",
    title: "Night Ambient",
    src: "/audio/night.mp3",
    scene: "night",
    defaultVolume: 0.3,
    loop: true,
  },
];
```

請根據實際音檔名稱更新，不要假設檔案已存在。

三首音樂應可透過設定檔管理，不要在元件內硬編碼路徑。

音量參數只是初始建議，請依實際三首音樂的響度調整。

04 — Audio Initialization

瀏覽器不一定允許自動播放有聲音的音訊。

請設計符合瀏覽器 Autoplay Policy 的啟動流程。

要求：

1. 初始狀態不自動播放有聲音的音樂。
2. 顯示清楚但不干擾場景的「開啟聲音」入口。
3. 使用者點擊後初始化或解鎖 AudioContext，並開始播放。
4. 不得使用隱藏按鈕、模擬點擊或其他方式繞過瀏覽器限制。
5. 初始化失敗時，UI 必須提供重新嘗試的方式。
6. 音樂載入期間應正確處理 Loading State。
7. 音樂載入失敗時不得造成整個 3D 場景錯誤。

聲音啟用前，神社場景應正常運作。

05 — Adaptive Music Switching

建立 Scene-Aware Music System。

預設對應：

* Day → Track 01
* Sunset → Track 02
* Night → Track 03

切換白天、黃昏、夜晚時，AudioManager 應收到新的場景狀態。

例如：

```
audioManager.setScene("sunset");
```

要求：

* 自動選擇對應音樂。
* 不重複播放相同音樂。
* 不因 React Re-render 重複初始化 Audio。
* 切換場景不會重新掛載 Three.js Canvas。
* 使用者暫停音樂時，場景切換不應強制恢復播放。
* 使用者尚未開啟聲音時，場景切換只更新待播放的音樂。

06 — Smooth Crossfade

實作兩首音樂之間的 Crossfade。

建議轉場時間：

* Day → Sunset：4 秒
* Sunset → Night：5 秒
* Night → Day：4 秒

建議採用 Equal-Power Crossfade：

`gainA = cos(t × π/2)`

`gainB = sin(t × π/2)`

其中 t 為 0–1 的轉場進度。

如果使用 Howler.js 的音量淡入淡出功能，請評估線性淡入淡出的實際聽感。

如需實現真正的 Equal-Power Crossfade，可在 Howler.js 的公開 API 能力範圍內建立音量排程，或評估額外的 Web Audio GainNode 控制方案，但不要依賴不穩定的內部屬性。

需要處理：

1. A 音樂漸弱。
2. B 音樂漸強。
3. 兩首音樂短暫重疊。
4. 轉場結束後停止 A。

5. B 持續循環。

特別注意：

* 使用者快速連續切換三種場景時，不得產生重疊的音樂實例。
* 新的切換命令必須能夠中止或接管前一個轉場。
* 音量不可突然跳動。
* 不得累積未停止的音軌。
* 相同音樂不應重複啟動。
* 淡入淡出必須同時考量 Master Volume。

07 — Manual Music Selection

除了場景自動選歌，也允許使用者手動切換三首音樂。

建議提供兩種模式：

Auto Mode

根據白天、黃昏、夜晚自動選歌。

Manual Mode

使用者自行選擇 Track 01、02、03。

當使用者手動選歌時，不應因場景變化立即覆蓋其選擇。

使用者可隨時切換回 Auto Mode。

請明確管理：

* Playback State
* Selected Track
* Audio Mode
* Scene State
* Master Volume
* Muted State

避免自動與手動控制互相衝突。

08 — Audio Controls UI

請依照目前 Shrine Diorama 的 UI / Typography 視覺規範設計。

Art Direction：

* Japanese Editorial Minimalism
* Calm
* Refined

* Minimal
* Premium
* Low Visual Noise

建立簡潔的音訊控制元件。
主要功能：

* 開啟 / 關閉聲音
* Play / Pause
* Master Volume
* 選擇音樂
* Auto / Manual Mode

建議在畫面左下角原有操作入口附近整合。
不要建立一個大型音樂播放器遮擋 3D 神社。
Desktop 可使用小型 Popover。
Mobile 可使用精簡的 Bottom Sheet 或 Popover。
請使用現有 Design Tokens。
需要支援：

* Keyboard Navigation
* Focus State
* Accessible Labels
* Touch Interaction
* Reduced Motion
* Responsive Layout

09 — Environmental Sound Effects
建立可擴充的 Sound Effects 系統。
第一階段不需要強制加入所有音效。
預留：

* Ocean Waves
* Wind Chimes
* Cherry Blossom Interaction
* Water Ripple
* Ema Open

* Ema Submit
* Wish Completion Bell

建議音效介面：

```
audioManager.playSfx("wish-complete");
```

環境聲應與背景音樂分開控制音量。
建立：

* Music Volume
* Ambient Volume
* SFX Volume
* Master Volume

音效不得因為短時間多次互動而造成過大的音量疊加。
加入適當的 Cooldown、Concurrency Limit 或 Voice Pool。
如果音效檔案尚未提供，請建立可擴充的介面，不要自行引用不存在的檔案。
10 — Audio Mixing
三首音樂的實際響度可能不同。
請避免僅使用相同 Volume 數值就假設聽感一致。
建議：

* 測試三首音樂的主觀響度。
* 在可行時分析 LUFS 或其他響度指標。
* 避免切歌時突然變大聲。
* 保留足夠的 Headroom。
* 不要讓多個音效同時播放造成削波或過度刺耳。

如果沒有音訊分析工具，請回報限制，不要假裝已完成響度標準化。
11 — Performance & Resource Management
請注意：

* AudioManager 應具有穩定的生命週期。
* 不要在每次 React Render 建立 Howl Instance。

* 避免重複載入同一個音檔。
* 適當釋放不再使用的音訊資源。
* 處理 Component Unmount。
* 處理頁面 Visibility Change。
* 確保切換音樂不影響 3D FPS。

在頁面隱藏時，應採取一致的播放策略，例如暫停或依使用者設定繼續播放。
頁面恢復時，必須尊重使用者原本的播放意圖。
不要擅自解除靜音。
12 — State Persistence
使用 localStorage 保存必要偏好：

* Sound Enabled Preference
* Master Volume
* Music Volume
* Audio Mode
* Selected Manual Track
* Muted State

但需要區分「記住使用者偏好」與「取得本次瀏覽器播放權限」。
即使使用者之前已開啟音樂，也不能假設下次開啟網站時瀏覽器一定允許自動播放。
請避免 SSR 或瀏覽器環境初始化問題。
13 — Testing Scenarios
請測試：
A. First Visit

* 網站初始安靜。
* 使用者可以明確開啟聲音。
* Audio Loading 正常。
* 不影響 3D Canvas。

B. Scene Switching

* 白天、黃昏、夜晚正確對應音樂。
* Crossfade 自然。
* 不出現突然的音量跳動。

C. Rapid Switching

* 快速切換 Day / Sunset / Night。
* 不產生多個重疊播放實例。
* 不會讓音量越來越大。

D. Manual Music Mode

* 手動選歌正常。
* 場景切換不覆蓋手動選擇。
* 返回 Auto Mode 後正確恢復場景對應音樂。

E. Pause / Mute

* 暫停後不會因日夜切換自行播放。
* 靜音後不會因 Crossfade 意外發聲。
* 恢復播放時音量正確。

F. Mobile

* iOS Safari。
* Android Chrome。
* 觸控操作正常。
* 音訊解鎖流程符合瀏覽器規則。
* 音訊控制不干擾 Camera Drag。

G. Long Session

* 至少測試 10 分鐘。
* 多次切歌。
* 多次調整音量。
* 多次切換靜音。
* 確保不出現音訊實例累積或記憶體異常。

14 — Acceptance Criteria

完成時必須達到：

1. 三首音樂都可以正常播放。
2. Day / Sunset / Night 自動切歌。
3. Crossfade 自然、無明顯斷裂。
4. 使用者可手動選歌。
5. 音量、暫停與靜音正常。
6. UI 符合現有神社設計語言。
7. 不影響 Three.js / R3F 效能。
8. 瀏覽器自動播放限制處理正確。
9. 支援桌面與手機。
10. 音效架構可以持續擴充。

15 — Deliverables

請完成：

* Howler.js Integration
* AudioManager
* Track Configuration
* Scene-Aware Music Switching
* Crossfade System
* Audio Controls UI
* Auto / Manual Mode
* Volume Persistence
* SFX Extension Interface
* Error / Loading Handling
* Desktop / Mobile Verification

並更新：

* `docs/IMPLEMENTATION_PLAN.md`
* 必要的專案架構文件
* 相關測試與驗收結果

請實際測試音樂播放，不要只確認 TypeScript Build 成功。

請記錄：

* 實際使用的三首音檔。
* 每首音樂對應的場景。

```
* 各首音樂的建議音量。
* Crossfade 實作方式。
* 目前已完成與尚未完成的功能。

禁止為此階段重構 Three.js 場景、水面 Shader、船隻物理、櫻花動畫或既有 UI 架構。
請優先完成穩定、簡潔、自然且符合神社氣質的聲音系統。
```

Claude 先盤點，列出規格和現況不一樣的地方，再動手：

規格寫的	實際做的	理由
React、TypeScript、R3F、Zustand	照現況：介面是 React、引擎是原生 three.js，沒有 TypeScript、R3F、Zustand	不為了聲音引入框架
Howler.js	不裝	環境音已經是 Web Audio；其他理由見〈⚖️ 動手前先想〉
初始不自動播放	保留階段 12-6 的「聲音預設打開」，第一次點擊畫面時才啟動	階段 12 已經做的決定優先；`docs/AUDIO.md` 寫明差異與改回去的方法

曲名：Claude 用 `ffprobe` 讀檔案的標籤。兩首 Floating Shrine 的標籤一樣，但內容不同。用頻譜重心分出亮暗：比較亮的放白天，比較暗的放夜晚，加上「昼 / 夜」副標區分。你的五首也可以用同一個方法分：最暖的放黃昏，其餘兩首當手動選。

響度：用 `ebur128` 量每一首。參考成品當時的黃昏曲子響度範圍 21 LU，從 −35 dB 漸強到 −13.5 dB，峰值 +3.6 dBFS。只調音量對不齊（開頭太小、後段太大），所以多過一個壓縮器。每一首用 `trim` 對到約 −18 LUFS，總輸出加限幅器。

這些是數字對齊，沒有用耳朵聽過。Claude 沒辦法聽，`docs/AUDIO.md` 把這一點寫成限制。

📋 貼給 AI（13-3：預覽畫面是空的）

```
貼給 AI
```

〈附上截圖〉

這裡都看不到

開發伺服器一直在跑，只是瀏覽器面板還沒有載入任何網址。Claude 把面板打開到 `http://localhost:4324/`。

📋 貼給 AI（13-4：部署成 Artifact）

貼給 AI

你只需要修改：沒有。Artifact 是 Claude 的私人分享頁面，這一步可以跳過。

```
幫我部署成 Artifact
```

Astro 的輸出要先轉換：絕對路徑改成相對路徑，`_astro/` 改名 `assets/`。步驟寫在 `docs/AUDIO.md` 第 10 節。 Claude 先用靜態伺服器跑一次轉換後的檔案，0 個錯誤才發布。

第 1 版拿掉了坂本龍一那首，黃昏改放白天那首。你的五首如果都是自己有權使用的，就沒有這個問題。

📋 貼給 AI（13-5：預設曲目與音效音量）

貼給 AI

你只需要修改：曲名換成你要預設播的那首。原本的指令指定的是坂本龍一那首，你換成自己的曲子，或改成「預設隨時段」。最後的成品是預設隨時段（〈上線之後〉）。

```
預設都幫我播放 Merry Christmas..

然後其他音效的聲音可以稍微小一點
```

- 預設改成手動模式、選那首，換時段不換歌
- 環境音（海浪、雨、雷）0.7 改成 0.45，音效 0.8 改成 0.6
- Artifact 第 2 版又把那首放回去，連結是私人的。見〈⚖️ 動手前先想〉

踩到的問題：只改預設值沒有用。瀏覽器裡已經存了舊的偏好（環境音 0.7、自動模式），會蓋過新的預設。另外模式的讀法寫死成「不是手動就是自動」，沒有存檔時根本不看預設值。修法是存偏好的名稱換成新的 `shrine.audio.v2`，模式改成讀預設。測試時先故意塞一份舊偏好，確認新預設會生效。

✅ 怎麼知道做對了

- [ ] 聲音面板的選曲有六個選項：隨時段，加上你的五首

- [ ] 選「隨時段」，切到白天、黃昏、夜晚，各自播你放在那個時段的曲子。手動選的兩首只在手動選曲時播
- [ ] 第一次點畫面之後，音樂慢慢淡入
- [ ] 黃昏切到夜晚，舊的那首慢慢淡出、新的那首慢慢淡入，中間音量沒有凹下去
- [ ] 快速連按時段按鈕十幾次，最後只剩一首在播
- [ ] 選了一首固定的曲子之後切時段，曲子不變；選回「隨時段」才換
- [ ] 重新整理之後，你調過的音量與模式都還在
- [ ] 請 Claude 跑 `npm test`，聲音的測試全部通過

這張是參考成品只有三首時的面板，所以選曲只有四個選項。你放五首會有六個。選中「隨時段」時，下面一行寫著現在播的是哪一首。三條滑桿是總音量、音樂、海浪與雨。

參考成品當時（三首）的實測（headless Chrome：沒有視窗、由程式操作的 Chrome。真的播放）：

情境	結果
黃昏切到夜晚，2.5 秒時	兩首的音量是 0.706 與 0.708，總功率 1.00
快速連切 12 次（每 150 ms 一次）	最多 3 首同時存在，最大總功率 1.00，最後只剩一首在播

情境	結果
10 分鐘、206 次隨機操作	記憶體 29 到 34 MB 來回，沒有往上爬；0 個錯誤
真的在播	4 秒內播放位置前進 4.00 秒

🆘 如果沒有成功

你可能會看到	通常是因為	怎麼辦
改了預設值，打開還是舊的設定	瀏覽器存了舊的偏好，蓋過新的預設	「改了預設值時，存偏好的名稱也要換一個新的」
有一首特別大聲或特別小聲	五首的響度不一樣	「請用 ebur128 量五首的響度，把它們對齊到同一個值」
「隨時段」播到手動選的那兩首	設定裡那兩首也標了時段	「手動選的兩首請不要對應任何時段，只在手動選曲時播放」
一首曲子開頭很小聲、後面很大聲	這首曲子本身從頭到尾漸強	「只調一個音量對不齊，請加一個壓縮器」
驗證腳本說「沒有任何音樂實例」	腳本在網頁裡找 `<audio>`，但音樂元素沒有放進網頁	「請改讀聲音系統自己的狀態，不要在網頁裡找元素」
Claude 開始安裝 Howler 或 Zustand	照規格的第 02 節做了	「先不要安裝。請先列出規格和現有架構不一樣的地方，說明每一個套件要解決什麼問題，再決定要不要裝」

這一步失敗可以跳過嗎：可以。後面的階段不依賴背景音樂。

📖 為什麼這樣做

🎯 這一步的重點

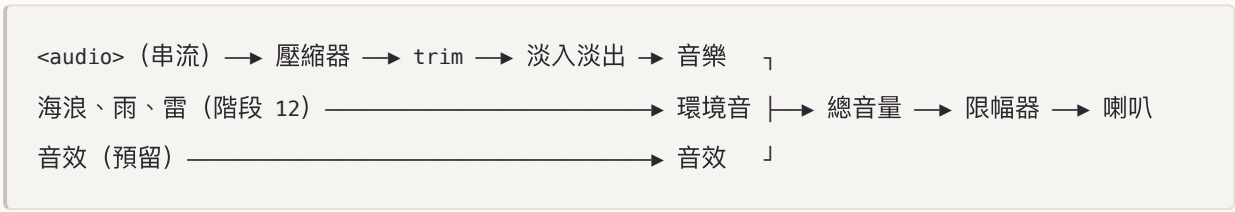
規格寫了某個套件，先問那個套件做不做得到規格要的事。

🧭 這一步在做什麼

> **名詞**：LUFS 是量「人耳聽起來多大聲」的單位。數字越接近 0 越大聲。
> 你需要知道到什麼程度：知道每一首曲子要對到差不多的 LUFS，換曲時才不會忽大忽小。量的工具是 ffmpeg 的 `ebur128`，由 Claude 跑。

五首都拉到約 −18 LUFS，留空間給環境音與音效。總輸出最後加一個限幅器，雷聲疊在音樂上時不會過載。真機上聽到哪一首特別大聲，改 `config.js` 的 `trim`。

🧩 它在建構流程的位置



💡 那段指令為什麼那樣寫

規格第 01 節寫了「優先使用既有架構，不要重新建立重複的場景狀態」。第 02 節卻要求採用 Howler.js，既有架構裡沒有它，兩節互相衝突。 Claude 照第 01 節的原則處理第 02 節：先列差異，說明 Howler 做不到等功率淡化，所以不裝。你看得到這張差異表，才有機會在安裝之前決定。

13–1 排在 13–2 之前，是因為授權問題在寫程式之前就能判斷。等網站做好、要部署了才想到，要換的就不只是一個檔案，還有測試、截圖和文件。

階段 14　做登入介面，後端先用假的

📖 讀完約 12 分鐘　🛠️ 動手約 40 分鐘

- **這一步一定要懂**：介面只呼叫六個函式，後端是真的還是假的，介面不知道。這一版的「需要登入」只是介面流程，不是權限控管。
- **這一步可以先不懂**：`<dialog>` 元素的細節、sessionStorage。
- **做完後你要能用自己的話回答**：為什麼按「一起祈願」時，沒登入的人會直接看到登入視窗，而不是先按住一秒才被擋？

⚖️ 動手前先想

「需要登入」算不算權限控管？

不算。假後端任何人都能登入。就算換成真的 Supabase，繪馬和祈願還是存在瀏覽器裡。任何人打開開發者工具都能略過介面直接改。

要擋得住，限制要寫在資料表的權限規則裡（階段 16）。沒登入不能新增繪馬與祈願，每人每塊繪馬只能祈願一次。前端的檢查擋不住繞過網頁的人。這一條寫在 `CLAUDE.md` 的最上面。

🛠️ 動手做

🧰 這一步的產出

```
src/services/auth/client.js    ← 介面只 import 這個檔案，裡面是六個函式
src/services/auth/mock.js      ← 假後端：不寄信，登入碼一律 123456
src/app/components/AuthDialog.jsx    ← 登入視窗、左下的帳號按鈕
```

畫面上應該看到：沒登入時，繪馬卡片的祈願按鈕寫「登入後一起祈願」。點了跳出登入視窗：輸入信箱、收 6 位數登入碼。

📋 貼給 AI（14–1：登入介面）

貼給 AI

你只需要修改：沒有。整段照貼。

> 接著可以來幫我實作會員登入的功能，先設計會員登入的介面。
>
> 如果寫一張繪碼要送出或者要一起祈願，就需要登入才行

Claude 做了幾個預設的選擇，實際做的時候沒有改：

問題	選擇	理由
登入方式	信箱收 6 位數登入碼＋Google，不用密碼	Supabase 都原生支援；沒有密碼，就沒有密碼外洩、忘記密碼這些事要處理
後端	先做假的	接 Supabase 要你的專案與確認，Claude 不能自己建雲端資源
寫繪馬什麼時候要登入	寫的時候不用，按「掛上繪馬」才要；登入後自動掛上	先寫完再要求登入，內容不會因為登入而不見
祈願什麼時候要登入	按鈕先換字，點了直接開登入；登入後不會自動祈願	不讓人按住一秒才被擋；祈願要自己按住
視窗	原生 `<dialog>`	開著時背後不能操作、焦點留在裡面、Esc 關閉，都不用自己寫

第一列的登入方式在階段 15 改掉了：真的 Supabase 改成點信裡的連結。假後端維持 6 位數登入碼。

> **名詞**：OTP（一次性密碼）是每次登入時寄給你的一組短碼，用過或過期就失效。這一步的「6 位數登入碼」就是它。
> 你需要知道到什麼程度：知道使用者不用設定密碼，每次登入收一次信。

Claude 寫的時候先想到一個問題。Esc 會同時被登入視窗和繪馬卡片收到。關登入視窗時卡片也會一起關，寫到一半的內容就沒了。修法是登入視窗裡的按鍵不往外傳。測試「Esc 只關登入視窗，卡片與內容都還在」通過。

測試時踩到兩個問題，都是測試環境造成的，登入流程本身沒問題：

症狀	原因	修法
「按住祈願 2.5 秒」沒成功	SwiftShader 下，卡片動畫每幀最多前進 0.05 秒，1.1 秒的集氣要更久的真實時間	改按 6 秒
「登入後票數沒變」偶爾失敗	頁面在模擬其他人祈願，票數本來就會變	改看按鈕的字

✅ 怎麼知道做對了

- [ ] 沒登入時，祈願按鈕寫「登入後一起祈願」；點了直接跳出登入視窗
- [ ] 輸入錯的信箱格式，有說人話的錯誤訊息
- [ ] 登入碼打錯，有錯誤訊息；打 `123456` 登入成功
- [ ] 寫一張繪馬、按「掛上繪馬」跳出登入；登入後繪馬自動掛上去
- [ ] 寫到一半的繪馬，按 Esc 關掉登入視窗，卡片和內容都還在
- [ ] 手機寬度下，登入視窗從畫面底部滑上來，按鈕都 ≥ 44 px

視窗下方的虛線框寫著這是原型、不會真的寄信、登入碼一律 123456。背後的繪馬卡片還開著，關掉登入視窗就回到它。

參考成品的驗證：瀏覽器流程 14 項全部通過，0 個錯誤。

🆘 如果沒有成功

你可能會看到	通常是因為	怎麼辦
按 Esc 關登入視窗，繪馬卡片也一起關了	Esc 同時被登入視窗和卡片收到	「登入視窗裡的按鍵不要往外傳，Esc 只關登入視窗」
測試說「按住祈願」沒成功	測試用的軟體顯示卡很慢	「這是測試環境的限制。請把按住的時間拉長，或改成等祈願完成的條件」
測試說「登入後票數沒變」偶爾失敗	頁面在模擬其他人祈願	「請改看按鈕的字，不要看票數」

這一步失敗可以跳過嗎：不行。階段 15 只換後端，介面就是這一步做的。

📖 為什麼這樣做

🎯 這一步的重點

介面只綁在六個函式上。換後端時，介面一行都不用改。

🧭 這一步在做什麼

先做登入的樣子，後端用一個假的。假後端關分頁就登出。

六個函式的名稱和 Supabase 的對應，寫在 `client.js` 的開頭。階段 15 換成真的 Supabase 時，只要換掉這一個資料夾裡的實作。

🧩 它在建構流程的位置

```
繪馬卡片 ──▶ requireLogin() ──▶ 登入視窗 ──▶ client.js 的六個函式 ─┬─▶ mock.js（現在）
                                                                    └─▶ supabase.js（階段 15）
```

💡 那段指令為什麼那樣寫

「先設計會員登入的介面」的「先」，讓這一步可以完全在自己電腦上完成。接真的後端需要你在 Supabase 建專案，那是要你自己點頭的事（操作卡 2 的第三層）。下一輪才說「真的接上」。

階段 15　接上 Supabase 的信箱連結登入

📖 讀完約 20 分鐘　🛠️ 動手約 60 分鐘

> - **這一步一定要懂**：`.env` 只能放可以公開的那把金鑰。`service_role` 那一把能繞過所有權限，不能放進前端。依賴第三方後台的設計，動手前先打開後台確認那個設定改得到。
> - **這一步可以先不懂**：PKCE 與 implicit 兩種登入流程的細節、Supabase 錯誤代碼的對照。
> - **做完後你要能用自己的話回答**：原本設計的是「輸入 6 位數登入碼」。為什麼改成「點信裡的連結」？

⚖️ 動手前先想

設計依賴第三方後台的某個設定，動手前要不要先確認？

要。下面 15-1 的「輸入 6 位數登入碼」是照舊版後台的印象設計的，前提是信件範本可以改。照著做下去，程式寫好、`.env` 填好，打開後台才會看到 Set up custom SMTP to edit templates。動手前先請你截一張範本頁，或先問「這個設定現在改得到嗎」。這樣就不會先做一版用不了的流程。

登入流程用 PKCE 還是 implicit？

下面 15-6 會先選 PKCE，15-9 再換成 implicit。

> **名詞**：**權杖**（access token，格式叫 JWT）是你登入成功時 Supabase 發的一段很長的字串。之後每一個請求都帶著它，資料庫靠它知道「現在是誰」。
> 你需要知道到什麼程度：知道它一小時後過期，supabase-js 會自動換新。它等同你一小時內的登入狀態，不要貼給別人、不要放進文件、不要截圖。

> **名詞**：**PKCE** 是一種登入流程的安全做法。跳轉回網站時，網址上只帶一次性的代碼，不帶登入權杖。
> 你需要知道到什麼程度：知道它的代價是連結要在同一個瀏覽器打開。好處是網址被別人看到，也拿不到你的登入狀態。

> **名詞**：**implicit** 是另一種登入流程：連結回到網站時，權杖直接放在網址 `#` 後面。
> 你需要知道到什麼程度：知道 `#` 後面的部分不會送到任何伺服器，supabase-js 讀完就從網址清掉。在哪個瀏覽器打開連結，就在那個瀏覽器登入。

兩種的差別：

	PKCE	implicit
連結回到網站時，網址上帶什麼	一次性的代碼	權杖，放在 `#` 後面
換一個瀏覽器打開連結	登入失敗	照樣登入
收到信之後又按了寄送	已經寄到的那封失效	不受影響
權杖經過伺服器、被記進伺服器的紀錄時	紀錄裡只有用過的代碼	沒有這層保護

PKCE 保護的情況，這個網站沒有。它沒有自己的伺服器，`#` 後面的部分也不會送到任何伺服器。換成 implicit 的風險很小。換來的是學員與使用者不會被「多按一次就失效」「換瀏覽器就失效」卡住。有自己伺服器的網站，或用的框架會在伺服器端處理登入，就要回頭用 PKCE。

開發用的密碼登入，會不會跟著上線？

不會，而且要驗證過。HTML 與 JS 都用 `import.meta.env.DEV` 判斷，`npm run build` 建置出來的網站裡搜不到這段。只寫在 JS 裡的話，HTML 的輸入框還是會出現在正式網站上。兩邊都要包，建置後再搜一次。

哪一把金鑰可以放前端？新資料表要不要自動對外？

> **名詞**：**anon key**（新後台叫 Publishable key）是可以公開的那一把金鑰。**service_role key**（新後台叫 Secret key）是不能外流的那一把。
> 你需要知道到什麼程度：知道 anon key 會被打包進網頁，任何人按 F12 都看得到。這是設計上就這樣。service_role key 無視所有權限規則，只能放在伺服器上。

> **名詞**：**RLS**（列層級安全）是資料表上的權限規則，管「這個人能碰這張表的哪幾列」。
> 你需要知道到什麼程度：知道公開金鑰拿得到什麼，完全由 RLS 決定。沒開 RLS 的資料表，拿著公開金鑰的人什麼都讀得到、寫得進去。

金鑰	放哪裡	被別人拿到會怎樣
anon / Publishable	`.env`，會被打包進網頁	能做的事由 RLS 決定。規則寫對，拿到也碰不到別人的資料
service_role / Secret	只能放伺服器	讀走、改掉、刪光所有資料

`.env` 已經在 `.gitignore` 裡，不會進版本控制，也不會進打包給學員的 zip。

> Automatically expose new tables 勾著時，之後建的每一張新資料表，一建好就能透過公開金鑰讀寫。那張表如果沒開 RLS，任何人都讀得到、寫得進去。參考成品建立時是勾著的，階段 16 的 SQL 在建表時開 RLS，並在一開始收回自動給的權限。這靠的是每一張都記得開。關掉它，是讓資料庫在你忘記時也不會對外。兩種做法你都要知道代價。

🛠️ 動手做

🧰 這一步的產出

```
.env                              ← 兩個值：Project URL 與可以公開的金鑰（不進版本控制）
src/services/auth/client.js       ← 依 .env 選後端：填了用 Supabase，沒填用假的
src/services/auth/supabase.js     ← Supabase Auth：點信裡的連結登入（EMAIL_MODE = 'link'，implicit）
src/services/auth/errors.js       ← Supabase 的錯誤代碼對照成介面的錯誤訊息
src/app/components/AuthDialog.jsx     ← 開發時多一個「用測試帳號的密碼登入」（建置後的網站沒有）
```

畫面上應該看到：和階段 14 一樣的登入視窗。差別是按鈕寫「寄登入連結」，信真的會寄到你的信箱，點信裡的連結回到網站就登入了。在哪個瀏覽器點，就在那個瀏覽器登入。用 `npm run dev` 開的網站，登入視窗下方多一個「開發用：用測試帳號的密碼登入」。

📋 貼給 AI（15-1：真的接上會員系統）

貼給 AI

你只需要修改：沒有。整句照貼。

> 協助我實作並真的接上會員系統！

Claude 先問兩件事，因為雲端資源與帳號是你的，它不能自己決定。實際的回答：

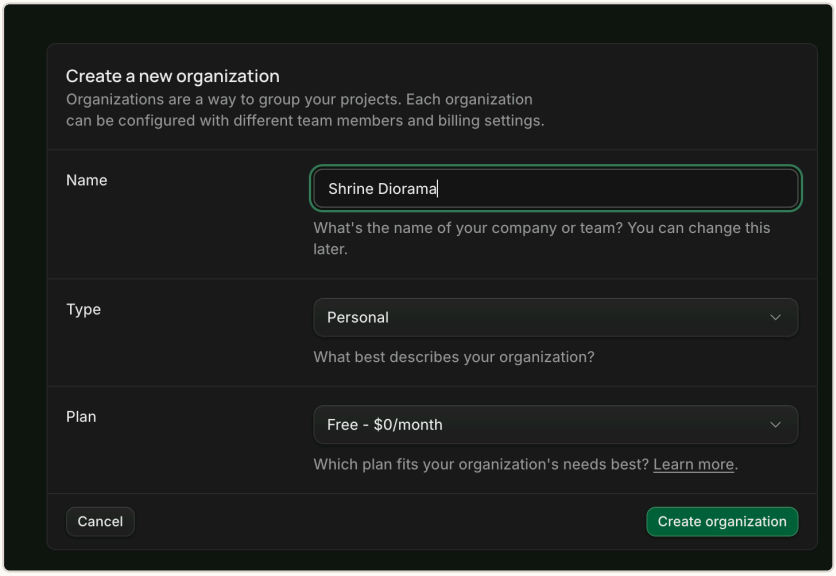
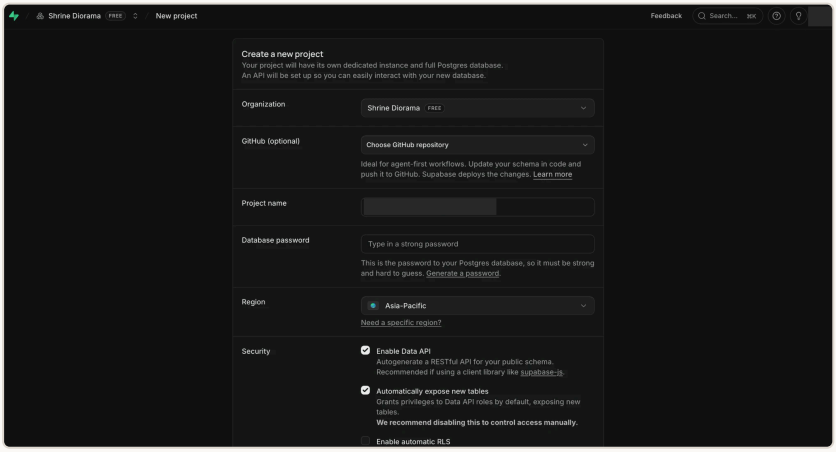
問題	實際的決定
用新的 Supabase 專案，還是沿用手上現有的	新的。會員與權限才不會和其他專案混在一起
先接哪一種登入	先只接信箱，Google 之後再接

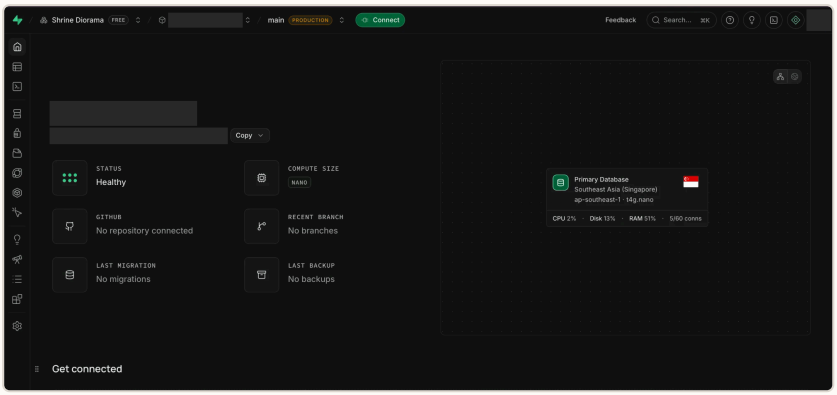
Claude 做的事：

- 安裝 `@supabase/supabase-js` 2.117.3，版本鎖死。最新的 2.118.0 發布不到兩週，先不用。`npm audit` 0 個問題
- `auth/` 拆成 `client.js`（依 `.env` 選後端）、`supabase.js`、`mock.js`、`errors.js`。介面完全沒改：階段 14 把介面只綁在六個函式上，這一輪換後端就只動這個資料夾
- Google 登入的程式寫好了，按鈕先藏起來（`.env` 的 `PUBLIC_AUTH_GOOGLE`）
- `.env.example` 寫清楚只能放公開金鑰，不能放 `service_role`

你自己做（15-2：建立 Supabase 專案）

到 supabase.com 登入。下面三張是實際建立時的畫面，專案名稱與網址已經遮掉。

步驟	參考畫面
一、建立組織	 Name 填一個你認得的名稱，Type 選 Personal，Plan 選 Free – $0/month，按右下角 Create organization。
二、建立專案	 Project name 自己取。Database password 按 Generate a password 產生。Region 選 Asia-Pacific。最下面的 Security 區塊見下方說明。

步驟	參考畫面
三、專案建好	 STATUS 是 Healthy 就代表建好了。右邊的 Primary Database 寫著 Southeast Asia (Singapore)，這是選 Asia-Pacific 之後實際落在的位置。

Database password 存進你的密碼管理器。不要寫進專案，也不要放進 `.env`，網站用不到它。

Region：實際選的是 Asia-Pacific，專案落在新加坡（`ap-southeast-1`）。東京也可以，選離你的使用者近的就好。

Security 區塊的三個選項，實際建立時的狀態：

選項	實際的設定	它做什麼
Enable Data API	勾	自動為資料表產生網路上讀寫用的介面。網站用 supabase-js 連資料庫需要它
Automatically expose new tables	勾	之後建的每一張新資料表，都自動透過上面那個介面對外。畫面上 Supabase 自己寫著建議關掉
Enable automatic RLS	沒勾	新資料表自動開啟 RLS

第二項的意思與代價，見本階段開頭的〈⚖️ 動手前先想〉。

📋 貼給 AI（15-3：對外 API 要不要另外架後台）

Claude 在接 Supabase 的時候，可以先問下一步要怎麼規劃：

貼給 AI

你只需要修改：沒有。整句照貼。先把模式切到 Plan。

```
我希望這個專案還可以有 API 對外開放使用；這樣的設計是不是要搭建後台？
```

Claude 的規劃寫在 `docs/IMPLEMENTATION_PLAN.md` 第 19 節。其中「只有登入才拿得到的資料」是階段 16，公開的 API 是階段 17。

📋 貼給 AI（15-4：.env 要填什麼）

Claude 如果回英文，貼：

貼給 AI

```
請都用中文跟我互動～.env 要填些什麼呢
```

兩個值都在 Supabase 專案首頁上方的 Connect，或 Project Settings → API：

- Project URL
- 可以公開的那一把金鑰：新後台叫 Publishable key（開頭 `sb_publishable_`），舊後台叫 anon public

> ⚠️ 絕對不要複製 `sb_secret_` 或 `service_role` 那一把。理由見本階段開頭的〈⚖️ 動手前先想〉。

Claude 從 `.env.example` 複製出 `.env`，三行都空著，等你填。公開金鑰本來就會出現在網頁上。可以貼給 Claude 請它寫進 `.env`，也可以自己用編輯器填。填好、重啟開發伺服器之後，登入視窗裡的「原型」說明和 Google 按鈕都藏起來了，代表已經改用 Supabase。

你也可以照下面這段請 Claude 做，金鑰由你自己貼在等號右邊（整理版指令）：

貼給 AI

你只需要修改：等號右邊兩個值。等號左邊的名稱一個字都不要改，等號前後不要加空白，也不要加引號。

```
請從 .env.example 複製出 .env，填入下面兩行，第三行留空。
填完之後重新啟動開發伺服器，告訴我網站現在用的是 Supabase 還是假後端。
不要把金鑰的內容印出來。

PUBLIC_SUPABASE_URL=剛剛複製的 Project URL
PUBLIC_SUPABASE_ANON_KEY=剛剛複製的那把可以公開的金鑰
```

接著 Claude 用一個收不到信的測試信箱 `shrine-test@example.com` 寄一次，修正了兩個錯誤訊息的對照。這兩個訊息的意思見〈🆘 如果沒有成功〉的最後兩列。

📋 貼給 AI（15-5：Email 與 URL Configuration 在哪裡）

要設定後台，但找不到位置時，貼：

貼給 AI

〈附上截圖〉

```
Email 要如何設置呀？還有 URL Configuration i在哪裏
```

「i在哪裏」是「在哪裡」。Claude 給了三個直接連結，比「左邊第幾個圖示」穩，後台改版也找得到：

```
https://supabase.com/dashboard/project/<專案 ID>/auth/url-configuration
https://supabase.com/dashboard/project/<專案 ID>/auth/templates
https://supabase.com/dashboard/project/<專案 ID>/auth/providers
```

Claude 也給了信件範本的內容。它要你把 Magic Link 與 Confirm signup 兩個範本，換成有 6 位數登入碼 `{{ .Token }}` 的版本。

📋 貼給 AI（15-6：後台截圖顯示範本改不了）

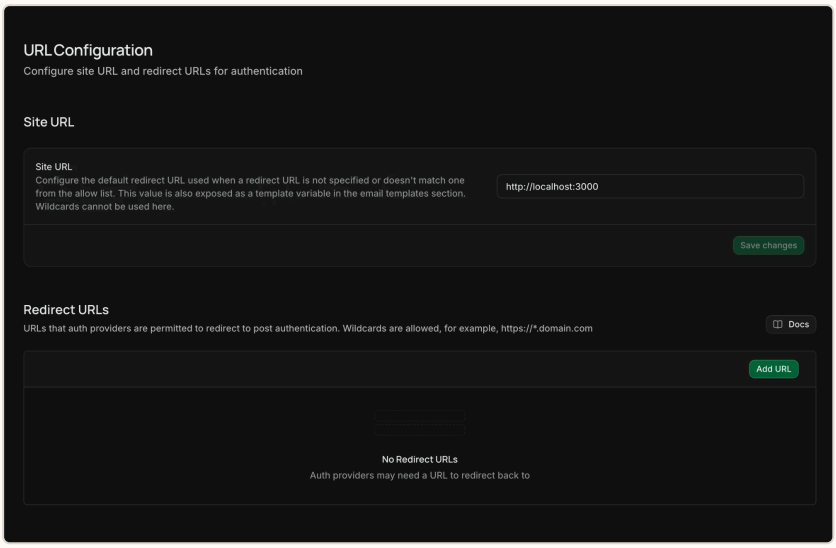
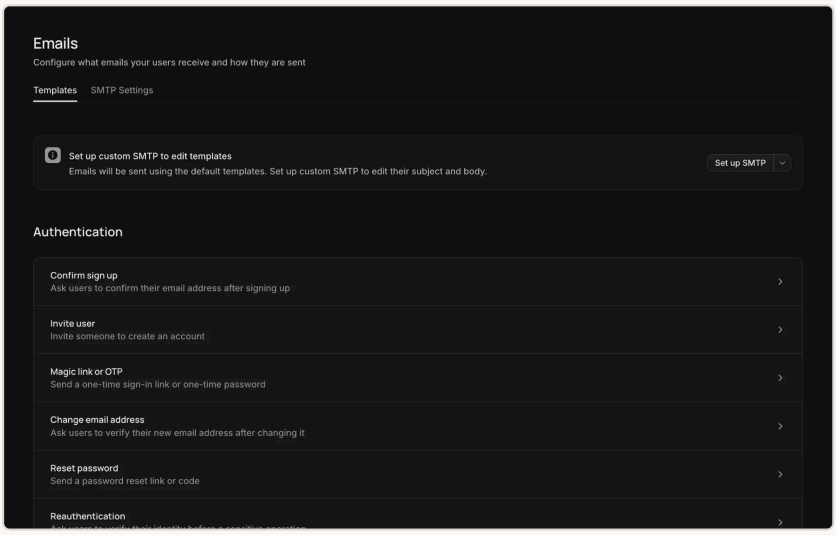
打開那幾頁，把三張截圖直接傳回去，不用加文字：

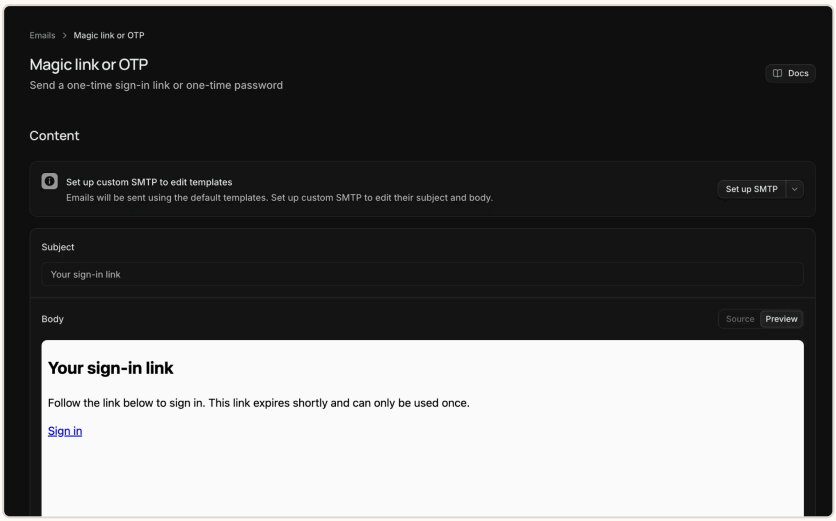
貼給 AI

〈附上截圖〉

〈附上截圖〉

〈附上截圖〉

截圖	畫面
URL Configuration	 Site URL 是預設的 `http://localhost:3000`，不是這個專案的 4324。Redirect URLs 是空的。
Emails → Templates	 最上面的提示框寫著 Set up custom SMTP to edit templates。沒有接自己的寄信服務，就不能改範本。

截圖	畫面
Magic link or OTP 範本	 內建範本只有一個 Sign in 連結，沒有 6 位數的碼。

這一輪推翻了 15–1 的設計。「輸入 6 位數登入碼」的前提是範本可以改，這個前提不成立。

Claude 列了兩個選項：接自己的寄信服務（SMTP）再改範本，或先改成點信裡的連結。選後者：不用設定寄信服務，馬上能測。公開上線時反正要接自己的寄信服務（內建的只寄給專案成員），到時候再切回 6 位數。

Claude 改的地方：

- `EMAIL_MODE`：預設 `'link'`。`.env` 加 `PUBLIC_AUTH_EMAIL=code` 切回 6 位數。兩種程式都留著，假後端維持 6 位數
- 寄信時帶 `emailRedirectTo`，連結回到這一頁
- PKCE：連結要在寄信的同一個瀏覽器打開。畫面上寫清楚「用手機收信、在手機上點，電腦這邊不會登入」。這一點在 15–9 被推翻了
- 在別的分頁點了連結：原本那一頁也會跟著登入，登入視窗自己關掉。接著做原本要做的事（掛繪馬、祈願）
- 連結過期或已經用過：回來的網址帶 `error_code=otp_expired`。登入視窗打開，顯示「登入連結過期或已經用過，請再寄一次」，網址清乾淨

已知限制：寫到一半的繪馬存在 sessionStorage。連結如果在新分頁打開，sessionStorage 不共用，寫到一半的內容拿不回來。這一點還沒改。

📋 貼給 AI（15–7：常常找不到 URL Configuration）

實際這一則傳了四張截圖，第四張是後台左側的選單：

貼給 AI

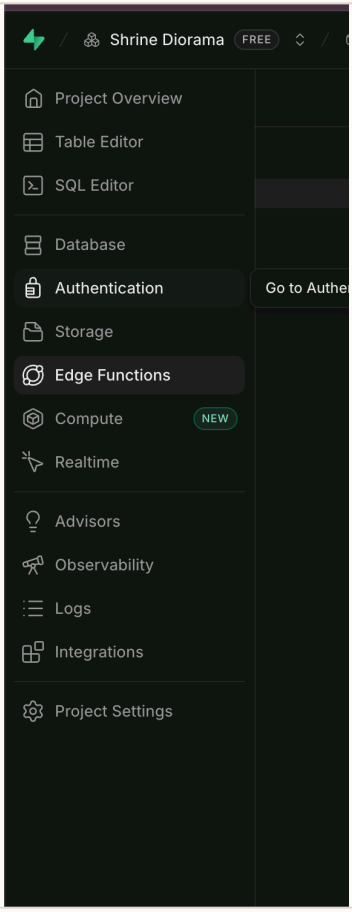
《附上截圖》

《附上截圖》

《附上截圖》

《附上截圖》

如附圖四，常常會找不到 `URL Configuration`

截圖	畫面
後台左側選單	 第一層選單裡沒有 URL Configuration。它在 Authentication 裡面。

Claude 給了兩種找法：

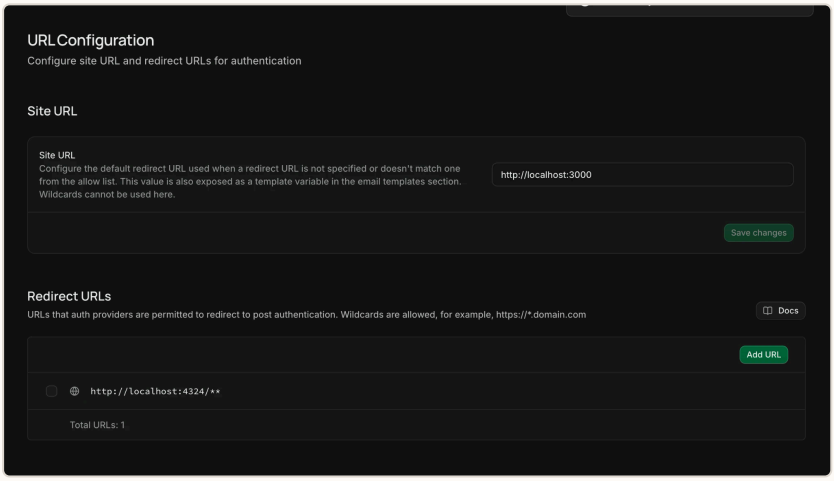
1. 直接開連結（`<專案 ID>` 換成你的）。後台改版、選單換位置，這個網址通常還是對的：

```
https://supabase.com/dashboard/project/<專案 ID>/auth/url-configuration
```

2. 從選單走：左側 Authentication → 第二層選單的 CONFIGURATION → URL Configuration。第二層的名稱和這裡寫的不一樣，就截一張 Authentication 打開後的畫面給 Claude。

你自己做（15-8：設定網址）

1. 打開 URL Configuration（15-7 的連結或選單路徑）
2. Site URL 從 `http://localhost:3000` 改成 `http://localhost:4324`，按 Site URL 那一格右下角的 Save changes
3. Redirect URLs 按 Add URL，填 `http://localhost:4324/**`，存檔

狀態	參考畫面
第一次存檔後	 Redirect URLs 加好了，Site URL 還是 3000。兩個區塊各有自己的 Save changes，要分開存。

寄信時程式已經指定「回到 4324」，所以 Site URL 沒改，大部分時候也能登入。指定的網址不在 Redirect URLs 名單裡，或之後信件範本用到網站網址時，就會退回 Site URL，跳到 `localhost:3000`。那裡沒有網站。

信件範本不用改。接上自己的寄信服務、要切回 6 位數登入碼時，請 Claude 照 `docs/AUTH.md` 第 1 節第 4 步做。

📋 貼給 AI（15-9：第一次真的寄信）

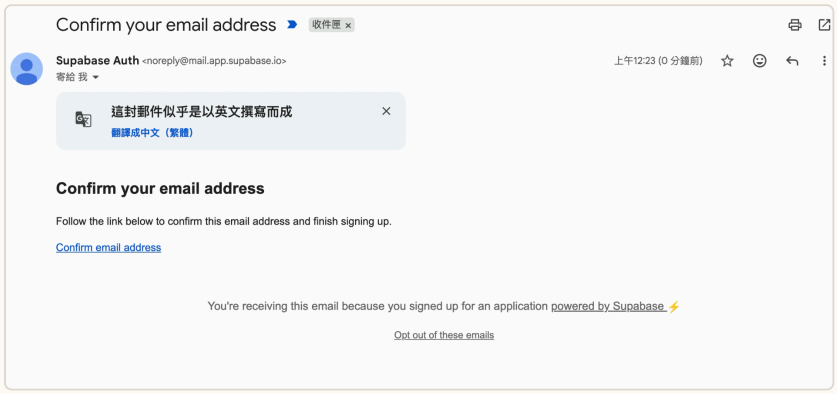
網址設好之後，在預覽視窗輸入你自己的信箱，按「寄登入連結」。收到第一封信之後又按了幾次：

貼給 AI

《附上截圖》

《附上截圖》

然後一直卡在寄得太頻繁，等一分鐘再寄。

截圖	畫面
第一封信	 新帳號第一次收到的是 Confirm your email address，連結文字是 Confirm email address。不是 Magic link 範本的 Sign in。

這一輪找到兩個問題。

問題一：錯誤訊息不對。先想一想：Supabase 回 429（太多請求）時，一定是「等一分鐘就好」嗎？

- 症狀：畫面一直說「寄得太頻繁，等一分鐘再寄」，等了一分鐘還是一樣
- 原因：內建寄信服務的上限是整個專案每小時約 2 封，不是一分鐘。Claude 把 Supabase 回的所有 429 都對到「等一分鐘」
- 修法：分成三種訊息。同一個信箱 60 秒內重寄，顯示「N 秒後再試」（秒數從錯誤訊息裡讀）。整個專案這一小時的額度用完（`over_email_send_rate_limit`），說明暫時寄不出去，已經收到的那封還能用。其他的照舊
- 你可以這樣請 Claude 查：「請列出 Supabase 登入可能回的錯誤代碼，每一種對應畫面上哪一句提示」

問題二：多按幾次寄送，第一封信的連結就失效了。這一個還沒有人回報，是 Claude 看到「又按了幾次」想到的。

- 原因：PKCE 在按下寄送的那一刻，會產生一組成對的驗證碼。一半送給 Supabase，另一半存進這個瀏覽器，點連結時兩半要對得上。之後每按一次寄送（就算被擋下），瀏覽器裡那一半就換掉一次。第一封信的連結就對不上了
- 另一個原因：預覽視窗和 Gmail 打開連結的 Chrome 是兩個不同的瀏覽器。PKCE 要在同一個瀏覽器打開，這裡也會失敗
- 修法：登入流程改成 implicit。在哪個瀏覽器打開連結，就在那個瀏覽器登入。多按幾次寄送，也不會讓已經寄到的信失效

這是被推翻的判斷。15-6 選了 PKCE，畫面上寫「請用這個瀏覽器打開連結」。換成 implicit 的理由見本階段開頭的〈⚖️ 動手前先想〉。

這種情況只能等。第一封信是在 PKCE 下寄的，改成 implicit 也救不回來。這一小時的額度又用完了，要等約一小時才能再寄。

📋 貼給 AI（15-10：沒辦法登入時，用開發用的密碼登入）

寄信額度用完、又急著測的時候，用這一招。

現在沒辦法登入

貼給 AI

那怎麼辦...我現在沒辦法登入了

Claude 的做法：不經過寄信，在 Supabase 後台直接建一個「已驗證、有密碼」的測試帳號。登入視窗加一個開發用的密碼登入。

- 只在 `npm run dev`、而且接上 Supabase 時出現。HTML 用 `{import.meta.env.DEV && …}` 包起來，JS 也用 `import.meta.env.DEV` 判斷。建置出來的網站裡沒有這段（`dist/index.html` 搜不到 `loginDev`）
- 測試帳號的密碼由你自己設，不經過 Claude，也不寫進專案
- 階段 19 的資安檢查要用兩個帳號互測「看不到別人的資料」，本來就需要測試帳號

密碼登入的按鈕跑版

貼給 AI

（附上截圖）

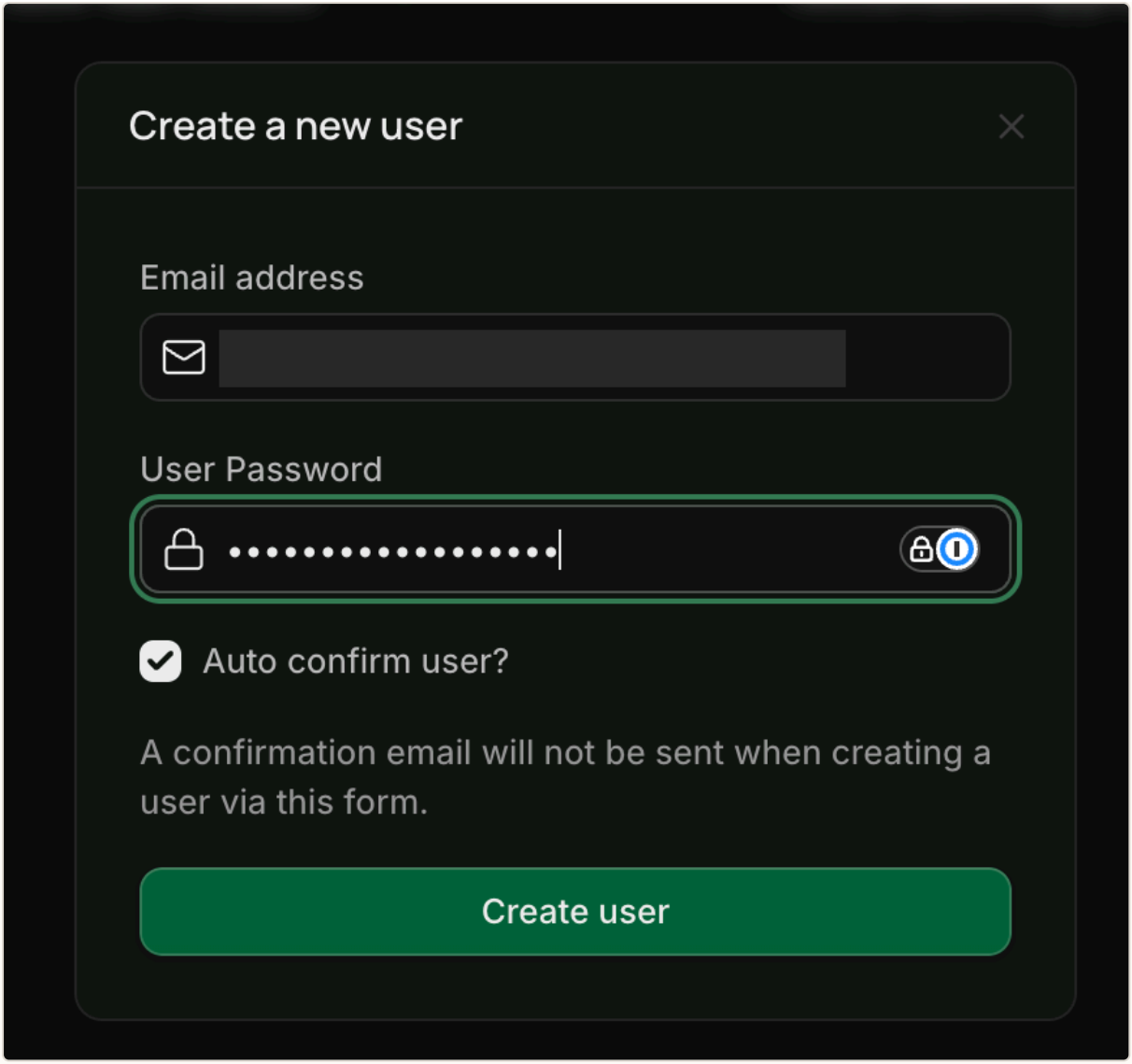
按鈕有點跑版

「用密碼登入」的按鈕改成和上面的「寄登入連結」一樣撐滿整列。說明文字的行距調成一般行距，虛線框底部多留一點空間，按鈕不再貼著虛線。

你自己做（15–11：建測試帳號、用密碼登入）

在 Supabase 後台：

1. 左側 Authentication → Users
2. 右上角 Add user → Create new user
3. Email address：用 Gmail 的話，可以在 `@` 前面加 `+shrine`，例如 `你的帳號+shrine@gmail.com`。信一樣寄到原本的信箱，Supabase 會把它當成另一個帳號，不會和還在等驗證的那個帳號衝突
4. User Password：自己設一組，存進密碼管理器。不要貼給 Claude，也不要寫進專案
5. 勾 Auto confirm user
6. 按 Create user



勾了 Auto confirm user，視窗下方寫著不會寄確認信。這個帳號完全不用收信。

在網站上：

1. 用 `npm run dev` 開的網址（`http://localhost:4324/`），點左下角「登入」
2. 電子郵件填剛才的信箱
3. 點開下方虛線框「開發用：用測試帳號的密碼登入」，輸入密碼，按「用密碼登入」

實測用這個方法登入成功，接著在階段 18 掛了第一塊附照片的繪馬（18–3）。

✅ 怎麼知道做對了

- [ ] Supabase 專案首頁的 STATUS 是 Healthy
- [ ] 你說得出 Security 區塊三個選項各自的意思，以及你的專案勾了哪幾個
- [ ] Database password 存在密碼管理器裡，專案資料夾裡找不到它
- [ ] Claude 回報網站改用 Supabase，而且沒有印出金鑰

- [ ] URL Configuration 的 Site URL 是 `http://localhost:4324`，Redirect URLs 有 `http://localhost:4324/**`
- [ ] 登入視窗的按鈕寫「寄登入連結」
- [ ] 用你自己的信箱寄一次，只按一次：新帳號收到的是 Confirm your email address。點信裡的連結，回到網站就登入了
- [ ] 測試帳號建好了，Auto confirm user 是勾的；用「開發用」的密碼登入成功，左下角出現帳號按鈕
- [ ] 請 Claude 執行 `npm run build`，確認建置出來的網站裡沒有開發用的密碼登入
- [ ] 請 Claude 把 `.env` 暫時改名成 `.env.bak` 再重啟：網站退回假後端（登入碼 123456）。看完請 Claude 改回來

已經實測過的：按鈕文字、過期連結回來時的訊息與網址、假後端登入流程 14 項、收到 Confirm your email address、測試帳號用密碼登入。

點信裡的連結登入，這一步當時沒有在本機再測一次。第一封信是 PKCE 時代寄的，改成 implicit 之後，先用測試帳號登入。同一個 implicit 的連結登入，後來在正式網站實測通過，見 20-7（正式網址）與 20-8（自己的網域）。

🆘 如果沒有成功

你可能會看到	通常是因為	怎麼辦
左側選單找不到 URL Configuration	它在 Authentication 的第二層	用 15-7 的直接連結
點了信裡的連結，跑到 `localhost:3000` 打不開	Site URL 還是預設值，或 Redirect URLs 沒有 4324	回 15-8 改好兩格，各自按 Save changes
「登入連結過期或已經用過，請再寄一次」	連結只能用一次，而且有時效	再寄一次，點最新那一封
畫面說這一小時寄不出去了	內建寄信服務整個專案每小時約 2 封	已經收到的信直接點。急著測就用 15-11 的測試帳號
「N 秒後再試」	同一個信箱 60 秒只能寄一次	等倒數結束
登入視窗沒有「開發用」那一塊	不是用 `npm run dev` 開的網站，或 `.env` 沒填（假後端不顯示）	確認網址是 `http://localhost:4324/`，`.env` 有填
密碼對，還是登不進去	建帳號時沒勾 Auto confirm user，帳號還在等驗證	在 Users 列表確認那個帳號的狀態，必要時重建一個，這次勾 Auto confirm user

你可能會看到	通常是因為	怎麼辦
填了 `.env`，網站還是用假後端	開發伺服器只在啟動時讀 `.env`	「請重新啟動開發伺服器」
重啟過了，還是用假後端	等號前後有空白，或值加了引號	「請檢查 .env 的格式：等號前後不能有空白，值不能加引號。不要把值印出來」
信箱格式明明對，畫面卻說「這個信箱收不到信」	`email_address_invalid` 也代表這個網域收不到信，例如 `example.com`	換一個真的收得到信的信箱
「網站還在測試，目前只寄給管理者的信箱」	內建寄信服務只寄給專案成員	自己測試用專案成員的信箱，或用 15–11 的測試帳號。要讓別人登入，要在 Authentication → Emails → SMTP Settings 接自己的寄信服務

這一步失敗可以跳過嗎：可以。`.env` 沒填時網站會用假後端，前面所有階段照常運作。但階段 16 之後都需要真的 Supabase。

📖 為什麼這樣做

🎯 這一步的重點

`.env` 只放 anon key：它會打包進網頁，拿到的人能做什麼由 RLS 決定。`service_role` 能繞過 RLS，不放進這個前端專案的任何檔案。

🧭 這一步在做什麼

把階段 14 的假後端換成真的 Supabase。介面沒改，只動 `auth/` 這個資料夾。

🧩 它在建構流程的位置

```
登入視窗 ──▶ client.js ──▶ .env 有填？ ──是──▶ supabase.js ─┬─▶ Supabase Auth（寄連結）
                                                            │      └ （npm run dev 才有）測試帳號的密碼
                                                            └否──▶ mock.js（登入碼 123456）
```

💡 那段指令為什麼那樣寫

15–4 整理的那段寫了「不要把金鑰的內容印出來」。公開金鑰被看到沒關係，但你要養成設定值不出現在對話與截圖裡的習慣。之後遇到不能外流的那一把，才不會順手貼出去。驗證只問「網站現在用的是 Supabase 還是假後端」。你要確認的是設定有沒有生效，不是金鑰長什麼樣。

15–6 沒有寫任何字，只傳截圖。Claude 能從截圖判斷，是因為它知道自己的設計依賴「範本可以改」。下一次設計要用到後台設定時，你可以一開始就請它列出：這個設計依賴後台哪幾個設定。然後你先去截圖確認。

15–9 照抄了畫面上的訊息「等一分鐘再寄」。只寫「寄不出去」，Claude 只知道有問題。有了訊息原文，才能拿去和實際的上限對照。 15–10 只有一句「我現在沒辦法登入了」。Claude 沒有叫你等一小時，而是找一條不經過寄信的路，而且這條路之後的資安檢查也用得到。

階段 16　只有登入才拿得到的資料

📖 讀完約 18 分鐘　🛠️ 動手約 50 分鐘

> - **這一步一定要懂**：權限寫在資料庫裡，不寫在網頁裡。API 用呼叫者自己的權杖查資料庫，不用能繞過權限的那把金鑰。同樣是 401，看回應內容就知道請求停在哪一層。
> - **這一步可以先不懂**：SQL 的語法、觸發器怎麼寫、Edge Function 的程式內容。照步驟貼就好。
> - **做完後你要能用自己的話回答**：「誰掛了哪一塊繪馬」為什麼不放在公開的繪馬表裡？

⚖️ 動手前先想

> **名詞**：**Edge Function** 是放在 Supabase 上、收到網路請求才執行的一小段程式。這一步要做的 `me` 就是一支。
> 你需要知道到什麼程度：知道它是「對外 API」的一種做法，網址是 `https://<專案ID>.supabase.co/functions/v1/<名稱>`。程式由 Claude 寫，你只負責貼上與部署。

帶著登入權杖呼叫 `me`，它回傳你自己掛的繪馬和祈願過的繪馬。帳號視窗裡的「我的參拜紀錄」就是用它拿資料。

> **名詞**：**觸發器**（trigger）是資料庫裡的一條規則：某張表新增一列時，自動執行一段程式。
> 你需要知道到什麼程度：知道這個案例的「記下誰掛的」和「票數加一」都是它做的。前端碰不到。

> **名詞**：**security definer** 是讓一段資料庫程式用「建立它的人」的權限執行，而不是用呼叫的人的權限。
> 你需要知道到什麼程度：知道它權限比較大，所以要搭配 `search_path = ''`。還要收回所有人直接呼叫它的權限。

誰掛了哪一塊，要不要放進公開的繪馬表？

不要。`wishes` 是公開的，任何人都讀得到每一列。如果它有一個 `user_id` 欄位，別人就能從牆上反查「這幾塊是同一個人掛的」。 RLS 管的是「哪幾列」，不是「哪幾欄」。同一列裡，公開的願望和私密的作者沒辦法分開管。所以作者另外放一張 `wish_owners`，只有本人讀得到。

新資料表的權限，要沿用自動給的，還是先全部收回？

先全部收回。階段 15 建專案時 Automatically expose new tables 是勾著的。新表會自動把所有權限給沒登入的人（anon）和登入的人（authenticated）。所以 SQL 一開始先 `revoke all`，再一個欄位一個欄位給：

- 沒登入：只能讀 `wishes`
- 登入：`wishes` 只能新增願望與署名兩個欄位；`prayers` 只能新增繪馬 id 一個欄位

沒有收回的話，前端可以直接寫 `votes`，也可以在 `prayers` 填別人的 `user_id`。

票數與「誰掛的」由誰來寫？

由資料庫自己寫，前端碰不到。前端寫得到的話，任何人都能把一塊繪馬的票數改成一萬。

資料	誰寫	前端為什麼改不了
`wish_owners`	掛繪馬時，觸發器用 `auth.uid()` 記下	前端對這張表沒有新增權限
`prayers.user_id`	預設值是 `auth.uid()`	前端沒有寫這個欄位的權限，policy 也檢查 `user_id = auth.uid()`
`wishes.votes`	有人祈願時，觸發器加一	前端對這個欄位沒有任何寫入權限

`auth.uid()` 是從請求帶的權杖讀出來的。權杖由 Supabase 簽發，前端偽造不了。觸發器用 `security definer`，才寫得進前端沒有權限的地方。搭配 `search_path = ''`，函式裡每個名稱都寫完整（`public.wishes`），別人沒辦法用同名的表偷換掉。

階段 15 問的是哪一把金鑰可以放前端。這裡換成 API 那一邊：用呼叫者的權杖，還是用 secret 金鑰查資料庫？

用呼叫者的權杖。Supabase 的預設範本支援 secret 金鑰，註解也寫了：帶 secret 金鑰的請求會繞過 RLS。 `me` 拿請求帶來的權杖，用公開金鑰加上這個權杖去查資料庫。資料庫看到的是「這位使用者」，查詢照樣經過 RLS。就算 `me` 的程式寫錯了，RLS 還在，拿不到別人的紀錄。用 secret 金鑰的話，程式裡任何一個條件寫錯，就會回傳所有人的資料。

同樣是 401，停在哪一層？

> **名詞**：**閘道**是請求進到 `me` 的程式之前，Supabase 先檢查的那一關。
> 你需要知道到什麼程度：知道閘道擋下的請求，`me` 的程式根本沒有執行。看回應內容就分得出是哪一邊回的。

回應內容	誰回的	代表什麼
`UNAUTHORIZED_NO_AUTH_HEADER`	Supabase 閘道	請求還沒進到 `me` 的程式
`missing_token`	`me` 的程式	進到程式了，但沒帶權杖
`invalid_token`	`me` 的程式	帶了權杖，但無效或過期

Verify JWT with legacy secret 開著時，閘道只接受舊版密鑰簽的權杖。新專案的登入權杖用新的簽章金鑰，可能被閘道一律擋掉。所以關掉它，改由 `me` 自己驗證權杖。

錯誤訊息的提示叫你 `GRANT … TO anon`，要不要照做？

不要。讀 `prayers` 被擋下時，錯誤回應裡附了一句提示，教你把讀取權限給沒登入的身分（anon）。那是資料庫的通用提示。它只知道「這個請求被拒絕了」。它不知道這個拒絕是你在上面「先全部收回」那一段設計好的。照抄那一行，沒登入的人就讀得到所有人的祈願紀錄。

看到權限錯誤時，先問：這個身分本來就該被擋嗎？該擋的話，錯誤就是正確的結果。把整段錯誤貼給 Claude 時，也加一句「不要照提示改權限，先告訴我這個拒絕是不是設計好的」。

🛠️ 動手做

🧰 這一步的產出

```
supabase/migrations/001_wishes_prayers.sql   ← 三張資料表、權限、觸發器、示範繪馬
supabase/functions/me/index.ts               ← 對外 API：GET /functions/v1/me，我的參拜紀錄
src/pages/api.astro                          ← API 文件頁 /api
```

畫面上應該看到：掛上新繪馬、祈願，都會寫進 Supabase 的資料表。左下帳號視窗多一個「我的參拜紀錄」。繪馬牆本身這時還是讀示範資料，不是讀資料庫。階段 17 換成讀資料庫（遷移計畫的最後一步）。

這一步有一半是你在 Supabase 後台親手做的。Claude 寫 SQL 與程式，你貼上、執行、部署。

📋 貼給 AI（16–1：只有登入才拿得到的資料）

實際這一則前面還有專案 ID 與金鑰兩行，這裡不放：

貼給 AI

你只需要修改：「在教學裡面」換成你的用途，例如「在我的網站上」。

> 另外如果我想要實作那種...只有登入才能取得特定資料，有 Token 那種設計在教學裡面

Claude 先問兩件事。實際的回答：

問題	實際的決定
受保護的資料是什麼	「我的參拜紀錄」：自己掛過、祈願過的繪馬。最像一般網站的「我的訂單」
資料表誰建	Claude 寫 SQL，你貼進 Supabase 的 SQL Editor 執行。不用把資料庫密碼或管理金鑰給 Claude

Claude 寫了 `001_wishes_prayers.sql` 與 `me/index.ts`，掛繪馬與祈願改成寫進資料庫。順手修了一件事：掛繪馬改成先存成功才掛上牆。反過來的話，存失敗的繪馬重新整理就不見了，使用者卻以為掛好了。

你自己做（16–2：在 SQL Editor 建資料表）

1. Supabase 後台左側選 SQL Editor
2. 請 Claude 給你 `supabase/migrations/001_wishes_prayers.sql` 的完整內容，整段貼進編輯器
3. 按 Run
4. 往下看結果：最後一段會列出每一張資料表與一個 `rls` 欄位

三張表的 `rls` 都要是 `true`：`prayers`、`wish_owners`、`wishes`。有任何一張是 `false`，先不要往下做，把結果貼給 Claude。

這段 SQL 可以重複執行。第二次跑不會重複建表，也不會重複匯入示範繪馬。

```sql
create table if not exists public.wishes (
  id uuid primary key default gen_random_uuid(),
  text text not null check (char_length(text) between 1 and 40),
  by_name text not null default '匿名' check (char_length(by_name) between 1 and 12),
  votes integer not null default 0 check (votes >= 0),
  created_at timestamptz not null default now()
);

create table if not exists public.wish_owners (
  wish_id uuid primary key references public.wishes (id) on delete cascade,
  user_id uuid not null references auth.users (id) on delete cascade,
  created_at timestamptz not null default now()
);
create index if not exists wish_owners_user_id on public.wish_owners (user_id);

create table if not exists public.prayers (
  wish_id uuid not null references public.wishes (id) on delete cascade,
```

Results Chart

資料表	rls
prayers	true
wish_owners	true
wishes	true

📋 貼給 AI（16-3：Edge Function 怎麼操作）

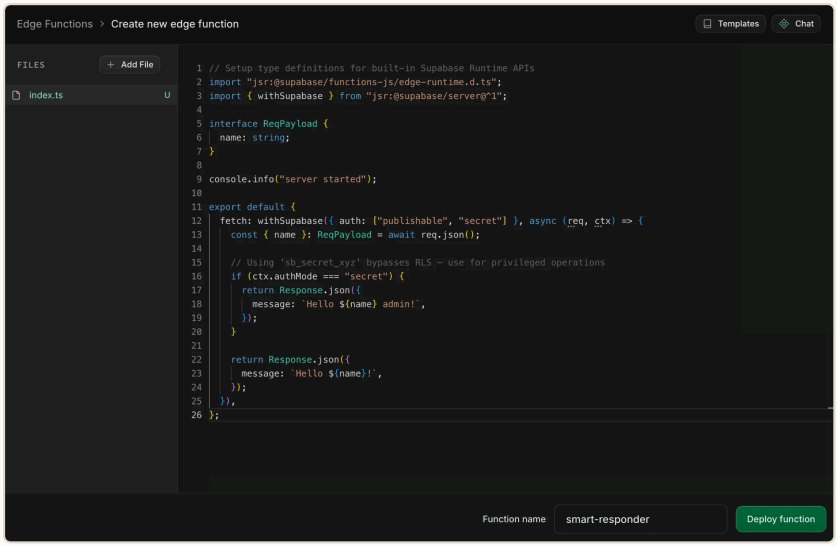
打開 Edge Functions → Create new edge function，截下編輯器的畫面，連同這句貼給 Claude：

> 貼給 AI
>
> **〈附上截圖〉**
>
> 附上這個截圖。然後具體要怎麼操作呀？

Claude 照截圖給了步驟，也說明為什麼不用畫面上的預設範本：

1. 編輯器裡已經有一段 Supabase 的預設範本。不要用它，理由見下表與本階段開頭的〈⚖️ 動手前先想〉
2. 在編輯器裡按 ⌘A（Windows 是 Ctrl + A）全選，刪掉範本
3. 請 Claude 給你 `supabase/functions/me/index.ts` 的完整內容，貼進編輯器
4. 右下角 Function name 從預設的 `smart-responder` 改成 `me`。網站用這個名字呼叫

5. 按 Deploy function
6. 部署完到 `me` 的 Settings 分頁，關掉 Verify JWT with legacy secret，按 Save changes（原因見 16-5）

狀態	參考畫面
刪掉之前的預設範本	 第 12 行的 `auth: ["publishable", "secret"]`，以及第 15 行的註解：用 secret 金鑰的請求會繞過 RLS。這就是不用範本的原因。右下角的 Function name 要改成 `me`。

📋 貼給 AI（16-4：程式貼不上）

貼給 AI

> SQL 已經完成囉
>
> 這個找不到「貼上程式：按 ⌘V，貼上我剛複製好的程式。第一行應該是 `// GET /functions/v1/me`：我的參拜紀錄…。」

原因：剪貼簿的環境不是 UTF-8，中文註解被弄壞了。Claude 改成把完整程式直接貼在對話裡，你從那裡複製。

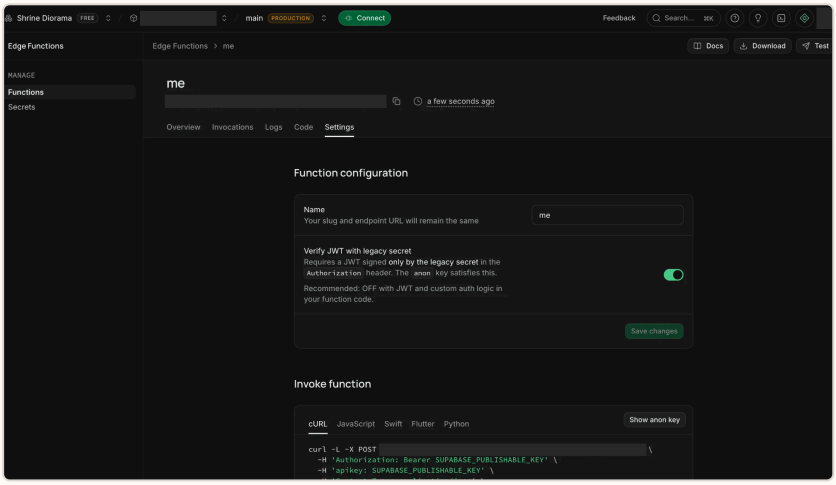
📋 貼給 AI（16-5：關掉舊版的 JWT 驗證）

部署完，進到 `me` 這支函式的 Settings 分頁，截圖貼給 Claude：

貼給 AI

（附上截圖）

補上截圖，然後這些步驟都要更新到教學手冊中喔！

狀態	參考畫面
部署後的設定頁	 Verify JWT with legacy secret 的開關部署後預設是開的。說明文字最後一行寫著建議關掉。專案網址已經遮掉。

開關開著時，沒帶權杖的請求回的是閘道的 `401 UNAUTHORIZED_NO_AUTH_HEADER`，請求還沒進到 `me` 的程式就被擋了。切了開關之後，再傳一張截圖：

貼給 AI

（附上截圖）

這裡的操作把它關閉

Claude 再試一次，回應還是閘道的錯誤。開關切了，但右下角的 Save changes 還沒按，設定沒有存進去。

📋 貼給 AI（16-6：按下 Save changes）

貼給 AI

按下囉

Claude 再測：

測試	結果
沒帶權杖	`401 {"error":"missing_token", …}`，這次是 `me` 的程式回的
假的權杖 `Bearer not-a-real-token`	`401 {"error":"invalid_token", …}`
瀏覽器的預檢請求（OPTIONS）	`200`，網站可以從頁面呼叫這支 API

接著它只用公開金鑰、不登入，直接打資料表。這是在看拿到公開金鑰的人實際上能做什麼。

動作	結果	符合設計嗎
讀 `wishes`	`200`，回示範繪馬（北海道那塊 42 票、阿嬤那塊 37 票）	是：繪馬牆是公開的
讀 `prayers`	`401`，`42501` permission denied	是：沒登入讀不到
讀 `wish_owners`	`42501` permission denied	是
新增繪馬	`42501` permission denied	是：要登入
把票數改成 9999	`42501` permission denied；再讀還是 42	是：票數只能由觸發器加

> **名詞**：`42501` 是資料庫「權限不足」的錯誤代碼，訊息是 `permission denied`。
> 你需要知道到什麼程度：看到它就知道是權限擋下來的。

你自己做（16-7：用 curl 親手試）

> **名詞**：curl 是在終端機裡送出一個網路請求、把回應印出來的工具。Mac 和 Windows 10 以上都內建。
> 你需要知道到什麼程度：知道 `-i` 會連狀態碼一起印出來，`-H` 是加一行標頭。Windows 的 PowerShell 要打 `curl.exe`，不是 `curl`。

網址裡的 `<專案 ID>` 換成你自己的。它在專案首頁的 Project URL 裡，`https://` 後面、`.supabase.co` 前面那一段。

第 1 步：沒帶權杖。開關已經關掉，回的應該是 `me` 自己的 `401`（`missing_token`），不是閘道的錯誤。這一步不涉及權杖，可以請 Claude 跑：

貼給 AI

請幫我在終端機執行下面這行，把狀態碼和回應內容給我看：

```
curl -i https://<專案 ID>.supabase.co/functions/v1/me
```

開關還開著時，實測的結果：

```
401 {"code":"UNAUTHORIZED_NO_AUTH_HEADER","message":"Missing
authorization header"}
```

關掉開關、按了 Save changes 之後，同一行會回 `401 {"error":"missing_token", …}`。狀態碼一樣，回應內容換成 `me` 的程式自己回的。

第 2 步：帶一個假的權杖：

貼給 AI

請再執行一次，這次加上一個假的權杖，把狀態碼和回應內容給我看：

```
curl -i -H "Authorization: Bearer not-a-real-token" https://<專案
ID>.supabase.co/functions/v1/me
```

你應該看到 `401 {"error":"invalid_token", …}`。

第 3 步：拿到自己的權杖。這一步你自己做，不要把權杖貼給 Claude。

1. 在 Chrome 打開 `http://localhost:4324/`，登入（階段 15 的信箱連結，或 15-11 的測試帳號）
2. 打開開發者工具（F12，Mac 是 Option + Command + I），切到 Console
3. 輸入下面這一行（`<專案 ID>` 換成你的），按 Enter

```
JSON.parse(localStorage.getItem('sb-<專案 ID>-auth-token')).access_token
```

印出來的那一長串就是權杖。

第 4 步：帶權杖。在你自己的終端機（Mac 是「終端機」，Windows 是 PowerShell）執行，`<權杖>` 換成第 3 步印出來的那一串：

```
curl -i -H "Authorization: Bearer <權杖>" https://<專案
ID>.supabase.co/functions/v1/me
```

你應該看到 `200`，回應裡只有你自己掛過、祈願過的繪馬。

實測沒有用 curl 跑這一步，改用 16-10 的「帶我的權杖呼叫」，結果見 16-10 的表。

第 5 步：換一個帳號。照 15-11 再建一個測試帳號（例如 `你的帳號+shrine2@gmail.com`），用它的權杖再跑一次第 4 步。

你應該看到（預期）：`me` 回 `200`，但只有那個帳號自己的繪馬與祈願。這一項沒有實際跑過，階段 19 的 `rls-check.mjs` 也會測。

📋 貼給 AI（16-8：拿到公開金鑰的人能做什麼）

16-6 的那張表，你可以自己跑一次。公開金鑰本來就公開，所以這一步可以交給 Claude（整理版指令）：

貼給 AI

你只需要修改：沒有。整段照貼。最後兩行一定要留。

```
請用 .env 裡的公開金鑰（不要印出來），不登入，直接對 Supabase 的資料表做下面五件事，
每一件都告訴我狀態碼和錯誤代碼：

1. 讀 wishes 的 text 與 votes
2. 讀 prayers
3. 讀 wish_owners
4. 新增一塊繪馬
```

5. 把 id 為 00000000-0000-4000-8000-000000000000 那一塊的 votes 改成 9999，改完再讀一次它的票數

如果錯誤回應裡有提示要我執行 GRANT，不要照做，只要把提示的內容告訴我。

這一步只看結果，不要修改任何權限或資料表。

結果應該和 16-6 的第二張表一樣。

📋 貼給 AI（16-9：API 文件頁）

貼給 AI

然後有 API 文件嗎？可以在操作說明裡面放一個

Claude 做成網站上的一頁 `/api`，用同一套 Design Token，操作說明裡加一行連結。頁面內容：

- 網址開頭（從 `.env` 來）、權杖怎麼帶
- 端點表、`GET /me` 的範例與錯誤代碼
- 閘道擋下來時的回應長什麼樣
- 還沒開放的項目
- 試呼叫：不帶權杖呼叫、帶我的權杖呼叫、複製我的權杖

權杖由 supabase-js 的 `auth.getAccessToken()` 取，不自己去讀瀏覽器存的格式。回應一律用 `textContent` 顯示（階段 7 的規則）。

你自己做（16-10：看 API 文件、試呼叫）

1. 打開首頁左下的「操作說明」，點 API 文件的連結（或直接開 `http://localhost:4324/api`）
2. 捲到〈試呼叫〉，先不要登入，按「不帶權杖呼叫」
3. 再按「帶我的權杖呼叫」
4. 回首頁登入，再回到這一頁，按「帶我的權杖呼叫」

狀態碼	error	意思與處理
401	missing_token	沒帶 Authorization 標頭。登入後帶上權杖再試
401	invalid_token	權杖是假的或已經過期。重新登入拿新的權杖
405	method_not_allowed	這支端點只接受 GET
500	query_failed 、server_misconfigured	伺服器的問題，稍後再試

回應如果是 {"code":"UNAUTHORIZED_…"} 這種格式，代表請求在 Supabase 的閘道就被擋下，還沒進到這支 API 的程式。通常是後台的「Verify JWT with legacy secret」還開著。

試呼叫

在這個網站登入之後，可以直接在這裡呼叫 /me ，比較帶與不帶權杖的差別。結果只顯示在你自己的畫面上。

不帶權杖呼叫 | 帶我的權杖呼叫 | 複製我的權杖

還沒登入：先回神社登入，再回到這一頁。

還沒開放

- 公開讀取繪馬牆（不需要登入）：規劃中，會是有版本號的端點，不直接開放資料表本身。
- 從外部掛繪馬、祈願：規劃中。
- 使用量限制與 API 金鑰：規劃中。

錯誤代碼表下面那一段淡色的框，說明大寫的 `UNAUTHORIZED_…` 是閘道擋的。這和本階段開頭〈⚖️ 動手前先想〉裡「同樣是 401，停在哪一層」是同一件事。〈試呼叫〉三個按鈕下面那一行，是沒登入時按「帶我的權杖呼叫」看到的提示。

實測的結果：

動作	結果
不帶權杖呼叫	顯示 `HTTP 401` 與 `missing_token`
沒登入時按「帶我的權杖呼叫」	提示「還沒登入：先回神社登入，再回到這一頁」
登入之後按「帶我的權杖呼叫」	`HTTP 200` ，回應只有自己的繪馬， `prayers: []`

你自己做（16–11：祈願、看我的參拜紀錄）

用 15–11 的測試帳號登入，掛一塊繪馬。實測掛的是「世界和平：)」(18–3)。接著祈願，把截圖傳回去，不用加文字。

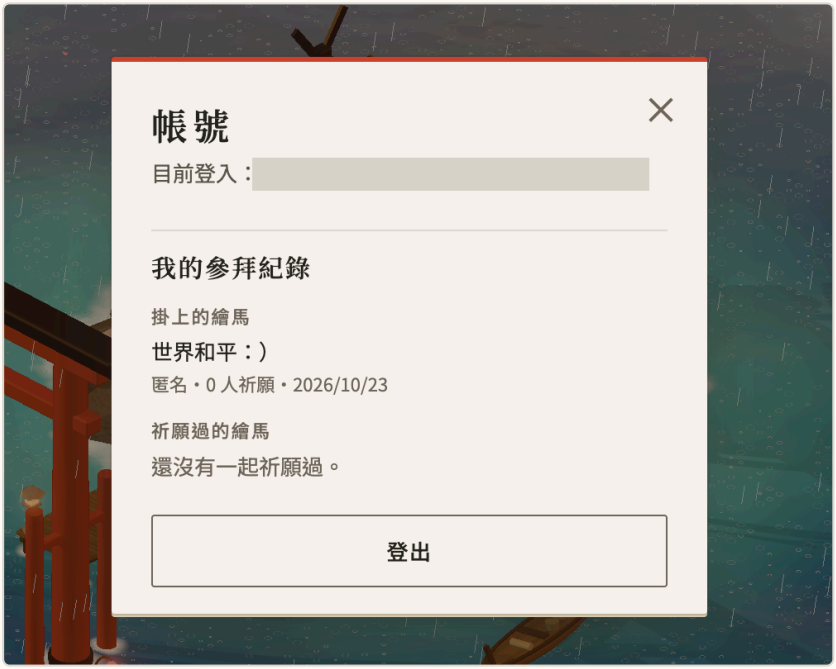
1. 登入之後，點牆上任何一塊繪馬，按住「一起祈願」
2. 再打開同一塊

3. 點左下角的帳號按鈕，看「我的參拜紀錄」

實測的結果：

動作	結果
祈願之前打開「我的參拜紀錄」	列出剛掛的「世界和平：)」，祈願那一欄寫「還沒有一起祈願過」
按住祈願	卡片顯示「你已經一起祈願了」、「1 人也這樣希望」
從外面讀 `/v1/wishes`（階段 17 的公開 API）	「世界和平：)」的票數是 1

票數 1 是觸發器加的，前端沒有寫 `votes` 的權限。從外面讀一次，確認的是資料庫裡的值，不是畫面上的數字。

狀態	參考畫面
祈願之前的「我的參拜紀錄」	 這一頁的資料是用登入權杖呼叫 `me` 拿到的。信箱已經遮掉。

狀態	參考畫面
祈願之後的卡片	 按鈕換成「你已經一起祈願了」，不能再按。同一個人再祈願，資料庫的主鍵也會擋下來。

✅ 怎麼知道做對了

- [ ] 16–2 的結果裡，三張表的 `rls` 都是 `true`
- [ ] Edge Functions 列表裡有一支叫 `me` 的函式
- [ ] `me` 的 Settings 裡，Verify JWT with legacy secret 是關的，而且按過 Save changes
- [ ] 開關開著時看到閘道回的 `UNAUTHORIZED_NO_AUTH_HEADER`；關掉之後同一行回 `401 missing_token`；假權杖回 `401 invalid_token`
- [ ] 16–8 的五件事，只有讀 `wishes` 成功，票數還是 42
- [ ] 16–7 第 4、5 步：帶自己的權杖是 `200`，只有自己的紀錄；換帳號拿到的不同
- [ ] 登入之後，帳號視窗的「我的參拜紀錄」列出你掛過、祈願過的繪馬
- [ ] 操作說明裡有 API 文件的連結。〈試呼叫〉不帶權杖顯示 `HTTP 401` 與 `missing_token`。沒登入時按「帶我的權杖呼叫」會提示先登入
- [ ] 登入之後，〈試呼叫〉帶我的權杖呼叫顯示 `HTTP 200`

除了第 6 項的「換帳號」，全部實測過。帶自己的權杖是用 16–10 的試呼叫按鈕測的。換帳號在階段 19 一起測，補上之前不要把它當成已經驗證。

掛繪馬、祈願有沒有真的寫進資料表，用階段 17 的公開 API 從外面讀一次（16–11 的最後一列）。也可以在後台 Table Editor 打開 `wishes` 看票數。

🆘 如果沒有成功

你可能會看到	通常是因為	怎麼辦
16-2 的結果有一張表 `rls` 是 `false`	SQL 沒有整段貼上，或中途出錯	把 SQL Editor 的整段錯誤訊息貼給 Claude，再加一句：「請先不要修改任何東西。先告訴我最可能的三個原因，以及你準備先檢查哪一個。」
貼進編輯器的程式，中文註解變成亂碼或不見	剪貼簿的環境不是 UTF-8	「請把完整程式直接貼在對話裡，我從對話複製」
關掉開關之後，回應還是 `UNAUTHORIZED_NO_AUTH_HEADER`	只切了開關，沒按 Save changes	回 16-5 按 Save changes，再跑一次 curl，確認回應變成 `missing_token`
帶權杖還是 `401 invalid_token`	權杖過期了（超過一小時），或複製時少了頭尾	重新整理網頁，回 16-7 第 3 步再拿一次
帶權杖之後回 `500 server_misconfigured`	函式環境裡找不到公開金鑰	「me 回 server_misconfigured，請檢查它找公開金鑰的順序」
API 文件頁的試呼叫結果跑到畫面右上角	頁面專用的 class 和首頁撞名	「試呼叫的 class 和首頁撞名了，請改一個這一頁專用的名字」
錯誤回應的提示寫著 `GRANT SELECT ON public.prayers TO anon`	這是資料庫的通用提示，它不知道這個拒絕是你設計好的	不要照做。原因見〈⚖️ 動手前先想〉的最後一題

這一步失敗可以跳過嗎：16-7 第 4、5 步可以，但第 1 步一定要做。那是這一步最重要的觀察。16-2 不行，沒有資料表，掛繪馬與祈願會失敗。

📖 為什麼這樣做

🎯 這一步的重點

資料庫在每一次查詢都檢查「這一列是不是你的」。網頁、API 都只是把你的權杖帶過去。

🧭 這一步在做什麼

把階段 14 留下的問題補上：「需要登入」從介面流程，變成資料庫擋得住的規則。再加一支要帶權杖才拿得到資料的 API。

表	誰讀得到	誰能新增	說明
`wishes`	所有人（繪馬牆是公開的）	登入的人，只能填願望與署名	願望 1 到 40 字、署名 1 到 12 字，在資料庫檢查。前端的字數限制不算數
`wish_owners`	只有本人	沒有人直接新增，掛繪馬時由資料庫自動記下	不放進 `wishes`，原因見〈⚖️ 動手前先想〉
`prayers`	只有本人	登入的人，只能用自己的身分	每人每塊一次

🧩 它在建構流程的位置

```
瀏覽器（登入後拿到權杖）
      │  Authorization: Bearer <權杖>
      ▼
Supabase 閘道 ──（Verify JWT 開著時，在這裡就可能被擋）
      │
      ▼
Edge Function me ──（沒帶權杖：401 missing_token）
      │  用同一個權杖查資料庫
      ▼
資料表 + RLS ──（只回 user_id = 這個權杖的人）
```

💡 那段指令為什麼那樣寫

16–1 只有一句話。設計是 Claude 問了兩個問題之後決定的。第二個問題「資料表誰建」的答案，決定了這一步的分工：Claude 寫 SQL，你貼進 SQL Editor。你不用把資料庫密碼或 secret 金鑰交給任何人。建資料表、部署 API 都是操作卡 2 第三層「一定要你點頭」的事，由你在後台親手按。

16–3 到 16–6 每做一步就截圖或回報一句，Claude 每一步都用 curl 確認回應。16–5 的開關沒生效，就是這樣抓到的。

16–7 第 3、4 步要你自己做，因為權杖等同你的登入狀態。它只出現在你自己的瀏覽器和終端機裡。 16–8 交給 Claude，因為它只用公開金鑰。兩者的差別就是階段 15 那一題：哪一把金鑰可以放前端。公開的可以交出去，等同登入狀態的不行。 16–8 最後兩行各擋一件事。「不要照做 GRANT」擋的是錯誤訊息裡那句 `GRANT … TO anon` 提示。「不要修改任何權限」讓這個實驗只看結果，不會順手把擋下來的東西放行。

階段 17　做對外 API，繪馬牆改讀資料庫

📖 讀完約 15 分鐘　🛠️ 動手約 45 分鐘

> - **這一步一定要懂**：公開的 API 任何人都能寫程式連續呼叫，前端的按鈕擋不住。使用量限制要放在資料庫。新部署的函式要關掉 Verify JWT，不然連公開的端點都會被擋。分頁要有不會重複的排序。
> - **這一步可以先不懂**：`v1` 的程式內容、base64url 怎麼編碼、觸發器的寫法。
> - **做完後你要能用自己的話回答**：七塊示範繪馬，用一頁兩塊往下翻，為什麼第一版會出錯？

⚖️ 動手前先想

公開 API 要不要直接用資料表自動產生的那一套？

不要。Supabase 本來就會為每張資料表產生一套讀寫的網址（階段 15 的 Data API），網站自己也在用。但那一套的欄位名稱就是資料表的欄位名稱。別人寫程式接上之後，你改一個欄位名稱，他的程式就壞了。內部才有的東西（例如作者的 id）也會跟著曝光。所以另外做 `v1`，回傳的欄位由 `v1` 決定（例如資料表的 `by_name` 對外叫 `by`）。之後要改格式，開一支 `v2`，`v1` 照舊。

使用量限制放在哪裡？

放在資料庫。網站上的按鈕可以做成「按一次之後變灰」。但 API 公開之後，任何人都能寫一個迴圈，一秒呼叫一百次，根本不經過那顆按鈕。下面 17-2 會在 SQL Editor 執行 `002_public_api.sql`，它建一個觸發器。新增繪馬之前，觸發器先數這個帳號 10 分鐘內掛了幾塊。不管請求從網站、`v1`，還是直接打資料表來，都會經過這個觸發器。祈願不用另外限制：`prayers` 的主鍵已經保證每人每塊只能一次。

階段 16 那一題再出現一次：`v1` 用什麼身分查資料庫？

和 `me` 一樣，用呼叫者的權杖。沒帶權杖就是沒登入的身分（anon）。`v1` 的程式寫錯，RLS 還在。

新部署的函式，Verify JWT 要不要關？

這個案例的兩支函式都要關，因為它們自己驗證權杖。開著的話：

函式	開著時的症狀
`me`（階段 16）	錯誤訊息從 `missing_token` 變成閘道的 `UNAUTHORIZED_…`，功能照樣是「沒權杖就不給」
`v1`（這一步）	讀繪馬牆這種公開端點也回 401，公開 API 完全不能用

每一支新部署的函式都要記得關，並按 Save changes。重新部署之後也要再看一次。

分頁用什麼當游標？

> **名詞：分頁的游標**（cursor）是上一頁最後一筆的位置，下一頁從它後面接著拿。這個 API 回的 `next` 就是游標。
> 你需要知道到什麼程度：知道游標裡的排序依據不能有兩筆一樣的值。否則一樣的那幾筆會被跳過或重複。只用時間不夠，要「時間＋id」。

用「時間＋id」。id 每一筆都不一樣，排序就不會有兩筆並列。游標放進網址之前，要嘛編碼，要嘛做成不需要編碼的代碼。`v1` 選後者，呼叫的人只要原樣帶回來，不用知道裡面是什麼。

🛠️ 動手做

🧰 這一步的產出

```
supabase/migrations/002_public_api.sql   ← 使用量限制（每個帳號 10 分鐘最多掛 5 塊）、繪馬牆排序用的索引
supabase/functions/v1/index.ts           ← 對外 API 第 1 版：一支函式、五個端點
src/services/auth/supabase.js               ← 繪馬牆改讀資料庫：最新 12 塊
src/pages/api.astro                      ← API 文件頁補上五個端點、分頁、錯誤代碼
```

畫面上應該看到：繪馬牆上的繪馬是從資料庫讀出來的。別人掛的繪馬，重新整理之後也會出現在你的牆上。網站不再假裝有人在祈願。祈願計數只會在真的有人祈願時改變。

這一步和階段 16 一樣，有一半是你在 Supabase 後台親手做的。

📋 貼給 AI（17-1：接著做階段 17 到 20）

階段 16 做完，請 Claude 接著往下做。實際的這一則前面還順便回答了聲音素材的處理方式：

貼給 AI

你只需要修改：第一行換成你自己對聲音素材的決定，或整行刪掉。

```
環境音不能放進去～但可以和學生說明可以去 Suno 生成或者任何 AI 工具搭配 Prompt 指令生成。
然後接著幫我實作階段 17-20
```

Claude 會先問你兩件事，從選項裡選：

問題	實際選的
階段 18「上傳」要讓使用者上傳什麼	繪馬附一張圖
階段 20「部署」要部署到哪裡（部署前會先列出指令給你確認）	Vercel

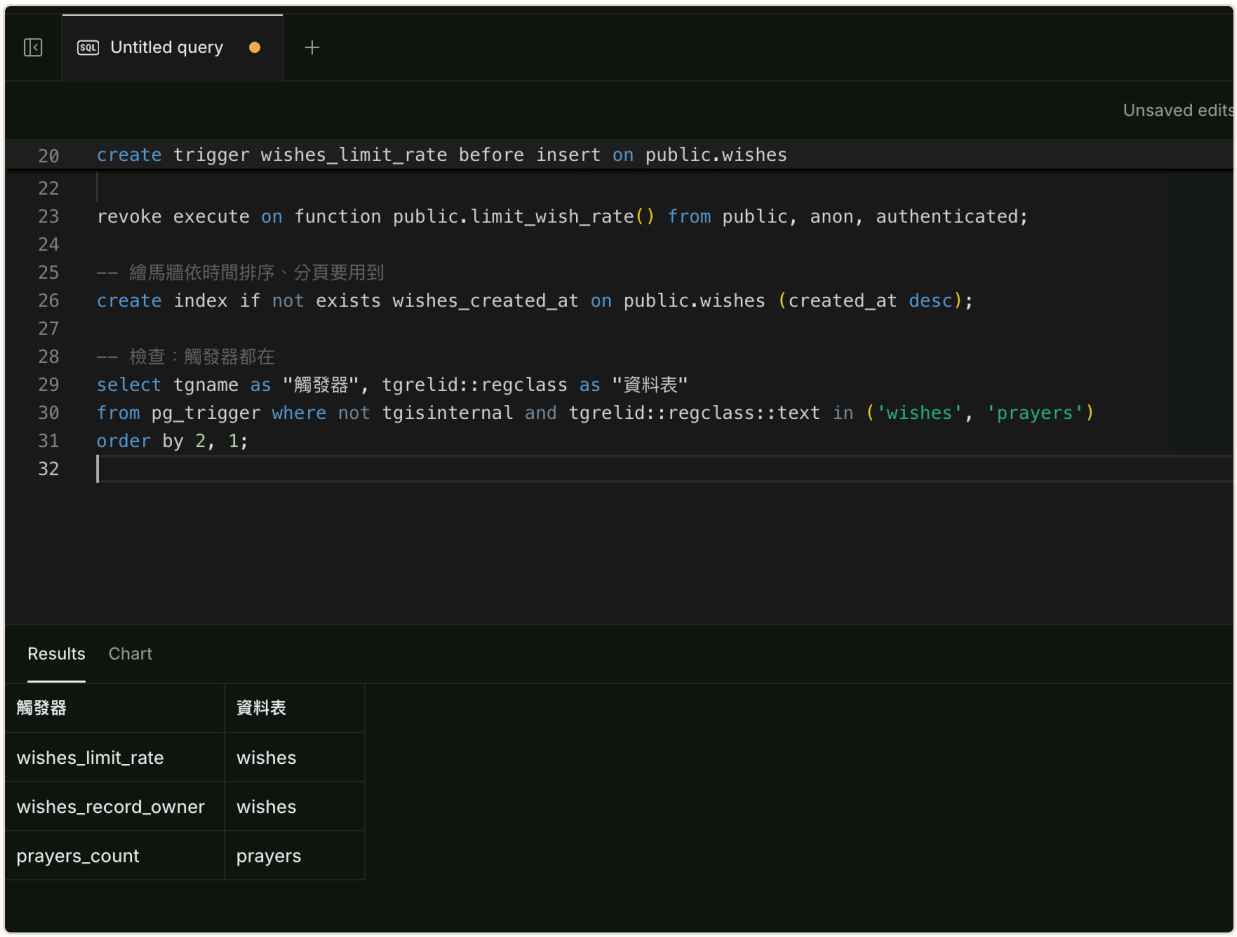
Claude 做的事：

- `v1` 一支函式、五個端點（見〈🧭〉的表）。讀繪馬牆不用登入，寫入要帶權杖
- 輸入先在 API 檢查一次，給清楚的 400 訊息。資料庫的 check 限制再擋一次
- 錯誤細節只記在後台的 Logs，不回給呼叫的人。500 只回 `server_error`
- 使用量限制寫在資料庫（`002_public_api.sql`），滿 5 塊就拒絕。API 回 429，網站顯示「10 分鐘內最多掛 5 塊繪馬」
- 繪馬牆改讀資料庫，讀最新 12 塊（12 是繪馬牆的掛勾數）。讀不到資料庫，或用的是假後端，才用示範資料
- 「模擬其他人在祈願」只留在假後端。票數是真的之後，不能再假裝有人祈願
- `/api` 文件頁補上五個端點的範例、分頁、錯誤代碼（400、404、409、429）

Claude 自己驗證過的：開站時讀繪馬牆的請求回 `200`。6 秒內祈願計數沒有變（沒有模擬祈願）。`npm test` 17 項、建置通過。

你自己做（17-2：在 SQL Editor 執行 002）

1. 請 Claude 給你 `supabase/migrations/002_public_api.sql` 的完整內容
2. SQL Editor 開一個新的查詢，整段貼上，按 Run
3. 結果應該列出三個觸發器



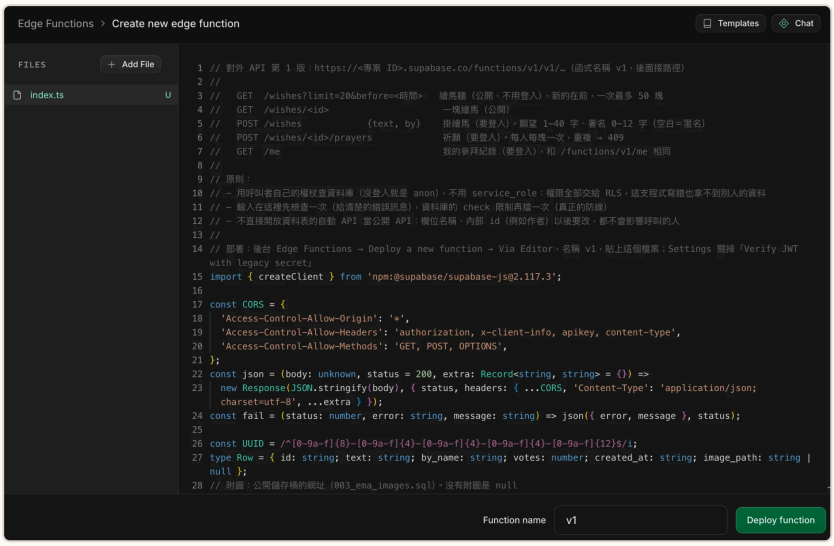
觸發器	資料表	做什麼	哪一份 SQL 建的
`wishes_limit_rate`	`wishes`	10 分鐘內滿 5 塊就拒絕	002
`wishes_record_owner`	`wishes`	記下誰掛的	001
`prayers_count`	`prayers`	票數加一	001

這段 SQL 可以重複執行。少了任何一個，先不要往下做，把結果貼給 Claude。

你自己做（17-3：部署 v1）

和階段 16 部署 `me` 是同一套步驟：

1. Edge Functions → Deploy a new function → Via Editor（打開的編輯器標題是 Create new edge function）
2. 刪掉預設範本，貼上 `supabase/functions/v1/index.ts` 的完整內容
3. 右下角 Function name 填 `v1`
4. 按 Deploy function
5. 部署完到 `v1` 的 Settings 分頁，關掉 Verify JWT with legacy secret，按 Save changes

狀態	參考畫面
按 Deploy 之前	 程式開頭的註解列了五個端點和三條原則。右下角 Function name 是 `v1`。

函式名稱是 `v1`，所以完整網址會出現兩次 v1：

```
https://<專案 ID>.supabase.co/functions/v1/v1/wishes
                                  └ Supabase 固定的 ┘└ 函式名稱，後面接路徑
```

📋 貼給 AI（17–4：部署了，但公開的端點也被擋）

貼給 AI

部署囉

Claude 跑了 10 項測試：

結果	項目數
閘道的 `401 UNAUTHORIZED_NO_AUTH_HEADER`	9 項，包括「讀繪馬牆」這種不用登入的公開端點
假權杖回 `UNAUTHORIZED_INVALID_JWT_FORMAT`	同上，也是閘道回的
瀏覽器的預檢請求（OPTIONS）	`200`，`access-control-allow-origin: *`

原因：新函式預設開著 Verify JWT with legacy secret。閘道要求每個請求都帶 JWT，請求根本進不到 `v1` 的程式。

📋 貼給 AI（17-5：關掉開關之後，分頁出錯）

關掉 `v1` 的開關並存檔之後：

> 貼給 AI
>
> 好囉

Claude 不帶權杖重測：

測試	結果
讀繪馬牆、讀一塊繪馬	`200`，每一塊都有 `image` 欄位（沒有附圖是 `null`）
沒帶權杖掛繪馬、祈願、讀我的紀錄	`401 missing_token`
帶假的權杖	`401 invalid_token`
不存在的繪馬、不是 id 的路徑	`404`「找不到這塊繪馬」
不存在的端點	`404`「沒有這個端點」
`before=abc`	`400 invalid_before`
拿第一頁的 `next` 去要第二頁	`400`，這是 bug

看原因之前先想一想：假設七塊繪馬是同一瞬間建立的。「比上一頁最後一塊的時間早」這個條件，會不會把其他六塊也跳過？

分頁的 bug 有兩個原因：

1. 七塊示範繪馬是同一瞬間匯入的，`created_at` 一模一樣。第一版的分頁條件只有「比這個時間早」，同一個時間的會整批跳過。就算沒報錯，也會漏資料
2. 時間字串裡的 `+00:00` 放進網址時沒有編碼。`+` 在網址裡代表空白，時間就解析不了

修法：

- 排序改成「先比時間，時間一樣再比 id」，兩個都由新到舊。網站自己讀繪馬牆也加上 id 排序，順序才固定
- `next` 改成「時間＋id」編成的一串代碼（base64url）。代碼只有英文字母、數字、`-`、`_`，放進網址不用編碼
- 下一頁的條件：時間比較早，或時間一樣但 id 比較小

規格可以先寫好：「分頁的排序要有不會重複的依據（時間＋id），用 limit=2 翻完全部，確認沒有重複、沒有漏掉」。

- API 文件改成「把 `next` 原樣帶回來」

這個 bug 只有好幾筆同時間的資料才會出現。正式上線之後，兩個人在同一毫秒掛繪馬的機會很小。所以只用正式資料測，可能很久都看不到。示範資料剛好是一起匯入的，第一次測就踩到了。

📋 貼給 AI（17-6：重新部署之後）

Claude 改好程式，請你重新部署 `v1`。重新部署可能會把 Verify JWT 打開，所以也請你部署完再看一眼 Settings。

貼給 AI

部署好了

測試	結果
`limit=2` 一頁一頁往下拿	4 頁拿完 7 塊，0 塊重複，0 塊漏掉。最後一頁的 `next` 是 `null`
`before=abc`	`400`，訊息改成「before 要用上一頁回傳的 next，原樣帶回來」
沒權杖掛繪馬	仍然是 `401`。重新部署之後 Verify JWT 還是關的

沒登入能測的部分全部通過。

📋 貼給 AI（17-7：自己翻一次分頁）

公開端點不用權杖，可以交給 Claude 跑（整理版指令）：

貼給 AI

你只需要修改：`<專案 ID>` 換成你的。

```
請在終端機用 curl 呼叫下面這個網址，每次只拿 2 塊繪馬：
https://<專案 ID>.supabase.co/functions/v1/v1/wishes?limit=2

拿到之後，用回應裡的 next 接著要下一頁（加上 &before=<next>），一直到 next 是 null。
告訴我：總共幾頁、幾塊、有沒有重複的 id、有沒有漏掉的。
最後再用 before=abc 呼叫一次，把狀態碼和訊息給我看。
這一步只看結果，不要修改任何程式或設定。
```

你自己做（17-8：看 API 文件頁）

打開 `http://localhost:4324/api`。和階段 16 比，多了五個端點的範例，每一個都寫明要不要登入。分頁寫著「把 `next` 原樣帶回來」，錯誤代碼表多了 400、404、409、429。

✅ 怎麼知道做對了

- [ ] 17-2 的結果列出三個觸發器
- [ ] Edge Functions 列表裡有 `v1`；它的 Settings 裡 Verify JWT with legacy secret 是關的，而且按過 Save changes
- [ ] 不帶權杖讀繪馬牆是 `200`；不帶權杖掛繪馬是 `401 missing_token`
- [ ] 17-7：用 `limit=2` 翻完所有頁，沒有重複、沒有漏掉，最後一頁 `next` 是 `null`
- [ ] 打開網站，繪馬牆上是資料庫裡的繪馬；用另一個瀏覽器掛一塊，這邊重新整理後看得到
- [ ] 不登入放著 6 秒，右上的祈願計數沒有自己變
- [ ] 帶權杖呼叫：掛繪馬 `201`、同一塊祈願兩次第二次 `409`、10 分鐘內掛第 6 塊 `429`

前四項在 17-2 到 17-6 實測過。第 5 項的前半與第 6 項是 Claude 在 17-1 驗證的：讀繪馬牆的請求回 `200`，6 秒內計數沒變。「另一個瀏覽器掛一塊」還沒有人實際試過。最後一項：網站上的掛繪馬、祈願已經在階段 16、18 用真的帳號測過，直接打 API 的這三項還沒有跑過。拿你自己的結果對照，階段 19 的 `rls-check.mjs` 也包含這三項。

🆘 如果沒有成功

你可能會看到	通常是因為	怎麼辦
`NOT_FOUND Requested function was not found`	還沒按 Deploy，或函式名稱不是 `v1`	回 17-3 按 Deploy function，確認名稱
讀繪馬牆也回 `UNAUTHORIZED_NO_AUTH_HEADER`	Verify JWT 還開著，或切了開關沒按 Save changes	回 17-3 第 5 步。重新部署之後也要再看一次
拿 `next` 要下一頁回 `400`	用的是第一版只有時間的分頁，或 `next` 被改過	確認部署的是最新的 `v1`；`next` 原樣帶回來，不要自己解碼或改寫
分頁拿到重複或少了幾塊	排序只有時間，同時間的資料被跳過	「請把排序改成時間、再 id，next 改成時間＋id 的代碼，再用 limit=2 翻一次給我看」
建置失敗：`Expected } but found :`	Astro 頁面的 HTML 裡有 `{…}`	「API 文件頁的範例裡有大括號，請用 Astro 的寫法包起來」
繪馬牆還是示範資料	`.env` 沒填，或讀資料庫失敗、退回示範資料	打開開發者工具的 Console，把 `listWishes` 開頭的警告貼給 Claude
掛繪馬時出現「10 分鐘內最多掛 5 塊繪馬」	使用量限制生效了	這是正確的結果。等 10 分鐘，或用另一個測試帳號

這一步失敗可以跳過嗎：17-7 可以。17-2、17-3 不行，階段 18 的照片 API 與網站讀繪馬牆都要用到。

📖 為什麼這樣做

🎯 這一步的重點

API 一公開，呼叫它的就不只是你的網站。每一條限制都要放在別人繞不過去的地方：資料庫。

🧭 這一步在做什麼

把階段 16 只給自己用的 `me`，擴充成別人也能用的 API。再把繪馬牆從示範資料換成資料庫。

方法與路徑	要登入嗎	做什麼
`GET /wishes?limit=20&before=<next>`	不用	繪馬牆，新的在前，一次最多 50 塊

方法與路徑	要登入嗎	做什麼
`GET /wishes/<id>`	不用	一塊繪馬
`POST /wishes`	要	掛繪馬。願望 1 到 40 字、署名 0 到 12 字（空白就是匿名）
`POST /wishes/<id>/prayers`	要	祈願。每人每塊一次，重複回 `409`
`GET /me`	要	我的參拜紀錄，和 `/functions/v1/me` 相同

路徑前面都要加 `https://<專案 ID>.supabase.co/functions/v1/v1`。

🧩 它在建構流程的位置

```
別人的程式 ┐
           ├──► Supabase 閘道（Verify JWT 要關掉）──► v1 ──用呼叫者的權杖──► 資料表 + RLS + 觸
你的網站 ──┘                                                                  └
```

💡 那段指令為什麼那樣寫

17–1 是一句「接著幫我實作階段 17–20」。四個階段要做什麼，在 `docs/IMPLEMENTATION_PLAN.md` 第 19 節已經寫好。Claude 只問了計畫裡沒決定、又不能由它決定的兩件事：上傳什麼、部署到哪。前面一行是另一件事（聲音素材），放在同一則訊息裡也沒關係。兩件事各自一句，Claude 分得開。

17–4 到 17–6，每一句都很短：「部署囉」「好囉」「部署好了」。Claude 每次都把整套測試重跑一次。分頁的 bug 和「重新部署後 Verify JWT 還是不是關的」，都是這樣抓到的。你可以在第一次部署時就請它：「之後我每說一次部署好了，你就把全部的端點重測一次」。

17–7 最後一行「不要修改任何程式或設定」，讓這一步只是觀察。翻頁出錯的話，你會先看到錯誤，再決定要不要改。

階段 18　繪馬附一張照片

📖 讀完約 12 分鐘　🛠️ 動手約 40 分鐘

- **這一步一定要懂**：檔案的類型、大小、能放在哪個資料夾，都要在伺服器擋。照片上傳之前要去掉 EXIF，裡面可能有拍攝地點。公開的儲存桶不要建 select policy。
- **這一步可以先不懂**：canvas 怎麼重新編碼、storage policy 的 SQL 寫法、卡片翻面的動畫。
- **做完後你要能用自己的話回答**：照片在公開的儲存桶裡，任何人都看得到。為什麼別人還是列不出「所有照片」？

⚖️ 動手前先想

照片要不要在上傳前去掉 EXIF？

> **名詞**：**EXIF** 是照片檔裡附帶的拍攝資訊：相機、拍攝時間，手機拍的通常還有 GPS 座標。
> 你需要知道到什麼程度：知道直接上傳手機照片，別人下載回去就可能讀到拍攝地點。這個案例在瀏覽器重新編碼一次，把它去掉。

要。繪馬牆是公開的，任何人都能下載照片。手機拍的照片通常帶著 GPS 座標，有人在家裡拍一張貓掛上去，就等於公開了家裡的位置。去 EXIF 放在前端，是因為照片在離開使用者的電腦之前就該去掉。驗證要從外面下載檔案來看（18–4 第 3 件），畫面上看不出來。

公開的儲存桶，要不要建 select policy？

> **名詞**：**Storage** 是 Supabase 放檔案的地方。**儲存桶**（bucket）是裡面的一個資料夾，可以設定公開或不公開、收哪些類型、單檔多大。
> 你需要知道到什麼程度：知道公開的儲存桶「知道網址就看得到」。上傳和刪除的權限另外用 storage policy 管，寫法和資料表的 RLS 一樣。

不要。公開的儲存桶「知道網址就看得到」，不需要 select policy。建了 select policy，反而開放了「列出這個儲存桶的所有檔案」。別人不用知道網址，就能把每一張照片的路徑列出來。連還沒掛上、掛上又刪掉的都在裡面。 18–3 會看到「列出所有檔案回空陣列」，那就是沒建這條 policy 的結果。

先傳照片，還是先寫繪馬？

先傳照片。繪馬要記照片的路徑，照片不在就沒有路徑可記。反過來的問題是：照片傳好了，繪馬寫失敗（例如超過使用量限制）。這時照片就變成沒有主人的檔案。所以寫失敗時把剛傳的照片刪掉。storage policy 也開了「刪除自己資料夾裡的檔案」。

階段 14 那一題再出現一次：照片的「只能上傳到自己的資料夾」算不算權限控管？

算，因為擋在 storage policy，不在前端。前端也是用自己的 id 組路徑，但那只是為了上傳成功，不是為了擋人。18–3 會用沒登入的身分上傳到 `hack/`，被拒絕就證明擋得住。

已知限制：照片沒有人工審核。公開之前要決定不當照片怎麼處理，目前只能在 Supabase 後台刪。

🛠️ 動手做

🧰 這一步的產出

```
supabase/migrations/003_ema_images.sql   ← 儲存桶 ema-images、上傳與刪除的權限、wishes.image_path 欄位與檢查
src/shell/photo.js                       ← 選照片、縮圖、重新編碼成 WebP（去掉 EXIF）
src/app/components/WishCard.jsx                          ← 寫繪馬時附照片；卡片多一顆「看背面的照片」
```

畫面上應該看到：寫繪馬時，操作列多一顆「附一張照片」。選了照片會出現縮圖。掛上之後打開那一塊，操作列多一顆「看背面的照片」。按下去卡片翻面，背面是那張照片，裁成繪馬的形狀。

📋 貼給 AI（18–1：上傳什麼）

這一步沒有單獨的指令。17–1 Claude 問「階段 18「上傳」要讓使用者上傳什麼？」，選「繪馬附一張圖」。

你可以照下面這段請 Claude 做（整理版指令）：

貼給 AI

你只需要修改：第一行換成你要讓使用者上傳的東西。

寫繪馬時可以附一張照片，掛上之後在卡片背面顯示。

檔案類型與大小要在伺服器檢查，不能只靠前端。

上傳之前先在瀏覽器縮圖，並去掉照片裡的 EXIF（含拍攝地點）。

先列出你打算怎麼設定儲存空間的權限，等我確認再動手。

Claude 的設計：照片放 Supabase Storage 的 `ema-images` 儲存桶，`wishes` 只記路徑。每一道限制都在伺服器：

1. 儲存桶只收 WebP 與 JPEG、單檔 2 MB。超過的 Storage 直接拒絕
2. 只能上傳到自己的資料夾 `<自己的 user id>/…`（storage policy）
3. 公開可看，但不建 select policy。理由見本階段開頭的〈⚖️ 動手前先想〉
4. `wishes.image_path` 有格式檢查，第一段一定是自己的 user id。不能把別人的照片掛在自己的繪馬上

前端（`src/shell/photo.js`）：

- 長邊縮到 1280，重新編碼成 WebP（舊版 Safari 退回 JPEG），品質 0.82
- 縮圖是把照片重新畫到 canvas 上再輸出。EXIF 不在畫面上，所以不會跟著輸出

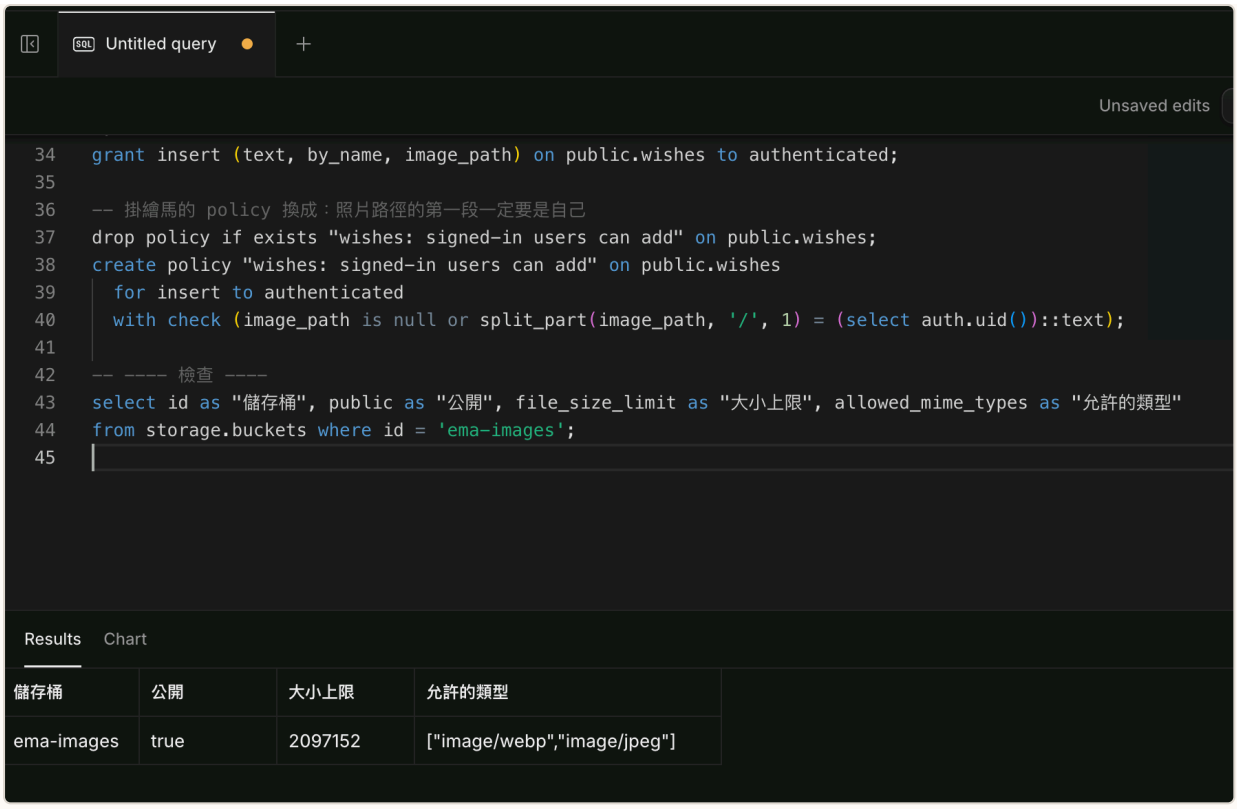
卡片：

- 照片貼在繪馬卡片的背面。卡片本來就有背面（階段 7）
- 翻面的速度和幀率無關（時間常數 0.12 秒）。開了「減少動態效果」就直接翻
- 照片網址用屬性設定，不經過 `innerHTML`（階段 7 的規則）

003 還沒執行之前，Claude 先用假後端測了一輪（暫時加 `PUBLIC_AUTH_MOCK=1`，測完移除）。2400×1600 的照片縮成 1280×853。不是圖片的檔案顯示「只能放照片」。登入後繪馬自動掛上，翻到背面看得到照片。Console 0 個錯誤。

你自己做（18-2：在 SQL Editor 執行 003）

1. 請 Claude 給你 `supabase/migrations/003_ema_images.sql` 的完整內容
2. SQL Editor 開一個新的查詢，整段貼上，按 Run
3. 結果應該只有一列：`ema-images` 、`true` 、`2097152` 、`["image/webp","image/jpeg"]`



`2097152` 是 2 MB 換算成位元組（2 × 1024 × 1024）。截圖上方那段 SQL 就是設計的第 4 點：`split_part(image_path, '/', 1)` 要等於自己的 user id。

📋 貼給 AI（18–3：第一次真的上傳）

用 15–11 的測試帳號登入，寫一塊繪馬、附一張圖、掛上，打開、翻到背面。把過程的截圖傳回去，不用加文字：

貼給 AI

《附上截圖》

《附上截圖》

《附上截圖》

《附上截圖》

《附上截圖》

貼給 AI

《附上截圖》

狀態	參考畫面
寫繪馬、附照片	 選了照片之後，按鈕變成「換一張照片」，旁邊是縮圖。縮圖右上的 × 可以拿掉照片。
掛上之後打開	 署名空白就寫「匿名」。操作列多了「看背面的照片」。

狀態	參考畫面
翻到背面	 照片裁成繪馬的形狀。按鈕變成「翻回正面」。

去 EXIF 有沒有生效，截圖看不出來，要從外面下載那個檔案來看。

Claude 接著從外面驗證。只用公開金鑰、不登入：

檢查	結果
`GET /v1/wishes?limit=1`	最新一塊就是「世界和平：)」、匿名、0 票。照片路徑是「使用者 id / 隨機 id.webp」
下載照片	`200`、66,504 bytes、`image/webp`、1024×1280。瀏覽器已經縮過、轉成 WebP
照片裡有沒有 EXIF	找不到 EXIF 區塊，也沒有 GPS 字樣
列出儲存桶裡的所有檔案	`200`，但回空陣列 `[]`
不登入，上傳到 `hack/test.jpg`	拒絕：`new row violates row-level security policy`
讀 `wish_owners`	`42501` 拒絕。沒有人能從外面查「這塊是誰掛的」

📋 貼給 AI（18–4：自己從外面驗證）

這幾件事只用公開金鑰，可以交給 Claude（整理版指令）：

貼給 AI

你只需要修改：沒有。整段照貼。最後兩行一定要留。

```
請用 .env 裡的公開金鑰（不要印出來），不登入，做下面五件事，每一件都告訴我結果：

1. 呼叫 v1 的 GET /wishes?limit=1，告訴我最新一塊的文字和照片路徑
2. 下載那張照片，告訴我狀態碼、檔案大小、格式、長寬
3. 檢查那個檔案裡有沒有 EXIF 區塊、有沒有 GPS 資訊
4. 列出 ema-images 儲存桶裡的所有檔案
5. 上傳一個小檔案到 ema-images 的 hack/test.jpg

如果有任何一件「成功」了而它不應該成功，先停下來告訴我，不要自己修。
這一步只看結果，不要修改任何權限或設定。
```

第 4 件應該回空的，第 5 件應該被拒絕。

✅ 怎麼知道做對了

- [ ] 18–2 的結果：`ema-images` 、 `true` 、 `2097152` 、 `["image/webp","image/jpeg"]`
- [ ] 寫繪馬時選一個不是圖片的檔案，畫面說「只能放照片」
- [ ] 附一張照片掛上；打開那一塊，按「看背面的照片」看得到照片，再按「翻回正面」翻回來
- [ ] 18–4：照片是 WebP、長邊不超過 1280、沒有 EXIF
- [ ] 18–4：列出儲存桶回空的；不登入上傳被拒絕

全部實測過。第 2 項是 Claude 用假後端測的，其他是實際的帳號在真的 Supabase 上測的。

還有一項沒有實際跑過：用第二個測試帳號上傳到第一個帳號的資料夾。你應該看到（預期）被拒絕。階段 19 的 `rls-check.mjs` 包含這一項。

🆘 如果沒有成功

你可能會看到	通常是因為	怎麼辦
掛繪馬時說照片上傳失敗	003 還沒執行，儲存桶不存在	回 18–2 執行 003

你可能會看到	通常是因為	怎麼辦
上傳被拒絕：`new row violates row-level security policy`	沒登入，或路徑的第一段不是自己的 user id	確認已經登入。路徑是程式產生的，自己改過程式的話，把錯誤整段貼給 Claude
選了照片，畫面說「只能放照片」	選到的檔案不是圖片	換一張 JPEG 或 PNG
18-4 的第 4 件列得出檔案	儲存桶建了 select policy	「ema-images 不應該有 select policy，請告訴我目前有哪些 policy，先不要刪」

這一步失敗可以跳過嗎：可以。沒有附照片的繪馬照常運作。階段 19 的資安檢查會用到這一步的權限設定，建議回頭補。

📖 為什麼這樣做

🎯 這一步的重點

使用者上傳的檔案要在伺服器擋三件事：是什麼類型、多大、放在誰的資料夾。照片本身也可能帶著上傳的人不知道的資訊。

🧭 這一步在做什麼

讓繪馬可以附一張照片。照片放在 Supabase Storage，繪馬只記路徑。

限制	在哪裡擋	繞過前端會怎樣
只收 WebP、JPEG	儲存桶的設定	Storage 直接拒絕
單檔 2 MB	儲存桶的設定	Storage 直接拒絕
只能放進自己的資料夾	storage policy	被拒絕，就是 18-3 的 `hack/test.jpg`
繪馬只能掛自己的照片	`wishes` 的 policy 與 check	被拒絕
去掉 EXIF、縮圖	前端	繞過的人上傳的是他自己的照片，影響的是他自己

只有最後一列放在前端。它保護的是上傳的人自己，不是保護網站。

🧩 它在建構流程的位置

```
選照片 ──► photo.js（縮圖、重新編碼、去 EXIF）
                │
                ▼
        Storage：ema-images/<自己的 id>/<隨機 id>.webp ──（類型、大小、資料夾，在這裡擋）
                │ 成功
                ▼
        wishes.image_path ──（路徑第一段一定是自己，在這裡擋）
                │ 失敗就刪掉剛傳的照片
                ▼
        卡片背面 ◄── 公開網址
```

💡 那段指令為什麼那樣寫

18–1 整理的那段寫了「檔案類型與大小要在伺服器檢查，不能只靠前端」。不寫的話，限制可能只做在前端：`<input type="file" accept="image/*">` 加上 JS 檢查大小。那兩個都在前端，直接呼叫 Storage 就繞過了。最後一句是「先列出你打算怎麼設定儲存空間的權限，等我確認再動手」。加這一句是因為儲存空間的權限設錯，後果是別人能讀到或覆蓋所有人的照片。這種設定要先看過再執行。

18–4 第 5 件是故意做一件應該失敗的事。只驗證「我自己上傳成功」，證明不了「別人上傳不了」。倒數第二行「有任何一件成功了而它不應該成功，先停下來告訴我」，讓 Claude 不會順手修掉。你要先知道哪裡沒擋住。

一輪修正：拖拉、帳號名稱、小雨、游標、漁夫

📖 讀完約 12 分鐘　🛠️ 動手約 40 分鐘

> - **這一步一定要懂**：回報 bug 時，「在哪裡」比「發生什麼」更能幫忙重現。看不到的東西不一定不在那裡：收起來的面板、沒有背景的外框，都可能接走滑鼠。
> - **這一步可以先不懂**：`user-select`、`pointer-events` 這些 CSS 屬性的細節；漁夫的建模方式。
> - **做完後你要能用自己的話回答**：左側一大片海拖不動，畫面上什麼都看不到。要怎麼找出是誰接走了滑鼠？

這一章沒有編號。它是階段 18 做完之後，用網站的時候回報的六件事，分散在介面、場景和帳號按鈕。放在一起，是因為它們是同一種工作：先重現，再找原因，修完再用同一個動作驗證一次。

⚖️ 動手前先想

點不到的地方，要先看還是先量？

先量。做到修正 5 時會遇到一片點不到的海，看起來什麼都沒有，用看的找不到原因。`elementFromPoint` 直接回答「那一點最上面是誰」，一次就找到 `.guide`。這和階段 2 起每一階段都在做的事一樣：出現奇怪的症狀時，先量，不要先猜。

小雨要調淡，還是少畫幾條？

少畫幾條。原本的小雨只調透明度，700 條全部照畫，看起來就是比較淡的大雨。雨的大小，眼睛看的是「有幾條」。條數跟著雨量變，小雨才像小雨，也少算了 455 條。

游標長什麼樣，誰決定？

介面決定。引擎只告訴介面「現在滑到了可以點的東西」（`data-hover`）。引擎直接改游標的話，介面換一套外觀（例如階段 11 的方案 A、C）時，還要去改引擎。

🛠️ 動手做

🧰 這一步的產出

```
src/styles/global.css                ← 介面文字不能選取；游標；左下按鈕列與兩個面板的定位
src/app/components/AuthDialog.jsx                ← 帳號按鈕的名字去掉 + 後面的別名、最多 9
個字寬
src/engine/sim/weather.js            ← 三級雨量：小雨、大雨、雷雨（雨聲的音量
在 src/shell/audio/config.js）
src/engine/scene/boats.js            ← 漁夫的蓑衣、臉、手臂、手、坐墊；釣線垂到水面、浮標
src/engine/input/pointer.js          ← 滑到可以點的東西時標 data-hover，不直接改游標
```

畫面上應該看到：

- 在介面上拖拉不會選到字
- 左側那片海拖得動
- 天氣多一種小雨
- 游標是朱紅的小圓圈
- 漁夫有臉、手臂和手，釣線垂到水面，水面上有一顆紅白浮標

📋 貼給 AI（修正 1：拖拉會選到字、有時候不能點）

可以連著傳三則，一則講一件：

貼給 AI

《附上截圖》

這應該是 BUG，有時候會不能點擊，或者拖拉畫面反而會變成全選螢幕上得鱒的等

貼給 AI

《附上截圖》

貼給 AI

> 尤其是在左下角拖拉什特別容易選取文字

「得鱒的等」是「上的字等」,「拖拉什」是「拖拉時」。截圖裡還有第二個問題。左下的帳號按鈕顯示完整的測試帳號名稱（含 `+shrine`），把旁邊的按鈕擠開。

Claude 先試著重現。用 headless 瀏覽器從場景中間拖，重現不出來。第三則說了「尤其是在左下角」，Claude 改成從介面的字開始拖，才重現。

症狀 → 原因：

- 拖拉從介面的字上開始：左下角三顆按鈕、操作說明面板，字最密的地方。瀏覽器把它當成選取文字
- 有文字被選取時，下一次在場景上按下去，瀏覽器可能當成「拖曳剛才選取的文字」。放開滑鼠的事件（pointerup）不會送到場景，點擊就不見了
- 「選到字」和「有時候不能點」是同一個原因

修法：

- 神社頁的介面文字設成不能選取（`user-select: none`）。輸入框照常可以選。API 文件頁不受影響，那裡要讓人複製範例
- 帳號按鈕的名字去掉 `+` 後面的別名，最多 9 個字寬，超過就截斷。完整信箱放在滑鼠停留的提示（`title`）與螢幕報讀器讀的名稱（`aria-label`）

> **名詞**：`user-select: none` 是一個 CSS 設定，讓那一塊的文字不能被滑鼠選取。
> 你需要知道到什麼程度：知道它適合用在按鈕、面板標題。這些是「拿來點、不拿來複製」的字。要讓人複製的地方（例如 API 文件的範例）不能加。

驗證：下面六種操作，選到的字全部是空的。從說明面板內拖、從面板拖到場景、從場景拖進左下角、從按鈕拖到右上、從標題拖到場景、按 ⌘A。

拖完馬上點本殿，鏡位照常切到本殿。API 文件頁仍然可以選字。

📋 貼給 AI（修正 2：更小的小雨）

貼給 AI

另外我希望雨有更小的小雨版本！目前只有比較大的大雨版本

原因：雨只有一個強度（見〈⚖️ 動手前先想〉的小雨那一題）。

修法：分成三級雨量。

天氣	雨量	雨絲條數	天色	雨聲
小雨（新的）	0.35	約 245 條	只暗一點	大雨的一半
大雨（原本的小雨改名）	1	700 條	和原本一樣	和原本一樣
雷雨	1.4	700 條	和原本一樣	和原本一樣

雨絲的條數是 700 × min(1, 雨量)，只更新畫出來的那幾條。透明度是雨量 × 0.55，上限 0.77。

天氣循環：晴 → 薄霧 → 晴 → 小雨 → 大雨 → 雷雨 → 小雨。

驗證：用 `?wx=小雨`、`?wx=大雨`、`?wx=雷雨` 各開一次，水面鏡位三張並排比較。`npm test` 17 項、建置通過。

截圖時踩到的：第一次每種天氣等 27 秒才截。天氣循環一輪約 28 秒，截到的已經是下一種天氣，標籤和畫面對不上。改成等 13 秒。

📋 貼給 AI（修正 3：場景的游標）

貼給 AI

我想要一個適合這個場景版本的游標！

Claude 用 SVG 畫游標，寫在 CSS 變數裡：

位置	游標
一般（海面、空白處）	朱紅的小圓圈加中心一點，像印章的外框。外圍一圈白邊，在深色的海和淺色的紙上都看得到。點的位置在圓心

位置	游標
可以點的東西（繪馬、本殿、鳥居、船、介面的按鈕）	實心朱紅圓點加中心白點
輸入框	照常是打字的游標
API 文件頁	一般的游標

改了一個邊界：引擎原本直接改畫布的 `style.cursor = 'pointer'`，現在只在畫布上標 `data-hover`。理由見〈⚖️ 動手前先想〉，和階段 8 切開引擎與介面是同一條規則。

驗證：海面上 `data-hover` 是空的，游標是圓圈。滑到繪馬 `data-hover=1`，換成實心圓點。按鈕是實心圓點。輸入框是打字游標。

📋 貼給 AI（修正 4：漁夫的細節）

貼給 AI

（附上截圖）

接著再幫我優化人物的細節！

截圖裡的漁夫是兩層圓錐、一顆黑球、一頂斗笠（階段 12 照「以形體為主」只做了輪廓）。

Claude 加的：

- 蓑衣三層，下緣每一片草往外、往下推一點。推多少用頂點位置的雜湊算，不呼叫 `rand()`，岩石和船不會換位置（`CLAUDE.md` 工程原則第 6 條）
- 臉、兩隻手臂與手（藍染的布）、盤坐的腿、坐墊；斗笠加頂端的小鈕與帽緣
- 釣竿握在前面那隻手，以手為支點隨風輕晃（和吊燈籠同一個 `swing()`），每 11 秒左右提一下竿

踩到的問題：

症狀	原因	修法
蓑衣像一座塔	三層疊得太開	三層互相蓋住一半以上，下緣的草加長
斗笠和身體分開	斗笠浮在頭的上方	斗笠壓到頭的一半

📋 貼給 AI（修正 5：左側一整片拖不動）

修正 1 之後，如果還有一塊地方拖不動，圈起來貼：

貼給 AI

（附上截圖）

畫面這個區塊，拖拉會沒反應。

然後你又變成英文了

截圖圈的是左下三顆按鈕上方的一大片海。

畫面上那裡什麼都沒有，所以 Claude 不是用看的，而是直接問瀏覽器。它用 `document.elementFromPoint(x, y)` 掃左側每一個點，看那一點最上面是誰。

> **名詞**：`document.elementFromPoint(x, y)` 是瀏覽器內建的一個函式，回傳畫面上某一點最上面的那個元素。
> 你需要知道到什麼程度：知道「點下去沒反應」時，可以請 Claude 用它查那一點被誰接走了。不用自己寫。

結果：y 從 200 到 600，拿到的都是 `.guide`（左下按鈕列的外框），不是 3D 場景的畫布。

原因：

- `.guide` 裡面有兩個收起來的面板（操作說明、聲音）。收起來時設成看不見（`visibility: hidden`），但還佔著高度
- 外框因此從畫面頂端一路延伸到按鈕。外框沒有背景，看不到，卻接走了整片左側的拖拉與點擊
- 修正 1 的「有時候不能點」，點在畫面左邊時也是這個原因

> **名詞**：`visibility: hidden` 讓元素看不見，但它還佔著原本的位置。`pointer-events: none` 讓元素不接收滑鼠，點下去會穿過它，交給下面的東西。
> 你需要知道到什麼程度：知道「看不見」和「不佔位置」「不接滑鼠」是三件不同的事。

同一個原因的第二個問題：打開操作說明時，面板頂端在 y = −4，超出畫面。收起來的聲音面板在它下面佔位置，把它往上推。這是階段 13 加聲音面板時沒注意到的。

修法：

- 外框本身不接收滑鼠（`pointer-events: none`），只有按鈕列與打開中的面板接收
- 兩個面板改成浮在按鈕列上方（絕對定位），不再佔位置

驗證：

檢查	結果
左側 (100, 400) 最上面是誰	畫布
在左側拖拉	鏡位從全景變成自由視角，拖拉有效
打開的面板	點得到；面板上方的空白處是畫布
兩個面板的頂端	桌機 375 / 270、手機 497 / 326，都在畫面內，底端緊貼按鈕上方

📋 貼給 AI（修正 6：漁夫的手太簡陋）

貼給 AI

（附上截圖）

附圖服裝沒問題，但手有點太鬧，太簡陋了啦

修正完畢後，接著引導我去完成後續的步驟！

手改了三版：

版本	做法	問題
修正 4	兩根從肩膀直戳到手的藍色棍子	回饋原文：「太簡陋了啦」
第一次重做	分成上臂、手肘、前臂，還是藍色	從側面看，手肘從蓑衣兩側戳出來，像兩根天線
第二次重做	袖子改成蓑衣的草色，和身體融在一起。肩膀往內，手肘往前收在身體前方。只有握拳的手是膚色。兩隻手一前一後都握在竿子上	無

釣線也一起改：

- 原本釣線拉到船底以下 0.8，水下那一段靠不透明的水面蓋住，線停在哪裡看不出來
- 改成每一幀算竿尖正下方的水面高度（用共用的 `getWaterHeight`，含波浪），釣線剛好垂到水面

- 水面上放一顆紅白浮標。12-2 的指令寫的是「釣竿的魚餌應該要在海裡面」，這一版才做到

驗證：側面截圖，手臂不再突出、浮標在水面上。`npm test`、建置通過。

✅ 怎麼知道做對了

- [ ] 從左下角的按鈕或操作說明的字開始拖到場景：沒有選到任何字。拖完馬上點本殿，鏡位切到本殿
- [ ] API 文件頁的範例仍然可以選取、複製
- [ ] 用一個很長的測試帳號登入，左下的帳號按鈕不會把旁邊擠開。滑鼠停在按鈕上看得到完整信箱
- [ ] `?wx=小雨` 的雨絲明顯比 `?wx=大雨` 少
- [ ] 海面上是朱紅圓圈游標，滑到繪馬變成實心圓點，輸入框是打字游標
- [ ] 左側按鈕列上方那片海拖得動
- [ ] 打開操作說明與聲音面板，頂端都在畫面內
- [ ] 從側面看漁夫：手臂沒有從蓑衣兩側戳出來，釣線垂到水面，有浮標

帳號按鈕與游標兩項以外，都是 Claude 驗證過的（截圖或 `elementFromPoint`）。帳號按鈕的截斷沒有另外截圖，是照 CSS 的設定寫的預期結果。游標截圖拍不到，只能在你自己的瀏覽器上看。

🆘 如果沒有成功

你可能會看到	通常是因為	怎麼辦
某一塊拖不動、點不到，畫面上什麼都沒有	有一個看不見的元素蓋在上面	「請用 document.elementFromPoint 掃這一區的點，告訴我最上面是哪個元素，先不要修改」
介面的字可以選，但輸入框不能打字了	`user-select: none` 加到了輸入框	「輸入框要可以選字，請把它排除」
小雨和大雨看起來一樣	只改了透明度，雨絲條數沒有跟著雨量變	「小雨的雨絲條數要跟著雨量減少，請用 ?wx=小雨 和 ?wx=大雨 各截一張並排給我看」
加了漁夫的細節之後，岩石和船換位置	新的細節用了共用的 `rand()`	「新加的東西請用自己的亂數，不要呼叫共用的 rand()」（`CLAUDE.md` 工程原則第 6 條）
截圖前後看起來一樣	存成同一個檔名，看到的是舊圖	請 Claude 換一個檔名重截

你可能會看到	通常是因為	怎麼辦
截到的天氣和標籤對不上	等太久，天氣已經循環到下一種	請 Claude 縮短等待的秒數，或用 `?wx=` 固定天氣

這一章失敗可以跳過嗎：可以。這六件都不影響後面的資安檢查與部署。修正 5 建議做，左側拖不動會讓使用者以為網站壞了。

📖 為什麼這樣做

🎯 這一步的重點

使用者回報的 bug，先重現再修。重現不出來，就是還少了一個條件，通常是「在哪裡」。

🧭 這一步在做什麼

用了網站一陣子，回報了六件事。兩件是 bug：拖拉選到字、左側拖不動。一件是排版：帳號名字太長。三件是想要更好：小雨、游標、漁夫。

🧩 它在建構流程的位置

```
回報（截圖＋一句話）
        |
        ▼
重現 —重現不出來—▶ 回頭問：在哪裡、從哪裡開始
        |
        ▼
找原因（elementFromPoint、截圖並排）—▶ 修 —▶ 用回報的那個動作再做一次
```

💡 那段指令為什麼那樣寫

六則回報都很短，而且都附了截圖。截圖指出是哪裡，文字說是什麼問題。

修正 1 的第三則「尤其是在左下角拖拉什特別容易選取文字」是重現的條件。修正 6 的「服裝沒問題，但手有點太鬧」先說了哪裡不用改。Claude 只改手，沒有動蓑衣和斗笠。

你也可以在回報時，直接要求先查再修（整理版指令，照修正 5 的做法）：

貼給 AI

畫面上這一區點了沒反應，但看起來什麼都沒有。

請先用 `document.elementFromPoint` 掃這一區的幾個點，告訴我最上面是哪個元素、它為什麼在那裡。

找到原因之前，先不要修改任何東西。

階段 19　資安檢查

📖 讀完約 15 分鐘　🛠️ 動手約 40 分鐘

- **這一步一定要懂**：資安檢查是用另一個人的身分，去做他不該做的事。每一項的通過條件是「被拒絕，而且資料沒有變」。只測自己做得到什麼，證明不了別人做不到。
- **這一步可以先不懂**：`rls-check.mjs` 的程式內容、每一條 policy 的 SQL 寫法。
- **做完後你要能用自己的話回答**：B 去查 A 的祈願紀錄。資料庫沒有回錯誤，只回了一個空的清單。為什麼這樣算擋住了？

⚠️ 這一階段的兩個帳號互測，參考成品沒有實際跑過。 19–2 到 19–7 是完整的步驟，學員照做得出來。裡面的「你應該看到」是照程式與 SQL 推出來的預期結果，都標了「預期」。 19–1 的靜態檢查，以及〈🧭〉表格裡前面階段做過的檢查，是實測結果。

⚖️ 動手前先想

「被拒絕」要怎麼確認？

被擋下的方式有兩種。一種是直接回錯誤，例如 `42501`：根本沒給這個權限。這個專案沒給任何人 update、delete，所以 B 改票數、刪繪馬會收到這種錯誤。另一種是沒有錯誤，但拿到 0 列：有權限，但 RLS 讓那幾列「對你不存在」。B 查 A 的祈願紀錄就是這種，回的是空的清單。這也是本階段開頭那個問題的答案。

只看「有沒有錯誤」，第二種會被誤判成攻擊成功。所以程式把「空的」也算成拒絕，改和刪之後再用沒登入的身分讀一次，確認繪馬還在。

互測要不要讓 Claude 登入去跑？

不要，Claude 也不會這樣做。它的安全規則不拿帳號密碼登入外部服務。這對你也有好處：密碼只在你的 `.env` 和密碼管理器裡，不會留在對話紀錄。程式只印通過與失敗，輸出貼給 Claude 也不會洩漏什麼。

React 會把願望當純文字顯示，還要檢查 `innerHTML` 嗎？

要。React 的 `{}` 會把值當純文字，但它也留了一個會當成程式碼的寫法：`dangerouslySetInnerHTML`。3D 引擎裡也可能有 `innerHTML`。這些地方放進去的只能是固定字串，不能是使用者寫的、或從資料庫讀來的東西。

測試帳號用完要不要刪？

測試帳號是真的帳號，有密碼就能登入。開發用的密碼登入不在正式網站上（階段 15），但密碼還是要夠長、存在密碼管理器裡。不再跑互測時，在後台 Authentication → Users 把兩個測試帳號刪掉，`.env` 的四行也一起刪。

階段 14 那一題再出現一次：「需要登入」算不算權限控管？

「需要登入」的按鈕、輸入框的字數限制，都在前端。這一步的每一項都繞過前端直接打伺服器，所以通過才算權限控管。

🛠️ 動手做

🧰 這一步的產出

```
tests/security/rls-check.mjs            ← 兩個測試帳號互相攻擊，共 22 項，只印通過 / 失敗
supabase/cleanup_security_test.sql      ← 清掉測試留下的「[資安測試]」繪馬
.env.example                            ← 多四個空的欄位：TEST_A_EMAIL、
TEST_A_PASSWORD、TEST_B_EMAIL、TEST_B_PASSWORD
src/app/components/WishCard.jsx                    ← 祈願數先轉成數字再顯示
```

畫面上不會有變化。這一步的產出是一份檢查結果：哪些攻擊被擋下來了。

📋 貼給 AI（19-1：先做不用登入的檢查）

一輪修正做完，請 Claude 帶你往下做。它會先做不用登入就能查的檢查。

你可以照下面這段請 Claude 做（整理版指令）：

貼給 AI

你只需要修改：沒有。整段照貼。

請做不需要登入的資安檢查。每一項告訴我怎麼查的、查到什麼：

1. npm run build 之後，在 dist/ 裡搜管理金鑰：sb_secret_、service_role、eyJ 開頭的長字串
2. 列出 src/ 裡所有用到 innerHTML 的地方，說明放進去的是固定字串，還是會變的資料
3. 執行 npm audit
4. 確認 .env 寫在 .gitignore 裡
5. 確認 dist/ 裡沒有開發用的東西：測試帳號、開發用的密碼登入、window.__ 開頭的除錯變數

搜到這些字串時，先看前後文，判斷它是真的金鑰，還是只是程式裡的一個名稱。

要修改任何東西之前，先列出來等我確認。

名詞：**npm audit** 是 npm 內建的指令，檢查專案裝的套件有沒有已知的安全問題。
你需要知道到什麼程度：知道結果是 0 代表目前沒有已知問題。有的話，把結果貼給 Claude 判斷要不要升級。

Claude 實測的結果：

檢查	結果
建置後的網站裡有沒有管理金鑰	搜 `sb_secret_`、`service_role` 有命中。看前後文，是 supabase-js 判斷金鑰種類的程式（`e.startsWith('sb_secret_')`），不是金鑰。搜不到任何 `eyJ` 開頭的 JWT
`innerHTML`	都只放固定字串：卡片和御神籤上的印章圖案（`dangerouslySetInnerHTML`）、開發用的除錯面板。祈願數由 React 當純文字顯示，另外先轉成數字（`Number(w.votes) || 0`）
`npm audit`	0 個問題
`.env`	在 `.gitignore` 裡
開發用的東西	建置後的網站裡沒有測試帳號、沒有開發用的密碼登入、沒有 `window.__` 開頭的變數

祈願數那一處：React 已經把它當純文字，不會被當成程式碼。還是先轉成數字，因為「這個值一定是數字」是資料庫保證的，不是這一行程式保證的。轉成數字之後，就算哪天資料來源變了，畫面上也只會出現數字。

📋 貼給 AI（19-2：請 Claude 寫互測的程式）

接下來要用兩個帳號互相攻擊。Claude 寫了 `tests/security/rls-check.mjs`，但沒有自己執行。

原因見〈⚖️ 動手前先想〉的「互測要不要讓 Claude 登入去跑」。程式由你在自己的終端機執行。

你可以照下面這段請 Claude 寫（整理版指令）：

貼給 AI

你只需要修改：沒有。整段照貼。

```
請寫一支資安檢查的程式 tests/security/rls-check.mjs，用兩個測試帳號 A、B 互相攻擊，確認資料庫、儲存空間、API 的權限都擋得住。
- 帳號密碼從 .env 的 TEST_A_EMAIL、TEST_A_PASSWORD、TEST_B_EMAIL、TEST_B_PASSWORD 讀。你不要讀 .env，也不要自己登入，程式由我執行
- 每一項只印「通過」或「失敗」，不印權杖、密碼、信箱
- 要測：A 掛繪馬、祈願、重複祈願；B 看不看得到 A 的紀錄；B 冒用 A（寫 A 的 user_id、改票數、刪繪馬、上傳到 A 的資料夾、用 A 的照片）；資料庫的字數限制；超過 2 MB 與 SVG 的上傳；含 HTML 的願望；沒登入的寫入與讀取；使用量限制
- 「被拒絕」有兩種：回錯誤，或沒有錯誤但拿到 0 列。改和刪之後要再讀一次，確認資料沒變
- 測試留下的繪馬，開頭都加「[資安測試]」，另外寫一段 SQL 清掉它們
寫完先告訴我每一項在測什麼、怎麼樣算通過，我確認之後自己執行。
```

你自己做（19-3：建第二個測試帳號）

A 用 15-11 建的那個測試帳號。B 照 15-11 的步驟再建一個，只有兩處不同：

- Email address 填另一個別名，例如 `你的帳號+shrine2@gmail.com`
- User Password 另外設一組，一樣存進密碼管理器

一樣要勾 Auto confirm user，這個帳號才不用收信就能用密碼登入。

你自己做（19–4：在 .env 加四個欄位）

用編輯器打開專案資料夾裡的 `.env`，在最後加四行：

```
TEST_A_EMAIL=你的帳號+shrine@gmail.com
TEST_A_PASSWORD=〈A 的密碼〉
TEST_B_EMAIL=你的帳號+shrine2@gmail.com
TEST_B_PASSWORD=〈B 的密碼〉
```

格式和 15–4 一樣：等號兩邊不要有空白，值不要加引號。

這四行自己打，不要貼給 Claude。`.env` 在 `.gitignore` 裡（19–1 第 4 項確認過），不會被推上 GitHub。

你自己做（19–5：執行互測）

在你自己的終端機，進到專案資料夾，執行：

```
node --env-file=.env tests/security/rls-check.mjs
```

`--env-file` 是 Node 20.6 之後才有的參數。先打 `node -v` 看版本，比 20.6 舊的話請 Claude 教你升級。

你應該看到（預期）：

```
兩個帳號都登入了，開始檢查

✔ 通過  A 用 API 掛繪馬（201）（HTTP 201）
✔ 通過  A 為自己的繪馬祈願（201）（HTTP 201）
✔ 通過  A 再祈願同一塊被擋（409）（HTTP 409）
✔ 通過  B 的「我的參拜紀錄」裡沒有 A 的繪馬與祈願
…
✔ 通過  10 分鐘內連續掛繪馬，超過 5 塊被擋（429）（第 5 次被擋）

共 22 項：22 項通過、0 項失敗
```

✔ 是通過，✖ 是失敗。最後一行的失敗數要是 0。

最後一項寫「第 5 次被擋」是預期的。B 在前面已經掛了一塊含 HTML 的繪馬，再掛 4 塊就到 5 塊的上限。

22 項分成七組。每一項失敗時通常是哪裡設定錯：

組別	項目	失敗時通常是	怎麼修
一、A 自己的正常動作（先確認帶權杖能用，後面的攻擊才有對照）	A 用 API 掛繪馬 `201`	`v1` 沒部署，或 Verify JWT 還開著（階段 17）	回 17-3 確認開關，按 Save changes
一	A 祈願 `201`	`prayers` 的 insert policy 沒建好（階段 16）	把結果貼給 Claude，請它對照 `001` 的 policy
一	同一塊祈願第二次 `409`	`prayers` 的主鍵不是（繪馬, 使用者）這一對	同上。這一項失敗，代表一個人可以無限灌票
二、B 讀 A 的紀錄（登入自己的帳號，去查別人的資料）	B 的「我的參拜紀錄」沒有 A 的東西	`me` 用了 secret 金鑰查資料庫，或查詢沒加「只查自己」的條件（階段 16）	請 Claude 確認 `me` 用的是呼叫者的權杖
二	B 查 `wish_owners` 查不到	`wish_owners` 的 select policy 沒限制「只看自己的」（見階段 16〈⚖️ 動手前先想〉）	policy 的條件要是 `auth.uid() = user_id`。這張表記的就是「誰掛了哪一塊」
二	B 查 A 的 `prayers` 查不到	`prayers` 的 select policy 沒限制「只看自己的」	policy 的條件要是 `auth.uid() = user_id`
三、B 冒用 A（不經過網站，用自己的權杖直接呼叫 Supabase，填別人的 id）	用 A 的 user_id 祈願	開放了寫入 `user_id` 欄位，或 policy 沒檢查 user_id 是不是自己	只開放寫 `wish_id`，`user_id` 由資料庫填成自己（預設值 `auth.uid()`）
三	把 A 的繪馬登記成 A 掛的	`wish_owners` 開放了 insert	擁有者只能由觸發器寫（階段 16）
三	改 A 那塊的票數	`wishes` 開放了 update	票數只能由觸發器加，任何人都不給 update
三	刪 A 的繪馬	`wishes` 開放了 delete	不給 delete。程式會再讀一次，確認繪馬還在
三	上傳照片到 A 的資料夾	storage policy 沒檢查路徑的第一段（階段 18）	路徑第一段要等於自己的 user id

組別	項目	失敗時通常是	怎麼修
三	掛繪馬時用 A 資料夾裡的照片	`wishes.image_path` 的檢查沒限制第一段（階段 18）	同上，寫在 `wishes` 的 policy
四、輸入限制（繞過網頁的輸入框，直接寫資料表或上傳檔案）	願望 41 個字	字數限制只做在前端	資料表加 check：願望最多 40 字
四	署名 13 個字	同上	署名最多 12 字
四	上傳 2.1 MB 的照片	儲存桶沒設大小上限	`ema-images` 的上限設 2 MB（18–2 的 `2097152`）
四	上傳 SVG	儲存桶沒限制類型	只收 `image/webp`、`image/jpeg`。SVG 裡可以藏程式碼
五、含 HTML 的願望	B 掛一塊願望是 `<img src=x onerror=alert(1)>`、署名是 `<b>測試</b>` 的繪馬	這一項「通過」只代表存得進去。資料庫存的就是一串文字	顯示出來的樣子，在 19–6 用瀏覽器看
六、沒登入（只拿網頁裡的公開金鑰）	沒登入用 API 掛繪馬 `401`	`v1` 沒檢查權杖	請 Claude 檢查 `v1` 掛繪馬的那一段
六	沒登入直接寫 `wishes`	給了 `anon` insert 的權限（見階段 16〈⚖️ 動手前先想〉）	收回。不要照錯誤訊息的提示 `GRANT … TO anon`
六	沒登入讀 `wish_owners`	同上	收回
六	沒登入列出所有照片	`ema-images` 建了 select policy（階段 18）	刪掉那條 policy。看照片用公開網址就夠了
七、使用量限制	B 連續掛繪馬，第 6 塊之前要被擋下來（`429`）	使用量限制只做在 `v1` 或前端，沒有放進資料庫的觸發器（階段 17）	放進資料庫的觸發器（`002`）

任何一項是 ✖ ，先不要改測試讓它通過。把那一行貼給 Claude：

貼給 AI

`rls-check.mjs` 的這一項失敗了：**（貼上那一行）**
請先不要修改任何東西。告訴我這一項對應哪一條 `policy` 或設定，最可能的原因是什麼，你準備先檢查哪一個。

你自己做（19-6：確認含 HTML 的願望只顯示成文字）

這一步要在 19-7 清掉測試繪馬之前做。

1. 用 `npm run dev` 開網站，重新整理
2. 在繪馬牆上找開頭是「[資安測試]」、後面接著 `<img src=x onerror=alert(1)>` 的那一塊，點開

你應該看到（預期）：

- 卡片上原樣顯示 `<img src=x onerror=alert(1)>` 這串字
- 署名原樣顯示 `<b>測試</b>`，字沒有變粗
- 沒有跳出任何對話框

如果跳出了對話框，代表某個地方把願望放進了 `innerHTML`，瀏覽器執行了裡面的程式。這種攻擊叫 XSS。

> **名詞**：**XSS**（跨站腳本攻擊）是把程式碼藏在文字裡。別人的瀏覽器顯示那段文字時，就會執行它。
> 你需要知道到什麼程度：知道使用者寫的東西只能用 `textContent` 或 canvas 放上畫面（階段 7）。`alert(1)` 是測試用的無害程式，真的攻擊會改成偷權杖。

你自己做（19-7：清掉測試資料）

`rls-check.mjs` 每跑一次，繪馬牆上會多幾塊開頭是「[資安測試]」的繪馬。

1. 請 Claude 給你 `supabase/cleanup_security_test.sql` 的完整內容
2. SQL Editor 開一個新的查詢，整段貼上，按 Run

這段 SQL 有三段。第一段列出要刪的繪馬，第二段刪掉，第三段數剩下幾塊。`wish_owners`、`prayers` 裡對應的資料會跟著刪（外鍵設了 on delete cascade）。

你應該看到（預期）：最後的結果是「剩下的測試繪馬」`0`。

這段 SQL 只刪資料表裡的繪馬。這一版的互測只上傳了被拒絕的照片，儲存桶裡不會留下檔案。

你自己做（19-8：看 Supabase 的 Security Advisor）

Supabase 會自己掃一次專案的設定，列出它找到的問題。

1. Supabase 後台左側 Advisors → Security Advisor
2. 看列出來的項目

你應該看到（預期）：沒有 ERROR 等級的項目，例如「某張表沒開 RLS」。三張資料表在階段 16 都開了 RLS。

WARN 的項目，一條一條貼給 Claude 判斷，不要直接照頁面上的建議改：

貼給 AI

```
Supabase 的 Security Advisor 列了這一項：【貼上標題與說明】
請先不要修改任何東西。告訴我它指的是哪個設定、對這個網站有沒有影響，如果要改，會不會影響現在能用的功能。
```

理由和階段 16 的 GRANT 那一題一樣：通用的建議不知道這個網站的設計。

✅ 怎麼知道做對了

- [ ] 19-1：建置後的網站沒有管理金鑰。`innerHTML` 只放固定字串或轉過的數字。`npm audit` 0。`.env` 在 `.gitignore`。沒有開發用的東西（實測）
- [ ] 19-5：最後一行是「22 項通過、0 項失敗」（預期）
- [ ] 19-6：含 HTML 的願望原樣顯示成文字，沒有跳出對話框（預期）
- [ ] 19-7：「剩下的測試繪馬」是 `0`（預期）
- [ ] 19-8：Security Advisor 沒有 ERROR（預期）

第 1 項是 Claude 實測的。第 2 到 5 項是預期結果，這一版沒有它們的實際輸出與截圖。拿你自己的畫面對照。

🆘 如果沒有成功

你可能會看到	通常是因為	怎麼辦
`.env 缺少 TEST_A_EMAIL`	沒加那四行，或名稱打錯	回 19-4，名稱要一字不差

你可能會看到	通常是因為	怎麼辦
`帳號 B 登不進去：Invalid login credentials`	信箱或密碼打錯，或建帳號時沒勾 Auto confirm user	到後台 Authentication → Users 確認帳號在、已確認；密碼不確定就在後台重設
`bad option: --env-file`	Node 太舊	「請教我把 Node 升級到 20.6 以上」
`Cannot find package '@supabase/supabase-js'`	不是在專案資料夾執行	先進到專案資料夾再執行
第二次跑，「含 HTML 的願望」失敗、`HTTP 429`	10 分鐘內 B 已經掛滿 5 塊	等 10 分鐘再跑。這也表示使用量限制擋住了
任何一項 ✖	那一項對應的權限沒設好	用 19–5 最後那段指令問 Claude。先找原因，不要改測試
打開含 HTML 的繪馬，跳出對話框	某處把願望放進了 `innerHTML`	「請列出所有用到 innerHTML 的地方，告訴我哪一處放了使用者寫的文字，先不要修改」

這一步失敗可以跳過嗎：19–1 不行。互測可以晚一點做，但要在讓別人登入之前做完。

📖 為什麼這樣做

🎯 這一步的重點

前面的階段都在確認「我做得到」。這一步確認「別人做不到」。兩件事要分開測，前者通過不代表後者通過。

🧭 這一步在做什麼

案例四一開始的五個目標之一是「可以經過資安驗證」。很多檢查在前面的階段已經做過，這一步補上要兩個帳號才能測的部分。

前面的階段已經實測過的：

檢查	結果	階段
沒登入讀 `prayers`、`wish_owners`，新增繪馬，改票數	都是 `42501` 拒絕；票數再讀還是原本的值	16

檢查	結果	階段
`me` 沒帶權杖、帶假權杖	`401 missing_token`、`401 invalid_token`	16
`v1` 沒權杖掛繪馬、祈願、讀紀錄	`401 missing_token`	17
`v1` 不存在的繪馬、不存在的端點、亂寫的 `before`	`404`、`404`、`400`	17
照片有沒有 EXIF	從外面下載，找不到 EXIF 與 GPS	18
沒登入上傳照片、列出所有照片	拒絕；列出來是空陣列	18
建置後的網站：金鑰、`innerHTML`、套件、`.env`、開發用的東西	見 19–1	19

這一步新加的，是要兩個帳號才能測的部分：B 能不能讀、改、冒用 A 的東西。

🧩 它在建構流程的位置

```
攻擊者（B，或沒登入的人）
      │  直接呼叫，不經過網頁
      ├──▶ v1 API ──────────▶ 檢查權杖
      ├──▶ 資料表 ──────────▶ RLS、check、觸發器（使用量、票數）
      └──▶ 儲存空間 ────────▶ 儲存桶的類型與大小、storage policy（資料夾）

網頁 ──▶ 顯示願望 ──────────▶ textContent、canvas（不執行裡面的程式）
```

除了最後一列，每一道限制都在伺服器上。`rls-check.mjs` 不經過網頁，直接打上面三條路。

💡 那段指令為什麼那樣寫

19–1 的「搜到這些字串時，先看前後文」：沒有這一句，Claude 可能看到 `sb_secret_` 就回報「有金鑰外流」。也可能反過來，看到了卻不報。要它說出判斷的依據。「要修改任何東西之前，先列出來等我確認」：資安檢查找到問題時，先知道問題在哪。然後再決定怎麼修。

19–2 的「你不要讀 .env，也不要自己登入」：寫明程式由你執行，Claude 就不會試著去拿密碼。「改和刪之後要再讀一次」：沒寫的話，程式可能只看有沒有錯誤。「拿到 0 列」會被當成攻擊成功，真的改成功了反而可能被當成通過。「開頭都加［資安測試］」：測試資料和真的繪馬放在同一張表。要有一個記號才刪得乾淨，不會刪到別人的繪馬。最後一句「先告訴我每一項在測什麼、怎麼樣算通過」：22 項的通過條件，要在執行之前先看過一次。

階段 20　部署到 Vercel

📖 讀完約 15 分鐘　🛠️ 動手約 40 分鐘

> - **這一步一定要懂**：推上 GitHub 之前，先看「會被推上去的檔案清單」。部署成功不等於網站接對了，要打開正式網站，看它實際發出了哪些請求。Astro 的 `PUBLIC_` 變數是建置那一刻寫進網頁的，補了變數一定要重新部署。
> - **這一步可以先不懂**：git 的指令、Vercel 怎麼建置 Astro。
> - **做完後你要能用自己的話回答**：Vercel 顯示部署成功，網站也打得開。為什麼正式網站其實在用假的登入？

⚖️ 動手前先想

GitHub 專案要公開還是私人？

私人。專案裡有授權不明的音檔，開發日誌記了很多內部的細節。公開的話，任何人都能複製。要給學員的版本另外打包，打包時會排除 `.env` 與音檔。

部署成功了，網站算不算接對了？

不算。20–3 會遇到這種情況：網站部署成功、每個檔案都 `200`、3D 場景正常，用的卻是假後端。看「接對了沒」，要看網站實際發出的請求：有沒有送到 `supabase.co`，狀態是多少。

沒填變數就退回假後端，這個設計要不要留？

本機要留：學員還沒建 Supabase 也能把網站跑起來。正式網站不該有：做到 20–3 時會看到，假的登入就這樣上線了，沒有任何地方報錯。所以提議的做法是只在 Vercel 上建置時檢查，少了就讓建置失敗。本機照舊。

環境變數的 Type 要選 Secret 還是 Config？

看這個值會不會出現在瀏覽器。`PUBLIC_` 開頭的會被寫進網頁，任何人打開開發者工具都看得到。選 Secret 也藏不住，選 Config 還方便之後確認值有沒有填對。真的不能外流的金鑰（secret、`service_role`），不應該放在 `PUBLIC_` 開頭的變數裡。這個前端專案的設定裡，也不應該出現它們。

🛠️ 動手做

🧰 這一步的產出

```
GitHub 私人專案（預設分支 main）          ← 程式碼，不含 .env、node_modules、dist
Vercel 專案（Preset：Astro）              ← 兩個環境變數：
PUBLIC_SUPABASE_URL、PUBLIC_SUPABASE_ANON_KEY
https://<你的網址>.vercel.app             ← 正式網站
https://www.<新網域>                      ← 選做：自己的網域，沒有 www 的轉過來（20-8）
Resend + Supabase 的 SMTP Settings        ← 選做：自己的寄信服務，誰都收得到登入信（20-9）
```

畫面上應該看到：用正式網址打開，3D 場景、繪馬牆、聲音都和本機一樣。登入視窗的按鈕寫「寄登入連結」，繪馬牆讀的是 Supabase 的資料。

> **名詞**：**GitHub** 是放程式碼的網站。**私人專案**只有你和你邀請的人看得到。
> 你需要知道到什麼程度：知道推上去的是「被 git 追蹤的檔案」，`.gitignore` 列的檔案不會上去。推上去之後，就算再刪掉，舊的版本還留在紀錄裡。

> **名詞**：**Vercel** 是把網站放上網路的服務。它從 GitHub 拿程式碼，幫你建置，再給你一個網址。
> 你需要知道到什麼程度：知道接一次之後，每次推上 GitHub 它就自動重新部署。網站的設定（環境變數）要在 Vercel 的後台另外填，它讀不到你電腦上的 `.env`。

📋 貼給 AI（20-1：推上 GitHub）

推上 GitHub，貼：

貼給 AI

```
先幫我連接 Github 並且進行推送
```

Claude 會先問你兩件事，從選項裡選：

問題	實際選的	選項上寫的後果
GitHub 專案要公開還是私人	私人	只有自己看得到；之後連 Vercel 一樣可以自動部署
音檔要不要推上去	全部推上去	包含 Merry Christmas。私人專案不算公開散布，但之後部署到 Vercel 就會公開播放

第二題的後果後來真的發生了，見〈上線之後〉。

推上去之前，Claude 做了四項檢查：

檢查	怎麼查	結果
不該上去的檔案	`.gitignore` 排除 `.env`、`node_modules`、`dist`、`.astro`；再用 `git ls-files` 列出實際要推的檔案	`.env`、`node_modules`、`dist` 都不在清單裡
金鑰	搜 `sb_secret_`、`service_role`、`eyJhbGci`、`TEST_A_PASSWORD=`	只有文件裡提到這些名稱的句子，沒有真的金鑰
個資	讀過要推的文件	開發日誌裡出現一次個人信箱（測試帳號的別名），改成「〈信箱〉+shrine」
大小	數檔案	169 個檔案、30 MB。音檔 15 MB，其餘大部分是教材的圖片

接著執行三個指令。`git init -b main` 建立版本紀錄，接著做第一個提交。最後用下面這一行建私人專案並推上去：

```
gh repo create <專案名稱> --private --source=. --remote=origin --push
```

你可以照下面這段請 Claude 做（整理版指令）：

貼給 AI

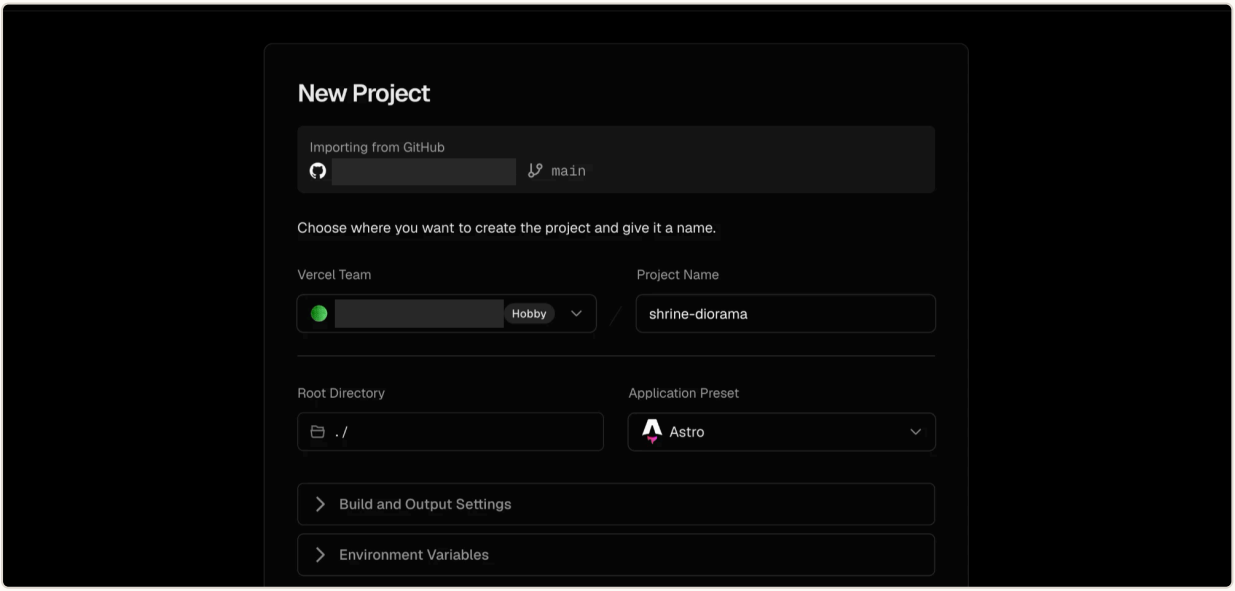
你只需要修改：沒有。整段照貼。

```
請把這個專案推上 GitHub，建成私人專案。推之前先做這幾件事，結果都告訴我：
1. 用 git status 和 git ls-files 列出會被推上去的檔案，確認 .env、node_modules、dist 都不在
2. 搜金鑰的樣式：sb_secret_、service_role、eyJhbGci、TEST_A_PASSWORD=。每一處告訴我是真的金鑰，還是只提到名稱
3. 看文件和開發日誌裡有沒有我的信箱或其他個資
4. 把要執行的 git 和 gh 指令列給我看，等我確認再執行
```

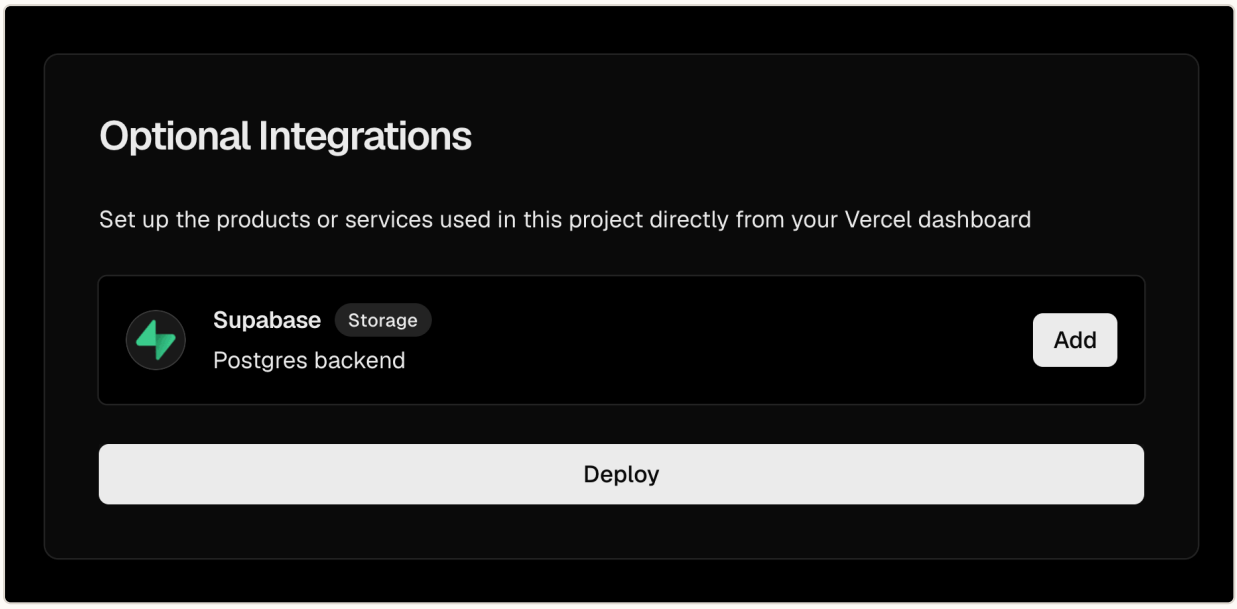
第 4 點是這個課程的規則：Claude 可以幫你推，但執行前你要看得到它要做什麼。

你自己做（20-2：在 Vercel 匯入專案）

1. 在 Vercel 新增一個專案，選 Import，找到剛才推上去的 GitHub 專案
2. Application Preset 會自動認出 Astro，Root Directory 維持 `./`
3. 展開下面的 Environment Variables，加兩個變數（名稱與值見 20-4）。實際做的時候沒有展開這一格，所以有了 20-3
4. 按 Create Project



接著會出現 Optional Integrations，裡面有一個 Supabase 的 Add：



不要按 Add。它會另外建一個新的 Supabase，不是你在階段 15 建的那個。直接按 Deploy。

📋 貼給 AI（20–3：部署好了，卻在用假的登入）

按了 Deploy、網站上線之後，把這一句和正式網址貼給 Claude，請它檢查：

貼給 AI

部署玩囉

「玩」是「完」。

Claude 先檢查正式網站：首頁、`/api`、網站圖示、音檔都是 `200`，3D 場景正常。

接著用瀏覽器打開，發現三件事：

- 整個頁面沒有任何一個請求送到 `supabase.co`
- 登入視窗的按鈕寫「寄登入碼」，「原型」的說明是打開的
- API 文件頁顯示「沒有接上 Supabase」

症狀 → 原因：

- 這三件事都是假後端的樣子
- 建置的時候沒有拿到 `PUBLIC_SUPABASE_URL` 和 `PUBLIC_SUPABASE_ANON_KEY`。拿不到，網站就退回假後端，這是階段 14、15 的設計

- Astro 的 `PUBLIC_` 變數是在建置的那一刻寫進網頁的。Vercel 上沒有這兩個變數，建出來的網頁裡就是空的

> **名詞**：**環境變數**是放在程式外面的設定值，例如網址和金鑰。本機放在 `.env`，Vercel 放在它的後台。
> 你需要知道到什麼程度：知道名稱以 `PUBLIC_` 開頭的，Astro 會在建置時直接寫進網頁。所以補了、改了之後，一定要重新部署一次，網頁裡的值才會變。

這個結果的風險：本機開發時，「沒填就退回假後端」很方便。到了正式網站，就變成悄悄上線一個任何人都能用 `123456` 登入的假介面。假後端寫不進真的資料庫，但使用者會以為自己登入了、繪馬掛上了。

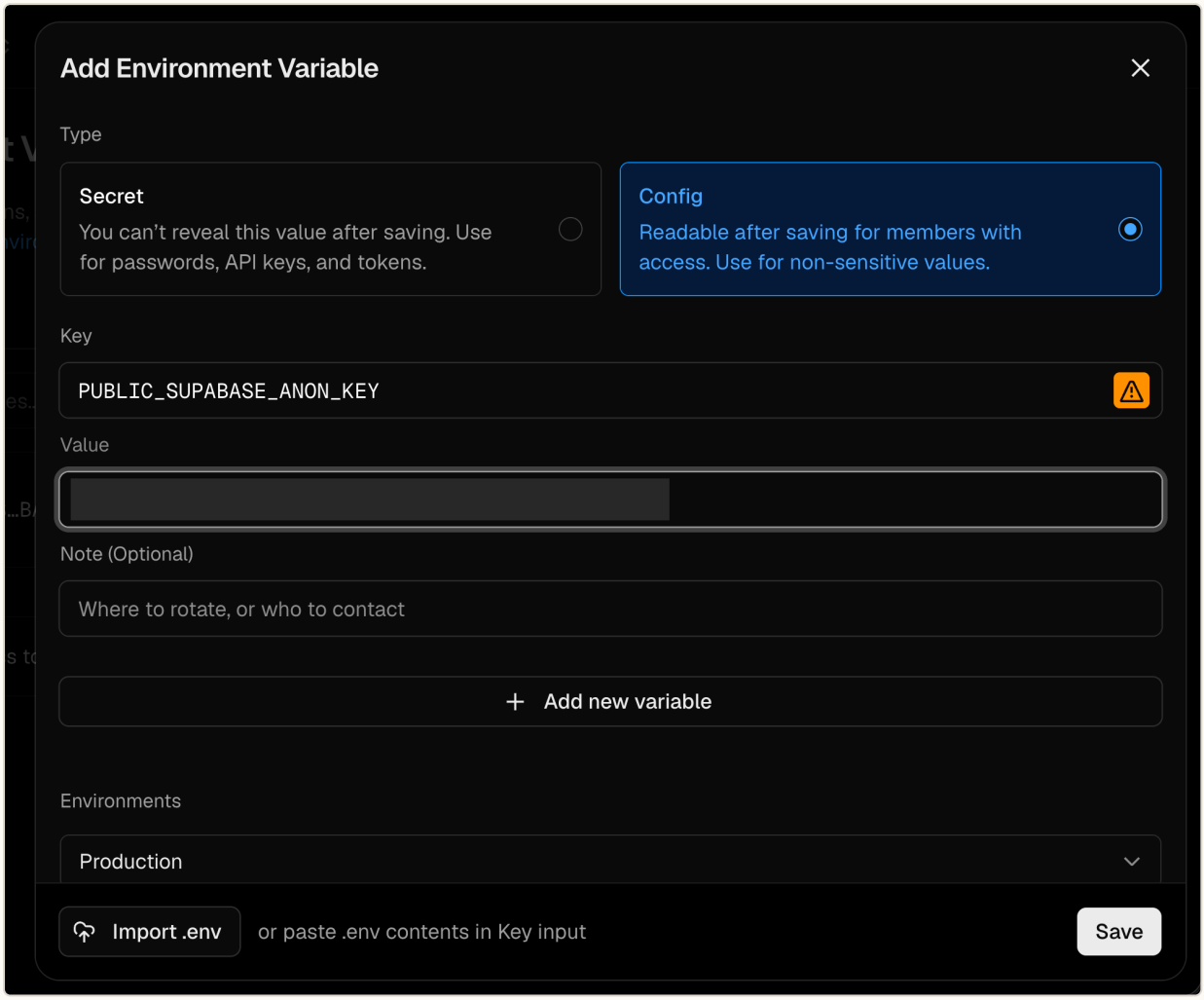
📋 貼給 AI（20–4：環境變數要在哪裡設）

Claude 會請你在 Vercel 補上兩個變數。不知道在哪裡，就問：

貼給 AI

```
我該在哪裡設置呢？
```

Claude 給的位置：Vercel 專案的 Settings → Environment Variables → Add Environment Variable。

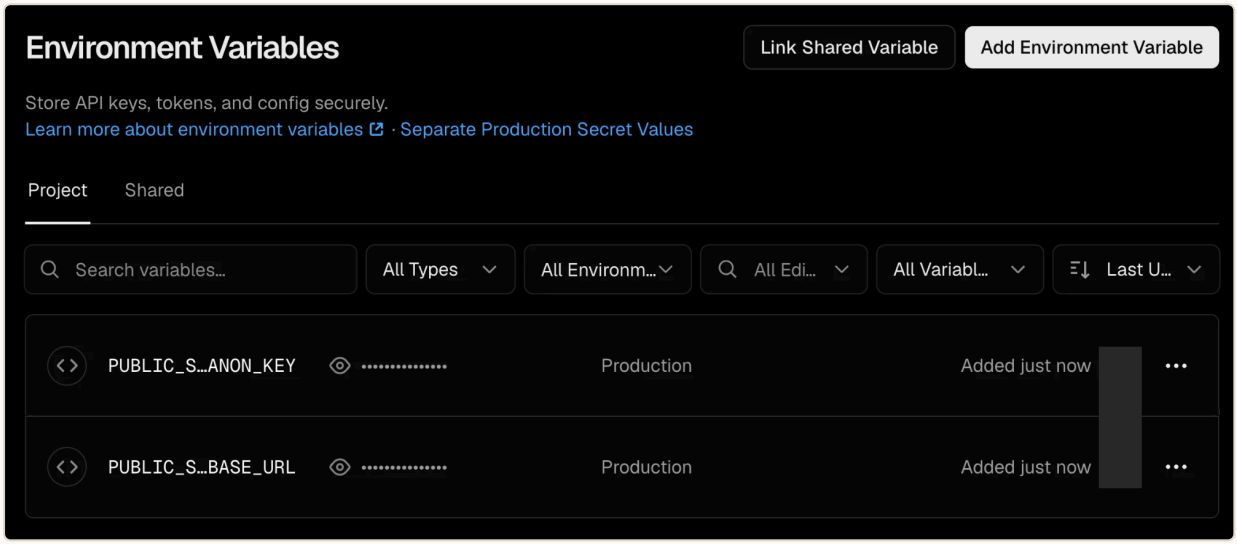


每一個變數這樣填：

欄位	填什麼	為什麼
Type	Config	這兩個值本來就會出現在瀏覽器裡。Secret 是給密碼、管理金鑰用的，存了之後看不到值
Key	`PUBLIC_SUPABASE_URL`、`PUBLIC_SUPABASE_ANON_KEY`	名稱要和 `.env` 一字不差。少了 `PUBLIC_`，Astro 不會把它放進網頁
Value	你 `.env` 裡對應的那一行等號後面的值	自己複製貼上，不用貼給 Claude
Environments	Production	正式網站用的。只選 Production 的話，Vercel 的預覽部署（Preview）還是會退回假後端

`PUBLIC_SUPABASE_ANON_KEY` 旁邊有一個黃色的警告圖示。名稱以 `PUBLIC_` 開頭、又有 `KEY`，Vercel 提醒你這個值會出現在瀏覽器裡。這把是公開金鑰，本來就會出現在瀏覽器（階段 15）。如果你要填的是 secret 或 `service_role` 金鑰，這個警告就是最後一次提醒：不要填。

兩個都存好之後，列表上會有兩列：



你自己做（20-5：重新部署）

變數加好了，網站還是舊的。網頁是在加變數之前建置的。

1. Vercel 專案的 Deployments
2. 最新那一筆右邊的「⋯」→ Redeploy

做完告訴 Claude，請它再檢查一次：

貼給 AI

```
已經 Redeploy 囉
```

📋 貼給 AI（20-6：確認正式網站真的接上了）

Claude 用瀏覽器重新檢查正式網站，下面指令裡的四項都正常。有一個 `200 /rest/v1/wishes` 的請求，按鈕寫「寄登入連結」，`/api` 不帶權杖回 `HTTP 401`，Console 0 個錯誤。

你自己也可以這樣確認：

1. 用 Chrome 打開正式網址，按 F12 打開開發者工具，切到 Network 分頁
2. 重新整理，在上方的篩選框打 `supabase`
3. 看得到一個 `wishes` 的請求，狀態是 `200`，就是接上了。一個都沒有，就是還在用假後端

也可以請 Claude 做（整理版指令）：

貼給 AI

你只需要修改：`<你的網址>` 換成你的正式網址。

```
請打開 https://<你的網址>.vercel.app/，告訴我：
1. 頁面有沒有送出任何請求到 supabase.co，狀態碼是多少
2. 登入視窗的按鈕寫什麼，有沒有開發用的密碼登入
3. /api 頁面不帶權杖試呼叫，回的是什麼
4. Console 有沒有錯誤
這一步只看結果，不要修改任何東西。
```

你自己做（20-7：Supabase 加上正式網址）

信裡的登入連結要能回到正式網站，Supabase 要知道這個網址。

1. 打開 URL Configuration（15-7 的連結或選單路徑）
2. Redirect URLs 按 Add URL，填 `https://<你的網址>.vercel.app/**`，存檔
3. Site URL 改成 `https://<你的網址>.vercel.app`，按那一格的 Save changes

`http://localhost:4324/**` 留在 Redirect URLs 裡，本機開發照常可以登入。寄信時程式會指定回到哪個網址，名單裡有就會回到那裡（15-8）。

2026-10-23 實測：在正式網址寄登入連結，點信裡的連結回到正式網站，登入成功。

你自己做（20-8：掛自己的網域，選做）

`<你的網址>.vercel.app` 可以一直用。想換成自己的網域（例如 `wishingshrine.com`）再做這一節。

先想兩個問題：網域的錢要付給誰？掛上去之後，專案裡還有哪些地方寫著舊網址？

可以先問 Claude：

貼給 AI

```
Vercel 如果要掛 Custom Domain 會需要額外付費嗎？
```

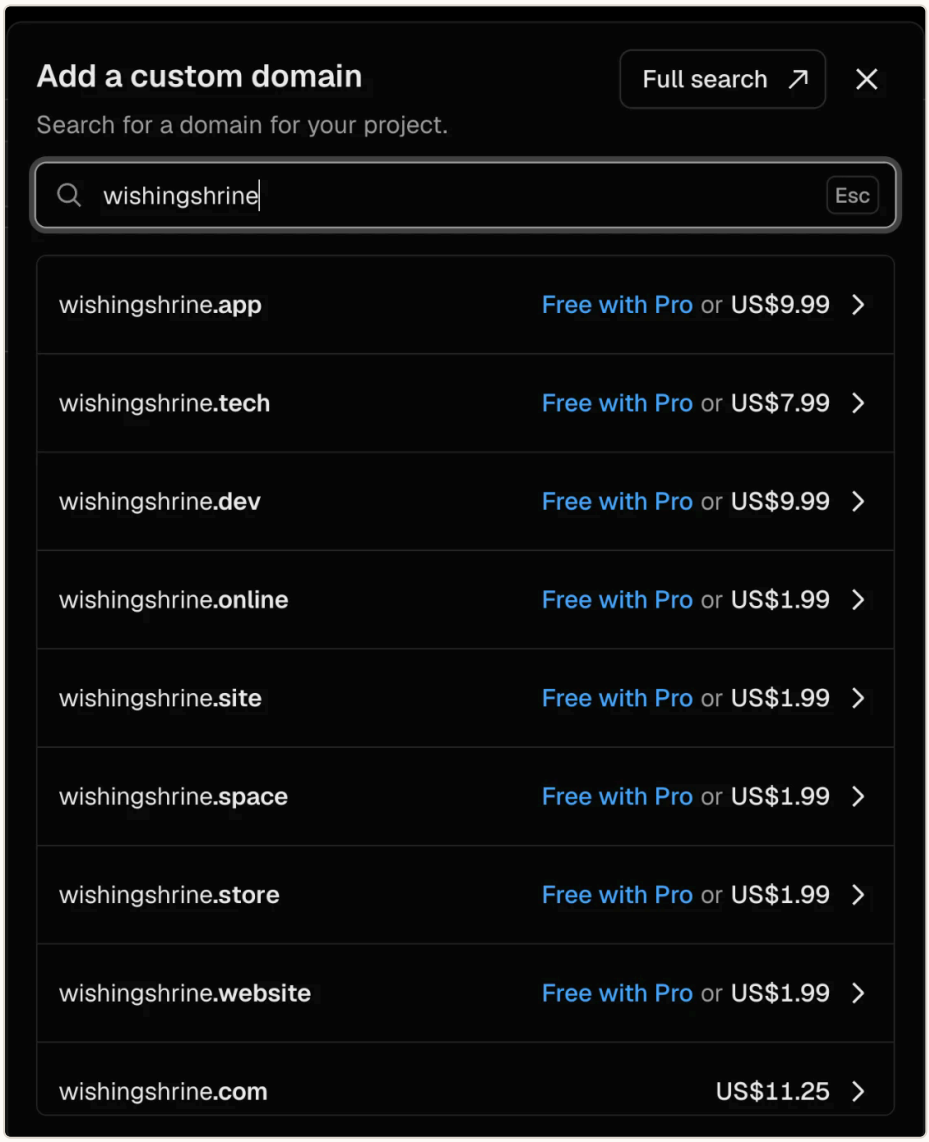
Claude 的回答：

- 把網域掛到 Vercel 專案上，Hobby 方案不另外收費，HTTPS 憑證也是自動、免費的。要付錢的是網域本身
- Hobby 方案限個人、非商業用途。拿來收費的課程或商業服務，要用 Pro。Pro 按每位成員每月計費，價格以 Vercel 官方頁為準

也可以問「有你覺得合適的網域嗎？」。Claude 查了幾個還沒被註冊的，推薦 `wishingshrine.com`：和分享預覽圖上的「WISHING SHRINE」一致，好唸也好拼。選名稱時避開連字號，口頭講網址時很容易漏掉。已經有自己的網域的話，用子網域（例如 `shrine.你的網域`）不用另外花錢。

步驟：

1. Vercel 專案的 Domains，按 Add a custom domain
2. 在搜尋框輸入名稱，看哪些後綴還能買、多少錢



截圖當時（2026-10-23）的價格。第一年的價格和續約價可能不同，以結帳頁為準：

後綴	價格	截圖上的標示
`.com`	US$11.25	無
`.app` 、 `.dev`	US$9.99	Free with Pro
`.tech`	US$7.99	Free with Pro
`.online` 、 `.site` 、 `.space` 、 `.store` 、 `.website`	US$1.99	Free with Pro

US$1.99 的那幾個特別要看清楚續約價。第一年便宜，第二年起可能貴很多。

3. 選好網域、付款之後，出現 Add Domains 對話框。實際的設定：
    - 網域填 `wishingshrine.com`
    - 「Redirect apex domains to www (recommended)」維持預設勾選：沒有 www 的網址會轉到有 www 的，正式網址就是 `www.wishingshrine.com`
    - 「Connect to an environment」選 Production
    - 按 Add Domain
4. 回到 Domains 列表，會看到三列：

網域	狀態
`wishingshrine.com`	308 轉到 `www.wishingshrine.com`
`www.wishingshrine.com`	Production
`<你的網址>.vercel.app`	Production，Valid Configuration

前兩列一開始顯示「Generating SSL Certificate」，HTTPS 憑證還在發，等它跑完再打開。

先自己想一題：為什麼要選一個正式網址（有 www 或沒有 www）？為什麼不讓兩個都直接接到網站？答案：分享預覽、登入信的連結都要寫完整網址，只能寫一個。另一個轉過去，使用者打哪一個都會到同一個地方。

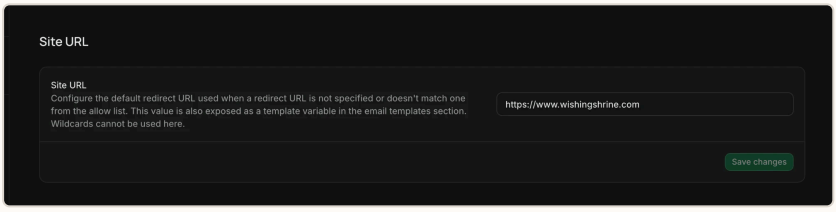
掛好之後要改的三個地方：

1. `astro.config.mjs` 的 `site` 改成正式網址。分享預覽的 `og:url` 、 `og:image` 用它算出完整網址，沒改的話，貼到社群的預覽圖還是指向舊網址。參考成品已經改成 `https://www.wishingshrine.com` ，建置後 `og:image` 是 `https://www.wishingshrine.com/og.jpg`
2. 到 Supabase → Authentication → URL Configuration（位置和 20-7 一樣）。下面寫的是參考成品的網域，換成你的：

- Site URL 改成 `https://www.wishingshrine.com`，再按那一格自己的 Save changes。按鈕還是亮綠色，就表示還沒存
- 在 Redirect URLs 按 Add URL，加上 `https://www.wishingshrine.com/**`。舊的 `https://<你的網址>.vercel.app/**` 那一筆先保留，已經寄出的登入信才還能用。每一筆都要從 `https://` 開頭、`/**` 結尾，少了 Supabase 可能比對不到
- 不改的話，在新網址寄出的登入信，連結會跳回舊網址或 `localhost`。見 🆘 的「掛了新網域之後，點登入信的連結還是跳回舊網址」那一列

3. 舊的 `<你的網址>.vercel.app` 不用刪，它還是可以用

改好之後的設定：

欄位	參考畫面
Site URL	 Save changes 變暗，表示已經存好。

Redirect URLs 改好之後是這三筆：

網址	給誰用
`http://localhost:4324/**`	本機開發（只有這一筆是 `http://`）
`https://www.wishingshrine.com/**`	新的正式網址
`https://<你的網址>.vercel.app/**`	舊網址，已經寄出的登入信還會用到

2026-10-23 實測：在 `www.wishingshrine.com` 寄登入連結，點信裡的連結回到 `www.wishingshrine.com`，登入成功。

第 1 點可以交給 Claude，第 2 點要你自己在後台改：

貼給 AI

你只需要修改：`<新網域>` 換成你買的網域。

```
我把網站掛到新網域 https://www.<新網域> 了（沒有 www 的會轉過來）。
請把 astro.config.mjs 的 site 改成這個網址，並搜尋專案裡還有沒有寫死舊網址的地方，列給我看。
另外告訴我 Supabase 的 URL Configuration 裡，Site URL 和 Redirect URLs 各要填什麼。改完推上去之前先給我看差異。
```

你自己做（20-9：接上 Resend 寄登入信，選做）

先想一題：登入信是誰寄出去的？為什麼你收得到，別人卻收不到？

到這一步為止，登入信都是 Supabase 內建的寄信服務寄的。它有兩個限制：整個專案每小時約 2 封，而且只寄給專案成員的信箱（階段 15）。所以一般訪客根本收不到登入信。內建的額度太少，所以要接上自己的寄信服務（自訂 SMTP）。接上之後，兩個限制都解除，網站的程式不用改。

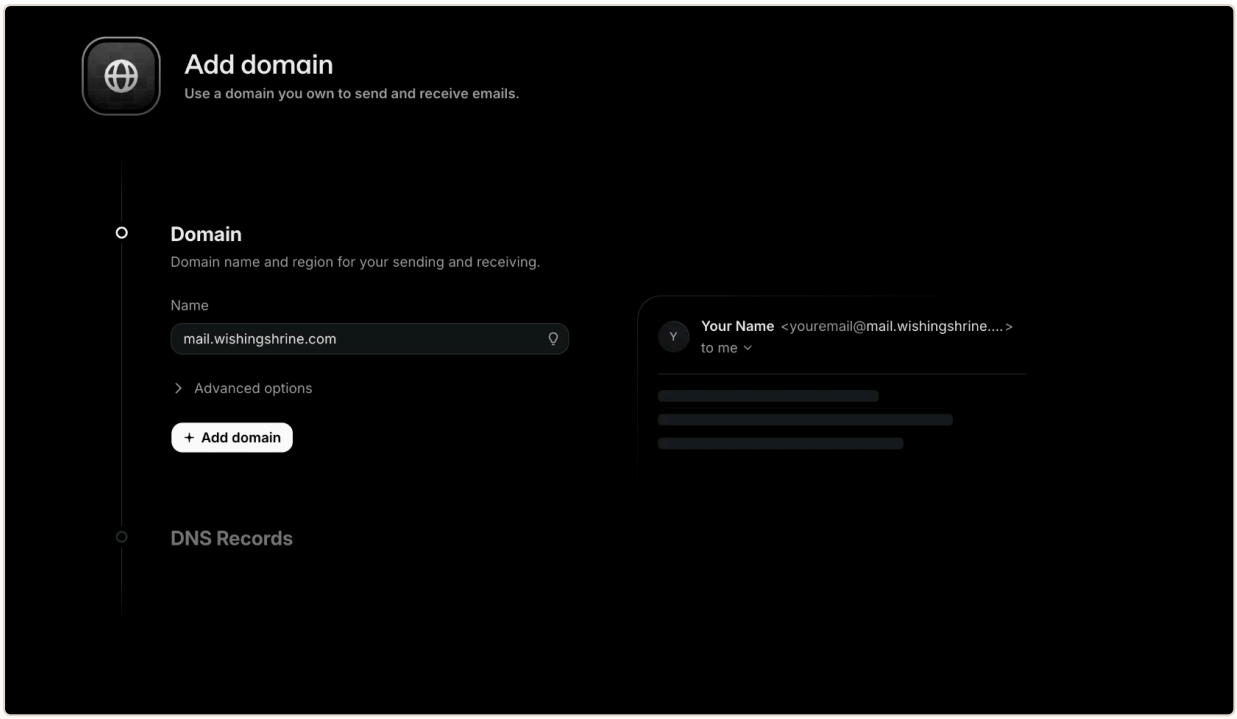
> **名詞**：**SMTP** 是寄電子郵件用的通訊方式。**自訂 SMTP** 是叫 Supabase 改用你指定的寄信服務寄登入信。
> 你需要知道到什麼程度：知道要填的是主機、埠號、帳號、密碼、寄件人這幾欄。值由寄信服務提供。

可以選的寄信服務（額度與價格以各家官網為準）：

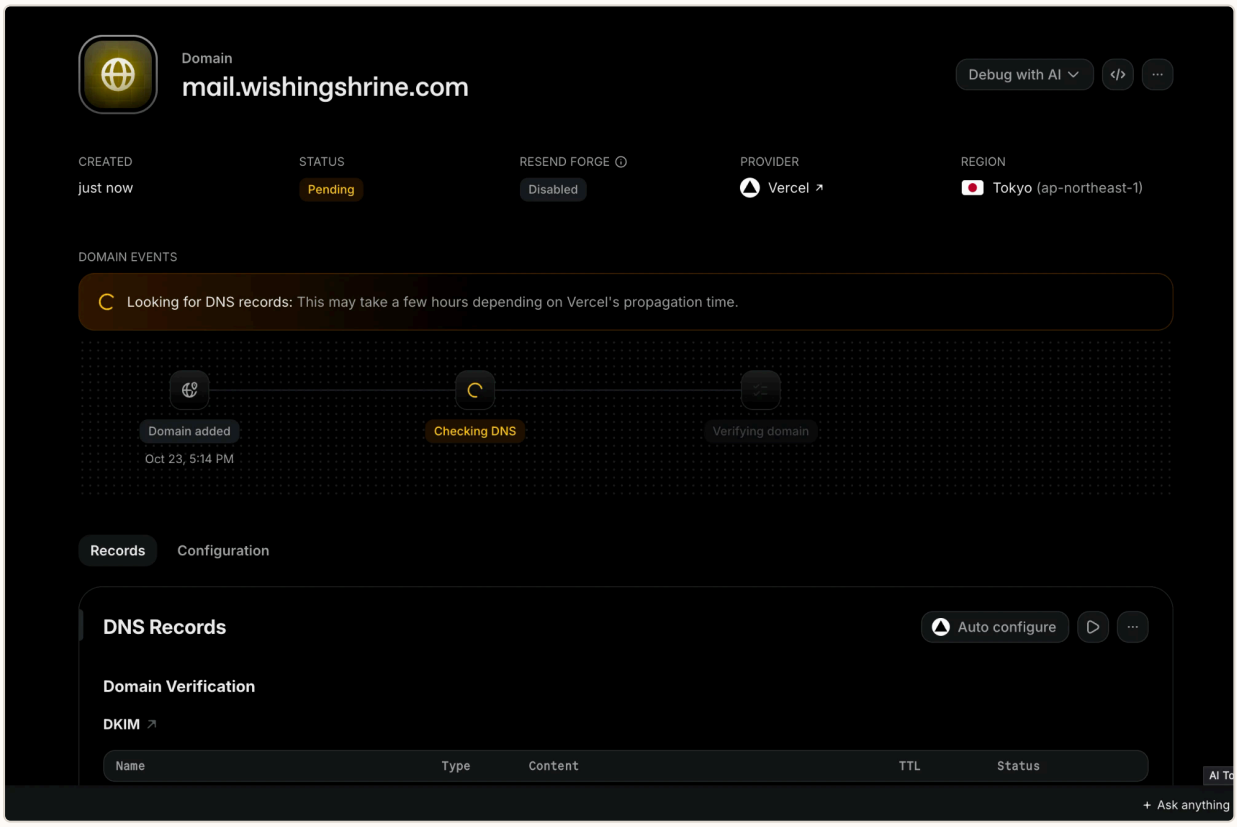
服務	免費方案（截至 2026-10）	備註
Resend	每月約 3,000 封、每天約 100 封、1 個網域	實際用的。網域在 Vercel 時可以自動設定 DNS，Supabase 官方也有接 Resend 的教學
Brevo、Amazon SES、Postmark	各家不同	也可以用，步驟類似：驗證網域、拿 SMTP 帳號密碼、填進 Supabase

步驟（2026-10-23 實測全部做完）：

1. 註冊 Resend，到 Domains → Add domain，Name 填子網域 `mail.wishingshrine.com`（換成你的網域），按 Add domain。用子網域寄信，以後主網域要收信或寄電子報，設定不會互相影響



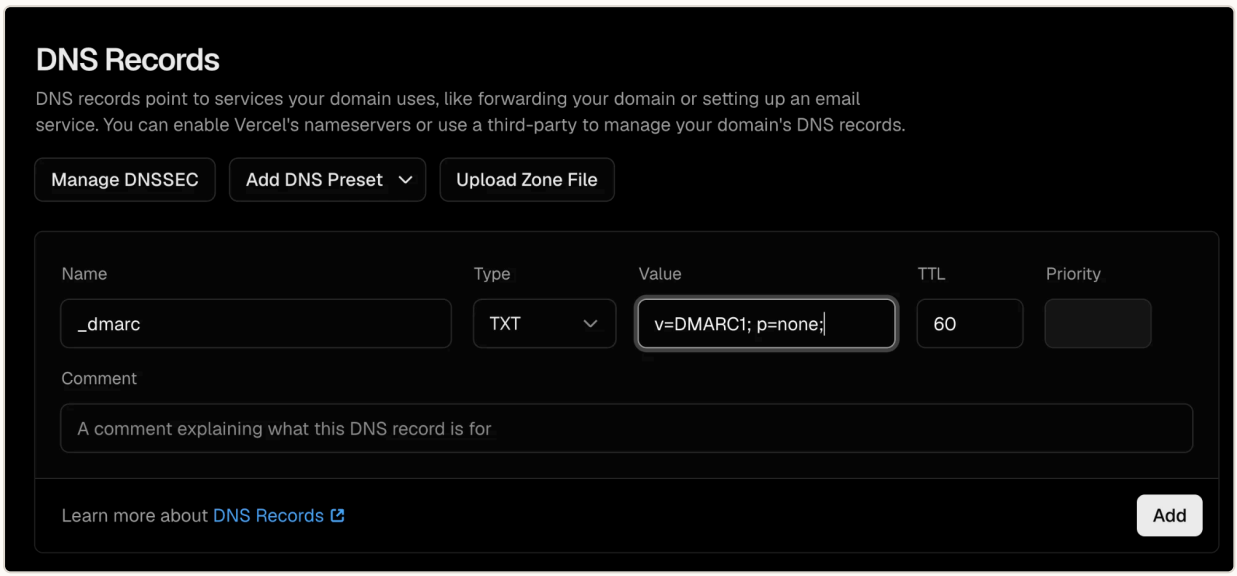
2. 加完之後，網域的狀態是 Pending，要把 Resend 給的 DNS 紀錄加到網域上，它才會驗證通過



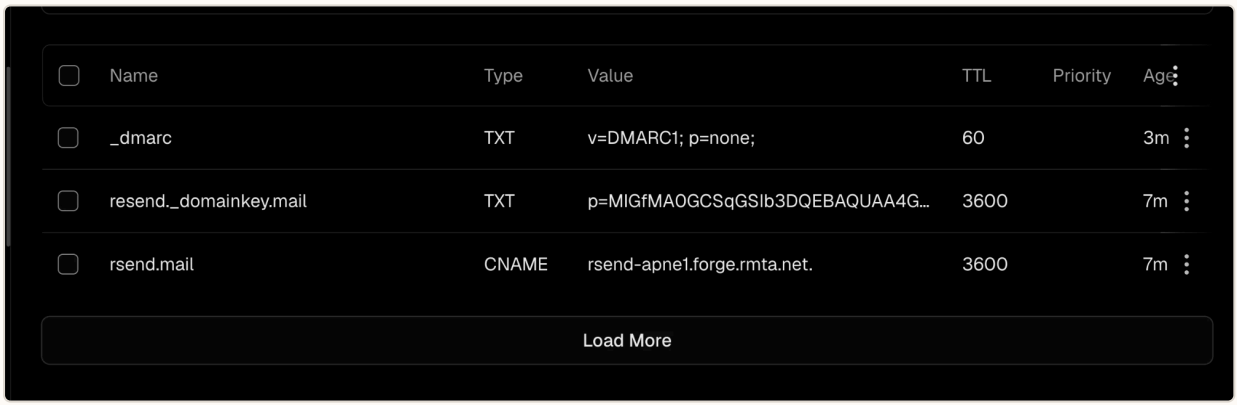
Resend 認出網域在 Vercel（PROVIDER 那一欄寫 Vercel），所以 DNS Records 旁邊有 **Auto configure**。按下去、授權 Vercel，它會自動把 DKIM、SPF 這些紀錄加進 Vercel。 不想授權的話，用手動的：到 Vercel 的網域頁，點最下面的「View DNS Records & More for

wishingshrine.com」。照 Resend 的表格一筆一筆加（Name、Type，Value 填 Resend 表格的 Content）。每一筆填完都要按右下角的 Add，才會存進去

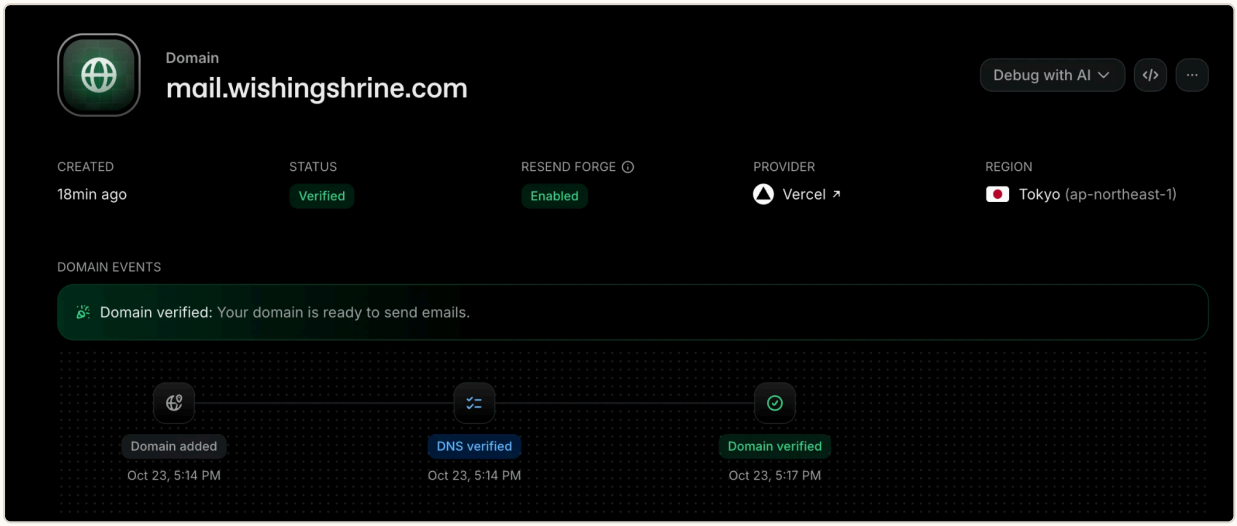
另外建議加一筆 `_dmarc` 的 TXT 紀錄，值是 `v=DMARC1; p=none;`，信比較不會被當成垃圾信



加完之後，Vercel 的 DNS 列表應該看得到這幾筆：

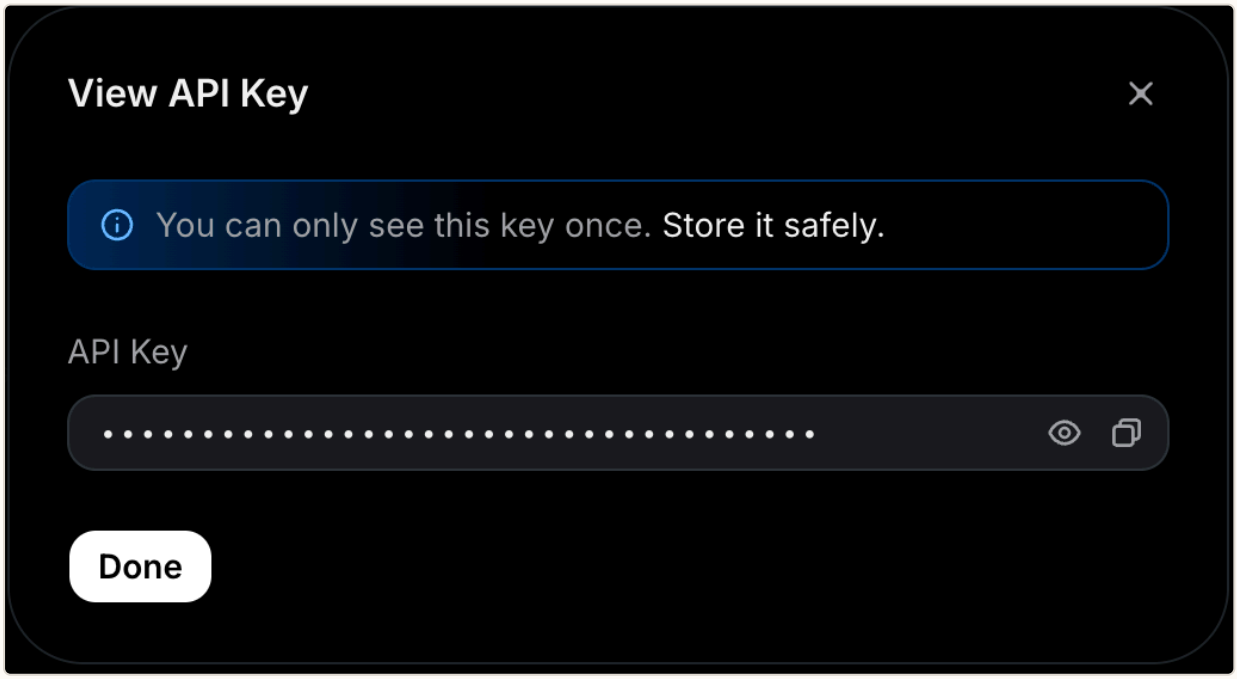


`resend._domainkey.mail` 是 DKIM，`_dmarc` 是剛才加的 DMARC。列表一頁放不下，Resend 表格裡其他的紀錄（例如 `send.mail`）按 Load More 才看得到。對照 Resend 的表格，每一筆都要在。 等 Resend 的狀態從 Pending 變成 Verified。畫面上寫著可能要幾個小時，實測約 3 分鐘



> **名詞**：**DNS 紀錄**是網域的設定表，告訴別人「這個網域的信由誰寄、網站在哪裡」。**DKIM**、**SPF** 是其中兩種，收信的一方用它們確認這封信真的是你寄的。
> 你需要知道到什麼程度：知道要照寄信服務給的表格原樣加，加完要等它驗證通過。不用自己算值。

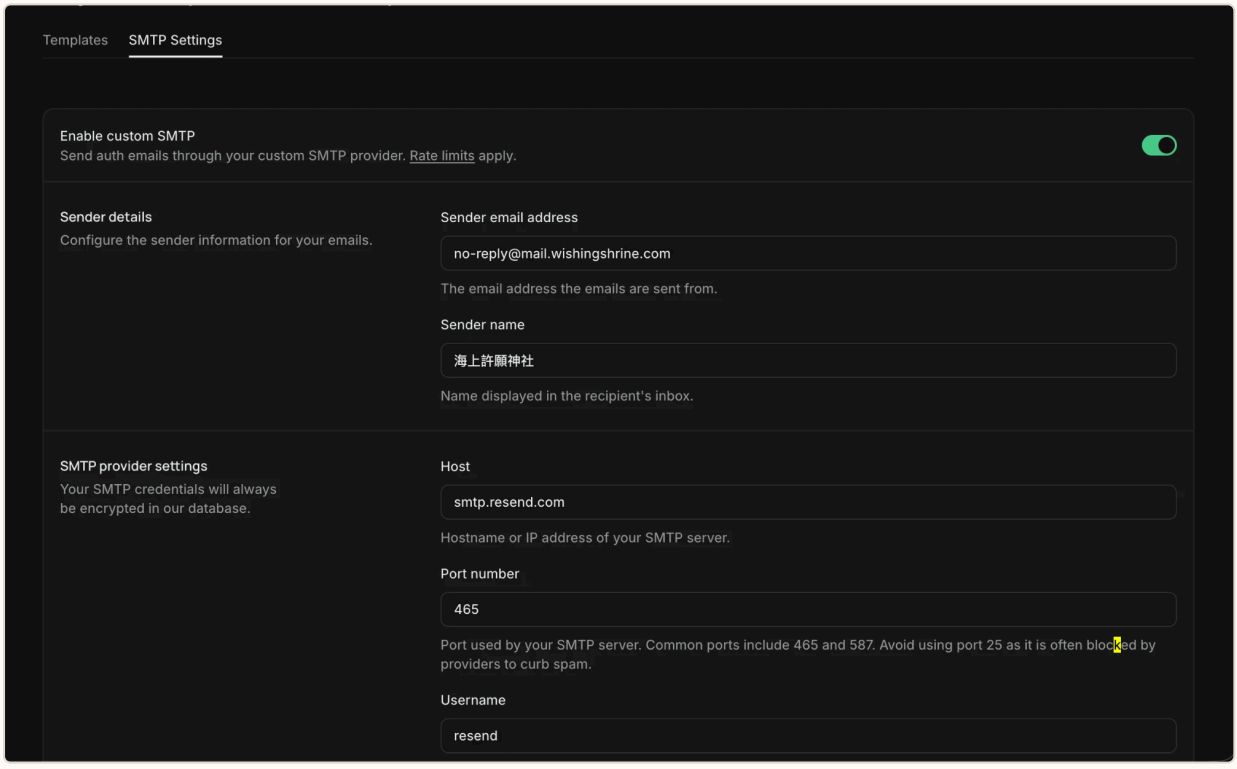
3. 在 Resend 建一把 API Key，權限只給寄信（Sending access）。這把 key 只貼到 Supabase，不要貼給 Claude，也不要放進 `.env` 或程式



這把 key 只會顯示這一次。按複製，直接貼進第 4 步的 Password，再按 Done。忘了複製就刪掉重建一把

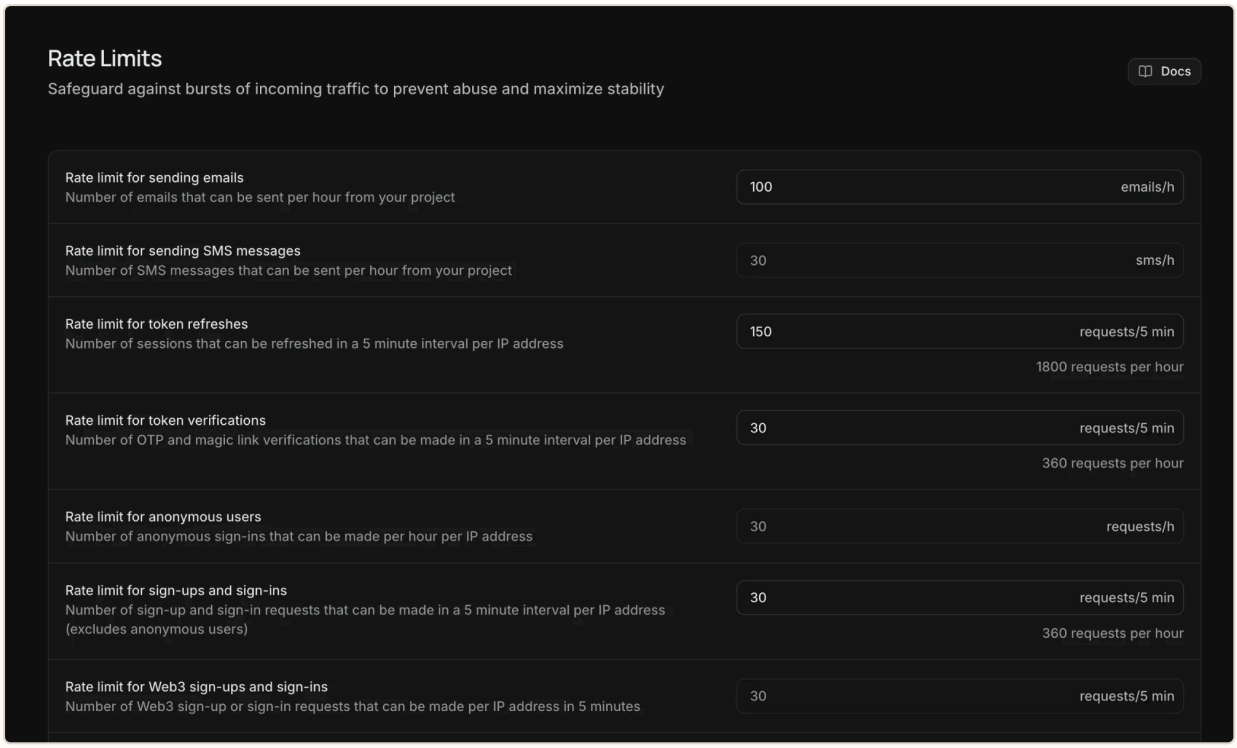
4. Supabase → Authentication → Emails → SMTP Settings，打開自訂 SMTP，填：

欄位	值
Host	`smtp.resend.com`
Port	`465`
Username	`resend`
Password	第 3 步的 API Key
Sender email	`no-reply@mail.wishingshrine.com`（換成你的子網域）
Sender name	`海上許願神社`



Password 欄在 Username 下面，畫面要往下捲才看得到。全部填完，整頁最下面還有一個 Save 要按，沒按就不會生效。

5. Supabase → Authentication → Rate Limits，把第一列 Rate limit for sending emails 調高，例如 100 emails/h



只改第一列。其他幾列（token verifications、sign-ups and sign-ins 等）是擋同一個人短時間內大量請求的，維持預設 6. 測試：用一個不是專案成員的信箱登入。確認收得到、沒有進垃圾信，點信裡的連結會回到 `https://www.wishingshrine.com`

2026-10-23 實測：用不是專案成員的信箱收到這封信，寄件人是自己設的名稱與 `no-reply@mail.wishingshrine.com`。點 Sign in 回到網站，登入成功。 信的內容是 Supabase 預設的英文範本

✅ 怎麼知道做對了

- [ ] 推上 GitHub 之前，`git ls-files` 的清單裡沒有 `.env`、`node_modules`、`dist`
- [ ] GitHub 專案是私人的
- [ ] Vercel 的 Environment Variables 有 `PUBLIC_SUPABASE_URL`、`PUBLIC_SUPABASE_ANON_KEY` 兩列，環境是 Production
- [ ] 正式網站的 Network 分頁看得到 `wishes` 的請求，狀態 `200`
- [ ] 正式網站的登入視窗寫「寄登入連結」，沒有開發用的密碼登入
- [ ] 正式網站的 `/api` 不帶權杖試呼叫，回 `401`
- [ ] Supabase 的 Redirect URLs 有正式網址
- [ ] 選做：Domains 列表裡，沒有 www 的網域 308 轉到 www；`astro.config.mjs` 的 `site` 是新網域；Supabase 的 Redirect URLs 有 `https://www.<新網域>/**`
- [ ] 選做：在新網址登入，點信裡的連結會回到新網址（2026-10-23 實測通過）
- [ ] 選做：Resend 的網域狀態是 Verified。Supabase 的 SMTP Settings 填好。用不是專案成員的信箱收得到登入信，點連結回到網站並登入（2026-10-23 實測通過）
- [ ] 選做：`https://www.<新網域>/` 回 `200`；沒有 www 的網址 `308` 轉到 www；`og:image` 是 `https://www.<新網域>/og.jpg`，回 `200`；舊的 vercel.app 網址仍然 `200`

全部在 2026-10-23 實測過。前六項在 20-6，第七項在 20-7，選做的四項在 20-8、20-9。

🆘 如果沒有成功

你可能會看到	通常是因為	怎麼辦
部署成功，但登入視窗寫「寄登入碼」、Network 裡沒有 `supabase`	建置時沒有拿到兩個變數	回 20-4 加變數，再照 20-5 Redeploy
加了變數、也 Redeploy 了，還是假後端	名稱少了 `PUBLIC_` 或拼錯；Environments 沒選 Production	對照 `.env` 的名稱一字一字比；確認環境
按了 Optional Integrations 的 Supabase Add	另外建了一個新的 Supabase	「我在 Vercel 按了 Supabase 的 Add，請帶我把它移除，先不要改任何程式」。網站要接的是你在階段 15 建的那個
在正式網站點信裡的連結，跳到 `localhost`	Redirect URLs 沒有正式網址，退回了 Site URL	照 20-7 加上正式網址

你可能會看到	通常是因為	怎麼辦
推上 GitHub 之後，正式網站沒有變；Redeploy 出現「Hobby teams do not support collaboration」	commit 作者的信箱沒有登記在你的 GitHub 帳號，Vercel 把它當成協作者，免費方案會擋下來	在 GitHub 帳號的 Emails 加上 commit 用的信箱並驗證。確認 Vercel 的 Login Connections 連的是這個 GitHub 帳號，再推一個新的提交。舊的提交按 Redeploy 可能還是被擋。不用按 Upgrade to Pro
掛了新網域之後，點登入信的連結還是跳回舊網址	Supabase 的 Redirect URLs 沒有加新網域，退回了 Site URL	照 20-8〈掛好之後要改的三個地方〉第 2 點，Redirect URLs 加 `https://<新網域>/**`，Site URL 改成新網域
Site URL 看起來改好了，登入信的連結還是跳到 `localhost:3000`	填完沒有按 Site URL 那一格自己的 Save changes	Save changes 還是亮綠色就是沒存。按下去，按鈕變暗才算存好
舊網址那一筆的登入信連結失效	Redirect URLs 那一筆少了 `https://` 或 `/**`（例如只寫 `<你的網址>.vercel.app`），Supabase 比對不到	刪掉那一筆，重新加 `https://<你的網址>.vercel.app/**`
打開新網域，瀏覽器說連線不安全	憑證還在發（Domains 顯示 Generating SSL Certificate）	等狀態變成正常再打開，不用改任何設定
分享到社群的預覽圖還是舊網址	`astro.config.mjs` 的 `site` 沒改	照 20-8〈掛好之後要改的三個地方〉第 1 點改 `site`，重新部署
別人說收不到登入信	內建寄信服務只寄給專案成員，每小時約 2 封	接上自己的寄信服務，見 20-9
找 DNS 時進到一個有 Redirect to Another Domain、308、Remove、Save 的畫面	那是網域本身的轉址設定，不是 DNS	按 Cancel，不要存。從同一頁最下面的「View DNS Records & More for …」進去
按了 Auto configure，跳出要授權 Vercel 的視窗	Resend 要透過 Vercel 幫你寫 DNS 紀錄	確認授權的是你自己的 Vercel 帳號再同意。不想授權就照 20-9 第 2 步手動加
加了 DMARC，用 `dig _dmarc.<你的網域> TXT` 卻查不到	這一筆沒有存進去（填完沒按 Add）。或是你用的 DNS 伺服器記住了先前「查不到」的結果，還沒更新	先看 Vercel 的 DNS 列表有沒有 `_dmarc` 這一筆，沒有就重填、按 Add。有的話，換一台查：`dig @8.8.8.8 _dmarc.<你的網域> TXT`，或直接問 Vercel 的伺服器：`dig @ns1.vercel-dns.com _dmarc.<你的網域> TXT`，等幾分鐘再試

你可能會看到	通常是因為	怎麼辦
Resend 的狀態一直是 Pending	DNS 還在生效，或紀錄沒加完、值貼錯	先等幾個小時。還是 Pending 的話，對照 Resend 表格逐筆檢查 Name、Type、Value
登入信進了垃圾信	收信的一方不確定這封信是你寄的	確認 DKIM、SPF 已經驗證通過，再加 `_dmarc` 的 TXT 紀錄
推上去之後才發現 `.env` 在 GitHub 上	`.gitignore` 沒寫，或是在寫之前就被追蹤了	刪掉那個檔案不夠，舊的版本還在。先到後台改掉兩個測試帳號的密碼，再請 Claude 把 `.env` 從追蹤裡移除。公開金鑰本來就公開，不用換

這一步失敗可以跳過嗎：可以。網站在本機一樣能用。要給別人用才需要這一步。

還沒做的事

自己的寄信服務已經接上（20–9），任何人都收得到登入信。接上之後也可以改信件範本、切回 6 位數登入碼（階段 15），這一版先不改。下面兩件還沒做：

1. 加上網站的安全標頭。正式網站目前只有 Vercel 預設的一個（HSTS，要求瀏覽器只用 https 連線）。其他的，例如限制網頁只能從哪些網域載入程式，要另外寫在 `vercel.json`
2. 建置時缺變數就直接失敗。在 Vercel 上建置時，少了這兩個變數就停下來，說出少了哪一個，不再悄悄退回假後端。Claude 提了這個做法，這一版還沒做

另外，照片還沒有人工審核（階段 18），兩個帳號互測還沒有跑（階段 19）。

📖 為什麼這樣做

🎯 這一步的重點

部署有兩個地方會悄悄出錯：推上去的檔案，和建置時拿到的設定。兩個都要自己打開來看，不能只看「成功」兩個字。

🧭 這一步在做什麼

把網站放上網路。程式碼放 GitHub，Vercel 從 GitHub 拿程式碼建置，給一個正式網址。Supabase 不用搬，正式網站和本機連的是同一個。

🧩 它在建構流程的位置

```
你的電腦 ──git push──▶ GitHub（私人）
                          │  自動
                          ▼
                    Vercel 建置 ◀── 環境變數（PUBLIC_SUPABASE_URL、PUBLIC_SUPABASE_ANON_KEY）
                          │  建置時寫進網頁
                          ▼
              https://<你的網址>.vercel.app ──▶ Supabase（和本機同一個）
```

💡 那段指令為什麼那樣寫

20–1 整理的那段，前三點都是「先列出來給我看」：要推的檔案、搜到的金鑰樣式、個資。`.gitignore` 寫對了不代表檔案沒被追蹤，要看實際的清單。第 2 點要 Claude 說出每一處「是真的金鑰，還是只提到名稱」。和 19–1 一樣，要它說出判斷的依據。第 4 點讓你在執行前看到指令。推上去的東西很難收回，這一步要先確認。

20–6 那段只問結果，不要修改。部署出了問題，你要先知道問題在哪一層（建置、變數、Supabase 設定），再決定改哪裡。

上線之後：換曲子、旗子的字、海浪聲

📖 讀完約 10 分鐘　🛠️ 動手約 30 分鐘

> - **這一步一定要懂**：「隱藏」一個檔案，要問它還在不在部署出去的資料夾。不在清單上，但還在 `public/`，任何人用網址都拿得到。
> - **這一步可以先不懂**：響度、頻譜重心怎麼量；canvas 怎麼載入字型。
> - **做完後你要能用自己的話回答**：兩段海浪聲，為什麼一段拿來循環，另一段只偶爾出現？

這一章沒有編號。它是網站上線之後，接著做的三件事：換背景音樂、改旗子上的字、換海浪聲。放在階段 20 後面，是因為第一件要用到階段 20 的知識：什麼東西會跟著部署出去。

⚖️ 動手前先想

階段 13 那一題再出現一次：這首曲子還能不能留在網站上？

不能。階段 13 已經判斷過它不能公開，部署成 Artifact 時也拿掉了（13–4）。到了階段 20，選了「全部推上去」。選項上寫著「之後部署到 Vercel 就會公開播放」，它就跟著上線了。同一個判斷，換一個部署的地方就要再做一次。

「隱藏」要做到哪裡？

做到檔案不在部署出去的資料夾。只從清單拿掉，網址還是有效的。驗證的方法是建置之後，在 `dist/` 裡搜那個檔名。

哪一段海浪拿來循環？

量頭尾的音量差。差得小，接回開頭聽不出來；差得大，每一輪接回開頭都會跳一下。下面〈上線後 3〉會量兩段新的海浪聲，實際差多少寫在那裡。不能循環的那段不用丟掉。如果它是一道浪從小漲到大，可以當成偶爾出現的大浪。

多畫一張貼圖，要先確認什麼？

確認它不會多用 `rand()`。這個場景的岩石、船都從同一條亂數排下來，多用一次，後面的位置全部換掉（階段 3）。

🛠️ 動手做

🧰 這一步的產出

```
_private/music/                          ← 拿掉的曲子移到這裡（.gitignore 排除，不部署、不推上 GitHub）
public/audio/music/                      ← 新增三首：glass-wind-chimes.mp3、frost-on-the-windowpane.mp3、after-the-rain.mp3
public/audio/                            ← sea-2.mp3 換成 sea-3.mp3；新增 wave-crash.mp3
src/shell/audio/config.js                ← 曲目清單、黃昏的曲子、存檔的鍵換成 v3、海浪的分工與大浪的間隔
src/engine/scene/flags.js                ← 兩面旗各畫一張貼圖，後面那面改成「潮音寄願」
src/engine/engine.js                     ← 建旗子之前先載入旗子要用的字
```

畫面上應該看到：聲音面板的選曲沒有 Merry Christmas，多了三首；黃昏播 Glass Wind Chimes。本殿前面兩面旗，一面「祈願成就」、一面「潮音寄願」。海浪聲偶爾會有一道大浪。

📋 貼給 AI（上線後 1：拿掉一首，加三首）

你在階段 13 已經準備了五首。這一節是當時的三首換成現在五首的過程，看判斷就好，不用再做一次。

附上新的曲子：

貼給 AI

```
@"/Users/《你的帳號》/Downloads/Glass Wind Chimes over the.mp3" @"/Users/《你的帳號》/Downloads/Frost on the Windowpane.mp3" @"/Users/《你的帳號》/Downloads/雨後の静けさ.mp3"
幫我隱藏坂本龍一，然後另外新增這三首
```

判斷：「隱藏」只把它從選曲清單拿掉，不夠。

- 檔案還在 `public/`，就會被部署出去。任何人知道網址 `/audio/music/…mp3` 就下載得到
- 20–1 選了「全部推上去」。所以階段 20 部署的正式網站裡有這首
- 這一章推上 GitHub 之後，Vercel 一開始沒有自動重新部署（原因見階段 20 的 🆘）。修好之後的提交部署成功，正式網址上的這首變成 404
- 只改本機、沒有重新部署，正式網站上的東西不會變。下架要在正式網址上確認

修法：

- 檔案移到 `_private/music/`，這個資料夾加進 `.gitignore`。不部署、不再推上 GitHub
- 建置之後，確認 `dist/` 裡沒有這個檔案
- 之前推上去的版本紀錄裡還有它。專案是私人的，不算公開散布；但要公開這個專案之前，要先把它從紀錄裡清掉

三首新曲子先量過再決定放哪裡：

曲目	響度	頻譜重心	放在	trim
Glass Wind Chimes over the	−14.6 LUFS	644 Hz（最溫暖）	黃昏	0.68
Frost on the Windowpane	−13.9 LUFS	1026 Hz（最明亮清冷）	手動選	0.62
雨後の静けさ	−14.8 LUFS	759 Hz	手動選	0.69

> **名詞**：**頻譜重心**是一段聲音「高音和低音的平均位置」，單位是 Hz。數字越低聽起來越暖、越暗，越高越亮。
> 你需要知道到什麼程度：知道 Claude 用它和 LUFS 決定哪一首放哪個時段。你用耳朵確認就好。

三首原本都比較大聲，用 `trim` 調到約 −18 LUFS，和白天、夜晚那兩首一樣（階段 13）。

其他跟著改的：

- 預設改回「隨時段」
- 存檔的鍵換成 `shrine.audio.v3`。舊的偏好是「手動：Merry Christmas」，不換鍵的話，開站會去找一首已經不存在的曲子。和 13-5 是同一個做法

驗證：舊偏好是「手動：Merry Christmas」，開站自動變成「隨時段」，黃昏播 Glass Wind Chimes。手動選雨後の静けさ、Frost on the Windowpane 都會播。選曲清單有六個選項（隨時段加五首）。`npm test` 17 項、建置通過。

📋 貼給 AI（上線後 2：旗子換一句字）

本殿前面兩面旗都寫「祈願成就」。想換一面，附上截圖：

貼給 AI

《附上截圖》

我想要其中一個調整到更有意境的文字

後面那面改成「潮音寄願」：把心願交給潮聲帶去，和海、浪聲扣在一起。原本兩面旗共用一張貼圖，改成各畫一張。

動手之前，Claude 先確認了兩件事：

要確認的事	為什麼要確認	結果
多畫一張貼圖，會不會多用 `rand()`	多用一次，後面的岩石、船就會換位置（`CLAUDE.md` 工程原則第 6 條）	畫字的程式和旗子的畫法都不用亂數，不會
新的字會不會用系統字型畫出來	網頁字型不會因為 canvas 要用就自動下載那幾個字。旗子用 600 的粗細，又是另一個字型檔	建旗子之前，先用 `document.fonts.load` 載入旗子要用的字與粗細

驗證：本殿鏡位截圖，兩面旗都是明體，岩石、船的位置不變。

這是階段 7 的同一個問題。繪馬上的字也是先載入字型，再畫到 canvas 上。

📋 貼給 AI（上線後 3：兩段新的海浪聲）

附上新的音效檔，請 Claude 評估怎麼用：

貼給 AI

`@"/Users/《你的帳號》/Downloads/sound-effect-3645.mp3" @"/Users/《你的帳號》/Downloads/sound-effect-186.mp3"`

另外提供兩種海浪的聲音，提供你交叉使用評估

Claude 把原本兩段和新的兩段一起量。量的是長度、響度、頻譜重心、頭尾 0.5 秒的音量差、每 2 秒的音量。

檔案	量到的	決定
sea-1（原本的）	安靜、起伏大	保留
sea-2（原本的）	頻譜重心 4000 Hz，聽起來沙沙的	拿掉
`sound-effect-186`	8 秒，頭尾音量只差 0.3 dB，中間湧起一道浪	最適合循環，取代 sea-2，存成 `sea-3.mp3`
`sound-effect-3645`	10 秒，從 −31 dB 漲到 −13 dB，最後突然結束，頭尾差 14 dB	不能循環。改成偶爾出現的大浪，存成 `wave-crash.mp3`

大浪的出現方式和雷聲一樣，間隔是隨機的：

天氣	間隔	音量
晴天、薄霧	20–45 秒一次	一般
大雨、雷雨	10–22 秒一次	比較大聲

兩段底層同時循環。一段 10 秒、一段 8 秒，合起來 40 秒才重複一次。大浪的間隔又是隨機的，聽不出規律。

驗證：開站後五個環境音都載入成功（`200`），0 個錯誤；`npm test`、建置通過。大浪聽起來自不自然，Claude 只能看數字。這一項要你自己聽。

授權：公開之前，要先確認這兩段音檔的出處與授權。下載音效時就把出處記下來，之後才查得到。

✅ 怎麼知道做對了

- [ ] 聲音面板的選曲有六個選項：隨時段，加上五首。沒有 Merry Christmas
- [ ] 時段切到黃昏，選曲選「隨時段」時，播的是 Glass Wind Chimes
- [ ] 請 Claude 執行 `npm run build`，在 `dist/` 裡搜不到 `merry-christmas`
- [ ] 本殿鏡位：兩面旗一面「祈願成就」、一面「潮音寄願」，都是明體。岩石、船的位置和改之前一樣
- [ ] 打開開發者工具的 Network 分頁，五個環境音（`sea-1`、`sea-3`、`wave-crash`、`rain`、`rain-thunder`）都是 `200`
- [ ] 聽兩分鐘，有偶爾的大浪，聽不出循環的接縫

前五項 Claude 實測過。最後一項 Claude 量不出來，要你自己聽。

🆘 如果沒有成功

你可能會看到	通常是因為	怎麼辦
清單上沒有那首了，但 `/audio/music/…mp3` 還下載得到	檔案還在 `public/`	「請把這個檔案移出 public/，加進 .gitignore，建置後確認 dist/ 裡沒有它」
開站沒有聲音，Console 說找不到某首曲子	舊的偏好還指著拿掉的那首	「請把聲音偏好的存檔名稱換一個新的」
旗子上的字是黑體，或是一格一格的方框	畫的時候字型還沒載入	「請在建旗子之前，用 document.fonts.load 載入旗子的字和粗細」
改了旗子之後，岩石和船換位置	新的貼圖用了共用的 `rand()`	「新加的東西請用自己的亂數，不要呼叫共用的 rand()」
海浪聲每隔幾秒就跳一下	頭尾音量差很多的那段被拿去循環	「請量每一段頭尾 0.5 秒的音量差，差很多的那段不要循環」

這一章失敗可以跳過嗎：可以。這三件都不影響網站的功能。用了別人的曲子，「移出 `public/`」那一項一定要做。

📖 為什麼這樣做

🎯 這一步的重點

上線之後，「從畫面上拿掉」和「從網站上拿掉」是兩件事。部署出去的是整個 `public/` 資料夾，不是畫面上看得到的東西。

🧭 這一步在做什麼

三件事都是換素材：曲子、旗子的字、海浪聲。換之前先量，換之後確認舊的東西真的不在了。

🧩 它在建構流程的位置

```
public/ ──建置──▶ dist/ ──部署──▶ 正式網站（任何人知道網址就拿得到）
   │
   └ 不要公開的檔案 ──移到──▶ _private/（.gitignore 排除）
```

💡 那段指令為什麼那樣寫

三則都很短。「幫我隱藏坂本龍一」沒有說隱藏到什麼程度，Claude 判斷要移出 `public/`，並在回報裡說明理由。如果你要的只是「先不要出現在清單上」，就要說清楚。反過來，如果你要的是「不要被下載到」，也要說清楚。

「提供你交叉使用評估」讓 Claude 先比較、再決定怎麼用，而不是直接替換。所以才有「一段循環、一段偶爾出現」的分工。

你也可以在要拿掉檔案時，直接寫出驗收的方法（整理版指令）：

貼給 AI

這首曲子不要再出現在網站上。不只是清單，檔案也要移出 `public/`，加進 `.gitignore`。

建置之後，在 `dist/` 裡搜這個檔名給我看，確認搜不到。

階段 21　延伸：御神籤

📖 讀完約 15 分鐘　🛠️ 動手約 60 分鐘

> - **這一步一定要懂**：抽到哪一支，由資料庫決定，畫面上的動畫只負責演出來。「一天一支」靠資料表的唯一約束，不是靠按鈕變灰。本機開發連的是正式的 Supabase，測試不要用真的帳號亂寫。
> - **這一步可以先不懂**：籤筒的搖晃怎麼用彈簧算、和紙的紋理怎麼畫。還有分享圖片怎麼用 canvas 產生。
> - **做完後你要能用自己的話回答**：訪客在瀏覽器裡抽到大吉，登入之後，為什麼不直接把這支籤存進帳號？

這一章是做完階段 20 之後加的延伸功能，網站已經上線。它把前面每一個階段的做法用在一個新功能上。這些做法是：先盤點、權限放在資料庫、測試不碰正式資料、上線前先問。

⚖️ 動手前先想

籤號誰決定？

資料庫。`omikuji_draw()` 用 `gen_random_uuid()` 的安全亂數選籤號，60 張等機率。前端不能指定籤號，也不能直接寫資料表。一天一支靠 `unique (user_id, draw_date)`。下面 21–5 的測試會同時送 12 個請求，確認資料表只有一筆。按鈕變灰只是讓畫面好看，擋不住直接打 API 的人。這是階段 14 那一題：「需要登入」算不算權限控管。

訪客的籤，登入後要不要匯入帳號？

不要。要安全地匯入，伺服器得先發一張證明「這支籤是我抽的」的票據。沒有票據，前端可以假造籤號，或一直重抽訪客籤，抽到大吉再匯入。所以按「登入收藏」時，先說明「登入後神社會為你的帳號抽出正式的籤，可能和這支不同」。帳號今天已經有籤就顯示帳號的；籤號不同時直接講。不在登入時悄悄換籤。

改籤文要不要覆寫？

不要。籤文只放在 `fortunes.js` 一個地方，資料庫的種子由它產生。改文字就升 `CONTENT_VERSION`，新版新增成新的列，舊版停用。已經抽過的紀錄指向舊版那一列，所以「我的御神籤」看到的是抽到當天的原文。被抽過的籤文刪不掉（外鍵設了 restrict）。

本機測試能不能用真的帳號？

> 不行。本機和正式網站連的是同一個 Supabase 專案，本機寫的就是正式資料。資料庫規則在本機暫時起的 PostgreSQL 測，介面用假後端測，正式環境只用自己的帳號抽一次。 21–3 會遇到的錯誤也說明了反過來的情況。本機測試從空的資料庫開始，測不到「正式資料庫裡已經有舊版」。改到正式資料庫的 SQL，測試要先放一份舊資料進去再跑。

🛠️ 動手做

🧰 這一步的產出

```
src/engine/scene/omikuji.js                ← 桌子、六角籤筒、木籤、托盤、立牌、結籤架；搖晃與揭曉的動畫
src/app/components/OmikujiDialog.jsx       ← React 御神籤介面
src/services/omikuji.js                    ← 訪客 / 會員抽籤資料
src/shell/omikuji/rules.js、share.js       ← 狀態機、台北日期與籤號、分享圖片
src/data/omikuji/fortunes.js、check.js     ← 60 張籤文（唯一來源）、籤文檢查
src/styles/omikuji.css                     ← 和紙、面板、手機版
supabase/migrations/004_omikuji.sql        ← 兩張資料表、RLS、抽籤與結籤的函式
supabase/migrations/005_omikuji_fortunes.sql ← 籤文種子（由 fortunes.js 產生，不手改）
supabase/functions/v1/index.ts             ← 對外 API 加 /omikuji/today、/draw、/history、/<id>/tie
tests/omikuji.test.mjs、omikuji-db.mjs、omikuji-e2e.mjs ← 單元、資料庫、端對端測試
docs/OMIKUJI.md                            ← 設計、規則、資料表、測試、部署與回滾
```

畫面上應該看到：本殿右前方多一張紅色小桌，上面有一個六角籤筒，旁邊是結籤架。點籤筒，鏡頭推近，下方出現說明與「開始搖籤」。

狀態	參考畫面
鏡頭推到籤筒	
搖籤中	

📋 貼給 AI（21–1：御神籤的規格）

這種大功能的規格很長，可以照〈用 ChatGPT 寫規格，再交給 Claude〉的做法，先請 ChatGPT 寫，再交給 Claude。實際用的規格包括這些部分：架構盤點、場景、互動、和紙、60 張籤文、每日規則。接著是收藏結籤分享、資料表與 API、狀態機、音效、效能、錯誤處理、測試、分階段交付。

先想一題：假設「抽到哪一支」是畫面上的動畫決定的。別人打開開發者工具，能不能讓自己每天都抽到大吉？

下面是精簡版，保留那份規格的結構與七條規則，可以直接貼。先把模式切到 Plan。

貼給 AI

你只需要修改：籤文的數量與吉凶分布可以換成你要的。七條規則都要留，其中第 2 條「籤號由伺服器決定」最不能刪。開頭的「先做架構盤點」與結尾的「不能用 Mock 冒充完成」也要留。

請在神社場景裡加一個「御神籤」功能。動手之前，先做架構盤點：
列出目前的前端、3D、動畫、介面、聲音、會員、資料庫、API 各是怎麼做的，御神籤要怎麼接，會影響哪些既有功能。
不能破壞已經能用的功能（繪馬、祈願、鏡位、天氣、聲音、登入）。

流程：點籤筒 → 鏡頭推近 → 搖籤（按鈕或左右拖曳）→ 木籤滑出 → 和紙展開 → 收藏、結籤、下載圖片、分享 → 關閉回到原本的鏡頭。

規則：
1. 一天一支，以台北時間 00:00 換日。登入的人以伺服器時間為準，不看裝置的時鐘
2. 籤號由伺服器用安全亂數決定。動畫只負責演出，前端不能指定籤號，也不能直接寫資料表
3. 同一個帳號同一天重送、開多個分頁、換裝置，都要拿到同一支
4. 未登入也能抽，結果存在這個瀏覽器，畫面上要寫清楚「只存在這個瀏覽器」
5. 籤文 60 張原創，只放在一個檔案裡；資料庫的籤文由它產生。改文字要升版本，不覆寫，舊紀錄保留抽到當天的原文
6. 資料表開 RLS，只讀得到自己的紀錄。寫入只能經過資料庫函式
7. 對外 API v1 加御神籤的端點，用呼叫者自己的權杖，不用 service_role

測試：
- 本機開發連的是正式的 Supabase，自動測試不要寫進正式資料。資料庫規則用本機暫時起的 PostgreSQL 測，介面流程用假後端跑
- 要測：一天一支、同時多個請求只有一筆、跨日、換版之後舊紀錄不變、訪客、鍵盤操作、減少動態效果、手機

以下要先問我，等我確認：會刪資料或改既有資料表的資料庫變更、在正式 Supabase 執行 SQL、部署。
不能用 Mock 冒充完成。哪些是假後端測的、哪些在正式環境測過，分開告訴我。
分階段交付，每一階段做完先給我看畫面再往下。

你應該看到：Claude 先交一份盤點，列出要新增的檔案。也列出要改的既有檔案（鏡位、點選、聲音、帳號視窗），以及整合風險。整合風險例如：新物件用了共用的 `rand()` 會讓岩石換位置、Esc 會被兩個地方攔。確認之後才動手。

📋 貼給 AI（21–2：做出來之後，檢查這幾件事）

用本機網址打開，抽一次、讀一次、關掉再開一次。一項一項看，有一樣的問題，附上截圖貼那一項；沒有就跳過。

1. 立牌上的字被柱子擋住

從預設的鏡頭看籤筒旁的立牌，字有沒有被柱子切掉？

貼給 AI

籤筒旁立牌上的字被柱子擋住了，請找原因，修完附一張同角度的特寫。

實際的原因：板子和柱子在同一個深度。修法是把板子移到柱子前面。

2. 揭曉時籤筒倒下去，穿進桌子

搖完之後，盯著籤筒倒過來的那一下：筒身有沒有穿進桌面？木籤有沒有掉在托盤裡？

貼給 AI

揭曉時籤筒倒下去穿進桌子，看起來卡住了。籤筒倒過來的時候，籤口要懸在托盤上方，木籤落進托盤，然後籤筒慢慢回正。請逐格截圖給我看。

實際的原因：籤筒以底部為轉軸倒 135°。修法是改成倒 115°，整個抬高。

3. 籤文從按鈕後面透出來

把籤紙捲到最下面：按鈕那一列後面，看不看得到籤文的字？

貼給 AI

籤紙捲到下面時，籤文從按鈕後面透出來。請找原因，先不要改。

實際的原因：按鈕列放在捲動區裡、設成 sticky，背景又是漸層。修法是把按鈕列移出捲動區。

4. 按鈕的焦點框怪怪的

用滑鼠點完之後，按鈕上是不是多了一圈框？

貼給 AI

用滑鼠點完，按鈕上多了一圈奇怪的框。滑鼠使用者不要看到，鍵盤使用者要看得到，請修。

實際的原因：程式把焦點移到按鈕上，滑鼠使用者也看到焦點框。修法是只有用鍵盤時才移焦點，焦點框改成貼著按鈕的金色。

5. 籤紙不像紙

貼給 AI

〈附上截圖〉

籤紙看起來像一張平面的卡片，不像和紙。請加上紙的纖維、摺痕、撕開的毛邊和朱印，影子要跟著毛邊。

6. 文字和框線太貼邊

貼給 AI

〈附上截圖〉

籤紙上的文字和框線太貼邊了，請加大內距，桌機和手機各截一張給我看。

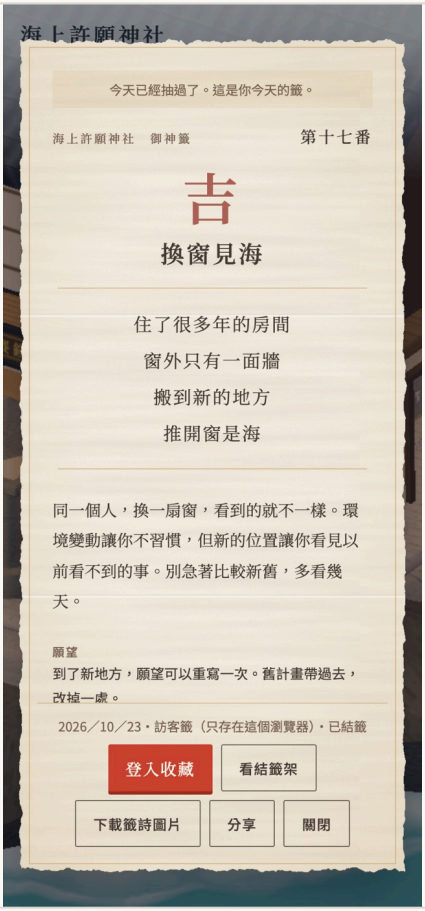
7. 籤文像生活提醒，不像籤

讀幾張籤文：詩是不是只在描述場景，籤意直接講結論？

貼給 AI

籤文我希望更有寓意。每首詩只寫一個意象的處境，籤意第一句先說破寓意，收尾改成短的偈語。

已經有人抽過舊版，請升內容版本，不要覆寫舊的籤文。

狀態	參考畫面
桌機（第 2 版籤文）	 同一天再打開，最上方會寫「今天已經抽過了」。
手機直式	 籤紙在畫面內，按鈕排成兩列。

狀態	參考畫面
另一支籤（小吉）	 還沒結籤時，按鈕是「結籤」；結過之後變成「看結籤架」。

你自己做（21–3：在 SQL Editor 建資料表與籤文）

1. 請 Claude 給你 `supabase/migrations/004_omikuji.sql` 的完整內容，貼進 SQL Editor，按 Run
2. 再請它給你 `005_omikuji_fortunes.sql`，一樣貼上、按 Run

你應該看到：

- 004 的結果：`omikuji_draws`、`omikuji_fortunes` 兩列，`rls` 都是 `true`
- 005 的結果：吉凶與張數

吉凶	張數
大吉	6
中吉	12
吉	15
小吉	12
末吉	9
凶	6

這兩段 SQL 會在正式的資料庫建表、寫入籤文。照指令的最後幾行，Claude 應該先問你，等你確認再給你貼。

改了籤文（升版本）之後，要再跑一次新的 005。實際第二次跑時出現：

```
ERROR: duplicate key value violates unique constraint
"omikuji_fortunes_one_active"
```

先想一想：本機的資料庫測試全部通過，為什麼正式的資料庫會失敗？

原因：新版的籤文直接以「啟用」狀態寫入，正式資料庫裡第 1 版也還在啟用。結果撞到「同一個籤號只能有一版啟用」。本機測試每次都從空的資料庫開始，沒有「已經有舊版在啟用」這種情況。那次失敗整段沒有執行，正式資料庫沒有變動。

貼給 AI

> 在正式資料庫跑新的 005 出現 duplicate key value violates unique constraint "omikuji_fortunes_one_active"。
> 請先不要改資料庫。告訴我原因，把 005 改成先以停用寫入、再停用舊版、最後啟用新版，並在資料庫測試補上「已經有另一版在啟用時跑 005」。

你自己做（21-4：重新部署對外 API）

網站本身不經過 `v1`，不部署也能抽籤。要讓對外 API 也有御神籤：

1. 請 Claude 給你最新的 `supabase/functions/v1/index.ts`
2. Supabase 後台 Edge Functions → `v1`，貼上，按 Deploy（做法和 17-3 一樣，部署完確認 Verify JWT 還是關的）

你應該看到：不帶權杖呼叫新端點，回 `401 missing_token`，不是 `404`。

```
curl -i https://<專案 ID>.supabase.co/functions/v1/v1/omikuji/today
```

回 `404` 代表部署的還是舊版；回大寫的 `UNAUTHORIZED_…` 代表 Verify JWT 又被打開了（階段 17）。

📋 貼給 AI（21-5：測試不要碰正式資料）

照這段請 Claude 跑測試（整理版指令）：

貼給 AI

請跑御神籤的三種測試，每一種告訴我結果：

1. `npm test`：單元測試（籤文、台北日期、籤號、訪客、狀態機）

2. `npm run test:db`：在本機暫時起一個 `PostgreSQL`，跑 `004`、`005`，測權限、一天一支、同時 12 個請求只有一筆、跨日、換版

3. 用假後端（`PUBLIC_AUTH_MOCK=1`）建置一份，跑端對端：訪客第一次抽、同一天再進來、連點五次、搖到一半按 `Esc`、鍵盤、減少動態效果、手機

這三種都不要連正式的 `Supabase`。

實測的結果：單元 24 項、資料庫 13 項、端對端 9 項全部通過。

最後用你自己的帳號，在正式網站抽一次：

- 抽到一支，重新整理之後還是同一支
- 按「結籤」，帳號視窗的「我的御神籤」顯示「已結籤」

實測抽到第十七番，重新整理仍是同一支，結籤之後顯示已結籤。

✅ 怎麼知道做對了

- [ ] 點籤筒，鏡頭推近；搖籤、揭曉、和紙展開；關閉之後回到原本的鏡頭
- [ ] 揭曉時籤筒不會穿進桌子，木籤落進托盤
- [ ] 同一天再打開，最上方寫「今天已經抽過了」，是同一支
- [ ] 訪客的籤紙上寫「只存在這個瀏覽器」
- [ ] 004 的 `rls` 兩列都是 `true`；005 的吉凶是 6 / 12 / 15 / 12 / 9 / 6
- [ ] `npm test`、`npm run test:db`、假後端的端對端都通過，而且都沒有連正式的 Supabase
- [ ] 不帶權杖呼叫 `/omikuji/today` 回 `401 missing_token`
- [ ] 用自己的帳號抽一次，重新整理是同一支；結籤後「我的御神籤」顯示已結籤
- [ ] 繪馬、祈願、鏡位、天氣、聲音都和加御神籤之前一樣

全部實測過。沒有測的：實機手機的 fps、iOS Safari 的分享選單、真機的觸控手感。

🆘 如果沒有成功

你可能會看到	通常是因為	怎麼辦
加了籤筒之後，岩石和船換位置	新物件用了共用的 `rand()`	「御神籤的物件請用自己的亂數，不要呼叫共用的 rand()」
按 Esc，籤的視窗和鏡位一起被關掉	好幾個地方都在攔 Esc	「御神籤的視窗收到 Esc 時，先處理自己的，不要再往外傳」
跑 005 出現 `omikuji_fortunes_one_active`	新版直接以啟用寫入，舊版也還在啟用	用 21–3 最後那段指令。那次失敗不會改到資料庫
同一個帳號在兩個分頁抽到不同的籤	一天一支只做在前端	「一天一支要靠資料表的唯一約束（user_id、日期），請確認 004 有這條」
呼叫 `/omikuji/today` 回 `404`	`v1` 還是舊版	回 21–4 重新部署
切到別的分頁再回來，動畫停住	分頁在背景時主迴圈停了，等動畫結束的程式一直等	「分頁切到背景時，動畫最多等兩秒就繼續」
本機測試寫進了正式資料庫	用 `npm run dev` 加真的帳號測	用 21–5 的三種測試。已經寫進去的紀錄，請 Claude 給你一段只刪自己帳號紀錄的 SQL，看過再跑

這一步失敗可以跳過嗎：可以。這是延伸功能，不影響前面的任何一個階段。

📖 為什麼這樣做

🎯 這一步的重點

抽籤的結果會被拿來比較、炫耀、重抽。誰決定結果、存在哪裡、能不能重來，都要放在別人改不到的地方：資料庫。

🧭 這一步在做什麼

在神社裡加一個完整的新功能：籤筒、搖籤、和紙、60 張籤文、一天一支。還有收藏、結籤、分享，以及對外 API。

誰	籤存在哪裡	能不能重抽
登入的人	資料庫 `omikuji_draws`，一個帳號一天一列	不能。重送、多分頁、多裝置都拿到同一支
訪客	這個瀏覽器的 localStorage	清掉瀏覽器資料、換瀏覽器就能重抽。只盡力而為，畫面上講清楚

🧩 它在建構流程的位置

```
點籤筒 ──▶ 搖籤（只是演出）──▶ omikuji_draw()（資料庫函式）
                                  │  安全亂數決定籤號
                                  │  unique (user_id, draw_date)：一天一列
                                  ▼
                              omikuji_draws ──▶ 和紙顯示 ──▶ 結籤：omikuji_tie()
```

💡 那段指令為什麼那樣寫

21–1 的「先做架構盤點」：御神籤要碰鏡位、點選、聲音、帳號視窗、資料庫、API。盤點時列出整合風險（共用亂數、Esc 被兩個地方攔），動手時才不會弄壞既有功能。「籤號由伺服器用安全亂數決定」寫在規則第 2 條。不寫的話，最直接的做法是動畫跑完時用 `Math.random()` 選一支，那就是前端決定。「以下要先問我」那一行：建表、寫入正式資料庫、部署都改得到正式環境。先看過再執行，這是操作卡 2 的第三層。「不能用 Mock 冒充完成」：假後端跑通，只代表介面流程對。要分開說哪些在正式環境測過，你才知道還差什麼。

21–2 的每一項都先寫看到的現象，再寫希望的樣子。第 2 項寫了「籤口懸在托盤上方，木籤落進托盤」。所以 Claude 才知道要改轉軸與角度，不只是調速度。

完成清單

- [ ] 二十個階段與〈一輪修正〉〈上線之後〉的 ✅ 驗收都通過了。標著「預期」的項目，拿你自己的結果對照
- [ ] 我的 `docs/DEVLOG.md` 每一個階段都有一筆，而且記了踩到的問題
- [ ] 我至少做過一次逐像素比對，看得懂 0% 的意思
- [ ] `CLAUDE.md` 的工程原則裡有一條：要接在一起的兩樣東西，位置從同一個常數算。底下列著本殿後排柱、碼頭木樁、繪馬架、船篷四個例子
- [ ] 我知道每一首背景音樂、每一段環境音的來源與授權狀態
- [ ] 我的 `.env` 只有可以公開的那把金鑰，Database password 不在專案裡
- [ ] 三張資料表的 `rls` 都是 `true`
- [ ] 我看過一次閘道回的 401，說得出它和 `missing_token` 的差別
- [ ] 我可以用自己的話說出：為什麼階段 14 的「需要登入」還不是權限控管
- [ ] 我用 `limit=2` 翻完過一次繪馬牆，沒有重複、沒有漏掉（階段 17）
- [ ] 我從外面下載過一張自己上傳的照片，確認沒有 EXIF（階段 18）
- [ ] 我說得出 Verify JWT 開著時，`me` 和 `v1` 的症狀差在哪裡
- [ ] 我跑過一次 `rls-check.mjs`，最後一行是 0 項失敗；跑完清掉了測試繪馬（階段 19）
- [ ] 推上 GitHub 之前，我看過 `git ls-files` 的清單（階段 20）
- [ ] 我在正式網站的 Network 分頁看過 `wishes` 的請求，確認不是假後端（階段 20）
- [ ] 建置之後，`dist/` 裡沒有我不想公開的檔案（〈上線之後〉）
- [ ] 延伸：御神籤的籤號由資料庫決定，同一個帳號一天只有一支。測試沒有寫進正式資料（階段 21）

離開這個範例，我會不會自己做？

回顧

#	問題	對應階段
1	給 AI 一個參考連結時，你會怎麼讓它有不照做的空間？	1
2	拆檔前後比對有 0.75% 不同。你會先做哪兩件事？	2
3	加了一張貼圖，岩石全部換位置。原因是什麼？	3
4	船的浮力為什麼要放在不碰 three.js 的檔案裡？	4
5	柱子沒接到屋頂。你會怎麼描述給 Claude，修完怎麼驗收？	6
6	換 React 之前，為什麼要先切開引擎與介面？	8
7	測試全過，畫面卻不對。你會先檢查什麼？	9
8	效能量兩次差 30%，你會怎麼回報？	10
9	字疊在 3D 場景上，對比要怎麼量？	11
10	規格寫了某個套件，你在安裝之前會先問什麼？	13
11	哪一把金鑰可以放前端？為什麼另一把不行？	15
12	你的設計要用到後台某個設定，動手之前你會先確認什麼？	15
13	收到登入信之後又按了幾次寄送，第一封信的連結就失效了。原因是什麼？	15
14	「誰掛了哪一塊」為什麼不放在公開的繪馬表？	16
15	API 為什麼用呼叫者的權杖查資料庫，不用 secret 金鑰？	16
16	網站上「掛繪馬」的按鈕已經會變灰，為什麼使用量限制還要放在資料庫？	17
17	七塊同時匯入的繪馬，一頁兩塊往下翻會漏掉。分頁要怎麼改？	17
18	公開的儲存桶，為什麼不建 select policy？	18
19	畫面上一塊地方拖不動，但什麼都看不到。你會怎麼找原因？	一輪修正
20	B 查 A 的祈願紀錄，拿到空的清單、沒有錯誤。這算擋住了嗎？為什麼？	19
21	Vercel 顯示部署成功，你怎麼確認正式網站接的是真的 Supabase？	20
22	在 Vercel 補了環境變數，網站還是舊的。為什麼？	20

#	問題	對應階段
23	一首曲子從選曲清單拿掉了，為什麼別人還下載得到？	上線之後
24	訪客在瀏覽器抽到大吉，登入後為什麼不直接存進帳號？	21

遷移

#	問題
25	你的下一個專案有兩樣東西要對齊（例如表格的欄寬和表頭）。你會怎麼確保它們不會分開？
26	你要把一個能用的網站換框架。你會怎麼證明換完之後「什麼都沒變」？
27	你拿到一份別人寫的長規格。在叫 AI 照做之前，你會先請它做什麼？
28	你的網站有一個按鈕寫著「需要登入」。你要怎麼確認它真的擋得住人？
29	畫面上有一個地方不對，但你不知道原因。你會怎麼寫給 AI？要附什麼？

第 26 題答不出來的話，回去看階段 2 與階段 8。

常見卡關

症狀	原因	解法
畫面一片空白，只有背景色	用檔案的方式打開了	請 Claude 給本機網址，在 Chrome 打開
程式改了，畫面沒變	看的是 Claude 桌面應用程式瀏覽器面板裡的舊快照	在 Chrome 打開 `http://localhost:…`
加了一個東西，岩石和船全部換位置	新東西用了共用的亂數	新東西用自己的亂數（階段 3）
柱子、木樁和它要接的東西之間有縫	兩邊各寫一個數字，例如木樁 ±1.5、碼頭板 ±1.3	用對方的常數算出位置，附接合處的特寫（階段 6）
逐像素比對有零點幾 % 的差異	畫面上有會自己變動的東西，或基準選錯了	先量雜訊底線，關掉模擬祈願，基準用同條件的截圖（階段 2、8）
測試偶爾失敗、重跑又過	測試等固定秒數，電腦忙的時候動畫還沒跑完	等條件成立，不要等秒數（階段 8）
效能數字每次都不一樣	SwiftShader 的雜訊約 ±25%	只做前後比較、單獨量；真機用 `?stats`（階段 10）
畫面縮小時下緣有空白	相機太近，前景被近裁切面切掉	拉遠相機，不是加大海面（階段 11）
換時段時介面先跳色	CSS 不能內插一串 RGB 數字	用 `@property` 註冊成顏色（階段 11）
改了預設值卻沒生效	瀏覽器存著舊的偏好	換一個新的存檔名稱（階段 13）
填了 `.env` 還是用假後端	沒重啟、等號旁有空白、加了引號	重啟開發伺服器，檢查格式（階段 15）
點了登入信的連結，沒有登入	Site URL 還是 `localhost:3000`、Redirect URLs 沒有 4324，或連結已經用過	改 URL Configuration；再寄一次，點最新那一封（階段 15）
一直顯示寄不出登入信	內建寄信服務整個專案每小時約 2 封	已經收到的信直接點；開發時用測試帳號的密碼登入（階段 15）
公開的 API 也回 `UNAUTHORIZED_NO_AUTH_HEADER`	新部署的函式 Verify JWT 還開著	關掉並按 Save changes，重新部署後再看一次（階段 17）

症狀	原因	解法
拿 `next` 要下一頁，回 400 或少了資料	分頁只用時間排序，或 `next` 沒有原樣帶回	排序用時間＋id，`next` 原樣帶回（階段 17）
能列出儲存桶裡的所有照片	公開儲存桶建了 select policy	不要建；看照片用公開網址就夠了（階段 18）
在介面上拖拉，選到一堆字	從介面的字開始拖，瀏覽器當成選取文字	介面文字設 `user-select: none`，輸入框除外（一輪修正）
某一塊拖不動，畫面上什麼都沒有	看不見的元素（收起來的面板、沒背景的外框）接走了滑鼠	用 `elementFromPoint` 查那一點最上面是誰（一輪修正）
`rls-check.mjs` 有一項 ✖	那一項對應的 policy 或設定沒有擋住	把那一行貼給 Claude，請它先說原因、不要修改，也不要改測試（階段 19）
正式網站部署成功，卻是「寄登入碼」、Network 裡沒有 `supabase`	建置時沒有拿到 `PUBLIC_` 變數，退回假後端	在 Vercel 加變數，再 Redeploy（階段 20）
從清單拿掉的檔案，用網址還下載得到	檔案還在 `public/`	移到 `public/` 外面並加進 `.gitignore`，建置後搜 `dist/`（〈上線之後〉）
API 回 401	看回應內容：大寫的 `UNAUTHORIZED_…` 是閘道擋的，`missing_token`、`invalid_token` 是 `me` 的程式回的	閘道擋的就關掉 Verify JWT；`invalid_token` 就重新拿一次權杖（階段 16）
卡超過 20 分鐘	原因還沒找到	整段錯誤訊息貼給 Claude，再加一句：「請先不要修改任何東西。先告訴我最可能的三個原因，以及你準備先檢查哪一個。」還是不行就先跳過，每個階段的 🆘 都寫了能不能跳過

名詞白話對照表

3D 與顯示卡

名詞	需要知道到什麼程度
WebGL / WebGPU	瀏覽器裡操作顯示卡的介面。WebGPU 比較新，不是每個瀏覽器都支援。這個案例全部用 WebGL
等角視角（正交相機）	沒有透視的鏡頭，遠的東西不會變小。畫面大小和相機距離無關
shader（著色器）	跑在顯示卡上、決定每個像素顏色的程式。和 JavaScript 是兩份程式
draw call	CPU 叫顯示卡畫一次東西的次數。變了代表多畫或少畫了東西
InstancedMesh	一次畫很多個相同形狀，只算一次 draw call
色調映射	把 3D 算出來的亮度壓進螢幕能顯示的範圍。換一種，整個畫面的顏色都變
選擇性泛光	只讓會自己發光的東西（燈籠、紙門）有光暈
近裁切面	比它更靠近相機的東西不畫。相機太近時前景會被切掉
臨界阻尼彈簧	追目標的方式：起步慢、中途加速、不會衝過頭。和幀率無關
SwiftShader	用 CPU 模擬顯示卡的軟體。量到的 fps 只能前後比較

驗證

名詞	需要知道到什麼程度
逐像素比對	兩張截圖每個像素拿來比，算出不同的比例。0% 代表完全相同
雜訊底線	同一個版本跑兩次的差異。比對之前要先量，確認是 0%
Node 測試	不開瀏覽器、直接用 Node 跑數學。只有不碰 three.js 的檔案能這樣測
對比	文字和背景的亮度比。4.5 以上一般文字才讀得清楚

聲音

名詞	需要知道到什麼程度
等功率交叉淡化	兩首交替時音量的平方加起來等於 1，正中間各約 0.707，聽起來不會凹下去

名詞	需要知道到什麼程度
LUFS	人耳聽起來多大聲的單位。每一首對到差不多的值，換曲時才不會忽大忽小
頻譜重心	一段聲音高音和低音的平均位置。越低聽起來越暖，越高越亮
壓縮器 / 限幅器	壓縮器把太大聲的部分壓下來；限幅器保證總輸出不超過上限

網站與會員

名詞	需要知道到什麼程度
Astro	把程式整理成網頁檔案的工具。這個案例用它產生首頁和 `/api`，階段 8 把專案搬進來
React / React Three Fiber	用元件組介面的工具，以及把 three.js 包成元件的寫法。這個案例的 3D 場景不改成後者
`textContent` / `innerHTML`	前者把文字當純文字放進畫面，後者當程式碼。使用者寫的內容只能用前者
`@property`	讓 CSS 知道一個自訂的值是顏色，才能慢慢過渡
OTP（一次性密碼）	每次登入寄一組短碼或一個連結，用過或過期就失效。假後端是 6 位數登入碼，真的 Supabase 目前寄連結
PKCE	跳轉回網站時網址上只帶一次性代碼，不帶登入權杖。登入連結要在寄信的同一個瀏覽器打開，再按一次寄送，舊的連結就失效。這個案例在 15–9 改掉了，現在用 implicit
implicit	跳轉回網站時，權杖放在網址 `#` 後面，不會送到伺服器。在哪個瀏覽器打開連結就在那裡登入。這個案例現在用這個
anon key / Publishable key	可以公開的金鑰，會被打包進網頁。能做什麼由 RLS 決定
service_role key / Secret key	不能外流的金鑰，無視所有權限規則。只能放伺服器
RLS（列層級安全）	資料表上的權限規則，管這個人能碰哪幾列。沒開的話，拿公開金鑰的人什麼都讀得到
`.env`	放網址與金鑰的設定檔。不進版本控制，改了要重啟開發伺服器
權杖（access token / JWT）	登入後 Supabase 發的字串，一小時過期，supabase-js 自動換新。等同一小時的登入狀態，不能貼給別人
Edge Function	放在 Supabase 上、收到請求才執行的程式。這個案例有 `me` 與 `v1` 兩支

名詞	需要知道到什麼程度
觸發器（trigger）	資料表新增一列時自動執行的程式。這個案例用它記下誰掛的、把票數加一
security definer	用建立者的權限執行的資料庫程式。權限比較大，要搭配 `search_path = ''`
curl	在終端機送出網路請求的工具。Windows PowerShell 要打 `curl.exe`
使用量限制	一段時間內最多能做幾次。這個案例是每個帳號 10 分鐘最多掛 5 塊，寫在資料庫的觸發器
分頁的游標	上一頁最後一筆的位置，下一頁從它後面接著拿。排序依據不能有兩筆一樣，所以用時間＋id
Storage / 儲存桶	Supabase 放檔案的地方。公開的儲存桶知道網址就看得到；上傳、刪除另外用 storage policy 管
EXIF	照片檔裡的拍攝資訊，手機照片常有 GPS 座標。上傳前要去掉
`elementFromPoint`	瀏覽器內建的函式，回傳畫面上某一點最上面的元素。點不到、拖不動時用它查
`user-select: none`	讓那一塊的文字不能被選取。按鈕、面板標題可以加，要讓人複製的地方不能加
`visibility: hidden` / `pointer-events: none`	前者看不見但還佔位置，後者不接收滑鼠。看不見、不佔位置、不接滑鼠是三件事
npm audit	檢查裝的套件有沒有已知安全問題的指令。0 代表目前沒有
XSS（跨站腳本攻擊）	把程式碼藏在文字裡，讓別人的瀏覽器執行。使用者寫的東西只用 `textContent` 或 canvas 放上畫面就不會發生
Security Advisor	Supabase 後台自動掃專案設定的頁面。列出的建議要先問清楚影響再改
GitHub / 私人專案	放程式碼的網站。私人專案只有自己和受邀的人看得到。推上去的檔案，刪掉之後舊版本還在紀錄裡
Vercel	從 GitHub 拿程式碼、建置、給網址的服務。之後每次推上去自動重新部署
環境變數	放在程式外面的設定值。本機在 `.env`，Vercel 在它的後台。`PUBLIC_` 開頭的在建置時寫進網頁，改了要重新部署
Redeploy	用同一份程式碼再建置、部署一次。補了環境變數之後要做

各階段預估時間

以下是依內容量估算的時間。

階段	主題	讀完	動手
1	先看成品，再做第一版海上神社	10 分	40 分
2	盤點、寫規格、拆檔	15 分	45 分
3	水面與光線	12 分	45 分
4	漣漪與船的浮力	15 分	50 分
5	時段與燈籠	8 分	30 分
6	接合處	10 分	35 分
7	繪馬卡片、字型與介面	12 分	45 分
8	搬進 Astro、介面改用 React	20 分	90 分
9	環境動態	12 分	45 分
10	時段轉場與鏡頭	12 分	45 分
11	介面方案比較	15 分	50 分
12	船型、雷雨、環境音	12 分	50 分
13	背景音樂	15 分	50 分
14	登入介面	12 分	40 分
15	接上 Supabase 的信箱連結登入	20 分	60 分
16	只有登入才拿得到的資料	18 分	50 分
17	對外 API、繪馬牆讀資料庫	15 分	45 分
18	繪馬附一張照片	12 分	40 分
一輪修正	拖拉、帳號名稱、小雨、游標、漁夫	12 分	40 分
19	資安檢查	15 分	40 分
20	部署到 Vercel	15 分	40 分
上線之後	換曲子、旗子的字、海浪聲	10 分	30 分

階段	主題	讀完	動手
21（延伸）	御神籤	15 分	60 分
	合計	**約 5.2 小時**	**約 17.8 小時**

不要跳過階段 2 和階段 8。跳掉它們，後面每一次修改都沒辦法證明其他東西沒壞。

合計超過十六小時，建議切成五次：

分段	階段	產出
第一次	階段 1 → 階段 6	有水面、船、時段，柱子、木樁、繪馬架都接上的場景
第二次	階段 7 → 階段 11	搬進 Astro、介面改用 React，有環境動態與鏡頭、介面定案
第三次	階段 12 → 階段 16	聲音、會員登入、需要權杖的 API
第四次	階段 17 → 一輪修正	公開的 API、照片上傳、使用者回報的修正
第五次	階段 19 → 上線之後	資安檢查、正式網站、上線後的素材調整
延伸	階段 21	御神籤（選做）